“十二五”普通高等教育本科国家级规划教材
中国大学MOOC课程和国家精品开放课程主讲教材配套参考书

“高等学校本科计算机类专业应用型人才培养研究”项目规划教材

C 语言程序设计学习指导（第 3 版）

Guide to Programming in C (Third Edition)

苏小红　王甜甜　车万翔　编著
王宇颖　主审

高等教育出版社

“十二五”普通高等教育本科国家级规划教材

中国大学MOOC课程和国家精品开放课程主讲教材配套参考书

“高等学校本科计算机类专业应用型人才培养研究”项目规划教材

C语言程序设计学习指导（第3版）

C Yuyan Chengxu Sheji Xuexi Zhidao

苏小红　王甜甜　车万翔　编著
王宇颖　主审

高等教育出版社·北京

内容简介

本书是国家精品开放课程、中国大学 MOOC 课程主讲教材《C 语言程序设计》(第 3 版)的配套参考书。

全书共 2 章，第 1 章为习题解答，包括主教材中的全部习题及解答。第 2 章为实验指导，共 6 部分。第 1 部分主要介绍 Visual C + + 6.0 和 Code::Blocks + gcc + gdb(为方便起见，本书将其简称为 Code::Blocks)集成开发环境下的标准 C 语言程序的调试方法；第 2 部分提供以程序设计方法和数据结构为主线设计的实验题目，内容既有趣味性，又有很强的应用背景，采用问题和任务驱动的方式，指导读者循序渐进地完成程序设计；第 3 部分介绍基于 C/S 结构的 C 语言编程题考试自动评分系统；第 4 部分介绍基于 B/S 结构的 C 语言编程题考试自动评分和题库管理系统；第 5 部分介绍在线评判网站的使用方法；第 6 部分介绍 C 语言程序设计的学习方法。

本书可作为高等学校各专业“C 语言程序设计”课程的教学参考书和计算机等级考试的参考书。

图书在版编目(CIP)数据

C 语言程序设计学习指导/苏小红，王甜甜，车万翔编著. --3 版. --北京：高等教育出版社，2015.8

ISBN 978-7-04-043320-3

Ⅰ. ①C… Ⅱ. ①苏… ②王… ③车… Ⅲ. ①C 语言-程序设计-高等学校-教学参考资料 Ⅳ. ①TP312

中国版本图书馆 CIP 数据核字(2015)第 155746 号

策划编辑 时 阳　　责任编辑 时 阳　　封面设计 张 志

插图绘制 杜晓丹　　责任印制 赵义民

出版发行 高等教育出版社

社 址 北京市西城区德外大街 4 号

邮政编码 100120

印 刷 北京东君印刷有限公司

开 本 850mm × 1168mm 1/16

印 张 21.25

字 数 530 千字

购书热线 010-58581118

咨询电话 400-810-0598

网 址 http://www.hep.edu.cn
http://www.hep.com.cn

网上订购 http://www.landraco.com
http://www.landraco.com.cn

版 次 2011 年 8 月第 1 版
2015 年 8 月第 3 版

印 次 2015 年 8 月第 1 次印刷

定 价 33.00 元

本书如有缺页、倒页、脱页等质量问题，请到所购图书销售部门联系调换

版权所有 侵权必究

物 料 号 43320-00

出版说明

信息化社会需要大量的计算机类专业人才。据统计，目前我国计算机类专业布点总数已逾2 800个，这些专业点为国家的现代化建设培养了大批计算机类专业人才，其中绝大多数是应用型人才。如何按照社会需求，确定合理的人才培养目标，并在其制导下培养特色突出的应用型人才，是提高教育质量和水平的重要任务。

为了更好地引导高校计算机类各专业点构建有特色的培养方案，例如，能够体现行业特色、区域需求，同时建设体现这些特色的学科基础课和专业课，促进本科计算机类专业应用型人才培养，出版一批体现应用型人才培养特色的新形态教材，教育部高等学校计算机类专业教学指导委员会、全国高等学校计算机教育研究会与高等教育出版社联合组建了“高等学校本科计算机类专业应用型人才培养研究”课题组，基于《计算机类专业教学质量国家标准》，围绕软件工程、网络工程、物联网工程等专业应用型人才培养的研究展开相关工作。

在研究的基础上，课题组汇聚80多所高校的教学经验，协同创新，开展了核心课程教学资源建设以及教材建设，这套教材作为课题研究的重要成果之一，具有以下几个显著特点。

- 以课题研制的《高等学校本科计算机类专业应用型人才培养指导意见》为指导，委托有丰富教学实践经验的教师编写，内容覆盖了不同专业的学科基础课、专业核心课及专业方向课。
- 教材内容基于理论适用，突出理论与实践相结合，强调“做中学”，引入丰富的实验案例，摒弃大而全、重理论轻实践的做法，结构新颖、努力突出专业特色。
- 采用纸质教材与数字资源相结合的形式，将教学内容与课程建设充分展示出来，使教师和学生借助网络实现全方位的个性化教学。

相信这套教材的出版能够起到推动各高校计算机类专业建设、提高教学水平和人才培养质量的作用。希望广大教师在教学过程中对教材提出宝贵的意见和建议，使其在使用过程中不断完善。

教育部高等学校计算机类专业教学指导委员会
全国高等学校计算机教育研究会
高等教育出版社
2015年3月

前　言

如何让学生通过“C 语言程序设计”课程的学习，轻松入门并掌握程序设计的精髓，养成良好的程序设计习惯，不让琐碎的语法细节成为学生理解程序设计方法和计算思维方法的障碍，一直是作者多年来孜孜以求的目标。多年 C 语言的教学经历，让作者将如下特点凝练于本书及其主教材《C 语言程序设计》(第 3 版)。

(1) 例题、习题和实验题兼顾趣味性和实用性，有实际的应用背景，面向工程实践和计算思维能力训练，绝非单纯为解释语法而设计。

(2) 将程序设计方法和数据结构作为两条脉络清晰的主线，避免语法堆砌、舍本求末、只见树木不见森林。

(3) 精心设计了一些含有隐蔽错误的分析改错题，注重错误程序的分析和讲解、程序设计风格、程序测试和调试方法。

希望上述风格独特的内容编排和设计能够让读者转变传统的学习观念，认识到学习 C 语言不是为了精通 C 语言的语法，而是以 C 语言为实践工具了解和掌握程序设计的思想和方法，达到今后无论使用什么语言编程，都具有灵活应用这些思想和方法解决实际问题的能力。

本书是《C 语言程序设计》(第 3 版)的配套教材，提供其全部习题的解答、实验指导。第 1 章习题解答中设计的习题是分阶梯的，包括改写例题的编程题、模仿例题的编程题、趣味游戏类编程题等，既有侧重程序阅读理解能力训练的写出程序运行结果的题目和程序填空题，还有侧重程序调试和排错能力训练的分析改错题以及侧重编程实践能力训练的任务递进式编程题。

第 2 章实验指导包括 6 部分内容。第 1 部分主要介绍 Visual C++ 6.0 和 Code::Blocks+gcc+gdb(本书后面将其简称为 Code::Blocks)集成开发环境下的标准 C 语言程序的调试方法。第 2 部分提供以程序设计方法和数据结构为主线设计的实验题目，内容既有趣味性，又有很强的应用背景，采用问题和任务驱动的方式，指导读者循序渐进地完成程序设计。其中，第 3 章~第 6 章的实验题主要采用自底向上的程序设计方法，侧重于 3 种基本控制结构的程序设计；第 7 章实验题开始采用自顶向下、逐步求精的模块化程序设计方法；第 8 章~第 13 章的实验题是一个贯穿全书、可作为课程设计内容的综合应用实例——学生成绩管理系统，从 V1.0 版本到 V6.0 版本所使用的数据结构从简单到复杂(从一维数组到二维数组，再到结构体数组或动态链表)，内容涵盖全书的所有知识点和算法(包括排序、查找等)。第 3 部分介绍基于 C/S 结构的 C 语言编程题考试自动评分系统。第 4 部分介绍基于 B/S 结构的 C 语言编程题考试自动评分和题库管理系统。第 5 部分介绍在线评判(Online Judge)。第 6 部分介绍 C 语言程序设计的学习方法。

本书与《C 语言程序设计》(第 3 版)是国家精品开放课程、中国大学 MOOC 课程使用教材，

与其配套的教学资源包括：

（1）面向读者的教材网站：http://book.sunner.cn；

（2）配合本书的中国大学 MOOC 课程网址：http://www.icourse163.org/course/hit-69005#/info（希望加入本课程 SPOC 的教师可与作者本人联系）；

（3）基于 Moodle 的交互式网络教学平台（乐学网）：https://cms.hit.edu.cn；

（4）面向学生竞赛的 ACM 程序设计竞赛网站：http://acm.hit.edu.cn/；

（5）基于 C/S 结构的 C 语言编程题考试自动评分系统（签署软件使用协议后可免费获取）；

（6）基于 B/S 结构的 C 语言编程题考试自动评分系统；

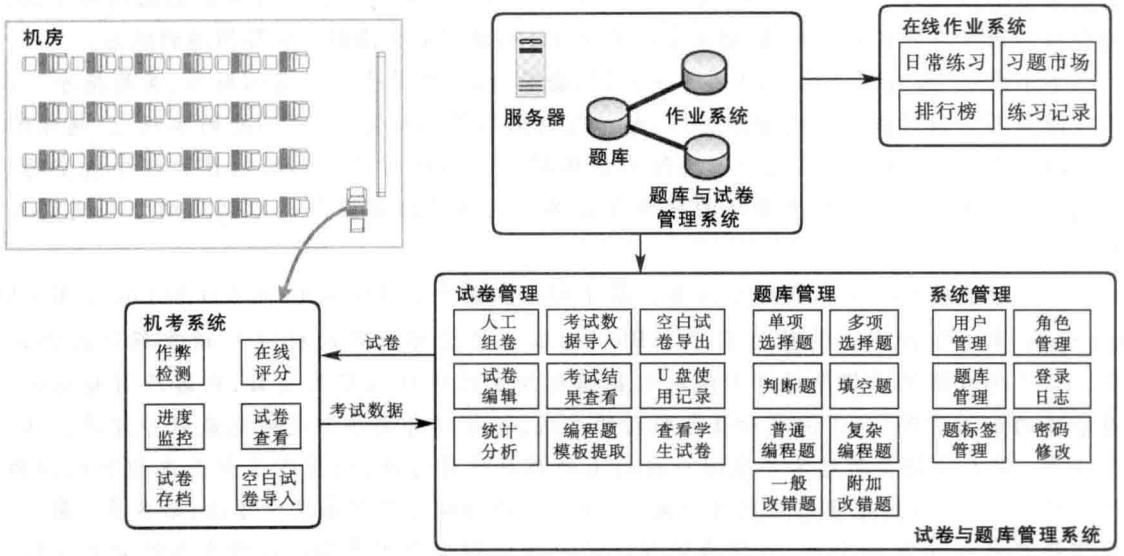
（7）C 语言试卷与题库管理系统（提供教材使用证明的教学单位可申请免费使用该系统的试用版）；

（8）面向学生自主学习的 C 语言在线作业和能力测试系统，使用《C 语言程序设计》（第 3 版）封四的刮刮卡可获得有效期一年的注册用户名，支持读者在线完成本书习题并获得系统自动评测结果，该系统从题库系统中随机抽题供读者练习；

（9）Code::Blocks+gcc+gdb 安装程序下载地址：http://www.codeblocks.org/downloads/26；

（10）教材的多媒体课件以及上述系统的简介和使用演示视频（扫描《C 语言程序设计》（第 3 版）的二维码查看）。

上述各种教学资源之间的关系如下图所示。有需要上述资源者可直接与作者本人联系（sxh@hit.edu.cn）。



全书的统稿工作由苏小红负责。Code::Blocks 集成开发环境的使用与调试方法简介、在线评测简介由车万翔编写，Visual C++集成开发环境的使用与调试方法简介由王甜甜编写，C 语言程序设计学习方法指南由苏小红和车万翔共同执笔，其余部分均由苏小红编写。

王宇颖教授在百忙之中审阅了全部书稿。赵玲玲、孙志岗、傅忠传、张彦航、孙承杰、张羽、郑贵滨、赵巍、郭萍、温东新、侯俊英、李希然、单丽莉、刘秉权、张卫、张洪志、王庆北、袁永峰、叶麟、

秦兵、李秀坤、孙大烈、刘旭东、郝惠馨、李漾、娄久、郭勇等参与了本书内容的校对工作。在此对他们的工作表示衷心的感谢。

因编者水平有限，书中错误在所难免，恳请读者来信批评指正，我们将在教材网站（网址为http://book.sunner.cn）上及时发布勘误信息。有索取教材相关资料者，请直接与作者联系，作者的E-mail地址为：sxh@hit.edu.cn。欢迎读者给我们发送电子邮件或在网站上留言，对教材提出宝贵意见，帮助我们不断完善本教材。

编著者

2015年于哈尔滨工业大学计算机科学与技术学院

郑重声明

高等教育出版社依法对本书享有专有出版权。任何未经许可的复制、销售行为均违反《中华人民共和国著作权法》，其行为人将承担相应的民事责任和行政责任；构成犯罪的，将被依法追究刑事责任。为了维护市场秩序，保护读者的合法权益，避免读者误用盗版书造成不良后果，我社将配合行政执法部门和司法机关对违法犯罪的单位和个人进行严厉打击。社会各界人士如发现上述侵权行为，希望及时举报，本社将奖励举报有功人员。

反盗版举报电话　(010)58581897　58582371　58581879

反盗版举报传真　(010)82086060

反盗版举报邮箱　dd@ hep.com.cn

通信地址　北京市西城区德外大街4号　高等教育出版社法务部

邮政编码　100120

目　录

第 1 章　习题解答 …… 1

1.1　习题 2 解答 …… 1

1.2　习题 3 解答 …… 1

1.3　习题 4 解答 …… 4

1.4　习题 5 解答 …… 6

1.5　习题 6 解答 …… 14

1.6　习题 7 解答 …… 39

1.7　习题 8 解答 …… 56

1.8　习题 9 解答 …… 98

1.9　习题 10 解答 …… 108

1.10　习题 11 解答 …… 124

1.11　习题 12 解答 …… 147

1.12　习题 13 解答 …… 160

第 2 章　实验指导 …… 169

2.1　集成开发环境简介 …… 169

2.1.1　Visual C++6.0 集成开发环境的使用与调试方法简介 …… 169

2.1.2　Code::Blocks 集成开发环境的使用与调试方法简介 …… 185

2.1.3　常见编译错误和警告信息的英汉对照 …… 202

2.2　实验题目与解答 …… 208

2.2.1　第 2 章实验题：熟悉上机实验环境和不同类型变量的定义与赋值 …… 208

2.2.2　第 3 章实验题：计算球的体积和表面积 …… 208

2.2.3　第 4 章实验题：大小写英文字母转换 …… 210

2.2.4　第 5 章实验题：身高预测、体型判断 …… 211

2.2.5　第 6 章实验题：国王的许诺、小学生计算机辅助教学系统 …… 213

2.2.6　第 7 章实验题：素数探求、小学生计算机辅助教学系统 …… 228

2.2.7　第 8 章实验题：学生成绩管理系统 V1.0 …… 254

2.2.8　第 9 章实验题：学生成绩管理系统 V2.0 …… 259

2.2.9　第 10 章实验题：学生成绩管理系统 V3.0 …… 266

2.2.10　第 11 章实验题：学生成绩管理系统 V4.0 …… 274

2.2.11　第 12 章实验题：学生成绩管理系统 V5.0 …… 284

2.2.12　第 13 章实验题：学生成绩管理系统 V6.0 …… 295

2.3　基于 C/S 结构的 C 语言编程题考试自动评分系统简介 …… 307

2.4　基于 B/S 结构的 C 语言编程题考试自动评分和题库管理系统简介 …… 314

2.5　在线评判简介 …… 319

2.6　C 语言程序设计学习方法指南 …… 324

参考文献 …… 327

第1章 习题解答

1.1 习题2解答

2.1 以下不正确的C语言标识符是(　　)。

A. AB1　　B. a2_b　　C. _ab3　　D. 4ab

答案:D

2.2 下面程序为变量x,y,z赋初值2.5,然后在屏幕上打印这些变量的值。程序中存在错误,请改正错误,并写出程序的正确运行结果。

```
#include <stdio.h>
main()
{
    printf("These values are :\n");
    int x = y = 2.5;
    printf("x = %d\n", X);
    printf("y = %d\n", y);
    printf("z = %d\n", Z);
}
```

错误原因分析略。修正错误后的参考程序如下:

```
#include <stdio.h>
main()
{
    float x = 2.5, y = 2.5, z = 2.5;
    printf("These values are :\n");
    printf("x = %f\n", x);
    printf("y = %f\n", y);
    printf("z = %f\n", z);
}
```

程序的运行结果如下:

```
These values are :
x = 2.500000
y = 2.500000
z = 2.500000
```

1.2 习题3解答

3.1 分析并写出下列程序的运行结果。

(1)

```
#include <stdio.h>
main()
{
    int a = 12, b = 3;
    float x = 18.5, y = 4.6;
    printf("%f\n", (float)(a * b) / 2);
    printf("%d\n", (int)x %(int)y);
}
```

(2)

```
#include <stdio.h>
main()
{
    int x = 32, y = 81, p, q;
    p = x++;
    q = --y;
    printf("%d %d\n", p, q);
    printf("%d %d\n", x, y);
}
```

参考答案:

(1) 程序的运行结果如下:

```
18.000000
2
```

(2) 程序的运行结果如下:

```
32 80
33 80
```

3.2 参考例3.1的程序,从键盘任意输入一个3位整数,编程计算并输出它的逆序数(忽略整数前的正负号)。例如,输入-123,则忽略负号,由123分离出其百位1、十位2、个位3,然后计算3×100+2×10+1=321,并输出321。

参考程序如下:

```
#include <math.h>
#include <stdio.h>
main()
{
```

```
5        int x, b0, b1, b2, y;
6        printf("Input x:");
7        scanf("%d", &x);
8        x = (int)fabs(x);
9        b2 = x / 100;                /* 计算百位数字 */
10       b1 = (x - b2 * 100) / 10;    /* 计算十位数字 */
11       b0 = x %10;                  /* 计算个位数字 */
12       y = b2 + b1 * 10 + b0 * 100;
13       printf("y = %d\n",y);
14   }
```

程序的运行结果如下：

```
Input x: -123↙
y = 321
```

3.3 设银行定期存款的年利率 *rate* 为2.25%，已知存款期为 *n* 年，存款本金为 *capital* 元，试编程计算并输出 *n* 年后的本利之和 *deposit*。

参考程序如下：

```
1    #include <math.h>
2    #include <stdio.h>
3    main()
4    {
5        int n;                       /* 存款期变量声明 */
6        double rate;                 /* 存款年利率变量声明 */
7        double capital;              /* 存款本金变量声明 */
8        double deposit;              /* 本利之和变量声明 */
9        printf("Please enter rate, year, capital:");
10       scanf("%lf,%d,%lf", &rate, &n, &capital);
11       deposit = capital * pow(1+rate, n); /* 计算存款利率之和 */
12       printf("deposit = %f\n", deposit); /* 打印存款利率之和 */
13   }
```

程序的运行结果如下：

```
Please enter rate, year, capital:0.0225,2,10000↙
deposit = 10455.062500
```

3.4 编程计算并输出一元二次方程 $ax^2+bx+c=0$ 的两个实根 $\frac{-b\pm\sqrt{b^2-4ac}}{2a}$，其中 a、b、c 的值由用户从键盘输入，假设 a、b、c 的值能保证方程有两个不相等的实根（即 $b^2-4ac>0$）。

参考答案：根据一元二次方程的求根公式：

$$x_{1,2}=\frac{-b\pm\sqrt{b^2-4ac}}{2a}=-\frac{b}{2a}\pm\frac{\sqrt{b^2-4ac}}{2a}$$

令

$$p=-\frac{b}{2a},\quad q=\frac{\sqrt{b^2-4ac}}{2a}$$

则有

$$x_1=p+q,\ x_2=p-q$$

参考程序如下：

```
#include <math.h>
#include <stdio.h>
main()
{
    float a, b, c, disc, p, q;
    printf("Please enter the coefficients a,b,c:");
    scanf("%f,%f,%f", &a, &b, &c);/* 要求输入满足方程有两个不相等实根的条件 */
    disc = b * b - 4 * a * c;        /* 计算判别式 */
    p = - b / (2 * a);
    q = sqrt(disc) / (2 * a);
    printf("x1=%7.4f, x2=%7.4f\n", p+q, p-q);
}
```

程序的运行结果如下：

```
Please enter the coefficients a,b,c:2,6,1↙
x1=-0.1771, x2=-2.8229
```

注意：本程序要求用户输入的数据满足 $b^2-4ac>0$ 这个约束条件，否则，如果 $b^2-4ac<0$，将会执行对负数开方的无效运算。运用第4章介绍的条件语句进行编程，可以有效解决这一问题。

1.3 习题4解答

4.1 分析并写出下面程序的运行结果。

(1)

```
#include <stdio.h>
main()
{
    char c1 = 'a', c2 = 'b', c3 = 'c';
    printf("a%cb%cc%c\n", c1, c2, c3);
}
```

(2)

```
#include <stdio.h>
main()
{
```

```
4         int a = 12, b = 15;
5         printf("a = %d%%, b = %d%% \n", a, b);
6     }
```

(3) 假设程序运行时输入123456。

```
1     #include <stdio.h>
2     main()
3     {
4         int a, b;
5         scanf("%2d%* 2s%2d", &a, &b);
6         printf("%d,%d\n", a, b);
7     }
```

参考答案:

(1) aabbcc

(2) a = 12%, b = 15%

(3) 123456↙

12,56

4.2 分析下面的程序,指出程序错在哪里和错误的原因,并改正错误。

错误原因分析略。修正错误后的参考程序如下:

```
1     #include <stdio.h>
2     main()
3     {
4         long a, b;
5         float x, y;
6         scanf("%d, %d", &a, &b);
7         scanf("%f, %f", &x, &y);
8         printf("a = %d, b = %d\n", a, b);
9         printf("x = %5.2f, y = %5.2f\n", x, y);
10    }
```

程序的运行结果如下:

```
12,3↙
12.345,56.789↙
a = 12, b = 3
x = 12.35, y = 56.79
```

4.3 填空题。

(1) 要使下面的程序在屏幕上显示1,2,34,则从键盘输入的数据格式应为_____。

```
1     #include <stdio.h>
2     main()
3     {
```

```
4        char a,b;
5        int c;
6        scanf("%c%c%d", &a, &b, &c);
7        printf("%c,%c,%d\n", a, b, c);
8    }
```

(2) 在与上面程序的输入相同的情况下,若将程序中的第7条语句修改为

```
printf("%-2c%-2c%d\n", a, b, c);
```

则程序的屏幕输出为_____。

(3) 要使上面程序的数据输入格式为1,2,34,输出语句在屏幕上显示的结果也为1,2,34,则应将程序中的第6条语句修改为_____。

(4) 在(3)的程序基础上,程序仍然输入1,2,34,若将程序中的第7条语句修改为

```
printf("\'%c\',\'%c\',%d\n", a, b, c);
```

则程序的屏幕输出为_____。

(5) 要使上面的程序无论用下面哪种格式输入数据,在屏幕上的输出结果都为1,2,34,则应将程序中的第6条语句修改为_____。

第1种输入方式:1,2,34↙(以逗号作为分隔符)

第2种输入方式:1 2 34↙(以空格作为分隔符)

第3种输入方式:1 2 34↙(以Tab键作为分隔符)

第4种输入方式:1↙

2↙

34↙(以回车符作为分隔符)

参考答案:

(1) 1234或者12 34

(2) 1 2 34

(3)

```
1  #include <stdio.h>
2  main()
3  {
4      char a,b;
5      int c;
6      scanf("%c,%c,%d", &a, &b, &c);
7      printf("%c,%c,%d\n", a, b, c);
8  }
```

(4) '1','2',34

(5) scanf("%c%*c%c%*c%d", &a, &b, &c);

1.4 习题5解答

5.1 从键盘任意输入一个实数,不使用计算绝对值函数编程计算并输出该实数的绝对值。

参考程序如下:

```
#include <stdio.h>
main()
{
    float x;
    printf("Input a float number:");
    scanf("%f", &x);
    if (x < 0) x = -x;
    printf("Absolute value of x is %f\n", x);
}
```

程序的运行结果如下:

```
Input a float number: -2.3↙
Absolute value of x is 2.300000
```

5.2 从键盘任意输入一个整数,编程判断它的奇偶性。

参考程序如下:

```
#include <stdio.h>
main()
{
    int a;
    printf("Input an integer number:");
    scanf("%d", &a);
    if (a%2 == 0)
        printf("a is an even number\n");
    else
        printf("a is an odd number\n");
}
```

程序的两次测试结果如下:

```
① Input an integer number: 2↙
  a is an even number
② Input an integer number: 5↙
  a is an odd number
```

5.3 在例3.8的基础上,从键盘任意输入三角形的三边长为 a,b,c,编程判断 a,b,c 的值能否构成一个三角形,若能构成三角形,则计算并输出三角形的面积,否则提示不能构成三角形。已知构成三角形的条件是:任意两边之和大于第三边。

参考程序如下：

```
#include <stdio.h>
#include <math.h>
main()
{
    float a, b, c, s, area;
    printf("Input a,b,c:");
    scanf("%f,%f,%f", &a, &b, &c);
    if (a+b>c && b+c>a && a+c>b)
    {
        s = (float)(a + b + c) / 2;
        area = sqrt(s * (s - a) * (s - b) * (s - c));
        printf("area = %f\n", area);
    }
    else
    {
        printf("It is not a triangle\n");
    }
}
```

程序的两次测试结果如下：

① Input a,b,c:3,4,5↙
area = 6.000000

② Input a,b,c:3,4,8↙
It is not a triangle

5.4 假设银行整存整取不同期限的年息利率分别为：

$$\text{年息利率}=\begin{cases}2.25\% & \text{期限 1 年}\\ 2.43\% & \text{期限 2 年}\\ 2.70\% & \text{期限 3 年}\\ 2.88\% & \text{期限 5 年}\\ 3.00\% & \text{期限 8 年}\end{cases}$$

要求输入存钱的本金和期限，求到期时能从银行得到的利息与本金的总合。

参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
main()
{
    int year;
```

```
7       double rate, capital, deposit;
8       printf("Please enter year,capital:");
9       scanf("%d,%lf", &year, &capital);
10      switch(year)
11      {
12          case 1:rate = 0.0225;break;
13          case 2:rate = 0.0243;break;
14          case 3:rate = 0.0270;break;
15          case 5:rate = 0.0288;break;
16          case 8:rate = 0.0300;break;
17          default:printf("Error rate! \n");
18                  exit(0);
19      }
20      deposit = capital * pow(1+rate, year);
21      printf("rate = %f, deposit = %f\n", rate, deposit);
22      return 0;
23  }
```

程序的两次测试结果如下：

① Please enter year,capital:2,10000↙
rate = 0.024300, deposit = 10491.904900

② Please enter year,capital:4,10000↙
Error rate!

5.5 阅读下面的程序，按要求在空白处填写适当的表达式或语句，使程序完整并符合题目要求。已知下面程序的功能是：从键盘任意输入一个年号，判断它是否是闰年。若是闰年，输出“Yes”；否则输出“No”。已知符合下列条件之一者是闰年：(1)能被4整除，但不能被100整除；(2)能被400整除。

```
1   #include <stdio.h>
2   main()
3   {
4       int year, flag;
5       printf("Input a year:");
6       scanf("%d", &year);
7       if (          ①          )
8           flag = 1;              /* 如果 year 是闰年,则标志变量 flag 置 1 */
9       else
10          flag = 0;              /* 否则,标志变量 flag 置 0 */
11      if (    ②    )
12          printf("%d is a leap year! \n", year);    /* 打印“是闰年” */
```

```
13        else
14            printf("%d is not a leap year! \n", year); /* 打印“不是闰年” */
15    }
```

参考答案：

① year%4 == 0 && year%100! = 0 ||year%400 == 0

② flag

5.6 将习题 5.5 程序中的第 7~10 行的 if 语句改用条件表达式实现，重新编写该程序。

参考程序如下：

```
1     #include <stdio.h>
2     main()
3     {
4         int year, flag;
5         printf("Input a year:");
6         scanf("%d", &year);
7         flag = (year%4 == 0 && year%100 ! = 0 ||year%400 == 0) ? 1 : 0;
8         if (flag)
9             printf("%d is a leap year! \n", year);     /* 打印“是闰年” */
10        else
11            printf("%d is not a leap year! \n", year); /* 打印“不是闰年” */
12    }
```

程序的 4 次测试结果如下：

① Input a year:2004↙

2004 is a leap year!

② Input a year:2009↙

2009 is not a leap year!

③ Input a year:2100↙

2100 is not a leap year!

④ Input a year:2000↙

2000 is a leap year!

5.7 在例 4.2 和第 4 章实验程序的基础上，从键盘输入一个英文字母，如果它是大写英文字母，则将其转换为小写英文字母，如果它是小写英文字母，则将其转换为大写英文字母，然后将转换后的英文字母及其 ASCII 码值显示到屏幕上；如果不是英文字母，则不转换直接将它及其 ASCII 码值输出到屏幕上。

参考程序如下：

```
1     #include <stdio.h>
2     main()
3     {
4         char ch;
```

```
5        printf("Press a key and then press Enter:");
6        ch = getchar();
7        if (ch >= 'A' && ch <= 'Z')
8            ch = ch + 32;
9        else if (ch >= 'a' && ch <= 'z')
10           ch = ch - 32;
11       printf("%c, %d\n", ch, ch);
12   }
```

程序的3次测试结果如下：

① Press a key and then press Enter: A↙
a, 97

② Press a key and then press Enter: a↙
A, 65

③ Press a key and then press Enter: 1↙
1, 49

5.8 从键盘任意输入一个字符，编程判断该字符是数字字符、大写字母、小写字母、空格还是其他字符。

参考程序如下：

```
1    #include <stdio.h>
2    main()
3    {
4        char ch;
5        printf("Press a key and then press Enter:");
6        ch = getchar();
7        if (ch >= 'a' && ch <= 'z' || ch >= 'A' && ch <= 'Z')
8            printf("It is an English character! \n");
9        else if (ch <= '9' && ch >= '0')
10           printf("It is a digit character! \n");
11       else if (ch == ' ')
12           printf("It is a space character! \n");
13       else
14           printf("It is other character! \n");
15   }
```

程序的5次测试结果如下：

① Press a key and then press Enter: A↙
It is an English character!

② Press a key and then press Enter: a↙
It is an English character!

③ Press a key and then press Enter: 9↙
It is a digit character!
④ Press a key and then press Enter: ↙
It is a space character!
⑤ Press a key and then press Enter: #↙
It is other character!

5.9 参考例 5.8 程序的测试结果,改用 if-else 语句编程,将输入的百分制成绩 score 转换成相应的五分制成绩 grade 后输出。已知转换标准为:

$$\text{grade}=\begin{cases}\text{A} & 90\leqslant \text{score}\leqslant 100\\ \text{B} & 80\leqslant \text{score}<90\\ \text{C} & 70\leqslant \text{score}<80\\ \text{D} & 60\leqslant \text{score}<70\\ \text{E} & 0\leqslant \text{score}<60\end{cases}$$

参考程序如下:

```
#include <stdio.h>
main()
{
    int score;
    printf("Please enter score:");
    scanf("%d", &score);
    if (score<0 || score>100)    printf("Input error! \n");
    else if (score>=90)          printf("%d——A \n", score);
    else if (score>=80)          printf("%d——B \n", score);
    else if (score>=70)          printf("%d——C \n", score);
    else if (score>=60)          printf("%d——D \n", score);
    else                         printf("%d——E \n", score);
}
```

程序的 15 次测试结果如下:

① Please enter score:0↙
0—E
② Please enter score:15↙
15—E
③ Please enter score:25↙
25—E
④ Please enter score:35↙
35—E
⑤ Please enter score:45↙
45—E

```
⑥ Please enter score:55↙
  55—E
⑦ Please enter score:65↙
  65—D
⑧ Please enter score:75↙
  75—C
⑨ Please enter score:85↙
  85—B
⑩ Please enter score:95↙
  95—A
⑪ Please enter score:100↙
  100—A
⑫ Please enter score:-10↙
  Input error!
⑬ Please enter score:200↙
  Input error!
⑭ Please enter score:105↙
  Input error!
⑮ Please enter score:-5↙
  Input error!
```

5.10 参考习题5.5中判断闰年的方法,从键盘输入某年某月(包括闰年),用switch语句编程输出该年的该月拥有的天数。要求考虑闰年以及输入月份不在合法范围内的情况。已知闰年的2月有29天,平年的2月有28天。

参考程序如下:

```
1    #include <stdio.h>
2    main()
3    {
4        int year, month;
5        printf("Input year,month:");
6        scanf("%d, %d", &year, &month);
7        switch (month)
8        {
9            case 1:
10           case 3:
11           case 5:
12           case 7:
13           case 8:
14           case 10:
```

```
15          case 12:printf("31 days\n");
16                  break;
17          case 2:if((year % 4== 0 && year % 100 != 0)||(year % 400 == 0))
18                 {
19                     printf("29 days\n");       /* 闰年的 2 月有 29 天 */
20                 }
21                 else
22                 {
23                     printf("28 days\n");       /* 平年的 2 月有 28 天 */
24                 }
25                 break;
26          case 4:
27          case 6:
28          case 9:
29          case 11:printf("30 days\n");
30                  break;
31          default:printf("Input error! \n");
32      }
33  }
```

程序的 6 次测试结果如下：

① Input year,month:2004,2↙
29 days
② Input year,month:2009,2↙
28 days
③ Input year,month:2100,2↙
28 days
④ Input year,month:2000,2↙
29 days
⑤ Input year,month:2000,11↙
30 days
⑥ Input year,month:2000,13↙
Input error!

1.5 习题 6 解答

6.1 分析并写出下列程序的运行结果。

(1)

```
1   #include <stdio.h>
```

```
2    main()
3    {
4        int i, j, k;
5        char space = ' ';
6        for (i=1;i<=4;i++)
7        {
8            for (j=1; j<=i; j++)
9            {
10               printf("%c",space);
11           }
12           for (k=1; k<=6; k++)
13           {
14               printf("* ");
15           }
16               printf("\n");
17       }
18   }
```

(2)

```
1    #include <stdio.h>
2    main()
3    {
4        int k = 4, n;
5        for (n=0; n<k; n++)
6        {
7            if (n%2 == 0) continue;
8            k--;
9        }
10       printf("k=%d, n=%d\n", k, n);
11   }
```

(3)

```
1    #include <stdio.h>
2    main()
3    {
4        int k = 4, n;
5        for (n=0; n<k; n++)
6        {
7            if (n%2 == 0) break;
8            k--;
```

```
9         }
10        printf("k=%d, n=%d\n", k, n);
11    }
```

参考答案:

```
(1)* * * * * *
     * * * * * *
       * * * * * *
         * * * * * *
```

(2) k=3, n=3

(3) k=4, n=0

6.2 阅读下面的程序,按要求在空白处填写适当的表达式或语句,使程序完整并符合题目要求,然后上机运行程序,写出程序的运行结果。

(1) 计算 1 + 3 + 5 + 7 + … + 99 + 101 的值。

```
1   #include <stdio.h>
2   main()
3   {
4       int i, sum = 0;
5       for (i=1; i<=101;    ①    )
6       {
7           ②    ;
8       }
9       printf("sum = %d\n", sum);
10  }
```

参考答案:① i = i + 2 ② sum = sum + i

程序的运行结果为:sum = 2601

(2) 计算 1×2×3 + 3×4×5 + … + 99 ×100×101 的值。

```
1   #include <stdio.h>
2   main()
3   {
4       long i ;
5       long term, sum = 0;
6       for (i=1;    ①    ; i=i+2)
7       {
8           term =    ②    ;
9           sum = sum + term;
10      }
11      printf("sum = %ld\n", sum);
12  }
```

参考答案:① i<=99 ② i * (i + 1) * (i + 2)

程序的运行结果为:sum = 13002450

(3) 计算 $a + aa + aaa + \cdots + aa\cdots a$($n$ 个 a)的值,n 和 a 的值由键盘输入。

```
1    #include <stdio.h>
2    main()
3    {
4        long    ①    , sum = 0;
5        int a , i, n;
6        printf("Input a,n:");
7        scanf("%d,%d", &a, &n);
8        for (i=1; i<=n; i++)
9        {
10           term =    ②    ;
11           sum = sum + term;
12       }
13       printf("sum = %ld\n", sum);
14   }
```

参考答案:① term = 0 ② term * 10 + a

程序的运行结果如下:

```
Input a,n:2,4↙
sum = 2468
```

(4) 计算 1-1/2+1/3-1/4+…+1/99-1/100+…,直到最后一项的绝对值小于 10^{-4} 为止。

```
1    #include <stdio.h>
2    #include <math.h>
3    main()
4    {
5        int n = 1;
6        float term = 1.0, sign = 1,sum = 0;
7        while (          ①          )
8        {
9            ____②____;
10           sum = sum + term;
11           sign =          ③          ;
12           n++;
13       }
14       printf("sum = %f\n", sum);
15   }
```

参考答案:① fabs(term)>= 1e-4 ② term = sign / n ③ -sign

程序的运行结果为:sum = 0.693092

(5) 利用 $\sin x \approx x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\frac{x^9}{9!}-\cdots$,计算 $\sin x$(x 为弧度值)的值,直到最后一项的绝对值小于 10^{-5}时为止,输出 $\sin x$ 的值并统计累加的项数。

```
1    #include <stdio.h>
2    #include <math.h>
3    main()
4    {
5        int n = 1, count = 1;
6        double x, sum, term;
7        printf("Input x:");
8        scanf("%lf", ①);
9        sum = x;
10       term = x;
11       do{
12           term = ②;
13           sum = sum + term;
14           n = n + 2;
15           ③;
16       }while (④);
17       printf("sin(x) = %f, count = %d\n", sum, count);
18   }
```

参考答案:① &x ② -term * x * x / ((n + 1) * (n + 2)) ③ count++
④ fabs(term) >= 1e-5

程序的运行结果如下:

```
Input x:10↙
sin(x) = -0.544022, count = 18
```

6.3 程序改错题。爱因斯坦曾出过这样一道数学题:有一个长阶梯,若每步跨 2 阶,最后剩下 1 阶;若每步跨 3 阶,最后剩下 2 阶;若每步跨 5 阶,最后剩下 4 阶;若每步跨 6 阶,则最后剩下 5 阶;只有每步跨 7 阶,最后才正好 1 阶不剩。参考例 6.15 程序,编写出的计算这个阶梯共有多少阶的程序如下所示,其中存在一些语法和逻辑错误,请找出并改正,然后上机运行程序并写出程序的运行结果。

```
1    #include <stdio.h>
2    main()
3    {
4        int x = 1, find = 0;
5        while (!find);
6        {
```

```
7           if (x%2=1 && x%3=2 && x%5=4 && x%6=5 && x%7=0)
8           {
9               printf("x = %d\n", x);
10              find = 1;
11              x++;
12          }
13      }
14  }
```

错误原因分析略。修正错误后的参考程序如下：

```
1   #include <stdio.h>
2   main()
3   {
4       int x = 1, find = 0;
5       while (!find)
6       {
7           if (x%2==1 && x%3==2 && x%5==4 && x%6==5 && x%7==0)
8           {
9               printf("x = %d\n", x);
10              find = 1;
11          }
12          x++;
13      }
14  }
```

程序的运行结果为：x = 119

6.4 参考例 6.4 程序，编程计算并输出 1～n 之间的所有数的平方和立方。其中，n 值由用户从键盘输入。

参考答案：这是一个计数控制的循环，适合用 for 语句实现。参考程序如下：

```
1   #include <stdio.h>
2   main()
3   {
4       int i, n;
5       printf("Please enter n:");
6       scanf("%d", &n);
7       for (i=1; i<=n; i++)
8       {
9           printf("%d* %d = %d\n", i, i, i * i); /* 输出所有数的平方 */
10      }
11      for (i=1; i<=n; i++)
```

```
12        {
13            printf("%d* %d* %d = %d\n", i, i, i, i * i * i);/* 输出所有数的
                                                              立方 */
14
15        }
16    }
```

程序的运行结果如下：

```
Please enter n:5↙
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
4 * 4 = 16
5 * 5 = 25
1 * 1 * 1 = 1
2 * 2 * 2 = 8
3 * 3 * 3 = 27
4 * 4 * 4 = 64
5 * 5 * 5 = 125
```

6.5 某人在国外留学，不熟悉当地天气预报中的华氏温度值，请编程按每隔 10°输出 -40°~110°之间的华氏温度与摄氏温度的对照表，以方便他对照查找。已知华氏温度和摄氏温度的转换公式为 $C=5/9\times(F-32)$，其中，C 表示摄氏温度，F 表示华氏温度。

参考答案：这是一个计数控制的循环，适合用 for 语句实现。参考程序如下：

```
1     #include <stdio.h>
2     main()
3     {
4         int   fahr;
5         float celsius;
6         for (fahr=-40; fahr<=110; fahr=fahr+10)
7         {
8             celsius = 5.0 / 9 * (fahr - 32);
9             printf("%4d\t%6.1f\n", fahr, celsius);
10        }
11    }
```

程序的运行结果如下：

```
-40   -40.0
-30   -34.4
-20   -28.9
-10   -23.3
  0   -17.8
```

```
 10  -12.2
 20   -6.7
 30   -1.1
 40    4.4
 50   10.0
 60   15.6
 70   21.1
 80   26.7
 90   32.2
100   37.8
110   43.3
```

6.6 假设银行一年整存零取的月息为 1.875%，现在某人手头有一笔钱，他打算在今后 5 年中，每年年底取出 1 000 元作为孩子来年的教育金，到第 5 年孩子毕业时刚好取完这笔钱，请编程计算第 1 年年初时他应存入银行多少钱。

参考答案：采用逆推法分析存钱和取钱的过程，然后采用迭代法求解。若第 5 年年底连本带息要取出 1 000 元，则第 5 年年初银行中的存款数额 y_5 应为

$$y_5 = 1000/(1+12\times0.01875)$$

按题意，由第 5 年年初银行中的存款数额 y_5 求得第 4 年年初银行中的存款数额 y_4 应为

$$y_4 = (y_5+1000)/(1+12\times0.01875)$$

同理，由第 $n+1$ 年年初银行中的存款数额 y_{n+1} 求得第 n 年年初银行中的存款数额 y_n 为

$$y_n = (y_{n+1}+1000)/(1+12\times0.01875)$$

以 0 作为 y_{n+1} 的初值，对上式进行逆推迭代求解，迭代 5 次的结果即为第 1 年年初银行中的存款数额 y_1，也就是他现在要存入银行的存款数额。参考程序如下：

```
1    #include <stdio.h>
2    #define RATE 0.01875
3    #define MONTHS 12
4    #define CAPITAL 1000
5    #define YEARS 5
6    main()
7    {
8        int i;
9        double deposit = 0;
10       for (i=0; i<YEARS; i++)
11       {
12           deposit = (deposit + CAPITAL) / (1 + RATE * MONTHS);
13       }
14       printf("He must save %.2f at the first year.\n", deposit);
15   }
```

程序的运行结果如下：

```
He must save 2833.29 at first year.
```

6.7 已知今年的工业产值为 100 万元，产值增长率从键盘输入，请编程计算工业产值过多少年可实现翻一番（即增加一倍）。

参考答案：用符号常量 CURRENT 表示今年的工业产值为 100 万元，用变量 growRate 表示产值增长率，用变量 year 表示产值翻番所需的年数，则计算年产值增长额的计算公式为：

```
output = output * (1 + growRate)
```

利用迭代法循环计算，直到 output >= 2 * CURRENT 时为止。当 output >= 2 * CURRENT 时，表示已实现产值翻番。此时，循环被执行的次数 year 即为产值翻番所需的年数。

参考程序 1 如下：

```
#include <stdio.h>
#define  CURRENT  100
main()
{
    int     year;                          /* 产值翻番所需年数 */
    double  growRate;                      /* 工业产值的增长率 */
    double  output;                        /* 工业总产值 */
    printf("Input grow rate:");
    scanf("%lf", &growRate);
    output = CURRENT;                      /* 当年产值为 100 万元 */
    for (year=0; output < 2 *CURRENT; year++)
    {
        output = output *  (1 + growRate);
    }
    printf("When grow rate is %.0f%%, the output can be doubled after
          %d years.\n", growRate*100, year);
}
```

参考程序 2 如下：

```
#include <stdio.h>
#define  CURRENT  100
main()
{
    int     year = 0;              /* 产值翻番所需年数，初始化为 0 */
    double  growRate;              /* 工业产值的增长率 */
    double  output = CURRENT;      /* 工业总产值，初始化为当年产值 100 万元 */
    printf("Input grow rate:");
    scanf("%lf", &growRate);
    do {
```

```
11          output = output *  (1 + growRate);
12          year++;
13      }while (output < 2 *CURRENT);
14      printf("When grow rate is %.0f%%, the output can be doubled after
15              %d years. \n", growRate*100, year);
16  }
```

程序的两次测试结果如下：

① Input grow rate: 0.08↙

When grow rate is 8%, the output can be doubled after 10 years.

② Input grow rate: 0.1↙

When grow rate is 10%, the output can be doubled after 8 years.

6.8 参考习题6.2(5)程序，利用$\frac{\pi}{4}=1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\cdots$，编程计算 π 的近似值，直到最后一项的绝对值小于10^{-4}时为止，输出 π 的值并统计累加的项数。

参考答案：利用计算累加项通式的方法来寻找累加项的构成规律，将累加项通式表示为：term = sign/n，分子 sign 按+1，-1，+1，-1，…以正负交替的形式变化，表示为 sign = -sign，取其初值为1.0，分母 n 按1，3，5，7，…变化，表示为 $n=n+2$，取其初值为1，用计数器变量 count 统计累加的项数，取其初值为0，每次循环执行1次加1运算。

循环累加的终止条件是：直到最后一项的绝对值小于10^{-4}时为止。显然，这是一个条件控制的循环，适合用 do-while 语句或者 while 语句编程实现。

参考程序1如下：

```
#include <math.h>
#include <stdio.h>
main()
{
    double pi, sum = 0, term, sign = 1.0;
    int count = 0, n = 1;
    do{
        term = sign / n;            /* 计算累加项 */
        sum = sum + term;           /* 累加 */
        count++;                    /* 记录累加的项数 */
        sign = -sign;               /* 累加项分子的变化规律 */
        n = n + 2;                  /* 累加项分母的变化规律 */
    }while (fabs(term) >= 1e-4);  /* 判断累加项是否满足循环终止条件 */
    pi = sum * 4;
    printf("pi = %f \ncount = %d \n", pi, count);
}
```

参考程序2如下：

```
#include <math.h>
#include <stdio.h>
main()
{
    double pi, sum = 0, term, sign = 1.0;
    int count = 0, n = 1;
    term = 1.0;                          /* term 在这里必须先初始化*/
    while (fabs(term) >= 1e-4)           /* 判断累加项是否满足循环终止条件 */
    {
        term = sign / n;                 /* 计算累加项 */
        sum = sum + term;                /* 累加 */
        count++;                         /* 记录累加的项数 */
        sign = -sign;                    /* 累加项分子的变化规律 */
        n = n + 2;                       /* 累加项分母的变化规律 */
    }
    pi = sum * 4;
    printf("pi = %f \ncount = %d \n", pi, count);
}
```

程序的运行结果如下：

```
pi = 3.141793
count = 5001
```

6.9 参考习题 6.2(5)程序，利用 $e=1+\frac{1}{1!}+\frac{1}{2!}+\frac{1}{3!}+\cdots+\frac{1}{n!}$，编程计算 e 的近似值，直到最后一项的绝对值小于 10^{-5} 时为止，输出 e 的值并统计累加的项数。

参考答案 1：利用计算累加项通式的方法来寻找累加项的构成规律，先计算 1!，2!，3!，…，再计算其倒数作为累加项 term。参考程序如下：

```
#include <math.h>
#include <stdio.h>
main()
{
    int n = 1, count =1;
    double e = 1.0, term = 1.0;
    long fac = 1;
    for (n=1; fabs(term) >= 1e-5; n++)
    {
        fac = fac * n;
        term = 1.0 / fac;                /* 计算累加项 */
        e = e + term;                    /* 累加 */
```

```
13            count++;                          /* 记录累加的项数 */
14        }
15        printf("e = %f, count = %d\n", e, count);
16    }
```

参考答案 2:考虑到$\frac{1}{2!}=\frac{1}{1!}\div 2,\frac{1}{3!}=\frac{1}{2!}\div 3,\cdots$,即前项与后项之间的关系为:term = term/n,因此可通过“利用前项计算后项”的方法来得到累加项的构成规律。参考程序如下:

```
1     #include <math.h>
2     #include <stdio.h>
3     main()
4     {
5         int n = 1, count =1;
6         double e = 1.0, term = 1.0;
7         do{
8             term = term / n;          /* 计算累加项 */
9             e = e + term;             /* 累加 */
10            n++;
11            count++;                  /* 记录累加的项数 */
12        }while (fabs(term) >= 1e-5);
13        printf("e = %f, count = %d\n", e, count);
14    }
```

参考答案 3:将参考答案 2 中的程序改用 while 语句实现。参考程序如下:

```
1     #include <math.h>
2     #include <stdio.h>
3     main()
4     {
5         int n = 1, count =1;
6         double e = 1.0, term = 1.0;
7         while (fabs(term) >= 1e-5)
8         {
9             term = term / n;          /* 计算累加项 */
10            e = e + term;             /* 累加 */
11            n++;
12            count++;                  /* 记录累加的项数 */
13        }
14        printf("e = %f, count = %d\n", e, count);
15    }
```

程序的运行结果如下:

```
e = 2.718282, count = 10
```

6.10 水仙花数是指各位数字的立方和等于该数本身的三位数。例如,153 是水仙花数,因为 $153 = 1^3+3^3+5^3$。请编程计算并输出所有的水仙花数。

参考答案 1:首先确定水仙花数 n 的可能的取值范围,因为 n 是一个三位数,所以其取值在 100~999 之间变化,显然,这是一个计数控制的循环。对于 n 的每一个可能的取值,首先分离出其百位数字 i、十位数字 j、个位数字 k,然后通过判定 n 与 $i*i*i+j*j*j+k*k*k$ 是否相等,即可确定 n 是否为水仙花数。参考程序如下:

```
#include <stdio.h>
main()
{
    int i, j, k, n;
    for (n=100; n<1000; n++)
    {
        i = n / 100;                /* 分离出百位数字 */
        j = (n - i * 100) / 10;     /* 分离出十位数字 */
        k = n %10;                  /* 分离出个位数字 */
        if (n == i * i * i + j * j * j + k * k * k) /* 判定是否满足水仙花数的条件 */
            printf("%6d", n);
    }
    printf("\n");
}
```

参考答案 2:设水仙花数的百位、十位、个位数字分别为 i、j、k,通过遍历 i、j、k 的所有可能取值(注意:对于三位数而言,其百位数字 i 的值不能为 0),并判定 $i*100+j*10+k$ 与 $i*i*i+j*j*j+k*k*k$ 是否相等,即可确定该三位数是否为水仙花数。参考程序如下:

```
#include <stdio.h>
main()
{
    int i, j, k;
    for (i=1; i<=9; i++)            /* 遍历百位数字的所有可能取值 */
    {
        for (j=0; j<=9; j++)        /* 遍历十位数字的所有可能取值 */
        {
            for (k=0; k<=9; k++)    /* 遍历个位数字的所有可能取值 */
            {
                /* 水仙花数判定 */
                if (i * 100+j * 10+k == i * i * i+j * j * j+k * k * k)
                    printf("%6d", i * 100+j * 10+k);
            }
```

```
15            }
16        }
17        printf("\n");
18    }
```

程序的运行结果如下：

```
   153  370  371  407
```

6.11 已知不等式：1! + 2! + … + m! < n，请编程对用户指定的 n 值计算并输出满足该不等式的 m 的整数解。

参考答案：计算累加和 1! + 2! + … + m! 的值，直到累加和变量 sum ≥ n 时结束循环，这是一个条件控制的循环。注意，由于要计算满足 sum < n 的 m 值，而退出循环时 m 值刚好使得 sum ≥ n，即多累加了 1 项，所以满足 sum < n 的 m 的整数解应为循环次数减 1 后的值。此外，由于将 n 定义为无符号长整型，所以输入、输出数据时应使用%lu 格式符。

参考程序 1 如下：

```
1   #include <stdio.h>
2   main()
3   {
4       unsigned long i, n, term = 1, sum = 0;
5       printf("Please enter n:");
6       scanf("%lu", &n);
7       for (i=1; ;i++)
8       {
9           term = term * i;
10          sum = sum + term;
11          if (sum >= n) break;
12      }
13      printf("m <= %lu\n", i-1);
14  }
```

参考程序 2 如下：

```
1   #include <stdio.h>
2   main()
3   {
4       unsigned long i = 0, n, term = 1, sum = 0;
5       printf("Please enter n:");
6       scanf("%lu", &n);
7       do{
8           i++;
9           term = term * i;
10          sum = sum + term;
```

```
11        }while(sum < n);
12        printf("m <= %lu\n", i-1);
13    }
```

参考程序 3 如下：

```
1     #include <stdio.h>
2     main()
3     {
4         unsigned long i, n, term = 1, sum = 0;
5         int flag = 0;
6         printf("Please enter n:");
7         scanf("%lu", &n);
8         for (i=1; !flag; i++)
9         {
10            term = term * i;
11            sum = sum + term;
12            if (sum >= n)
13            {
14                flag = 1;
15                printf("m <= %lu\n", i-1);
16            }
17        }
18    }
```

程序的运行结果如下：

```
Please enter n: 1000000↙
m <= 9
```

6.12 输入一些整数，编程计算并输出其中所有正数的和，输入负数或零时，表示输入数据结束。要求最后统计出累加的项数。

参考答案：由于负数或零是输入数据结束的标志，因此这是一个标记控制的循环，适合用 while 语句编程实现。

参考程序如下：

```
1     #include <stdio.h>
2     main()
3     {
4         int i = 0, n, sum = 0;
5         printf("Input a number:");
6         scanf("%d", &n);
7         while(n > 0)
8         {
```

```
9            sum = sum + n;
10           printf("Input a number:");
11           scanf("%d", &n);
12           i++;
13       }
14       printf("sum = %d, count = %d\n", sum, i);
15   }
```

程序的运行结果如下：

```
Input a number: 1↙
Input a number: 2↙
Input a number: 3↙
Input a number: 4↙
Input a number: 5↙
Input a number: -1↙
sum = 15, count = 5
```

6.13 参考例6.14程序，输入一些整数，编程计算并输出其中所有正数的和，输入负数时不累加，继续输入下一个数。输入零时，表示数据输入结束。要求最后统计出累加的项数。

参考程序如下：

```
1   #include <stdio.h>
2   main()
3   {
4       int i = 0, n, sum = 0;
5       printf("Input a number:");
6       scanf("%d", &n);
7       while(n ! = 0)
8       {
9           if (n > 0)
10          {
11              sum = sum + n;
12              i++;
13          }
14          printf("Input a number:");
15          scanf("%d", &n);
16      }
17      printf("sum = %d, count = %d\n", sum, i);
18  }
```

程序的运行结果如下：

```
Input a number: 1↙
```

```
Input a number: 2↙
Input a number: 3↙
Input a number: -4↙
Input a number: 4↙
Input a number: 5↙
Input a number: 0↙
sum = 15, count = 5
```

6.14 马克思手稿中有这样一道趣味数学题:男人、女人和小孩总计 30 个人,在一家饭店里吃饭,共花了 50 先令,每个男人各花 3 先令,每个女人各花 2 先令,每个小孩各花 1 先令,请用穷举法编程计算男人、女人和小孩各有几人。

参考答案 1:设有男人、女人和小孩各 x、y、z 人,按题目要求可列出如下方程组:

$$\begin{cases} x+y+z=30 \\ 3x+2y+z=50 \end{cases}$$

由于上述方程组中有三个未知数,因此这是一个不定方程,存在多解,需要采用例 6.15 中介绍的穷举法来求解。采用三重循环,令 x、y、z 分别从 0 变化到 30,穷举 x、y、z 的全部可能取值的组合,然后判断 x、y、z 的每一种组合是否满足方程组的解的条件。参考程序如下:

```
1    #include <stdio.h>
2    main()
3    {
4        int x, y, z;
5        printf("Man \tWomen \tChildren \n");
6        for (x=0; x<=30; x++)
7        {
8            for (y=0; y<=30; y++)
9            {
10               for (z=0; z<=30; z++)
11               {
12                   if (x+y+z == 30 && 3*x + 2*y + z == 50)
13                       printf("%3d \t%5d \t%8d \n", x, y, z);
14               }
15           }
16       }
17   }
```

参考答案 2:为了减少参考答案 1 中程序的循环次数,提高程序的运行效率,可通过增加一些启发式知识来优化程序,缩小需要穷举的范围。由于每个男人花 3 先令,所以在只花 50 先令的情况下,最多只有 16 个男人;同样,在只花 50 先令的情况下,最多只有 25 个女人,而小孩的人数可由方程式 $x+y+z=30$ 计算得到。参考程序如下:

```
1    #include <stdio.h>
```

```
2    main()
3    {
4        int x, y, z;
5        printf("Men \tWomen \tChildren \n");
6        for (x=0; x<=16; x++)
7        {
8            for (y=0; y<=25; y++)
9            {
10               z = 30 - x - y;
11               if (3*x + 2*y + z == 50)
12                   printf("%3d \t%5d \t%8d \n", x, y, z);
13           }
14       }
15   }
```

程序的运行结果如下：

```
 Men  Women  Children
  0     20       10
  1     18       11
  2     16       12
  3     14       13
  4     12       14
  5     10       15
  6      8       16
  7      6       17
  8      4       18
  9      2       19
 10      0       20
```

6.15 鸡兔同笼，共有98个头，386只脚，请用穷举法编程计算鸡、兔各为多少只。

参考答案1：设鸡为x只，兔为y只，根据题意有$x+y=98$，$2x+4y=386$。采用穷举法，令x和y分别从1变化到98，若x和y同时满足$x+y=98$和$2x+4y=386$，则打印x和y的值。参考程序如下：

```
1    #include <stdio.h>
2    main()
3    {
4        int x, y;
5        for (x=1; x<=98; x++)
6        {
7            for (y=1; y<98; y++)
```

```
8           {
9               if (x+y == 98 && 2*x+4*y == 386)
10                  printf("x = %d, y = %d\n", x, y);
11          }
12      }
13  }
```

参考答案 2:采用穷举法,令 x 从 1 变化到 98,$y=98-x$,此时只要判断 x 和 y 是否满足 $2x+4y=386$ 即可。参考程序如下:

```
1   #include <stdio.h>
2   main()
3   {
4       int x, y;
5       for (x=1; x<=98; x++)
6       {
7         y = 98 - x;
8         if (2*x+4*y == 386)
9             printf("x = %d, y = %d\n", x, y);
10      }
11  }
```

程序的运行结果如下:

```
x = 3, y = 95
```

6.16 古代《张丘建算经》中有一道百鸡问题:鸡翁一,值钱五;鸡母一,值钱三;鸡雏三,值钱一。百钱买百鸡,问鸡翁、母、雏各几何? 其意为:公鸡每只 5 元,母鸡每只 3 元,小鸡 3 只 1 元。请用穷举法编程计算,若用 100 元买 100 只鸡,则公鸡、母鸡和小鸡各能买多少只。

参考答案 1:设公鸡、母鸡、小鸡分别为 x、y、z 只,根据题意有 $x+y+z=100$,$5x+3y+z/3=100$,采用穷举法求解,令 x、y、z 分别从 0 变化到 100,若 x、y、z 同时满足 $x+y+z=100$ 和 $5x+3y+z/3=100$,则打印 x、y、z 的值。参考程序如下:

```
1   #include <stdio.h>
2   main()
3   {
4       int x, y, z;
5       for (x=0; x<=100; x++)
6       {
7           for (y=0; y<=100; y++)
8           {
9               for (z=0; z<=100; z++)
10              {
11                  if (x+y+z == 100 && 5*x+3*y+z/3.0 == 100)
```

```
12                      printf("x=%d, y=%d, z=%d\n", x, y, z);
13                  }
14              }
15          }
16      }
```

参考答案2：采用穷举法求解，因100元买公鸡最多可买20只，买母鸡最多可买33只，所以，可令 x 从0变化到20，y 从0变化到33，$z=100-x-y$，此时只要判断 x、y、z 是否满足 $5x+3y+z/3.0=100$ 即可。注意：$z/3.0$ 不能写成 $z/3$，由于 $z/3$ 是整除运算，会导致多个 z 值具有相等的 $z/3$ 计算结果，这样就会使得输出的解多出几组。参考程序如下：

```
#include <stdio.h>
main()
{
    int x, y, z;
    for (x=0; x<=20; x++)
    {
        for (y=0; y<=33; y++)
        {
            z = 100 - x - y;
            if (5*x+3*y+z/3.0 == 100)
                printf("x=%d, y=%d, z=%d\n", x, y, z);
        }
    }
}
```

程序的运行结果如下：

```
x=0, y=25, z=75
x=4, y=18, z=78
x=8, y=11, z=81
x=12, y=4, z=84
```

6.17 用100元人民币兑换10元、5元和1元的纸币（每一种都要有）共50张，请用穷举法编程计算共有几种兑换方案，每种方案各兑换多少张纸币。

参考答案1：设10元、5元和1元的纸币各换 x、y、z 张，根据题意有 $x+y+z=50$，$10x+5y+z=100$。由于每一种纸币都要有，所以令 x、y、z 分别从1变化到49，若 x、y、z 同时满足 $x+y+z=50$ 和 $10x+5y+z=100$，则打印 x、y、z 的值，对每一组满足条件的 x、y、z 值，用计数器计数即可得到兑换方案的数目。参考程序如下：

```
#include <stdio.h>
main()
{
    int x, y, z, count = 0;
```

```
    for (x=1; x<50; x++)
    {
        for (y=1; y<50; y++)
        {
            for (z=1; z<50; z++)
            {
                if (x+y+z == 50 && 10*x+5*y+z == 100)
                {
                    count++;
                    printf("x = %d, y = %d, z = %d\n", x, y, z);
                }
            }
        }
    }
    printf("count = %d\n", count);
}
```

参考答案 2:由于每一种纸币都要有,故 10 元纸币最多可换 9 张,5 元纸币最多可换 19 张,1 元纸币可换 $50-x-y$ 张,此时只要判断 x、y、z 是否满足 $10x+5y+z=100$ 即可。参考程序如下:

```
#include <stdio.h>
main()
{
    int x, y, z, count = 0;
    for (x=1; x<=9; x++)
    {
        for (y=1; y<=19; y++)
        {
            z = 50 - x - y;
            if (10*x+5*y+z == 100)
            {
                count++;
                printf("x = %d, y = %d, z = %d\n", x, y, z);
            }
        }
    }
    printf("count = %d\n", count);
}
```

程序的运行结果如下:

```
x = 2, y = 8, z = 40
```

```
count = 1
```

6.18 分别按如下三种形式编程输出九九乘法表。

```
1  2  3  4  5  6  7  8  9
2  4  6  8  10 12 14 16 18
3  6  9  12 15 18 21 24 27
4  8  12 16 20 24 28 32 36
5  10 15 20 25 30 35 40 45
6  12 18 24 30 36 42 48 54
7  14 21 28 35 42 49 56 63
8  16 24 32 40 48 56 64 72
9  18 27 36 45 54 63 72 81
```

```
1
2  4
3  6  9
4  8  12 16
5  10 15 20 25
6  12 18 24 30 36
7  14 21 28 35 42 49
8  16 24 32 40 48 56 64
9  18 27 36 45 54 63 72 81
```

```
1  2  3  4  5  6  7  8  9
   4  6  8  10 12 14 16 18
      9  12 15 18 21 24 27
         16 20 24 28 32 36
            25 30 35 40 45
               36 42 48 54
                  49 56 63
                     64 72
                        81
```

(1) 参考程序如下：

```
#include <stdio.h>
main()
{
    int m, n;
    for (m=1; m<10; m++)          /* 外层循环控制行数(被乘数)的变化 */
    {
        for (n=1; n<10; n++)      /* 内层循环控制列数(乘数)的变化 */
        {
            printf("%4d", m * n); /* 输出第 m 行第 n 列中的 m×n 的值 */
        }
        printf("\n");             /* 输出换行符,准备打印下一行 */
    }
}
```

(2) 参考程序如下：

```
#include <stdio.h>
main()
{
    int m, n;
    for (m=1; m<10; m++)          /* 外层循环控制行数(被乘数)的变化 */
```

```
6       {
7           for (n=1; n<=m; n++)        /* 内层循环控制列数(乘数)的变化 */
8           {
9               printf("%4d", m * n); /* 输出第 m 行第 n 列中的 m×n 的值 */
10          }
11          printf("\n");              /* 输出换行符,准备打印下一行 */
12      }
13  }
```

(3) 参考程序如下:

```
1   #include <stdio.h>
2   main()
3   {
4       int m, n, k;
5       for (m=1; m<10; m++)            /* 外层循环控制行数(被乘数)的变化 */
6       {
7           for (k=1; k<=4 * m-4; k++)/* 在第 n 行先打印 4×m-4 个空格 */
8           {
9               printf(" ");           /* 输出相应空格,使数字右对齐 */
10          }
11          for (n=m; n<10; n++)        /* 内层循环控制列数(乘数)的变化 */
12          {
13              printf("%4d", m * n); /* 输出第 m 行第 n 列中的 m×n 的值 */
14          }
15          printf("\n");              /* 输出换行符,准备打印下一行 */
16      }
17  }
```

6.19 (选做)有一天,一位百万富翁遇到一个陌生人,陌生人找他谈一个换钱的计划,陌生人对百万富翁说:"我每天给你 10 万元,而你第一天只需给我 1 分钱,第二天我仍给你 10 万元,你给我 2 分钱,第三天我仍给你 10 万元,你给我 4 分钱,依此类推。你每天给我的钱是前一天的 2 倍,直到满一个月(30 天)为止",百万富翁很高兴,欣然接受了这个契约。请编程计算在这一个月中陌生人总计给百万富翁多少钱,百万富翁总计给陌生人多少钱。

参考程序如下:

```
1   #include <stdio.h>
2   main()
3   {
4       int j;
5       double toStranger = 0;      /* 富翁给陌生人的钱,以元为单位 */
6       double toRichman = 0;       /* 陌生人给富翁的钱,以元为单位 */
```

```
7         double term = 0.01;          /* 百万富翁第一天给陌生人 0.01 元钱 */
8         for (j=1; j<=30; j++)
9         {
10            toRichman += 100000;  /* 陌生人每天给富翁 10 万元 */
11            toStranger += term;
12            term = term * 2;          /* 富翁每天给陌生人的钱是前一天的 2 倍 */
13        }
14        printf("百万富翁给陌生人: %f 元\n",toStranger);
15        printf("陌生人给百万富翁: %f 元\n",toRichman);
16    }
```

程序的运行结果如下:

```
百万富翁给陌生人: 10737418.230000 元
陌生人给百万富翁: 3000000.000000 元
```

6.20 (选做)一辆卡车违反了交通规则,撞人后逃逸。现场有三人目击该事件,但都没有记住车号,只记住车号的一些特征。甲说:车号的前两位数字是相同的;乙说:车号的后两位数字是相同的,但与前两位不同;丙是位数学家,他说:4 位的车号正好是一个整数的平方。请根据以上线索编程协助警方找出车号,以便尽快破案,抓住交通肇事犯。

参考答案:假设这个 4 位数的前两位数字都是 i,后两位数字都是 j,则这个可能的 4 位数 $k = i * 1000 + i * 100 + j * 10 + j$。其中,$i$ 和 j 都在 0~9 之间变化。此外,k 还须满足的条件是:k 是一个整数 m 的平方。由于 k 是一个 4 位数,所以 m 值不可能小于 31。因此,可采用穷举法,从 31 开始试验 k 与 $m * m$ 是否相等,若不相等,则 m 加 1 后再试,直到找到与 $m * m$ 相等的 k 值为止,结束测试。参考程序如下:

```
#include <stdio.h>
main()
{
    int i, j, k, m;
    for (i=0; i<=9; i++)
    {
        for (j=0; j<=9; j++)
        {
            if (i != j)
            {
                k = i * 1000 + i * 100 + j * 10 + j;
                for (m=31; m*m<=k; m++)
                {
                    if (m*m == k)
                        printf("k=%d, m=%d\n", k, m);
                }
```

```
17              }
18          }
19      }
20  }
```

程序的运行结果如下：

```
k = 7744, m = 88
```

6.21 （选做）在海军节开幕式上，有 A、B、C 三艘军舰要同时开始鸣放礼炮各 21 响。已知 A 舰每隔 5 s 放 1 次，B 舰每隔 6 s 放 1 次，C 舰每隔 7 s 放 1 次。假设各炮手对时间的掌握非常准确，请编程计算观众总共可以听到几次礼炮声。

参考答案：用 n 作为听到的礼炮声响的计数器，用 t 表示时间，从第 0 s 开始放第 1 响，A 舰到放完最后一响，最长时间为 20×5 s，B 舰到放完最后一响，最长时间为 20×6 s，C 舰到放完最后一响，最长时间为 20×7 s。因此，可以用一个 for 循环来模拟每一秒的时间变化，即 t 从 0 开始循环到 t>20×7 s 时结束。在循环体中判断：如果时间 t 是 5 的整数倍且 21 响未放完，则 A 舰放一响，计数器 n 加 1；如果时间 t 是 6 的整数倍且 21 响未放完，则 B 舰放一响，计数器 n 加 1；如果时间 t 是 7 的整数倍且 21 响未放完，则 C 舰放一响，计数器 n 加 1。注意：当有两舰或三舰同时鸣放时，应作为一响统计，即 n 不能同时计数，只要有一个执行了计数，其他两个就不能再进行计数。参考程序如下：

```
#include <stdio.h>
main()
{
    int n = 0, t;
    for (t = 0; t <= 20*7; t++)
    {
        if (t%5 == 0 && t <= 20*5)      /* 控制 A 舰每隔 5 s 放 1 次 */
        {
            n++;
            continue;                   /* 继续下一次循环 */
        }
        if (t%6 == 0 && t <= 20*6)      /* 控制 B 舰每隔 6 s 放 1 次 */
        {
            n++;
            continue;                   /* 继续下一次循环 */
        }
        if (t%7 == 0)                   /* 控制 C 舰每隔 7 s 放 1 次 */
        {
            n++;
        }
    }
```

```
22        printf("n = %d\n", n);
23    }
```

程序的运行结果如下：

```
n = 54
```

1.6 习题 7 解答

7.1 分析并写出下面程序的运行结果。

```
1     #include <stdio.h>
2     int Square(int i)
3     {
4         return i * i;
5     }
6     int main()
7     {
8         int i = 0;
9         i = Square(i);
10        for ( ; i<3; i++)
11        {
12            static int i = 1;
13            i += Square(i);
14            printf("%d,", i);
15        }
16        printf("%d\n", i);
17        return 0;
18    }
```

程序运行结果:2,6,42,3

7.2 用全局变量编程模拟显示一个数字式时钟,然后上机验证。

```
1     #include <stdio.h>
2     int hour, minute, second;          /* 定义全局变量 */
3     void Update()
4     {
5         second++;
6         if (second == 60)
7         {
8             ____①____;
9             minute++;
10        }
```

```
11        if (    ②    )
12        {
13            minute = 0;
14            hour++;
15        }
16        if (hour == 24)
17            ③ ;
18    }
19    void Display()
20    {
21        printf("    ④    ", hour, minute, second);
22    }
23    void Delay()
24    {
25        int t;
26        for (t=0; t<100000000; t++);  /* 用循环体为空语句的循环实现延时 */
27    }
28    int main()
29    {
30        int i;
31        ⑤ ;
32        for(i=0; i<1000000; i++)      /* 利用循环结构,控制时钟运行的时间 */
33        {
34            Update();                 /* 更新时、分、秒显示值 */
35            Display();                /* 显示时、分、秒 */
36            Delay();                  /* 模拟延迟时间为 1s */
37        }
38        return 0;
39    }
```

参考答案：① second = 0;② minute == 60;③ hour = 0;④ %2d:%2d:%2d\r;⑤ hour = minute = second = 0

7.3 用函数编程计算两整数的最大值，在主函数中调用该函数计算并输出从键盘任意输入的两整数的最大值。

参考程序 1 如下：

```
1    #include <stdio.h>
2    int Max(int a, int b);
3    int main()
4    {
```

```
    int a, b;
    printf("Input a,b:");
    scanf("%d,%d", &a, &b);
    printf("max = %d\n", Max(a, b));
    return 0;
}
/* 函数功能:计算 a 和 b 的最大值 */
int Max(int a, int b)
{
    return a > b ? a : b;
}
```

参考程序2如下:

```
#include <stdio.h>
int Max(int a, int b);
int main()
{
    int a, b, c;
    printf("Input a,b:");
    scanf("%d,%d", &a, &b);
    c = Max(a, b);
    printf("max = %d\n", c);
    return 0;
}
/* 函数功能:计算 a 和 b 的最大值 */
int Max(int a, int b)
{
    int c;
    if (a > b)
        c = a;
    else
        c = b;
    return c;
}
```

程序的运行结果如下:

```
Input a,b:5,8↙
max = 8
```

7.4 采用穷举法,用函数编程实现计算两个正整数的**最小公倍数(Least Common Multiple,LCM)**的函数,在主函数中调用该函数计算并输出从键盘任意输入的两整数的最小公

倍数。

参考答案:可以从正整数 a 和 b 的公倍数中来寻找其最小公倍数。首先,从 a 的倍数中寻找 b 的倍数(或者从 b 的倍数中寻找 a 的倍数),由于 $b\times a$ 一定是 a 和 b 的公倍数,所以寻找 a 和 b 最小公倍数的范围不会超过 $b\times a$。然后,在所有的 a 的倍数 a,$2\times a$,$3\times a$,…,$b\times a$ 中,从小到大依次判断该数是否是 b 的倍数,a 的倍数中第一个能被 b 整除的数必然是 a 和 b 的最小公倍数。

参考程序如下:

```
#include <stdio.h>
int Lcm(int a, int b);
int main()
{
    int a, b, x;
    printf("Input a,b:");
    scanf("%d,%d", &a, &b);
    x = Lcm(a, b);
    if (x ! = -1)
        printf("Least Common Multiple of %d and %d is %d\n", a, b, x);
    else
        printf("Input error! \n");
    return 0;
}
/* 函数功能:计算 a 和 b 的最小公倍数,输入负数时返回-1 */
int Lcm(int a, int b)
{
    int i;
    if (a <= 0 || b <= 0)  return -1;
    for (i=1; i<b; i++)
    {
        if (i * a % b == 0)  return i * a;
    }
    return b * a;
}
```

程序的两次测试结果如下:

① Input a,b: 16,24↙

Least Common Multiple of 16 and 24 is 48

② Input a,b: -16,24↙

Input error!

7.5 参考例 7.4,利用求阶乘函数 Fact(),编程计算并输出从 1 到 n 之间所有数的阶乘值。

参考程序如下：

```
#include <stdio.h>
unsigned long Fact(unsigned int n);
int main()
{
    unsigned int i, n;
    printf("Input n(n>0):");
    scanf("%u", &n);
    for (i = 1; i<=n; i++)
    {
        printf("%d! = %lu\n", i, Fact(i));
    }
    return 0;
}
/* 函数功能:用迭代法计算无符号整型变量 n 的阶乘 */
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    for (i=2; i<=n; i++)
    {
        result *= i;
    }
    return result;
}
```

程序的运行结果如下：

```
Input n(n>0):10↙
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

7.6 参考例 7.4，利用求阶乘函数 Fact()，编程计算并输出 1! + 2! + … + n! 的值。

参考程序如下：

```
#include <stdio.h>
unsigned long Fact(unsigned int n);
int main()
{
    unsigned int i, n;
    unsigned long sum = 0;
    printf("Input n(n>0):");
    scanf("%u", &n);
    for (i = 1; i<=n; i++)
    {
        sum = sum + Fact(i);
    }
    printf("sum = %lu\n", sum);
    return 0;
}
/* 函数功能:用迭代法计算无符号整型变量 n 的阶乘 */
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    for (i=2; i<=n; i++)
    {
        result *= i;
    }
    return result;
}
```

程序的运行结果如下：

```
Input n(n>0):10 ↙
sum = 4037913
```

7.7 两个正整数的**最大公约数(Greatest Common Divisor, GCD)**是能够整除这两个整数的最大整数。请分别采用如下 3 种方法编写计算最大公约数的函数 Gcd()，在主函数中调用该函数计算并输出从键盘任意输入的两整数的最大公约数。

(1) **穷举法**。由于 a 和 b 的最大公约数不可能比 a 和 b 中的较小者还大，否则一定不能整除它，因此，先找到 a 和 b 中的较小者 t，然后从 t 开始逐次减 1 尝试每种可能，即检验 t 到 1 之间的所有整数，第一个满足公约数条件的 t 就是 a 和 b 的最大公约数。

(2) **欧几里得算法**，也称辗转相除法。对正整数 a 和 b，连续进行求余运算，直到余数为 0 为止，此时非 0 的除数就是最大公约数。设 $r=a \bmod b$ 表示 a 除以 b 的余数，若 $r \neq 0$，则将 b 作

为新的 a,r 作为新的 b,即 Gcd(a, b)= Gcd(b, r),重复 a mod b 运算,直到 r=0 为止,此时 b 为所求的最大公约数。例如,50 和 15 的最大公约数的求解过程可表示为:Gcd(50, 15) = Gcd(15, 5)= Gcd(5, 0)= 5。

(3) **递归方法**。对正整数 a 和 b,当 $a>b$ 时,若 a 中含有与 b 相同的公约数,则 a 中去掉 b 后剩余的部分 $a-b$ 中也应含有与 b 相同的公约数,对 $a-b$ 和 b 计算公约数就相当于对 a 和 b 计算公约数。反复使用最大公约数的如下3条性质,直到 a 和 b 相等为止,这时,a 或 b 就是它们的最大公约数。

性质1 如果 $a>b$,则 a 和 b 与 $a-b$ 和 b 的最大公约数相同,即 Gcd(a, b) = Gcd($a-b$, b)。

性质2 如果 $b>a$,则 a 和 b 与 a 和 $b-a$ 的最大公约数相同,即 Gcd(a, b) = Gcd(a, $b-a$)。

性质3 如果 $a=b$,则 a 和 b 的最大公约数与 a 值和 b 值相同,即 Gcd(a, b) = a = b。

(1) 参考程序如下:

```
#include <stdio.h>
int Gcd(int a, int b);
int main()
{
    int a, b, c;
    printf("Input a,b:");
    scanf("%d,%d", &a, &b);
    c = Gcd(a,b);
    if (c ! = -1)
        printf("Greatest Common Divisor of %d and %d is %d\n", a, b, c);
    else
        printf("Input number should be positive! \n");
    return 0;
}
/* 函数功能:计算 a 和 b 的最大公约数,输入负数时返回-1 */
int Gcd(int a, int b)
{
    int i, t;
    if (a <= 0 || b <= 0)  return -1;
    t = a < b ? a : b;
    for (i=t; i>0; i--)
    {
        if (a%i==0 && b%i==0) return i;
    }
    return 1;
}
```

(2) 主函数同(1),函数 Gcd()的参考程序1(非递归实现)如下:

```
19  /* 函数功能:计算 a 和 b 的最大公约数,输入负数时返回-1 */
20  int Gcd(int a, int b)
21  {
22      int r;
23      if (a <= 0 ||b <= 0)  return -1;
24      do{
25          r = a % b;
26          a = b;
27          b = r;
28      } while (r ! = 0);
29      return a;
30  }
```

函数 Gcd( )的参考程序 2(递归实现)如下:

```
19  /* 函数功能:计算 a 和 b 的最大公约数,输入负数时返回-1 */
20  int Gcd(int a, int b)
21  {
22      if (a <= 0 ||b <= 0)return -1;
23      if (a % b == 0)
24          return b;
25      else
26          return Gcd(b, a % b);
27  }
28
```

(3) 主函数同(1),根据最大公约数的 3 条性质,编写非递归函数 Gcd( )如下:

```
19  /* 函数功能:计算 a 和 b 的最大公约数,输入负数时返回-1 */
20  int Gcd(int a, int b)
21  {
22      if (a <=0 ||b <=0)  return -1;
23      while (a ! = b)
24      {
25          if (a > b)
26              a = a - b;
27          else if (b > a)
28              b = b - a;
29      }
30      return a;
31  }
```

此外,根据最大公约数的 3 条性质,还可以编写递归函数 Gcd( )如下:

```
19    /* 函数功能:递归方法计算 a 和 b 的最大公约数,输入负数时返回-1 */
20    int Gcd(int a, int b)
21    {
22        if (a <=0 ||b <=0)  return -1;
23        if (a == b)
24            return a;
25        else if (a > b)
26            return Gcd(a-b, b);
27        else
28            return Gcd(a, b-a);
29    }
```

程序的两次测试结果如下:

① Input a,b: 16,24↙

Greatest Common Divisor of 16 and 24 is 8

② Input a,b: -16,24↙

Input number should be positive!

7.8 (选做)5个水手在岛上发现一堆椰子,先由第1个水手把椰子分为等量的5堆,还剩下1个给了猴子,自己藏起1堆。然后,第2个水手把剩下的4堆混合后重新分为等量的5堆,还剩下1个给了猴子,自己藏起1堆。以后第3、第4个水手依次按此方法处理。最后,第5个水手把剩下的椰子分为等量的5堆后,同样剩下1个给了猴子。请用迭代法编程计算并输出原来这堆椰子至少有多少个。

参考答案:若某水手面对的椰子数是 y 个,则他前一个水手面对的椰子数是 $y\times5/4+1$ 个,依此类推。若对某一个整数 y 经上述5次迭代都是整数,最后的结果即为所求。让 x 从1开始取值,y 从 $5x+1$ 开始取值,在按 $y\times5/4+1$ 进行的4次迭代中,若某一次 y 不是整数,则返回 x 增1再试,直到5次迭代的 y 值全部为整数时,打印输出 y 值即为所求。一般地,对于 $n(n>1)$ 个水手,按 $y\times n(n-1)+1$ 进行 n 次迭代可得 n 个水手分椰子问题的解。参考程序如下:

```
1     #include <stdio.h>
2     int Coconut(int n);
3     int main()
4     {
5         printf("y = %d\n", Coconut(5));
6         return 0;
7     }
8     int Coconut(int n)
9     {
10        int i = 1;
11        float x = 1, y;
12        y = n * x + 1;
```

```
13        do{
14            y = y * n / (n-1) + 1;
15            i++;
16            if (y != (int)y)
17            {
18                x = x + 1;
19                y = n * x + 1;
20                i = 1;
21            }
22        } while (i < n);
23        return (int)y;
24    }
```

程序的运行结果如下：

```
y = 3121
```

7.9 （选做）有 5 个人围坐在一起，问第 5 个人多大年龄，他说比第 4 个人大 2 岁；问第 4 个人多大年龄，他说比第 3 个人大 2 岁；问第 3 个人多大年龄，他说比第 2 个人大 2 岁；问第 2 个人多大年龄，他说比第 1 个人大 2 岁。第 1 个人说自己 10 岁。假设有 n 个人围坐在一起，请利用递归法编程计算并输出第 n 个人的年龄。

参考答案：这个问题可以用递归方法求解，递归公式为：

$$\text{age}(n)=\begin{cases}10 & \text{当 } n=1\\ \text{age}(n-1)+2 & \text{当 } n>1\end{cases}$$

参考程序如下：

```
#include<stdio.h>
unsigned int ComputeAge(unsigned int n);
int main()
{
    unsigned int n;
    printf("Input total persons:");
    scanf("%u", &n);
    printf("The person's age is %d \n", ComputeAge(n));
    return 0;
}
/* 函数功能：用递归算法计算年龄 */
unsigned int ComputeAge(unsigned int n)
{
    unsigned int age;
    if (n==1)
        age=10;
```

```
17        else
18            age=ComputeAge(n-1)+2;
19        return age;
20    }
```

程序的运行结果如下：

```
Input total persons: 5↙
The person's age is 18
```

7.10 （选做）在一种室内互动游戏中，魔术师要每位观众心里想一个三位数 abc（a、b、c 分别是百位、十位和个位数字），然后魔术师让观众心中记下 acb、bac、bca、cab、cba 5 个数以及这 5 个数的和值。只要观众说出这个和是多少，则魔术师一定能猜出观众心里想的原数 abc 是多少。例如，观众甲说他计算的和值是 1999，则魔术师立即说出他想的数是 443，而观众乙说他计算的和值是 1998，则魔术师说："你算错了！"。请编程模拟这个数字魔术游戏。

参考答案：经分析，显然有 $m=acb+bac+bca+cab+cba=122a+212b+221c$。已知 m，求解不定方程：

$$122a + 212b + 221c = m$$

其中，a、b、c 为一个数字，$a \neq 0$。采用穷举法，求解满足该方程的解值 a、b、c，然后打印输出。

参考程序如下：

```
1     #include <stdio.h>
2     int Magic(int m);
3     int main()
4     {
5         int m, ret;
6         printf("Input a sum:");
7         scanf("%d", &m);
8         ret = Magic(m);
9         if (ret == -1)
10            printf("The sum you calculated is wrong! \n");
11        else
12            printf("The number is %d \n",ret);
13        return 0;
14    }
15    int Magic(int m)
16    {
17        int a, b, c, n;
18        for (a=1; a<10; a++)
19        {
20            for (b=0; b<10; b++)
21            {
```

```
22              for(c=0; c<10; c++)
23              {
24                  n = 122 * a + 212 * b + 221 * c;
25                  if (m == n)
26                  {
27                      return 100 * a + 10 * b + c;
28                  }
29              }
30          }
31      }
32      return -1;
33  }
```

程序的两次测试结果如下：

```
① Input a sum:1998↙
  The sum you calculated is wrong!
② Input a sum:1999↙
  The number is 443
```

7.11 （选做）中国古代民间有这样一个游戏：两个人从1开始轮流报数，每人每次可报一个数或两个连续的数，谁先报到30，谁为胜方。若要改成游戏者与计算机做这个游戏，则首先需要决定谁先报数，可以通过生成一个随机整数来决定计算机和游戏者谁先报数。计算机报数的原则为：若剩下数的个数除以3，余数为1，则报1个数；若剩下数的个数除以3，余数为2，则报2个数；否则随机报1个或2个数。游戏者通过键盘输入自己报的数，所报的数必须符合游戏的规则。如果计算机和游戏者都未报到30，则可以接着报数。先报到30者即为胜方。请编程实现这个游戏，看一看游戏者和计算机谁能获胜。

参考答案：玩这个游戏获胜的必要条件是“牢牢抓住3的倍数”。即把对方报的数字对3求余，若值为1，则报的数字为对方的数字加2；若值为2，则报的数字为对方的数字加1；否则就随意加1或者2。只要在游戏中紧紧抓住3的倍数，并坚持到最后就一定能获胜。参考程序如下：

```
1   #include <time.h>
2   #include <stdlib.h>
3   #include <stdio.h>
4   int Input(int t);
5   int ControlComputer(int s);
6   int Rnd();
7   int main()
8   {
9       int tol = 1;                    /* 从1开始报数 */
10      if (Rnd() == 1)                 /* 取随机数决定计算机和游戏者谁先报数 */
```

```
11      {
12          tol = Input(tol);       /* 若随机数为1,则游戏者先报数 */
13      }
14      while (tol < 30)            /* 若未到达游戏结束条件,则继续报数 */
15      {
16          tol = ControlComputer(tol);
17          if (tol == 30)          /* 计算机先报到30,则计算机获胜 */
18          {
19              printf("Player lose! \n");
20          }
21          else                    /* 游戏者先报到30,则游戏者获胜 */
22          {
23              tol = Input(tol);
24              if (tol >= 30) /* 若游戏者报的数是30(或超过30),则游戏者获胜 */
25              {
26                  printf("Computer lose! \n");
27              }
28          }
29      }
30      printf("Game over! \n");
31      return 0;
32  }
33  /* 函数功能:游戏者报数 */
34  int Input(int t)
35  {
36      int a;
37      do{
38          printf("please count:");
39          scanf("%d", &a);  /* 输入报数的个数,将决定报的数是加1还是加2 */
40          if (a>2 ||a<1)
41              printf("Error input, again!");
42          else
43              printf("you count:%d\n", t+a);
44      } while (a>2 ||a<1);
45      return t+a;                 /* 返回报数已经报到的数值 */
46  }
47  /* 函数功能:控制计算机报数 */
48  int ControlComputer(int s)
```

```
{
    int c;
    printf("computer count:");
    if (s%3 == 1)                    /* 若余下数的个数除以 3,余数为 1,则取 1 */
    {
        s++;
        printf("%d\n", s);
    }
    else if (s%3 == 2)               /* 若余下数的个数除以 3,余数为 2,则取 2 */
    {
        s += 2;
        printf("%d\n", s);
    }
    else
    {                                /* 随机报数 1或 2个 */
        c = Rnd() + 1;
        s += c;
        printf("%d\n", s);
    }
    return s;                        /* 返回报数已经报到的数值 */
}
/* 函数功能:生成随机数,返回值为 0 或者 1 */
int Rnd()
{
    srand(time(NULL));               /* 也可以用 srand((unsigned)time(0));*/
    return rand()%2;
}
```

程序的两次测试结果如下：

① please count:2↙
you count:3
computer count:4
please count:2↙
you count:6
computer count:7
please count:2↙
you count:9
computer count:10
please count:2↙

```
you count:12
computer count:13
please count:2↙
you count:5
computer count:16
please count:2↙
you count:18
computer count:19
please count:2↙
you count:21
computer count:23
please count:2↙
you count:25
computer count:26
please count:1↙
you count:27
computer count:28
please count:2↙
you count:30
Computer lose!
Game over!
```

②

```
computer count:2
please count:1↙
you count:3
computer count:4
please count:2↙
you count:6
computer count:8
please count:1↙
you count:9
computer count:11
please count:1↙
you count:12
computer count:14
please count:1↙
you count:15
computer count:16
please count:2↙
```

```
you count:18
computer count:20
please count:1↙
you count:21
computer count:22
please count:2↙
you count:24
computer count:26
please count:1↙
you count:27
computer count:29
please count:1↙
you count:30
Computer lose!
Game over!
```

7.12 （选做）**汉诺塔（Hanoi）**是必须用递归方法才能解决的经典问题。它来自于印度神话。上帝创造世界时造了 3 根金刚石柱子，在第一根柱子上从下往上按大小顺序摞着 64 片黄金圆盘。上帝命令婆罗门把圆盘从下面开始按大小顺序重新摆放到第二根柱子上，并且规定每次只能移动一个圆盘，在小圆盘上不能放大圆盘。请编程求解 $n(n>1)$ 个圆盘的汉诺塔问题。

【问题求解方法分析】图 1-1 是 $n(n>1)$ 个圆盘的汉诺塔的初始状态。首先需要将问题简化，这是求解复杂问题的基本方法。

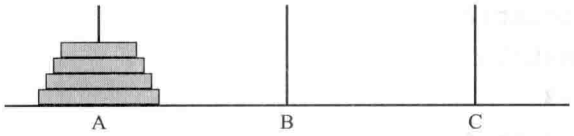


图 1-1 “汉诺塔”初始状态图

先考虑最简单的问题：对于只有 1 个圆盘的汉诺塔问题，只要直接将一个圆盘从一根柱子移到另一根柱子上即可求解，可用下面的函数实现：

```
Move(int n, char a, char b);
```

其含义为：表示将第 n 个圆盘从柱子 a 移到柱子 b 上。

接下来再考虑递归问题的表示：将 n 个圆盘借助于柱子 c 从柱子 a 移到柱子 b 上，可用下面的递归函数表示：

```
Hanoi(int n, char a, char b, char c);
```

为了求解 n 个圆盘的汉诺塔问题，需要采用数学归纳法来分析求解：假设 $n-1$ 个圆盘的汉诺塔问题已经解决，利用这个已解决的问题来求解 n 个圆盘的汉诺塔问题。具体方法是：将“上面的 $n-1$ 个圆盘”看成一个整体，即将 n 个圆盘分成两部分：上面的 $n-1$ 个圆盘和最下面的第 n 个圆盘。于是，移动 n 个圆盘的汉诺塔问题可简化如下：

（1）如图 1-2 所示，将前 $n-1$ 个圆盘从第一根柱子移到第三根柱子上，即 A→C；

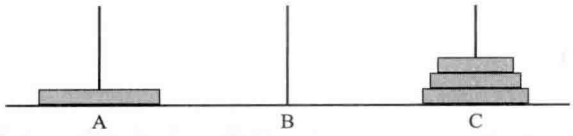


图 1-2 将前 $n-1$ 个圆盘从第一根柱子移到第三根柱子上

(2) 如图 1-3 所示,将第 n 个圆盘从第一根柱子移到第二根柱子上,即 A→B;

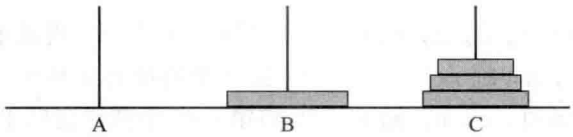


图 1-3 将第 n 个圆盘从第一根柱子移到第二根柱子上

(3) 如图 1-4 所示,将前 $n-1$ 个圆盘从第三根柱子移到第二根柱子上, 即 C→B。

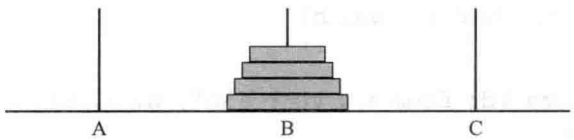


图 1-4 将前 $n-1$ 个圆盘从第三根柱子移到第二根柱子上

根据前面的分析,n 个圆盘汉诺塔问题的求解可用如下 3 步实现。

```
step 1  Hanoi(n-1, a, c, b);
step 2  Move(n, a, b);
step 3  Hanoi(n-1, c, b, a);
```

根据上述的分析,编写程序如下:

```
#include <stdio.h>
void Hanoi(int n, char a, char b, char c);
void Move(int n, char a, char b);
int main()
{
    int n;
    printf("Input the number of disks:");
    scanf("%d", &n);
    printf("Steps of moving %d disks from A to B by means of C:\n", n);
    Hanoi(n, 'A', 'B', 'C'); /* 将 n 个圆盘借助于 C 由 A 移到 B*/
    return 0;
}
/*  函数功能:用递归方法将 n 个圆盘借助于柱子 c 从源柱子 a 移动到目标柱子 b 上 */
void Hanoi(int n, char a, char b, char c)
```

```
15    {
16        if (n == 1)
17        {
18            Move(n, a, b);          /* 将第 n 个圆盘由 a 移到 b */
19        }
20        else
21        {
22            Hanoi(n-1, a, c, b);   /* 将第 n-1 个圆盘借助于 b 由 a 移动到 c*/
23            Move(n, a, b);          /* 第 n 个圆盘由 a 移到 b */
24            Hanoi(n-1, c, b, a);   /* 将第 n-1 个圆盘借助于 a 由 c 移动到 b*/
25        }
26    }
27    /* 函数功能:将第 n 个圆盘从源柱子 a 移到目标柱子 b 上 */
28    void Move(int n, char a, char b)
29    {
30        printf("Move %d: from % c to % c\n", n, a, b);
31    }
```

程序的运行结果如下：

```
Input the number of disks:3↙
Steps of moving 3 disks:
Move 1:from A to B
Move 2:from A to C
Move 1:from B to C
Move 3:from A to B
Move 1:from C to A
Move 2:from C to B
Move 1:from A to B
```

1.7 习题 8 解答

8.1 分析并写出下面程序的运行结果。

(1)

```
1    #include <stdio.h>
2    void Func(int x)
3    {
4        x = 20;
5    }
6    int main()
```

```
7     {
8         int x = 10;
9         Func(x);
10        printf("%d", x);
11        return 0;
12    }
```

(2)

```
1     #include <stdio.h>
2     void Func(int b[])
3     {
4         int j;
5         for (j=0; j<4; j++)
6         {
7             b[j] = j;
8         }
9     }
10    int main()
11    {
12        static int a[] = {5,6,7,8}, i;
13        Func(a);
14        for (i=0; i<4; i++)
15        {
16            printf("%d", a[i]);
17        }
18        return 0;
19    }
```

参考答案:

(1) 10

(2) 0123

8.2 阅读程序,按要求在空白处填写适当的表达式或语句,使程序完整并符合题目要求。

(1) 下面的函数用于统计 10 个整数中正数的个数。

```
1     int PositiveNum(int a[], int n)
2     {
3         int i, count ____①____ ;
4         for (i=0; i<n; i++)
5         {
6             if (a[i] > 0) ____②____ ;
7         }
```

```
8         return ___③___;
9     }
```

（2）下面的函数使用迭代法计算 Fibonacci 数列前 n 项的值。

```
1     void Fib(long f[], ___①___)
2     {
3         int i;
4         f[0] = 0;
5         f[1] = 1;
6         for (i=2; i<n; i++)
7         {
8             f[i] = ___②___;
9         }
10    }
```

（3）从键盘输入 10 个整数，编程计算并输出其最大值、最小值及其所在元素的下标位置。

```
1     #include <stdio.h>
2     int main()
3     {
4         int a[10], n, max, min, maxPos, minPos;
5         for (n=0; n<10; n++)
6         {
7             scanf("%d", &a[n]);
8         }
9         max = min = a[0];
10        maxPos = minPos = ___①___;
11        for (n=0; n<10; n++)
12        {
13            if (___②___)
14            {
15                max = a[n];
16                maxPos = ___③___;
17            }
18            else if (___④___)
19            {
20                min = a[n];
21                minPos = ___⑤___;
22            }
23        }
24        printf("max=%d, pos=%d\n", max, maxPos);
```

```
25        printf("min=%d, pos=%d\n", min, minPos);
26        return 0;
27    }
```

(4) 利用矩阵相乘公式 $c_{ij} = \sum_{k=1}^{n} a_{ik} \times b_{kj}$，编程计算 $m \times n$ 阶矩阵 $\boldsymbol{A}$ 和 $n \times m$ 阶矩阵 $\boldsymbol{B}$ 之积。

```
1     #include <stdio.h>
2     #define ROW 2
3     #define COL 3
4     /* 函数功能:计算矩阵相乘之积,结果存于二维数组 c 中 */
5     MultiplyMatrix(int a[ROW][COL], int b[COL][ROW], int ____①____ )
6     {
7         int i, j, k;
8         for (i=0; i<ROW; i++)
9         {
10            for (j=0; j<ROW; j++)
11            {
12                c[i][j] = ____②____;
13                for (k=0; k<COL; k++)
14                {
15                    c[i][j] = ____③____;
16                }
17            }
18        }
19    }
20    /* 函数功能:输出矩阵 A 中的元素 */
21    void PrintMatrix(int a[ROW][ROW])
22    {
23        int i , j ;
24        for (i=0; i<ROW; i++)
25        {
26            for (j=0; j<ROW; j++)
27            {
28                printf("%6d", a[i][j]);
29            }
30            ____④____;
31        }
32    }
33    int main()
```

```
34    {
35        int a[ROW][COL], b[COL][ROW], c[ROW][ROW], i, j;
36        printf("Input 2 * 3 matrix a:\n");
37        for (i=0; i<ROW ;i++)
38        {
39            for (j=0; j<COL; j++)
40            {
41                scanf("%d", ⑤ );
42            }
43        }
44        printf("Input 3 * 2 matrix b:\n");
45        for (i=0; i<COL; i++)
46        {
47            for (j=0; j<ROW; j++)
48            {
49                scanf("%d", ⑥ );
50            }
51        }
52        MultiplyMatrix( ⑦ );
53        printf("Results:\n");
54        PrintMatrix(c);
55        return 0;
56    }
```

参考答案：

(1) ① =0 ② count++ ③ count

(2) ① int n ② f[i-1]+f[i-2]

(3) ① 0 ② a[n]>max ③ n ④ a[n]<min ⑤ n

(4) ① int c[ROW][ROW] ② 0 ③c[i][j]+a[i][k] * b[k][j]
④ printf ("\n") ⑤ &a[i][j] ⑥ &b[i][j] ⑦ a,b,c

8.3 分析下面的函数能否实现“返回一个数组中所有元素被第一个元素除的结果”的功能。代码中存在怎样的错误隐患？请编写正确的程序。

```
1    void DivArray(int *pArray, int n)
2    {
3        int i;
4        for (i=0; i<n; i++)
5        {
6            pArray[i] /= pArray[0];
```

```
7        }
8    }
```

参考答案:应该将第4行语句

```
    for (i=0; i<n; i++)
```

改成

```
    for (i=n-1; i>0; i--)
```

否则,程序第一次循环后pArray[0]的值就变成了1,这样后面的数组元素的值就不再是被第一个元素除的结果。为了得到数组中所有元素被第一个元素除的结果,应该从数组最后一个元素开始依次向前计算。

8.4 输入某班学生某门课程的成绩(最多不超过40人,具体人数由用户从键盘输入),用函数编程统计不及格的人数。

参考程序如下:

```
1    #include <stdio.h>
2    #define N 40
3    int GetFailNum(int score[], int n);
4    int main()
5    {
6        int i, n, fail, score[N];
7        printf("How many students?");
8        scanf("%d", &n);
9        for (i=0; i<n; i++)
10       {
11           scanf("%d", &score[i]);
12       }
13       fail = GetFailNum(score, n);
14       printf("Fail students = %d\n", fail);
15       return 0;
16   }
17   /* 函数功能:统计不及格人数 */
18   int GetFailNum(int score[], int n)
19   {
20       int i, count = 0;
21       for (i=0; i<n; i++)
22       {
23           if (score[i] < 60) count++;
24       }
25       return count;
26   }
```

程序的运行结果如下：

```
How many students? 5↙
45 56 78 99 100↙
Fail students = 2
```

8.5 参考例 8.6 程序中的函数 ReadScore( ) 和 Average( )，输入某班学生某门课程的成绩（最多不超过 40 人），当输入为负值时，表示输入结束。用函数编程统计成绩高于平均分的学生人数。

参考程序如下：

```
#include <stdio.h>
#define N 40
int Average(int score[], int n);
int ReadScore(int score[]);
int GetAboveAver(int score[], int n);
int main()
{
    int score[N], m, n;
    n = ReadScore(score);        /* 输入成绩,返回学生人数 */
    printf("Total students are %d\n", n);
    m = GetAboveAver(score, n);/* 统计成绩在平均分及以上的学生人数 */
    if(m != -1)printf("Students of above average is %d\n", m);
    return 0;
}
/* 函数功能:若 n>0,则计算并返回 n 个学生成绩的平均分,否则返回-1 */
int Average(int score[], int n)
{
    int i, sum = 0;
    for (i=0; i<n; i++)
    {
        sum += score[i];
    }
    return n>0 ? sum/n : -1;
}
/* 函数功能:输入学生某门课程成绩,当输入成绩为负值时,结束输入,返回学生人数 */
int ReadScore(int score[])
{
    int i = -1;
    do{
```

```
30          i++;
31          printf("Input score:");
32          scanf("%d", &score[i]);
33      } while (score[i] >= 0);
34      return i;
35  }
36  /* 函数功能:若n>0,则统计并返回成绩在平均分及平均分之上的学生人数,否则返回-1 */
37  int GetAboveAver(int score[], int n)
38  {
39      int i, count = 0, aver;
40      aver = Average(score, n);/* 计算并打印平均分 */
41      if(aver == -1)return -1;
42      printf("Average score is %d\n", aver);
43      for (i=0; i<n; i++)
44      {
45          if (score[i] >= aver) count++;
46      }
47      return count;
48  }
```

程序的运行结果如下：

```
Input score:45↙
Input score:56↙
Input score:78↙
Input score:99↙
Input score:100↙
Input score:-1↙
Total students are 5
Average score is 75
Students of above average is 3
```

8.6 参考例8.7程序中的函数ReadScore()和FindMax()，从键盘输入某班学生某门课程的成绩和学号(最多不超过40人)，当输入为负值时，表示输入结束。用函数编程通过返回数组中最大元素的下标，查找并输出成绩的最高分及其所属的学生学号。

参考程序如下：

```
1   #include <stdio.h>
2   #define N 40
3   int ReadScore(int score[], long num[]);
4   int FindMax(int score[], int n);
5   int main()
```

```
{
    int score[N], maxNum, n;
    long num[N];
    n = ReadScore(score, num);    /* 输入成绩,返回学生人数 */
    printf("Total students are %d\n", n);
    maxNum = FindMax(score, n);  /* 计算并返回最高分所在数组的下标 */
    printf("The highest is %ld, ID is %d\n", num[maxNum], score[maxNum]);
    return 0;
}
/* 函数功能:输入学生某门课程的成绩,当输入为负值时,结束输入,返回学生人数 */
int ReadScore(int score[], long num[])
{
    int i = -1;
    do{
        i++;
        printf("Input student's ID and score:");
        scanf("%ld%d", &num[i], &score[i]);
    } while (score[i] >= 0 && num[i] >= 0);
    return i;
}
/* 函数功能:计算并返回最高分所在数组的下标 */
int FindMax(int score[], int n)
{
    int max = score[0], i, maxNum = 0;
    for (i=1; i<n; i++)
    {
        if (score[i] > max)
        {
            max = score[i];
            maxNum = i;
        }
    }
    return maxNum;
}
```

程序的运行结果如下:

```
Input student's ID and score:120310122 84 ↙
Input student's ID and score:120310123 83 ↙
Input student's ID and score:120310124 88 ↙
```

```
Input student's ID and score:120310125 87↙
Input student's ID and score:120310126 61↙
Input student's ID and score:-1 -1↙
Total students are 5
The highest is 120310124,ID is 88
```

8.7 参考例 8.7 程序中的函数 FindMax( ),输入 10 个整数,用函数编程将其中最大数与最小数的位置互换,然后输出互换后的数组。

参考程序 1 如下:

```
#include <stdio.h>
void MaxMinExchange(int a[], int n);
int FindMaxPos(int s[], int n);
int FindMinPos(int s[], int n);
int main()
{
    int i, a[10];
    printf("Input 10 numbers:");
    for (i=0; i<10; i++)
    {
        scanf("%d", &a[i]);
    }
    MaxMinExchange(a, 10);
    printf("Exchange results:");
    for (i=0; i<10; i++)
    {
        printf("%4d", a[i]);
    }
    printf("\n");
    return 0;
}
/* 函数功能:将数组中最大数与最小数的位置互换 */
void MaxMinExchange(int a[], int n)
{
    int maxPos, minPos, temp;
    maxPos = FindMaxPos(a, n);
    minPos = FindMinPos(a, n);
    temp = a[maxPos];
    a[maxPos] = a[minPos];
    a[minPos] = temp;
```

```
}
/* 函数功能:计算数组中的最大值在数组中的下标位置 */
int FindMaxPos(int s[], int n)
{
    int maxPos = 0, max = s[0], i;
    for (i=1; i<n; i++)
    {
        if (s[i] > max)
        {
            max = s[i];
            maxPos = i;
        }
    }
    return maxPos;
}
/* 函数功能:计算数组中的最小值在数组中的下标位置 */
int FindMinPos(int s[], int n)
{
    int minPos = 0, min = s[0], i;
    for (i=1; i<n; i++)
    {
        if (s[i] < min)
        {
            min = s[i];
            minPos = i;
        }
    }
    return minPos;
}
```

参考程序 2 如下:

```
#include <stdio.h>
void MaxMinExchange(int a[], int n);
int main()
{
    int i, a[10];
    printf("Input 10 numbers:");
    for (i=0; i<10; i++)
    {
```

```
9           scanf("%d", &a[i]);
10      }
11      MaxMinExchange(a, 10);
12      printf("Exchange results:");
13      for (i=0; i<10; i++)
14      {
15          printf("%4d", a[i]);
16      }
17      printf("\n");
18      return 0;
19  }
20  /* 函数功能:将数组中最大数与最小数的位置互换 */
21  void MaxMinExchange(int a[], int n)
22  {
23      int max = a[0], min = a[0], maxPos = 0, minPos = 0;
24      int i, temp;
25      for (i=1; i<n; i++)
26      {
27          if (a[i] > max)
28          {
29              max = a[i];
30              maxPos = i;
31          }
32          if (a[i] < min)
33          {
34              min = a[i];
35              minPos = i;
36          }
37      }
38      temp = a[maxPos];
39      a[maxPos] = a[minPos];
40      a[minPos] = temp;
41  }
```

程序的运行结果如下:

```
Input 10 numbers:1 2 3 4 5 6 7 8 9 10↙
Exchange results: 10  2  3  4  5  6  7  8  9  1
```

8.8 假设有 40 个学生被邀请来给餐厅的饮食和服务质量打分,分数划分为 1~10 这 10 个等级(1 表示最低分,10 表示最高分),编程统计并按如下格式输出餐饮服务质量调查结果。

```
Grade      Count      Histogram
  1          5        * * * * *
  2          10       * * * * * * * * * *
  3          7        * * * * * * *
 ...
```

参考程序 1 如下：

```
#include <stdio.h>
#define M 40
#define N 11
int main()
{
    int i, j, grade, feedback[M], count[N] = {0};
    printf("Input the feedbacks of 40 students: \n");
    for (i=0; i<M; i++)
    {
        scanf("%d", &feedback[i]);
    }
    for (i=0; i<M; i++)
    {
        switch (feedback[i])
        {
            case 1: count[1]++;break;
            case 2: count[2]++;break;
            case 3: count[3]++;break;
            case 4: count[4]++;break;
            case 5: count[5]++;break;
            case 6: count[6]++;break;
            case 7: count[7]++;break;
            case 8: count[8]++;break;
            case 9: count[9]++;break;
            case 10:count[10]++;break;
            default:printf("input error! \n");
        }
    }
    printf("Feedback \tCount \tHistogram \n");
    for (grade=1; grade<=N-1; grade++)
    {
        printf("%8d \t%5d \t", grade, count[grade]);
```

```
33          for (j=0; j<count[grade]; j++)
34          {
35              printf("%c",'*');
36          }
37          printf("\n");
38      }
39      return 0;
40  }
```

参考程序 2 如下：

```
1   #include <stdio.h>
2   #define M 40
3   #define N 11
4   int main()
5   {
6       int i, j, grade, feedback[M], count[N] = {0};
7       printf("Input the feedbacks of 40 students: \n");
8       for (i=0; i<M; i++)
9       {
10          scanf("%d", &feedback[i]);
11      }
12      for (i=0; i<M; i++)
13      {
14          count[feedback[i]]++;
15      }
16      printf("Feedback \tCount \tHistogram \n");
17      for (grade=1; grade<=N-1; grade++)
18      {
19          printf("%8d \t%5d \t", grade, count[grade]);
20          for (j=0; j<count[grade]; j++)
21          {
22              printf("%c",'*');
23          }
24          printf("\n");
25      }
26      return 0;
27  }
```

程序的运行结果如下：

```
Input the feedbacks of 40 students:
```

```
10 9 10 8 7  6 5  10 9 8↙
8  9 7  6 10 9 8  8  7 7↙
6  6 8  8 9  9 10 8  7 7↙
9  8 7  9 7  6 5  9  8 7↙
Feedback        Count        Histogram
     1              0
     2              0
     3              0
     4              0
     5              2        * *
     6              5        * * * * *
     7              9        * * * * * * * * *
     8              10       * * * * * * * * * *
     9              9        * * * * * * * * *
     10             5        * * * * *
```

8.9 在习题8.8的基础上,用一个整型数组feedback保存调查的40个反馈意见。用函数编程计算反馈意见的**平均数(Mean)**、**中位数(Median)**和**众数(Mode)**。中位数指的是排列在数组中间的数。如果原始数据的个数是偶数,那么中位数等于中间那两个元素的算术平均值。众数是数组中出现次数最多的那个数(不考虑两个或两个以上的反馈意见出现次数相同的情况)。

参考答案:在调查数据分析(Survey Data Analysis)中经常需要计算平均数、中位数和众数。计算中位数时,首先要调用排序函数对数组按升序进行排序,然后取出排序后数组中间位置的元素answer[$n/2$],就得到了中位数。如果数组元素的个数是偶数,那么中位数就等于数组中间那两个元素的算术平均值。

众数就是40个反馈意见中出现次数最多的那个数。计算众数时,首先要统计不同类型的反馈意见出现的次数,然后找出出现次数最多的那个反馈意见,这个反馈意见就是众数(这里没有考虑两个或者两个以上的反馈意见出现次数相同的情况)。习题8.8的运行结果以直方图的形式打印出了这一统计结果,可以帮助用户直观地判断众数的计算结果是否正确。

参考程序如下:

```
1    #include <stdio.h>
2    #define M 40
3    #define N 11
4    int Mean(int answer[], int n);
5    int Median(int answer[], int n);
6    int Mode(int answer[], int n);
7    void DataSort(int a[], int n);
8    int main()
9    {
```

```
10      int i, feedback[M];
11      printf("Input the feedbacks of 40 students:\n");
12      for (i=0; i<M; i++)
13      {
14          scanf("%d", &feedback[i]);
15      }
16      printf("Mean value = %d\n", Mean(feedback, M));
17      printf("Median value = %d\n", Median(feedback, M));
18      printf("Mode value = %d\n", Mode(feedback, M));
19      return 0;
20  }
21  /*  函数功能:若 n>0,则计算并返回 n 个数的平均数,否则返回-1 */
22  int Mean(int answer[], int n)
23  {
24      int i, sum = 0;
25      for (i=0; i<n; i++)
26      {
27          sum += answer[i];
28      }
29      return n>0? sum/n:-1;
30  }
31  /*  函数功能:计算 n 个数的中位数 */
32  int Median(int answer[], int n)
33  {
34      DataSort(answer, n);
35      if (n%2 == 0)
36          return (answer[n/2] + answer[n/2-1]) / 2;
37      else
38          return answer[n/2];
39  }
40  /*  函数功能:计算 n 个数的众数 */
41  int Mode(int answer[], int n)
42  {
43      int i, grade, max = 0, modeValue = 0, count[N] = {0};
44      for (i=0; i<n; i++)
45      {
46          count[answer[i]]++;
47      }
```

```
48        for (grade=1; grade<=N-1; grade++)
49        {
50            if (count[grade] > max)
51            {
52                max = count[grade];
53                modeValue = grade;
54            }
55        }
56        return modeValue;
57    }
58    /* 函数功能:按选择法对数组 a 中的 n 个元素进行排序 */
59    void DataSort(int a[], int n)
60    {
61        int i, j, k, temp;
62        for (i=0; i<n-1; i++)
63        {
64            k = i;
65            for (j=i+1; j<n; j++)
66            {
67                if (a[j] < a[k])  k=j;
68            }
69            if (k != i)
70            {
71                temp = a[k];
72                a[k] = a[i];
73                a[i] = temp;
74            }
75        }
76    }
```

程序的运行结果如下:

```
Input the feedbacks of 40 students:
10 9 10 8 7  6 5  10 9 8↙
8  9 7  6 10 9 8  8  7 7↙
6  6 8  8 9  9 10 8  7 7↙
9  8 7  9 7  6 5  9  8 7↙
Mean value = 7
Median value = 8
Mode value = 8
```

8.10 输入 $n \times n$ 阶矩阵,用函数编程计算并输出其两条对角线上的各元素之和。

参考程序如下:

```
#include <stdio.h>
#define N 10
void InputMatrix(int a[N][N], int n);
int AddDiagonal(int a[N][N], int n);
int main()
{
    int a[N][N], n, sum;
    printf("Input n:");
    scanf("%d", &n);
    InputMatrix(a, n);
    sum = AddDiagonal(a, n);
    printf("sum = %d\n", sum);
    return 0;
}
/* 函数功能:输入 n×n 矩阵的元素值,存于数组 a 中 */
void InputMatrix(int a[N][N], int n)
{
    int i, j;
    printf("Input %d * %d matrix:\n", n, n);
    for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}
/* 函数功能:计算 n×n 矩阵中两条对角线上的元素之和 */
int AddDiagonal(int a[N][N], int n)
{
    int i, j, sum = 0;
    for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            if (i == j ||i+j == n-1)
```

```
37                  sum = sum + a[i][j];
38              }
39          }
40          return sum;
41      }
```

程序的运行结果如下：

```
Input n:5↙
Input 5 * 5 matrix:
1 2 3 4 5↙
2 3 4 5 6↙
3 4 5 6 7↙
4 5 6 7 8↙
5 6 7 8 9↙
sum = 45
```

8.11 输入 $m \times n$ 阶矩阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$，用函数编程计算并输出 $\boldsymbol{A}$ 与 $\boldsymbol{B}$ 之和。

参考程序如下：

```
1   #include<stdio.h>
2   #define ROW 10
3   #define COL 10
4   void InputMatrix(int a[ROW][COL], int m, int n);
5   void AddMatrix(int a[ROW][COL], int b[ROW][COL], int c[ROW][COL],
6                  int m, int n);
7   void PrintMatrix(int a[ROW][COL], int m, int n);
8   int main()
9   {
10      int a[ROW][COL], b[ROW][COL], c[ROW][COL], m, n;
11      printf("Input m, n:");
12      scanf("%d,%d", &m, &n);
13      printf("Input %d * %d matrix a:\n", m, n);
14      InputMatrix(a, m, n);
15      printf("Input %d * %d matrix b:\n", m, n);
16      InputMatrix(b, m, n);
17      AddMatrix(a, b, c, m, n);
18      printf("Results:\n");
19      PrintMatrix(c, m, n);
20      return 0;
21  }
22  /* 函数功能：输入 m×n 矩阵的元素值，存于数组 a 中 */
```

```
23  void InputMatrix(int a[ROW][COL], int m, int n)
24  {
25      int i, j;
26      for (i=0; i<m; i++)
27      {
28          for (j=0; j<n; j++)
29          {
30              scanf("%d", &a[i][j]);
31          }
32      }
33  }
34  /* 函数功能:计算 m×n 矩阵之和,即对应位置元素之和,结果存于数组 c 中 */
35  void AddMatrix(int a[ROW][COL], int b[ROW][COL], int c[ROW][COL],
36                 int m, int n)
37  {
38      int i, j;
39      for (i=0; i<m; i++)
40      {
41          for (j=0; j<n; j++)
42          {
43              c[i][j] = a[i][j] + b[i][j];
44          }
45      }
46  }
47  /* 函数功能:输出 m×n 矩阵的元素值 */
48  void PrintMatrix(int a[ROW][COL], int m, int n)
49  {
50      int i, j;
51      for (i=0; i<m; i++)
52      {
53          for (j=0; j<n; j++)
54          {
55              printf("%6d", a[i][j]);
56          }
57          printf("\n");
58      }
59  }
```

程序的运行结果如下：

```
Input m, n:2,3↙
Input 2 * 3 matrix a:
1  2  3↙
4  5  6↙
Input 2 * 3 matrix b:
11  12  13↙
14  15  16↙
Results:
    12    14    16
    18    20    22
```

8.12 用函数编程计算并输出如图 1-5 所示的杨辉三角形。

```
1
1  1
1  2  1
1  3  3  1
1  4  6  4  1
1  5  10 10 5  1
1  6  15 20 15 6  1
```

图 1-5 杨辉三角形

参考程序 1 如下：

```
#include<stdio.h>
#define N 20
void CaculateYH(int a[][N], int n);
void PrintYH(int a[][N], int n);
int main()
{
    int a[N][N] = {0}, n;
    printf("Input n(n<20):");
    scanf("%d", &n);
    CaculateYH(a, n);
    PrintYH(a, n);
    return 0;
}
/* 函数功能:计算杨辉三角形前 n 行元素的值 */
void CaculateYH(int a[][N], int n)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        a[i][0] = 1;
        a[i][i] = 1;
    }
    for (i=2; i<n; i++)
    {
        for (j=1; j<=i-1; j++)
        {
```

```
            a[i][j] = a[i-1][j-1] + a[i-1][j];
        }
    }
}
/* 函数功能:输出杨辉三角形前 n 行元素的值 */
void PrintYH(int a[][N], int n)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        for (j=0; j<=i; j++)
        {
            printf("%4d", a[i][j]);
        }
        printf("\n");
    }
}
```

参考程序 2 如下:

```
#include<stdio.h>
#define N 20
void CaculateYH(int a[][N], int n);
void PrintYH(int a[][N], int n);
int main()
{
    int a[N][N] = {0}, n;
    printf("Input n(n<20):");
    scanf("%d", &n);
    CaculateYH(a, n);
    PrintYH(a, n);
    return 0;
}
/* 函数功能:计算杨辉三角形前 n 行元素的值 */
void CaculateYH(int a[][N], int n)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        for (j=0; j<=i; j++)
```

```
            {
                if (j==0 ||i==j)
                    a[i][j] = 1;
                else
                    a[i][j] = a[i-1][j-1] + a[i-1][j];
            }
        }
    }
    /* 函数功能:输出杨辉三角形前 n 行元素的值 */
    void PrintYH(int a[][N], int n)
    {
        int i, j;
        for (i=0; i<n; i++)
        {
            for (j=0; j<=i; j++)
            {
                printf("%4d", a[i][j]);
            }
            printf("\n");
        }
    }
```

程序的运行结果如下:

```
Input n(n<20):7↙
   1
   1   1
   1   2   1
   1   3   3   1
   1   4   6   4   1
   1   5   10  10  5   1
   1   6   15  20  15  6   1
```

8.13 (选做)兔子生崽问题。假设一对小兔的成熟期是一个月,即一个月可长成成兔,那么如果每对成兔每个月都可以生一对小兔,一对新生的小兔从第二个月起就开始生兔子,试问从一对兔子开始繁殖,一年以后可有多少对兔子? 请编程求解该问题。

参考答案:依题意,兔子的繁殖情况示意图如图 1-6 所示。图中实线表示成兔仍是成兔或者小兔长成成兔;虚线表示成兔生小兔。观察分析此图可发现如下规律:

(1) 每个月小兔对数 = 上个月成兔对数。

(2) 每个月成兔对数 = 上个月成兔对数 + 上个月小兔对数。

综合(1)和(2)有:每个月成兔对数 = 前两个月成兔对数之和。

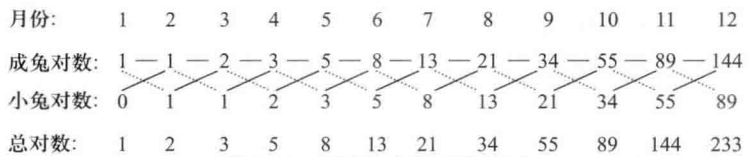


图 1-6 兔子繁殖情况示意图

用 $f_n(n=1,2,\cdots)$ 表示第 n 个月成兔对数，于是可将上述规律表示为如下递推公式：

$$f_1 = 1 \qquad (n=1)$$

$$f_2 = 1 \qquad (n=2)$$

$$f_n = f_{n-1} + f_{n-2} \qquad (n \geqslant 3)$$

依次令 $n=1,2,3,\cdots$，可由上述公式递推求出每个月成兔对数为：

$$1,1,2,3,5,8,13,21,34,55,89,144,\cdots$$

这就是著名的 Fibonacci 数列。同理，可得每个月小兔的对数为：

$$0,1,1,2,3,5,8,13,21,34,55,89,\cdots$$

因此，每个月兔子的总对数为：

$$1,2,3,5,8,13,21,34,55,89,144,233,\cdots$$

每个月兔子的总对数以及一年以后兔子总对数的参考程序如下：

```
#include <stdio.h>
#define N 12
void Fibonacci(int f[], int n);
int main()
{
    int f[N], i;
    Fibonacci(f, N);
    printf("\nTotal = %d\n", f[N-1]);
    return 0;
}
/* 函数功能:计算并打印 Fibonacci 数列的前 n 项 */
void Fibonacci(int f[], int n)
{
    int i;
    f[0] = 1;
    f[1] = 2;
    for (i=2; i<n; i++)
    {
        f[i] = f[i-1] + f[i-2];
    }
    for (i=0; i<N; i++)
    {
```

```
23          printf("%4d", f[i]);
24      }
25  }
```

程序的运行结果如下：

```
   1  2  3  5  8  13  21  34  55  89  144  233
   Total = 233
```

8.14 （选做）模拟骰子的 6 000 次投掷，编程统计并输出骰子的 6 个面各自出现的概率。

参考程序如下：

```
#include <stdlib.h>
#include <time.h>
#include <stdio.h>
int main()
{
    int face, roll, frequency[7] = {0};
    srand(time (NULL));
    for (roll=1; roll<=6000; roll++)
    {
        face = rand()%6 + 1;
        frequency[face]++;
    }
    printf("%4s%17s\n", "Face", "Frequency");
    for (face=1; face<=6; face++)
    {
        printf("%4d%17d\n", face, frequency[face]);
    }
    return 0;
}
```

程序的运行结果如下：

```
Face          Frequency
  1                997
  2                998
  3                972
  4               1033
  5                971
  6               1029
```

8.15 （选做）模拟文曲星上的猜数游戏。先由计算机随机生成一个各位相异的 4 位数字，由用户来猜，根据用户猜测的结果给出提示：$xAyB$。其中，A 前面的数字表示有几位数字不仅数字猜对了，而且位置也正确，B 前面的数字表示有几位数字猜对了，但是位置不正确。最多允许

用户猜的次数由用户从键盘输入。如果猜对,则提示“Congratulations!”;如果在规定次数以内仍然猜不对,则给出提示“Sorry, you haven't got it,see you next time!”。程序结束之前,在屏幕上显示这个正确的数字。

参考答案:首先要随机生成一个各位相异的 4 位数,方法是:将 0~9 这 10 个数字顺序放入数组 a(应足够大)中,然后将其排列顺序随机打乱 10 次,取前 4 个数组元素的值,即可得到一个各位相异的 4 位数。然后,用数组 a 存储计算机随机生成的 4 位数,用数组 b 存储用户猜的 4 位数,对 a 和 b 中相同位置的元素进行比较,可得 A 前面待显示的数字;对 a 和 b 的不同位置的元素进行比较,可得 B 前面待显示的数字。参考程序如下:

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
void MakeDigit(int a[]);
int InputGuess(int b[]);
int IsRightPosition(int magic[], int guess[]);
int IsRightDigit(int magic[], int guess[]);
int main()
{
    int a[10];                          /* 记录计算机所想的数 */
    int b[4];                           /* 记录人猜的数 */
    int count;                          /* 记录已经猜的次数 */
    int rightDigit;                     /* 猜对的数字个数 */
    int rightPosition;                  /* 数字和位置都猜对的个数 */
    int level;                          /* 最多允许猜的次数 */
    srand(time(NULL));
    MakeDigit(a);                       /* 随机生成一个各位相异的 4 位数 */
    printf("How many times do you want to guess?");
    scanf("%d", &level);
    count = 0;
    do{
        printf("No.%d of %d times:\n", count+1, level);
        printf("Please input a number:");
        if (InputGuess(b) != 0)     /* 读入用户的猜测 */
        {
        count++;
        rightPosition = IsRightPosition(a, b);/* 数字和位置都猜对的个数*/
        rightDigit = IsRightDigit(a, b); /* 用户猜对的数字个数*/
        rightDigit = rightDigit - rightPosition;
        printf("%dA%dB\n", rightPosition, rightDigit);
```

```
        }
    } while (count < level && rightPosition != 4);
    if (rightPosition == 4)
        printf("Congratulations, you got it at No.%d\n", count);
    else
        printf("Sorry, you haven't got it, see you next time! \n");
    printf("Correct answer is:%d%d%d%d\n", a[0], a[1], a[2], a[3]);
    return 0;
}
/* 函数功能:随机生成一个各位相异的 4 位数 */
void MakeDigit(int a[])
{
    int j, k, temp;
    for (j=0; j<10; j++)
    {
        a[j] = j;
    }
    for (j=0; j<10; j++)
    {
        k = rand() % 10;
        temp = a[j];
        a[j] = a[k];
        a[k] = temp;
    }
}
/* 函数功能: 读入用户猜的数,读入失败返回 0,否则返回非 0 */
int InputGuess(int b[])
{
    int i, ret = 1;
    for (i=0; i<4; i++)
    {
        ret = scanf("%1d", &b[i]);
        if (ret != 1)                          /* 如果输入非法 */
        {
            printf("Input Data Type Error! \n");
            while (getchar() != '\n');         /* 清除输入缓冲区中的内容 */
            return 0;
        }
```

```
69        }
70        if (b[0] == b[1] || b[0] == b[2] ||  b[0] == b[3] ||
71            b[1] == b[2] || b[1] == b[3] ||  b[2] == b[3])
72        {
73            printf("The digits must be different from each other! \n");
74            return 0;
75        }
76        else
77        {
78            return 1;
79        }
80    }
81    /*  函数功能: 统计 guess 和 magic 数组中数字和位置都一致的个数 */
82    int IsRightPosition(int magic[],int guess[])
83    {
84        int rightPosition = 0;
85        int j;
86        for (j=0; j<4; j++)
87        {
88            if (guess[j] == magic[j])
89                rightPosition = rightPosition + 1;
90        }
91        return rightPosition;
92    }
93    /*  函数功能:统计 guess 和 magic 数组中数字一致(不管位置是否一致)的个数 */
94    int IsRightDigit(int magic[],int guess[])
95    {
96        int rightDigit = 0;
97        int j, k;
98        for (j=0; j<4; j++)
99        {
100           for (k=0; k<4; k++)
101           {
102               if (guess[j] == magic[k])
103                   rightDigit = rightDigit + 1;
104           }
105       }
106       return rightDigit;
```

```
107     }
```

程序的运行结果如下：

```
How many times do you want to guess? 7↙
No.1 of 7 times:
Please input a number:1234↙
2A0B
No.2 of 7 times:
Please input a number:2304↙
0A3B
No.3 of 7 times:
Please input a number:0235↙
3A0B
No.4 of 7 times:
Please input a number:0239↙
3A0B
No.5 of 7 times:
Please input a number:0237↙
4A0B
Congratulations, you got it at No.5
Correct answer is:0237
```

8.16　（选做）用函数编程实现在一个按升序排序的数组中查找 x 应插入的位置，将 x 插入数组中，使数组元素仍按升序排列。

参考答案：插入（Insertion）是数组的基本操作之一。插入排序算法的关键在于要找到正确的插入位置，然后依次移动插入位置及其后的所有元素，腾出这个位置放入待插入的元素。插入排序的算法示意如图 1-7 所示。

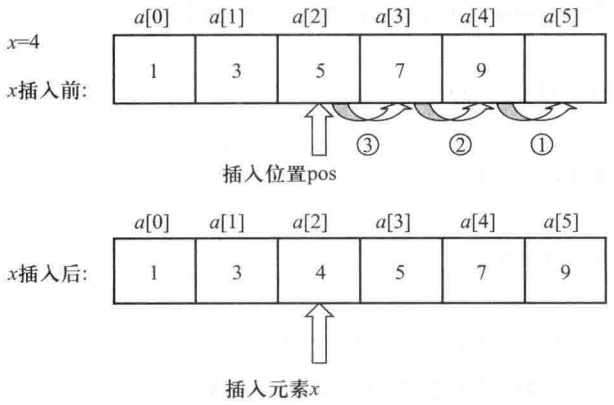


图 1-7　插入排序算法示意

参考程序如下：

```
#include<stdio.h>
#define N 20                        /* 插入前数组最大元素个数 */
void Insert(int a[], int n, int x);
int main()
{
    int a[N+1];                     /* 定义数组长度为插入前的数组元素个数加 1 */
    int x, i, n;
    printf("Input array size:");
    scanf("%d", &n);                /* 输入插入前数组元素个数 */
    printf("Input array:");
    for (i=0; i<n; i++)
    {
        scanf("%d", &a[i]);         /* 输入插入前已按升序排序的数组元素 */
    }
    printf("Input x:");
    scanf("%d", &x);                /* 输入待插入的元素 x */
    Insert(a, n, x);                /* 插入元素 x 到已排序数组中 */
    printf("After insert %d:\n", x);
    for (i=0; i<n+1; i++)
    {
        printf("%4d", a[i]);        /* 输出插入 x 后的数组元素 */
    }
    printf("\n");
    return 0;
}
/* 函数功能:将 x 插入到一个已按升序排序的数组中 */
void Insert(int a[], int n, int x)
{
    int i = 0, pos;
    while (i < n && x > a[i])       /* 查找待插入位置 */
    {
        i++;
    }
    pos = i;                        /* 记录元素 x 应插入的数组下标位置 pos */
    for (i = n-1; i>= pos; i--)     /* 从尾部开始移动 pos 及其后所有的元素 */
    {
        a[i+1] = a[i];              /* 向后复制数组元素 */
```

```
38          }
39          a[pos] = x;                    /* 插入元素 x 到位置 pos */
40      }
```

程序的运行结果如下：

```
Input array size: 5↙
Input array:1  3  5  7  9↙
Input x: 4↙
After insert 4:
  1  3  4  5  7  9
```

8.17　（选做）冒泡排序（Bubble Sort），也称为沉降排序（Sinking Sort），之所以称其为冒泡排序，是因为算法中值相对较小的数据会像水中的气泡一样逐渐上升到数组的最顶端。与此同时，较大的数据逐渐地下沉到数组的底部。这个处理过程需要在整个数组范围内反复执行多遍。每一遍执行时，比较相邻的两个元素，若顺序不对，则将其位置交换，当没有数据需要交换时，数据也就排好序了。在例 8.8 程序的基础上，编程将排序函数 DataSort( ) 改用冒泡法实现。

参考答案：如图 1-8 所示，采用冒泡法进行升序排序的基本原理如下。

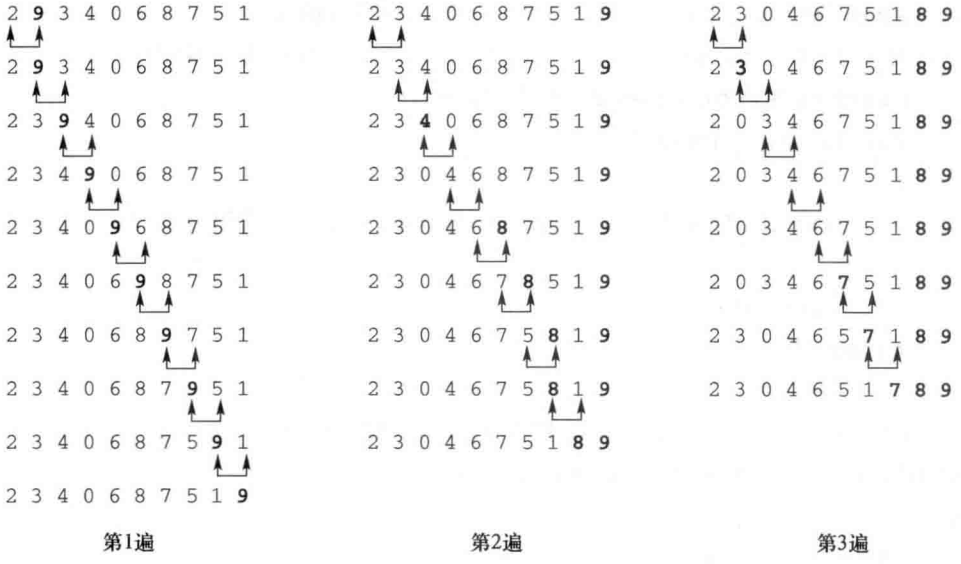


图 1-8　冒泡法排序原理

对数组中的 n 个数执行 $n-1$ 遍检查操作，在每一遍执行时，对数组中剩余的尚未排好序的元素进行如下操作：对相邻的两个元素进行比较，若排在后面的数小于排在前面的数，则交换其位置，这样每一遍操作中都将参与比较的数中的最大的数沉到数组的底部，经过 $n-1$ 遍操作后，就将全部 n 个数按从小到大的顺序排好序了。冒泡排序法的优点是易于理解、实现简单。但缺点是运行效率较低，这是因为在每一次交换中，一个元素只能向它的最终目标位置移动一个位置。在对一个较大的数组进行排序时，这一点表现得尤为明显。

参考程序如下：

```
#include<stdio.h>
#define N 10
void BubbleSort(int a[], int n);
int main()
{
    int i, n, a[N];
    printf("Input n:");
    scanf("%d", &n);
    printf("Input %d numbers:", n);
    for (i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
    BubbleSort(a, n);
    printf("Sorting results:");
    for (i=0; i<n; i++)
    {
        printf("%4d", a[i]);
    }
    printf("\n");
    return 0;
}
/* 函数功能:使用冒泡法实现数组 a 的 n 个元素的升序排序 */
void BubbleSort(int a[], int n)
{
    int i, j, temp;
    for (i=0; i<n-1; i++)
    {
        for (j=1; j<n-i; j++)
        {
            if (a[j] < a[j-1])
            {
                temp = a[j];
                a[j] = a[j-1];
                a[j-1] = temp;
            }
        }
```

```
38        }
39    }
```

程序的运行结果如下：

```
Input n: 10↙
Input 10 numbers:2 9 3 4 0 6 8 7 5 1↙
Sorting results:  0  1  2  3  4  5  6  7  8  9
```

8.18 （选做）挑战类型表示的极限——大数的存储问题。编程计算并输出 1～40 之间的所有数的阶乘。提示：用一个包含 50 个元素的数组存储一个大数，每个数组元素存储大数中的一位数字。

参考答案：用一个数组存储一个大数，每个数组元素存储大数中的一位数字，然后采用逐位相乘、后位向前进位的方法计算大数的阶乘值，只要数组长度定义得足够大，就可以计算足够大的数的阶乘值。不用函数编写的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 51
int main()
{
    int data[SIZE]={0}; /* 存储 50 位数,元素全部初始化为 0,不使用 data[0] */
    int index = 1;      /* 数组元素个数,表示阶乘值的位数 */
    int n;              /* 准备计算的阶乘中的最大数 */
    int i, j, k;
    data[1] = 1;        /* 初始化,令 1! =1 */
    printf("Input n:");
    scanf("%d", &n);
    for (i=1; i<=n; i++)          /* 计算 1～n 之间所有数的阶乘值 */
    {
        for (j=1; j<=index; j++)  /* 计算阶乘 i! */
        {
            data[j] = data[j] *i;  /* 每一位数字都乘以 i */
        }
        for (k=1; k<index; k++)
        {
            if (data[k] >= 10) /* 若阶乘值的每位数字大于或等于 10,则进位 */
            {
                data[k+1] = data[k+1] + data[k]/10;    /* 当前位向前进位 */
                data[k] = data[k] % 10;                /* 进位之后的值 */
            }
        }
```

```
27          /* 单独处理最高位,若计算之后的最高位大于或等于 10,则位数 index 加 1 */
28          while (data[index] >= 10 && index <= SIZE-1)
29          {
30              data[index+1] = data[index] / 10;       /* 向最高位进位 */
31              data[index] = data[index] % 10;         /* 进位之后的值 */
32              index++;                                /* 位数 index 加 1 */
33          }
34          if (index <= SIZE-1)  /* 检验数组是否溢出,若未溢出,则打印阶乘值 */
35          {
36              printf("%d! = ", i);
37              for (j=index; j>0; j--) /* 从最高位开始打印每一位阶乘值 */
38              {
39                  printf("%d", data[j]);
40              }
41              printf("\n");
42          }
43          else                        /* 若大于 50,数组溢出,则提示错误信息 */
44          {
45              printf("Overflow! \n");
46              exit(1);
47          }
48      }
49      return 0;
50  }
```

用函数编写的参考程序如下:

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #define SIZE 51
4   int BigFact(int m, int data[]);
5   int main()
6   {
7       int data[SIZE]={0};  /* 存储 50 位数,元素全部初始化为 0,不使用 data[0] */
8       int index;           /* 数组元素个数,表示阶乘值的位数 */
9       int n;               /* 准备计算的阶乘中的最大数 */
10      int i, j;
11      printf("Input n:");
12      scanf("%d", &n);
13      for (i=1; i<=n; i++)/* 计算 1~n 之间所有数的阶乘值 */
```

```
14      {
15          index = BigFact(i, data);/* 计算阶乘 i!,返回阶乘值的位数 */
16          if (index ! = 0)       /* 检验数组是否溢出,若未溢出,则打印阶乘值 */
17          {
18              printf("%d! = ", i);
19              for (j=index; j>0; j--)/* 从最高位开始打印每一位阶乘值 */
20              {
21                  printf("%d", data[j]);
22              }
23              printf("\n");
24          }
25          else                        /* 若大于 50,数组溢出,则提示错误信息 */
26          {
27              printf("Overflow! \n");
28              exit(1);
29          }
30      }
31      return 0;
32  }
33  /* 函数功能:计算 m!,存于数组 data 中,若数组未溢出,则返回阶乘值的位数,否则返回 0 */
34  int BigFact(int m, int data[])
35  {
36      int i,j,k;
37      int index = 1;                  /* 表示数组中的位数 */
38      for (i=0; i<SIZE; i++)
39      {
40          data[i] = 0;  /* 每个数组元素存储阶乘值的每一位数字,全部初始化为 0 */
41      }
42      data[1] = 1;                    /* 初始化,令 1! =1 */
43      for (i=1; i<=m; i++)            /* 计算阶乘 m! */
44      {
45          for (j=1; j<=index; j++)
46          {
47              data[j] = data[j] * i; /* 每一位数字都乘以 i */
48          }
49          for (k=1; k<index; k++)
50          {
51              if (data[k] >= 10)/* 若阶乘值的每位数字大于或等于 10,则进位 */
```

```
52              {
53                  data[k+1] = data[k+1] + data[k]/10;  /* 当前位向前进位 */
54                  data[k] = data[k] % 10;              /* 进位之后的值 */
55              }
56          }
57          /* 单独处理最高位,若计算之后的最高位大于或等于 10,则位数 index 加 1 */
58          while (data[index] >= 10 && index <= SIZE-1)
59          {
60              data[index+1] = data[index] / 10;        /* 向最高位进位 */
61              data[index] = data[index] % 10;          /* 进位之后的值 */
62              index++;                                 /* 位数 index 加 1 */
63          }
64      }
65      if (index <= SIZE-1)  /* 检验数组是否溢出,若未溢出,则返回阶乘值的位数 */
66          return index;
67      else                  /* 若大于 50,数组溢出,则返回 0 值 */
68          return 0;
69  }
```

程序的运行结果如下：

```
Input n: 40↙
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 6227020800
14! = 87178291200
15! = 1307674368000
16! = 20922789888000
17! = 355687428096000
18! = 6402373705728000
```

```
19! = 121645100408832000
20! = 2432902008176640000
21! = 51090942171709440000
22! = 1124000727777607680000
23! = 25852016738884976640000
24! = 620448401733239439360000
25! = 15511210043330985984000000
26! = 403291461126605635584000000
27! = 10888869450418352160768000000
28! = 304888344611713860501504000000
29! = 8841761993739701954543616000000
30! = 265252859812191058636308480000000
31! = 8222838654177922817725562880000000
32! = 263130836933693530167218012160000000
33! = 8683317618811886495518194401280000000
34! = 295232799039604140847618609643520000000
35! = 10333147966386144929666651337523200000000
36! = 371993326789901217467999448150835200000000
37! = 13763753091226345046315979581580902400000000
38! = 523022617466601111760007224100074291200000000
39! = 20397882081197443358640281739902897356800000000
40! = 815915283247897734345611269596115894272000000000
```

8.19 （选做）大奖赛现场统分。已知某大奖赛有 n 个选手参赛，$m(m>2)$ 个评委为参赛选手评分（最高 10 分，最低 0 分）。计分规则为：在每个选手的 m 个得分中，去掉一个最高分和一个最低分后，取平均分作为该选手的最后得分。要求编程实现：

（1）根据 n 个选手的最后得分，从高到低输出选手的得分名次表，以确定获奖名单。

（2）根据各选手的最后得分与各评委给该选手所评分数的差距，对每个评委评分的准确性和评分水准给出一个定量的评价，从高到低输出各评委得分的名次表。

参考答案：首先设计如下 5 个数组。

（1）sh[i]，存放第 i 个选手的编号。

（2）sf[i]，存放第 i 个选手的最后得分，即去掉一个最高分和一个最低分以后的平均分。

（3）ph[j]，存放第 j 个评委的编号。

（4）f[i][j]，存放第 j 个评委给第 i 个选手的评分。

（5）pf[j]，存放代表第 j 个评委评分水准的得分。

解决本问题的关键在于计算选手的最后得分和评委的得分。

先计算选手的最后得分。外层循环控制参赛选手的编号 i 从 1 变化到 n，当第 i 个选手上场时，输入该选手的编号 sh[i]。内层循环控制给选手评分的评委的编号 j 从 1 变化到 m，依次输入第 j 个评委给第 i 个选手的评分 f[i][j]，并将其累加到 sf[i]中，同时求出最高分 max 和最低

分 min。当第 i 个选手的 m 个得分全部输入并累加完毕后，去掉一个最高分 max，去掉一个最低分 min，于是第 i 个选手的最后得分为：

```
sf[i] = (sf[i] - max - min)/(m-2);
```

当 n 个参赛选手的最后得分 sf[0]，sf[1]，…，sf[n]全部计算完毕后，再将其从高到低排序，打印参赛选手的名次表。

下面计算评委的得分。评委给选手评分存在误差，即 f[i][j] ≠ sf[i]是正常的，也是允许的。但如果某个评委给每个选手的评分与各选手的最后得分都相差太大，则说明该评委的评分有失水准。可用下面的公式来对各个评委的评分水平进行定量评价：

$$pf[j] = 10 - \sqrt{\frac{\sum_{i=1}^{n}(f[i][j] - sf[i])^2}{n}}$$

显然，pf[j]值越高，说明评委的评分水平越高，因此可依据 m 个评委的 pf[j]值打印出评委评分水平高低的名次表。参考程序如下：

```
#include <stdio.h>
#include <math.h>
#define ATHLETE 40          /* 选手人数最高限 */
#define JUDGE 20            /* 评委人数最高限 */
void CountAthleteScore(int sh[], float sf[], int n, float f[], int m);
void Sort(int h[], float f[], int n);
void Print(int h[], float f[], int n);
void CountJudgeScore(int ph[], float pf[], int m, float sf[], float f[],
                     int n);
int main()
{
    int j, m, n;
    int sh[ATHLETE];                        /* 选手的编号 */
    int ph[JUDGE];                          /* 评委的编号 */
    float sf[ATHLETE];                      /* 选手的最后得分 */
    float pf[JUDGE];                        /* 评委的得分 */
    float f[ATHLETE][JUDGE];                /* 评委给选手的评分 */
    printf("How many Athletes?");
    scanf("%d", &n);                        /* 输入选手人数 */
    printf("How many judges?");
    scanf("%d", &m);                        /* 输入评委人数 */
    for(j=1; j<=m; j++)
    {
        ph[j] = j;
```

```
25      }
26      printf("Scores of Athletes:\n");
27      CountAthleteScore(sh, sf, n, *f, m);    /* 现场为选手统计分数 */
28      CountJudgeScore(ph,pf,m,sf,*f,n);       /* 为各个评委打分* /
29      printf("Order of Athletes:\n");
30      Sort(sh, sf, n);                        /* 选手得分排序 */
31      Print(sh, sf, n);                       /* 打印选手名次表 */
32      printf("Order of judges:\n");
33      Sort(ph, pf, m);                        /* 评委得分排序 */
34      Print(ph, pf, m);                       /* 打印评委名次表 */
35      printf("Over! Thank you! \n");
36      return 0;
37  }
38  /* 函数功能:统计参赛选手的得分 */
39  void CountAthleteScore(int sh[], float sf[], int n, float f[], int m)
40  {
41      int i, j;
42      float max, min;
43      for (i=1; i<=n; i++)                    /* 第 i 个选手 */
44      {
45          printf("\nAthlete %d is playing.", i);
46          printf("\nPlease enter his number code:");
47          scanf("%d", &sh[i]);
48          sf[i] = 0;
49          max = 0;                            /* 最高分初值设为最小值 */
50          min = 100;                          /* 最低分初值设为最大值 */
51          for (j=1; j<=m; j++)                /* 第 j 个评委 */
52          {
53              printf("Judge %d gives score:", j);
54              scanf("%f", &f[i*m+j]);
55              sf[i] = sf[i] + f[i*m+j];    /* 累加评委对第 i 个选手的评分 */
56              if (max < f[i*m+j])             /* 找出最高分 */
57                  max = f[i*m+j];
58              if (min > f[i*m+j])             /* 找出最低分 */
59                  min = f[i*m+j];
60          }
```

```
61          printf("Delete a maximum score:%.1f\n", max);
62          printf("Delete a minimum score:%.1f\n", min);
63          sf[i] = (sf[i] - max - min)/(m - 2);   /* 去掉一个最高分和一个最低分 */
64          printf("The final score of Athlete %d is %.3f\n", sh[i], sf[i]);
65      }
66  }
67  /* 函数功能:对分数从高到低排序 */
68  void Sort(int h[], float f[], int n)
69  {
70      int i, j, k, temp2;
71      float temp1;
72      for (i=1; i<=n-1; i++)
73      {
74          k = i;
75          for (j=i+1; j<=n; j++)
76          {
77              if (f[j] > f[k]) k = j;
78          }
79          if (k ! = i)
80          {
81              temp1 = f[k]; f[k] = f[i]; f[i] = temp1;
82              temp2 = h[k]; h[k] = h[i]; h[i] = temp2;
83          }
84      }
85  }
86  /* 函数功能:打印名次表 */
87  void Print(int h[], float f[], int n)
88  {
89      int i;
90      printf("order\tfinal score\tnumber code\n");
91      for (i=1; i<=n; i++)
92      {
93          printf("%5d\t%11.3f\t%6d\n", i, f[i], h[i]);
94      }
95    }
96    /* 函数功能:统计评委的得分 */
97    void CountJudgeScore(int ph[], float pf[], int m, float sf[],
98                         float f[], int n)
```

```
99      {
100        int i, j;
101        for (j=1; j<=m; j++)                    /* 第 j 个评委 */
102        {
103            pf[j] = 0;
104            for (i=1; i<=n; i++)                /* 第 i 个选手 */
105            {
106              pf[j] = pf[j] + (f[i*m+j] - sf[i]) *  (f[i*m+j] - sf[i]);
107            }
108            pf[j] = 10 - sqrt(pf[j]/n);
109        }
110    }
```

程序的运行结果如下：

```
How many Athletes? 5↙
How many judges? 5↙
Scores of judges:

Athlete 1 is playing.
Please enter his number code:11↙
Judge 1 gives score:9.5↙
Judge 2 gives score:9.6↙
Judge 3 gives score:9.7↙
Judge 4 gives score:9.4↙
Judge 5 gives score:9.0↙
Delete a maximum score:9.7
Delete a minimum score:9.0
The final score of Athlete 11 is 9.500

Athlete 2 is playing.
Please enter his number code:12↙
Judge 1 gives score:9.0↙
Judge 2 gives score:9.2↙
Judge 3 gives score:9.1↙
Judge 4 gives score:9.3↙
Judge 5 gives score:8.9↙
Delete a maximum score:9.3
Delete a minimum score:8.9
The final score of Athlete 12 is 9.100
```

```
Athlete 3 is playing.
Please enter his number code:13↙
Judge 1 gives score:9.6↙
Judge 2 gives score:9.7↙
Judge 3 gives score:9.5↙
Judge 4 gives score:9.8↙
Judge 5 gives score:9.4↙
Delete a maximum score:9.8
Delete a minimum score:9.4
The final score of Athlete 13 is 9.600

Athlete 4 is playing.
Please enter his number code:14↙
Judge 1 gives score:8.9↙
Judge 2 gives score:8.8↙
Judge 3 gives score:8.7↙
Judge 4 gives score:9.0↙
Judge 5 gives score:8.6↙
Delete a maximum score:9.0
Delete a minimum score:8.6
The final score of Athlete 14 is 8.800

Athlete 5 is playing.
Please enter his number code:15↙
Judge 1 gives score:9.0↙
Judge 2 gives score:9.1↙
Judge 3 gives score:8.8↙
Judge 4 gives score:8.9↙
Judge 5 gives score:9.2↙
Delete a maximum score:9.2
Delete a minimum score:8.8
The final score of Athlete 11 is 9.000
Order of Athletes:
order  final score  number code
    1        9.600      13
    2        9.500      11
    3        9.100      12
```

```
        4          9.000      15
        5          8.800      14
    Order of judges:
    order   final score   number code
        1          9.937      1
        2          9.911      2
        3          9.859      3
        4          9.833      4
        5          9.714      5
    Over! Thank you!
```

1.8 习题 9 解答

9.1 请结合例 9.6 程序分析下面两个程序能否实现两数交换功能,并说明为什么。

(1)

```
1   void Swap(int *x, int *y)
2   {
3       int *pTemp;
4       *pTemp = *x;
5       *x = *y;
6       *y = *pTemp;
7   }
```

(2)

```
1   void Swap(int *x, int *y)
2   {
3       int *pTemp;
4       pTemp = x;
5       x = y;
6       y = pTemp;
7   }
```

参考答案:

(1) 不能实现两数交换功能,因为指针变量 pTemp 未初始化,在指针 pTemp 指向未知的情况下,在第 4 行就对 pTemp 指向的未知单元进行写操作,这是很危险的。

(2) 不能实现两数交换功能,虽然指针变量 pTemp 在第 4 行被赋了一个初值,但第 4~6 行语句交换的是地址值,不是指针指向存储单元中的内容。

9.2 利用例 9.6 程序中的函数 Swap( ),用函数编程实现两个数组中对应元素值的交换。

参考程序如下:

```
1   #include <stdio.h>
2   #define N 10
3   void ReadData(int a[], int n);
4   void PrintData(int a[], int n);
5   void Swap(int *x, int *y);
6   int main()
7   {
8       int a[N], b[N], i, n;
```

```
9       printf("Input array size(n<=10):");
10      scanf("%d", &n);
11      printf("Input array a:");
12      ReadData(a, n);
13      printf("Input array b:");
14      ReadData(b, n);
15      for (i=0; i<n; i++)
16      {
17          Swap(&a[i], &b[i]);
18      }
19      printf("Output array a:");
20      PrintData(a, n);
21      printf("Output array b:");
22      PrintData(b, n);
23      return 0;
24  }
25  /* 函数功能:输入数组 a 的 n 个元素值 */
26  void ReadData(int a[], int n)
27  {
28      int i;
29      for (i=0; i<n; i++)
30      {
31          scanf("%d", &a[i]);
32      }
33  }
34  /* 函数功能:输出数组 a 的 n 个元素值 */
35  void PrintData(int a[], int n)
36  {
37      int i;
38      for (i=0; i<n; i++)
39      {
40          printf("%5d", a[i]);
41      }
42      printf("\n");
43  }
44  /* 函数功能:两整数值互换 */
45  void Swap(int *x, int *y)
46  {
```

```
47        int temp;
48        temp = *x;
49        *x = *y;
50        *y = temp;
51    }
```

程序的运行结果如下：

```
Input array size(n<=10):5↙
Input array a:1 2 3 4 5↙
Input array b:6 7 8 9 10↙
Output array a:    6    7    8    9    10
Output array b:    1    2    3    4    5
```

9.3 利用例 9.6 程序中的函数 Swap( )，从键盘输入 10 个整数，用函数编程实现计算其最大值和最小值，并互换它们所在数组中的位置。

参考程序如下：

```
1     #include <stdio.h>
2     void ReadData(int a[], int n);
3     void PrintData(int a[], int n);
4     void MaxMinExchang(int a[], int n);
5     void Swap(int *x, int *y);
6     int main()
7     {
8         int a[10], n;
9         printf("Input n(n<=10):");
10        scanf("%d", &n);
11        printf("Input %d numbers:", n);
12        ReadData(a, n);
13        MaxMinExchang(a, n);
14        printf("Exchange results:");
15        PrintData(a, n);
16        return 0;
17    }
18    /* 函数功能:输入数组 a 的 n 个元素值 */
19    void ReadData(int a[], int n)
20    {
21        int i;
22        for (i=0; i<n; i++)
23        {
```

```
24          scanf("%d", &a[i]);
25      }
26  }
27  /* 函数功能:输出数组 a 的 n 个元素值 */
28  void PrintData(int a[], int n)
29  {
30      int i;
31      for (i=0; i<n; i++)
32      {
33          printf("%5d", a[i]);
34      }
35      printf("\n");
36  }
37  /* 函数功能:将数组 a 中最大数与最小数的位置互换 */
38  void MaxMinExchang(int a[], int n)
39  {
40      int maxValue = a[0], minValue = a[0], maxPos = 0, minPos = 0;
41      int i;
42      for (i=1; i<n; i++)
43      {
44          if (a[i] > maxValue)
45          {
46              maxValue = a[i];
47              maxPos = i;
48          }
49          if (a[i] < minValue)
50          {
51              minValue = a[i];
52              minPos = i;
53          }
54      }
55      Swap(&a[maxPos],&a[minPos]);
56  }
57  /* 函数功能:两整数值互换 */
58  void Swap(int *x, int *y)
59  {
60      int temp;
61      temp = *x;
```

```
62          *x = *y;
63          *y = temp;
64      }
```

程序的运行结果如下：

```
Input n(n<=10):10↙
Input 10 Numbers:1 4 3 0  2 6 7 2 9 -1↙
Exchange results:    1    4    3    0    9    6    7    2    -2    -1
```

9.4 按如下函数原型，用函数编程解决如下的日期转换问题（要求考虑闰年）。

（1）输入某年某月某日，计算并输出它是这一年的第几天。

```
/*  函数功能：对给定的某年某月某日，计算它是这一年的第几天
    函数参数：整型变量 year、month、day，分别代表年、月、日
    函数返回值：这一年的第几天 */
int DayofYear(int year, int month, int day);
```

（2）输入某一年的第几天，计算并输出它是这一年的第几月第几日。

```
/*  函数功能：    对给定的某一年的第几天，计算它是这一年的第几月第几日
    函数入口参数：整型变量 year，存储年
                  整型变量 yearDay，存储这一年的第几天
    函数出口参数：整型指针 pMonth，指向存储这一年第几月的整型变量
                  整型指针 pDay，指向存储第几日的整型变量
    函数返回值：  无 */
void MonthDay(int year, int yearDay, int *pMonth, int *pDay);
```

（3）输出如下菜单，用 switch 语句实现根据用户输入的选择执行相应的操作。

```
1. year/month/day -> yearDay
2. yearDay -> year/month/day
3. Exit
Please enter your choice:
```

参考答案：首先需要建立一张存放 12 个月每月天数的表格，考虑到二月份的天数在平年和闰年是不同的，所以需要定义一个 2 行 13 列的二维数组 dayTab（分别用第 1～12 列的元素存放第 1～12 个月的天数），将其定义为全局数组类型，用于存放每个月的天数，第 0 行对应于平年各月份的天数（其中，二月份天数为 28 天），第 1 行对应于闰年各月份的天数（其中，二月份天数为 29 天）。若是闰年，则采用数组中第 1 行存放的每月天数计算；否则，采用数组中第 0 行存放的每月天数计算。

（1）函数 DayofYear()将某年某月某日转换为这一年的第几天，并将函数值返回。其实现算法为：若给定的月是 month，则将 1，2，3，…，month－1 月的各月天数依次累加，再加上指定的日，即可得到它是这一年的第几天。参考程序如下：

```
1    #include<stdio.h>
2    int DayofYear(int year, int month, int day);
3    int dayTab[2][13] = {{0,31,28,31,30,31,30,31,31,30,31,30,31},
```

```
4                           {0,31,29,31,30,31,30,31,31,30,31,30,31}
5     int main()
6     {
7         int year, month, day, yearDay;
8         printf("Please enter year, month, day:");
9         scanf("%d,%d,%d", &year, &month, &day);
10        yearDay = DayofYear(year, month, day);
11        printf("yearDay = %d\n", yearDay);
12        return 0;
13    }
14    /* 函数功能:对给定的某年某月某日,计算并返回它是这一年的第几天 */
15    int DayofYear(int year, int month, int day)
16    {
17        int i, leap;
18        /* 若 year 为闰年,即 leap 值为 1,则用第 1 行元素 dayTab[1][i]计算;
19           否则 leap 值为 0,用第 0 行 dayTab[0][i]计算 */
20        leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
21        for (i=1; i<month; i++)
22        {
23            day = day + dayTab[leap][i];
24        }
25        return day;              /* 返回计算出的 day 的值 */
26    }
```

程序的两次测试结果如下:

① Please enter year, month, day:2000,3,1↙
yearDay = 61

② Please enter year, month,day:2011,3,1↙
yearDay = 60

(2) 函数 MonthDay()将某年的第几天转换为某月某日。由于该函数需要计算两个值,无法同时用 return 语句返回,所以将相应的形参定义为指针类型。其实现算法为:对给定的某年的第几天 yearDay,只要从 yearDay 中依次减去 1,2,3,…各月的天数,直到正好减为 0 或不够减时为止,若已减了 i 个月的天数,则月份 month 的值为 i+1。这时,yearDay 中剩下的天数即为第 month 月的 day 的值。参考程序如下:

```
1     #include<stdio.h>
2     void MonthDay(int year, int yearDay, int *pMonth, int *pDay);
3     int dayTab[2][13] = {{0,31,28,31,30,31,30,31,31,30,31,30,31},
4                          {0,31,29,31,30,31,30,31,31,30,31,30,31}}
5     int main()
```

```
{
    int year, month, day, yearDay;
    printf("Please enter year, yearDay:");
    scanf("%d,%d", &year, &yearDay);
    MonthDay(year, yearDay, &month, &day);
    printf("month = %d, day = %d\n", month, day);
    return 0;
}
/* 函数功能:对给定的某一年的第几天,计算它是这一年的第几月第几日 */
void MonthDay(int year, int yearDay, int * pMonth, int * pDay)
{
    int i, leap;
    leap = ((year %4 == 0) && (year %100 ! = 0)) || (year %400 == 0);
    for (i=1; yearDay>dayTab[leap][i]; i++)
    {
        yearDay = yearDay - dayTab[leap][i];
    }
    *pMonth = i;          /* 将计算出的月份值赋值给 pMonth 所指向的变量 */
    *pDay = yearDay;      /* 将计算出的日号赋值给 pDay 所指向的变量 */
}
```

程序的两次测试结果如下:

① Please enter year, yearDay:2000,61↙
month = 3, day = 1

② Please enter year, yearDay:2011,60↙
month = 3, day = 1

(3) 参考程序如下:

```
#include <stdio.h>
#include <stdlib.h>
int DayofYear(int year, int month, int day);
void MonthDay(int year, int yearDay, int * pMonth, int * pDay);
void Menu(void);
int dayTab[2][13] = {{0,31,28,31,30,31,30,31,31,30,31,30,31},
                     {0,31,29,31,30,31,30,31,31,30,31,30,31}}
int main()
{
    int year, month, day, yearDay;
    char c;
    Menu();                  /* 调用 Menu 函数显示一个固定式菜单 */
```

```
13      c = getchar();          /* 输入选择 */
14      switch (c)              /* 判断选择的是何种操作 */
15      {
16          case '1':printf("Please enter year, month, day:");
17                   scanf("%d,%d,%d", &year, &month, &day);
18                   yearDay = DayofYear(year, month, day);
19                   printf("yearDay = %d\n", yearDay);
20                   break;
21          case '2':printf("Please enter year, yearDay:");
22                   scanf("%d,%d", &year, &yearDay);
23                   MonthDay(year, yearDay, &month, &day);
24                   printf("month = %d,day = %d\n", month, day);
25                   break;
26          case '3':exit(0);         /* 退出程序的运行 */
27          default:printf("Input error!");
28      }
29      return 0;
30  }
31  /* 函数功能:对给定的某年某月某日,计算并返回它是这一年的第几天 */
32  int DayofYear(int year, int month, int day)
33  {
34      int i, leap;
35      /* 若 year 为闰年,即 leap 值为 1,则用第 1 行元素 dayTab[1][i]计算;
36         否则 leap 值为 0,用第 0 行 dayTab[0][i]计算 */
37      leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
38      for (i=1; i<month; i++)
39      {
40          day = day + dayTab[leap][i];
41      }
42      return day;         /* 返回计算出的 day 的值 */
43  }
44  /* 函数功能:对给定的某一年的第几天,计算它是这一年的第几月第几日 */
45  void MonthDay(int year, int yearDay, int *pMonth, int *pDay)
46  {
47      int i, leap;
48      leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
49      for (i=1; yearDay>dayTab[leap][i]; i++)
50      {
```

```
51              yearDay = yearDay - dayTab[leap][i];
52          }
53          *pMonth = i;        /* 将计算出的月份值赋值给 pMonth 所指向的变量 */
54          *pDay = yearDay;    /* 将计算出的日号赋值给 pDay 所指向的变量 */
55      }
56      /* 函数功能:显示菜单 */
57      void Menu(void)
58      {
59          printf("1. year/month/day -> yearDay\n");
60          printf("2. yearDay -> year/month/day\n");
61          printf("3. Exit\n");
62          printf("Please enter your choice:");
63      }
```

程序的 4 次测试结果如下:

① 1. year/month/day -> yearDay
2. yearDay -> year/month/day
3. Exit
Please enter your choice: 1↙
Please enter year, month,day:2000,3,1↙
yearDay = 61

② 1. year/month/day -> yearDay
2. yearDay -> year/month/day
3. Exit
Please enter your choice: 1↙
Please enter year, month,day:2011,3,1↙
yearDay = 60

③ 1. year/month/day -> yearDay
2. yearDay -> year/month/day
3. Exit
Please enter your choice: 2↙
Please enter year, yearDay:2000,61↙
month = 3, day = 1

④ 1. year/month/day -> yearDay
2. yearDay -> year/month/day
3. Exit
Please enter your choice: 2↙
Please enter year, yearDay:2011,60↙
month = 3, day = 1

9.5 （选做）按如下函数原型，采用如图1-9所示的梯形法编程实现，在积分区间[a,b]内计算函数 $y_1=\int_0^1(1+x^2)\,dx$ 和 $y_2=\int_0^3\frac{x}{1+x^2}dx$ 的定积分。其中，指向函数的指针变量f用于接收被积函数的入口地址。

```
Integral(float (*f)(float), float a, float b);
```

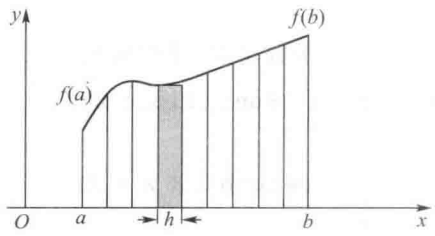


图1-9 梯形法求函数定积分

参考答案：在区间[a,b]内计算连续函数的定积分，就是要计算函数$f(x)$、直线$x=a$、$x=b$与x轴所围成的曲边梯形的面积。用梯形法近似计算定积分的基本思想为：将区间[a,b]划分成n等份（由编程者自己设定，本例设定n为100），等分区间的长度为$h=(b-a)/n$，将每个小曲边梯形的面积用n个直边小梯形的面积近似，求出n个直边小梯形的面积累加和，当n的取值足够大时，直边梯形的面积之和就近似等于定积分的值。具体计算公式为：

$$\begin{aligned}\int_a^b f(x)\,dx &= \frac{h}{2}[f(a)+f(a+h)]+\frac{h}{2}[(f(a+h)+f(a+2h))]+\cdots+\\ &\quad \frac{h}{2}[f(a+(n-i)h)+f(b)]\\ &= \frac{h}{2}[f(a)+2f(a+h)+2f(a+2h)+\cdots+2f(a+(n-1)h)+f(b)]\\ &= h\left[\frac{1}{2}(f(a)+f(b))+\sum_{i=1}^{n-1}f(a+ih)\right]\end{aligned}$$

为了设计一个通用的计算连续函数定积分的函数，需要使用函数指针类型的形参指向需要计算定积分的函数f(x)，即编写一个函数Integral(float (*f)(float), float a, float b)，其中形参a和b分别代表积分下限和积分上限，第一个被声明为函数指针类型的形参f用于接收被积函数的入口地址。因此，无须在函数Integral()内部显式地给出被积函数的解析表达式，当被积函数变化时，也无须修改函数代码，只要将被积函数的名字（即入口地址）作为函数实参来调用函数Integral()即可计算出不同函数的定积分。而如果不使用函数指针的话，就不得不分别针对不同的被积函数编写不同的计算定积分函数，不但编码效率低，而且程序的结构也不够简洁。参考程序如下：

```
1    #include <stdio.h>
2    float Fun1(float x);
3    float Fun2(float x);
4    float Integral(float (*f)(float), float a, float b);
5    int main()
6    {
7        float y1, y2;
8        y1 = Integral(Fun1, 0.0, 1.0);
9        y2 = Integral(Fun2, 0.0, 3.0);
10       printf("y1=%f\ny2=%f\n", y1, y2);
11       return 0;
```

```
12    }
13    /* 函数功能:计算函数 1+ x * x 的函数值 */
14    float Fun1(float x)
15    {
16        return 1 + x * x;
17    }
18    /* 函数功能:计算函数 x/(1+ x * x)的函数值 */
19    float Fun2(float x)
20    {
21        return x / (1 + x * x);
22    }
23    /* 函数功能:用梯形法计算函数的定积分 */
24    float Integral(float (*f)(float), float a, float b)
25    {
26        float s, h;
27        int n = 100, i;
28        s = ((*f)(a) + (*f)(b)) / 2;
29        h = (b - a) / n;
30        for (i=1; i<n; i++)
31        {
32            s += (*f)(a + i * h);
33        }
34        return s * h;
35    }
```

程序的运行结果如下:

```
y1 = 1.333350
y2 = 1.151212
```

1.9 习题 10 解答

10.1 函数 MyStrcpy( )可用下面更为简洁的形式编写,请读者分析这个程序是如何执行的,然后写出第 3~5 行的 while 语句的等价形式,使其变为循环体不为空的语句。

```
1    void MyStrcpy(char *dstStr, const char *srcStr)
2    {
3        while ((*dstStr++ = *srcStr++) != '\0')
4        {
5        }
```

```
6    }
```

参考答案:第 3~5 行的 while 语句的等价形式为:

```
while (*srcStr != '\0')
{
    *dstStr =*srcStr;
    dstStr++;
    srcStr++;
}
```

从上述等价形式,读者不难分析出第 3~5 行的 while 语句是如何执行的。

10.2 C 语言的高效和高能主要来自于指针,大多数语言都有无数的“不可能”,而 C 语言则是“一切皆有可能”。请按下列格式输入程序(注意不要在程序中随意加空格和换行),并上机运行程序,然后分析为什么下面程序的运行结果与源代码一模一样。

```
main(){char*a="main(){char*a=%c%s%c;printf(a,34,a,34);}";
printf(a,34,a,34);}
```

参考答案:略

10.3 阅读程序,按要求在空白处填写适当的表达式或语句,使程序完整并符合题目要求。

(1) 下面函数的功能是计算指针 p 所指向的字符串的长度(即实际字符个数)。

```
1    unsigned int MyStrlen(char *p)
2    {
3        unsigned int len;
4        len = 0;
5        for (; *p != ____①____; p++)
6        {
7            len ____②____;
8        }
9        return ____③____;
10   }
```

(2) 下面的函数计算字符数组 s 中字符串长度的方法与(1)有所不同。

```
1    unsigned int MyStrlen(char s[])
2    {
3        char *p = s;
4        while (*p != ____①____)
5        {
6            p++;                    /* 移动指针 p 使其指向下一个字符 */
7        }
8        return ____②____;           /* 返回指针 p 与字符串首地址之间的差值 */
9    }
```

(3) 下面函数的功能是比较两个字符串的大小,将字符串中第 1 个出现的不相同字符的 ASCII 码值之差作为比较结果返回。若第 1 个字符串大于第 2 个字符串,则返回正值;若第 1 个字符串小于第 2 个字符串,则返回负值;若两个字符串完全相同,则返回 0 值。

```
int MyStrcmp(char *p1, char *p2)
{
    for (; *p1 == *p2; p1++,p2++)
    {
        if (*p1 == '\0'  ) return    ①    ;
    }
    return    ②    ;
}
```

(4) 下面的函数同样实现函数 strcmp( )的功能,比较两个字符串 s 和 t,然后将两个字符串中第 1 个不相同字符的 ASCII 码值之差作为函数值返回。

```
int MyStrcmp(char s[], char t[])
{
    int i;
    for (i=0; s[i] == t[i]; i++ )
    {
        if (s[i] ==    ①    ) return 0 ;
    }
    return (    ②    );
}
```

(5) 下面的程序比较用户输入的密码 userInput 与内设的密码 password 是否相同。若相同,则输出“Correct password! Welcome to the system...”;若 password<userInput,则输出“Invalid password! user input<password”;否则输出“Invalid password! user input>password”。

```
#include <stdio.h>
#include <string.h>
int main()
{
    char password[7] = "secret";
    char userInput[81];
    printf("Input Password:");
    scanf("%s", userInput);
    if (        ①        )
        printf("Correct password! Welcome to the system...\n");
    else if (        ②        )
        printf("Invalid password! user input<password\n");
    else
```

```
14          printf("Invalid password! user input>password\n");
15          return 0;
16      }
```

参考答案：

(1) ① '\0'　② ++　③ len

(2) ① '\0'　② p-s

(3) ① 0　② *p1- *p2

(4) ① '\0'　② s[i]-t[i]

(5) ① strcmp(userInput, password) == 0 ② strcmp(userInput, password) < 0

10.4　输入一行字符，用函数编程统计其中有多少个单词。假设单词之间以空格分开。

参考答案：当前被检验字符不是空格，而前一个被检验字符是空格，则表示有新单词出现。根据这一判断是否有新单词出现的方法，编写程序如下：

```
1   #include <stdio.h>
2   int CountWords(char str[]);
3   int main()
4   {
5       char str[20];
6       printf("Input a string:");
7       gets(str);
8       printf("Numbers of words = %d\n", CountWords(str));
9       return 0;
10  }
11  int CountWords(char str[])
12  {
13      int i, num;
14      num = (str[0] != ' ') ? 1 : 0;
15      for (i=1; str[i]!='\0'; i++)
16      {
17          if (str[i]!= '' && str[i-1] == ' ')
18          {
19              num++;
20          }
21      }
22      return num;
23  }
```

程序的运行结果如下：

```
Input a string: How are you↙
Numbers of words = 3
```

10.5 参考例 10.5,分别用字符数组和字符指针作函数参数两种方法编程实现如下功能:在字符串中删除与某字符相同的字符。

参考程序 1:用字符数组作函数参数。

```
#include <stdio.h>
void Squeeze(char s[], char c);
int main()
{
    char str[20], ch;
    printf("Input a string:");
    gets(str);
    printf("Input a character:");
    ch = getchar();
    Squeeze(str, ch);
    printf("Results:%s\n", str);
    return 0;
}
void Squeeze(char s[], char c)
{
    int i, j;
    for (i=j=0; s[i]!='\0'; i++)
    {
        if (s[i] != c)
        {
            s[j] = s[i];
            j++;
        }
    }
    s[j] = '\0'; /* 在字符串 s 的末尾添加字符串结束标志 */
}
```

参考程序 2:用字符指针作函数参数。

```
#include <stdio.h>
#include <string.h>
#define N 100
void Squeeze(char *s, char c);
int main()
{
    char str[20], ch;
    printf("Input a string:");
```

```
9        gets(str);
10       printf("Input a character:");
11       ch = getchar();
12       Squeeze(str, ch);
13       printf("Results:%s\n", str);
14       return 0;
15   }
16   void Squeeze(char *s, char c)
17   {
18       char str[N];
19       char *t = str;
20       strcpy(t, s);
21       for (; *t != '\0'; t++)
22       {
23           if (*t != c)
24           {
25               *s = *t;
26               s++;
27           }
28       }
29       *s = '\0'; /* 在字符串 s 的末尾添加字符串结束标志 */
30   }
```

参考程序 3:用字符指针作函数参数。

```
1    #include <stdio.h>
2    void Squeeze(char *s, char c);
3    int main()
4    {
5        char str[20], ch;
6        printf("Input a string:");
7        gets(str);
8        printf("Input a character:");
9        ch = getchar();
10       Squeeze(str, ch);
11       printf("Results:%s\n", str);
12       return 0;
13   }
14   void Squeeze(char *s, char c)
```

```
15    {
16        char *t1 = s, *t2 = s;
17        for (; *t1 != '\0'; t1++)
18        {
19            if (*t1 != c)
20            {
21                *t2 = *t1;
22                t2++;
23            }
24        }
25        *t2 = '\0'; /* 在字符串 t2 的末尾添加字符串结束标志 */
26    }
```

程序的运行结果如下：

```
Input a string:Hello! How are you! ↙
Input a character:! ↙
Results:Hello How are you
```

10.6 参考例 10.5，分别用字符数组和字符指针作函数参数两种方法编程实现在字符串每个字符间插入一个空格的功能。

参考程序 1：用字符数组作函数参数。

```
1     #include <stdio.h>
2     #include <string.h>
3     #define N 100
4     void Insert(char s[]);
5     int main()
6     {
7         char str[N];
8         printf("Input a string:");
9         gets(str);
10        Insert(str);
11        printf("Insert results:%s\n", str);
12        return 0;
13    }
14    void Insert(char s[])
15    {
16        char t[N];
17        int i, j;
18        strcpy(t, s);
```

```
19        for (i=0, j=0; t[i] != '\0'; i++, j++)
20        {
21            s[j] = t[i];
22            j++;
23            s[j] = ' ';
24        }
25        s[j] = '\0';              /* 在字符串 s 的末尾添加字符串结束标志 */
26    }
```

参考程序 2:用字符指针作函数参数。

```
1     #include <stdio.h>
2     #include <string.h>
3     #include <stdlib.h>
4     #define N 100
5     void Insert(char *s);
6     int main()
7     {
8         char str[N];
9         printf("Input a string:");
10        gets(str);
11        Insert(str);
12        printf("Insert results:%s\n", str);
13        return 0;
14    }
15    void Insert(char *s)
16    {
17        char str[N];
18        char *t = str;
19        strcpy(t, s);
20        for (; *t != '\0'; s++, t++)
21        {
22            *s = *t;
23            s++;
24            *s = ' ';
25        }
26        *s = '\0';          /* 在字符串 s 的末尾添加字符串结束标志 */
27    }
```

程序的运行结果如下:

```
Input a string:Howareyou↙
Insert results:H o w a r e y o u
```

10.7 参考例 10.5,分别用字符数组和字符指针作函数参数两种方法编程实现字符串逆序存放功能。

参考程序 1:用字符数组作函数参数,利用两个数组实现字符串的逆序存放。

```
#include <stdio.h>
#include <string.h>
#define N 80
void Inverse(char str[], char ptr[]);
int main()
{
    char a[N], b[N];
    printf("Input a string:");
    gets(a);
    Inverse(a, b);
    printf("Inversed results:%s \n", b);
    return 0;
}
/* 函数功能: 实现将字符数组中的字符串逆序存放 */
void Inverse(char str[], char ptr[])
{
    int len, i, j;
    len = strlen(str);
    for (i=0, j=len-1; str[i] != '\0'; i++, j--)
    {
        ptr[j] = str[i];
    }
    ptr[i]='\0'; /* 在字符串 ptr 的末尾添加字符串结束标志 */
}
```

参考程序 2:用字符数组作函数参数,利用一个数组实现字符串的逆序存放。借助于中间变量 temp,将数组中首尾对称位置的元素互换。i 指向数组首部的元素,从 0 依次加 1 变化;j 指向数组尾部的元素,从 $n-1$ 依次减 1 变化;当 $i>j$ 时,停止互换操作。

```
#include <stdio.h>
#include <string.h>
#define N 80
void Inverse(char str[]);
int main()
{
```

```
7       char a[N];
8       printf("Input a string:");
9       gets(a);
10      Inverse(a);
11      printf("Inversed results:%s\n", a);
12      return 0;
13  }
14  /* 函数功能：实现将字符数组中的字符串逆序存放 */
15  void Inverse(char str[])
16  {
17      int len, i, j;
18      char temp;
19      len = strlen(str);
20      for (i=0, j=len-1; i<j; i++, j--)
21      {
22          temp = str[i];
23          str[i] = str[j];
24          str[j] = temp;
25      }
26  }
```

参考程序 3：用字符指针作函数参数。定义两个指针分别指向字符串的两端，同时向前和向后分别移动指针，边移动指针边交换指针指向的字符。

```
1   #include <stdio.h>
2   #include <string.h>
3   #define N 80
4   void Inverse(char *pStr);
5   int main()
6   {
7       char a[N];
8       printf("Input a string:");
9       gets(a);
10      Inverse(a);
11      printf("Inversed results:%s\n", a);
12      return 0;
13  }
14  /* 函数功能：实现将字符数组中的字符串逆序存放 */
15  void Inverse(char *pStr)
16  {
```

```
    int len;
    char temp;
    char *pStart;          /* 指针变量 pStart 指向字符串的第一个字符 */
    char *pEnd;            /* 指针变量 pEnd 指向字符串的最后一个字符 */
    len = strlen(pStr);    /* 求出字符串长度 */
    for (pStart=pStr,pEnd=pStr+len-1; pStart<pEnd; pStart++,pEnd--)
    {
        temp = *pStart;
        *pStart = *pEnd;
        *pEnd = temp;
    }
}
```

参考程序 4：

```
#include<stdio.h>
void Inverse(char str[]);
int main()
{
    char str[80];
    printf("Input a string:\n");
    gets(str);
    Inverse(str);
    printf("Inversed results:\n");
    puts(str);
    return 0;
}
void Inverse(char str[])
{
    int i,n=0;
    char c;
    for(i=0;str[i]!='\0';i++)
    {
        n++;
    }
    for(i=0;i<n/2;i++)
    {
        c=str[n-i-1];
        str[n-i-1]=str[i];
```

```
25          str[i]=c;
26      }
27      return;
28  }
```

程序的运行结果如下：

```
Input a string:ABCDEFGHI↙
Inversed results:IHGFEDCBA
```

10.8 参考例 10.7，不用返回指针值的函数编程实现字符串连接函数 strcat( )的功能。

参考答案：用 i 和 j 分别作为字符数组 srcStr 和字符数组 dstStr 的下标，先将 i 和 j 同时初始化为 0，然后改变 i 的值使其位于字符 dstStr 的尾部，即字符串结束标志处，最后将字符数组 srcStr 中的字符依次复制到字符数组 dstStr 中。

参考程序 1：用字符数组作函数参数。

```
#include <stdio.h>
#include <string.h>
#define N 80
void MyStrcat(char dstStr[], char strStr[]);
int main()
{
    char s[N], t[N];
    printf("Input a string:");
    gets(s);
    printf("Input another string:");
    gets(t);
    MyStrcat(s,t);
    printf("Concatenate results:%s\n", s);
    return 0;
}
/* 函数功能:将字符串 srcStr 连接到字符串 dstStr 之后 */
void MyStrcat(char dstStr[], char srcStr[])
{
    unsigned int i, j;
    i = strlen(dstStr);              /* 将下标移动到字符串 dstStr 的末尾 */
    for (j=0; j<=strlen(srcStr); j++, i++)
    {
        dstStr[i] = srcStr[j];
    }
}
```

参考程序 2：用字符数组作函数参数。用 i 和 j 分别作为字符数组 srcStr 和字符数组 dstStr 的

下标,先将 i 和 j 同时初始化为 0,然后移动 i 使其位于字符 dstStr 的尾部,即字符串结束标志处,再将字符数组 srcStr 中的字符依次复制到字符数组 dstStr 中。

```
#include <stdio.h>
#include <string.h>
#define N 80
void MyStrcat(char dstStr[], char srcStr[]);
int main()
{
    char s[N], t[N];
    printf("Input a string:");
    gets(s);
    printf("Input another string:");
    gets(t);
    MyStrcat(s,t);
    printf("Concatenate results:%s\n", s);
    return 0;
}
/* 函数功能:将字符串 srcStr 连接到字符串 dstStr 之后 */
void MyStrcat(char dstStr[], char srcStr[])
{
    int i=0, j=0;
    while (dstStr[i] != '\0') /* 将下标移动到字符串 dstStr 的末尾 */
    {
        i++;
    }
    while (srcStr[j] != '\0')
    {
        dstStr[i] = srcStr[j];
        i++;
        j++;
    }
    dstStr[i] = '\0'; /* 在字符串 dstStr 的末尾添加字符串结束标志 */
}
```

参考程序 3:用字符指针作函数参数。

```
#include <stdio.h>
#include <string.h>
#define N 80
void MyStrcat(char *dstStr, char *srcStr);
```

```
5    int main()
6    {
7        char s[N], t[N];
8        printf("Input a string:");
9        gets(s);
10       printf("Input another string:");
11       gets(t);
12       MyStrcat(s,t);
13       printf("Concatenate results:%s\n", s);
14       return 0;
15   }
16   /* 函数功能:将字符串 srcStr 连接到字符串 dstStr 之后 */
17   void MyStrcat(char *dstStr, char *srcStr)
18   {
19       while (*dstStr != '\0')
20       {
21           dstStr++;
22       }
23       while (*srcStr != '\0')  /* 若 srcStr 所指字符不是字符串结束标志 */
24       {
25           *dstStr = *srcStr;/* 将 srcStr 所指字符复制到 dstStr 所指内存中 */
26           srcStr++;            /* 使 srcStr 指向下一个字符 */
27           dstStr++;            /* 使 dstStr 指向下一个存储单元 */
28       }
29       *dstStr = '\0';   /* 在字符串 dstStr 的末尾添加字符串结束标志 */
30   }
```

程序的运行结果如下:

```
Input a string: Hello↙
Input another string: China! ↙
Concatenate results:Hello China!
```

10.9 参考例 10.4,输入 5 个国名,编程找出并输出按字典顺序排在最前面的国名。

参考答案:所谓字典顺序就是将字符串按由小到大的顺序排列,因此找出按字典顺序排在最前面的国名指的就是找出最小的字符串。

参考程序 1 如下:

```
1    #include <stdio.h>
2    #include <string.h>
3    #define N 80                              /* 字符串最大长度 */
```

```
4    int main()
5    {
6        int n;
7        char str[N], min[N];
8        printf("Input five countries' names:\n");
9        gets(str);                          /* 输入一个字符串 */
10       strcpy(min, str);                   /* 将其作为最小字符串暂存 */
11       for (n=1; n<5; n++)
12       {
13           gets(str);                      /* 每次输入一个字符串 */
14           if (strcmp(str, min) < 0)       /* 比较两个字符串的大小 */
15           {
16               strcpy(min, str);
17           }
18       }
19       printf("The minimum is:%s\n", min);/* 输出最小字符串 min */
20       return 0;
21   }
```

参考程序 2 如下：

```
1    #include <stdio.h>
2    #include <string.h>
3    #define N 80
4    int MinString(char str[][N], int n);
5    int main()
6    {
7        int n=5, min, i;
8        char str[5][N];
9        printf("Input five countries'names:\n");
10       for (i=0; i<n; i++)
11       {
12           gets(str[i]);
13       }
14       min = MinString(str, n);
15       printf("The minimum is:%s\n", str[min]);
16       return 0;
17   }
18   /* 函数功能:找出并返回按字典顺序排在最前面的字符串 */
19   int MinString(char str[][N], int n)
```

```
20    {
21        int i, minIndex;
22        char min[N];
23        strcpy(min, str[0]);
24        minIndex = 0;
25        for (i=1; i<n; i++)
26        {
27            if (strcmp(str[i], min) < 0)
28            {
29                strcpy(min, str[i]);
30                minIndex = i;
31            }
32        }
33        return minIndex;        /* 返回最小的字符串在二维字符数组中的位置 */
34    }
```

程序的运行结果如下：

```
Input five countries'names:
America↙
China↙
Japan↙
England↙
Sweden↙
The minimum is:America
```

10.10 （选做）任意输入英文的星期几，通过查找如图 1-10 所示的星期表，输出其对应的数字，若查到表尾仍未找到，则输出错误提示信息。

0	Sunday
1	Monday
2	Tuesday
3	Wednesday
4	Thursday
5	Friday
6	Saturday

图 1-10 星期表的内容

参考答案：用一个二维字符数组 weekDay 来存放如图 1-10 所示的星期表的内容（字符串）。输入待查找的字符串，然后在星期表中顺序查找与输入字符串相匹配的字符串，找到的字符串在星期表数组中的第一维下标（行号）即为题目所求。

需要注意的是，不能直接使用关系运算符比较输入的字符串 x 和星期表中的字符串 weekDay[i]是否相等，必须使用字符串处理函数，即用 if(strcmp(x,weekDay[i]) == 0)；其中，调用函数 strcmp()时的实参数组名 x 代表其中输入字符串的首地址，而 weekDay[i]代表二维字符数组 weekDay 的第 i 行第 0 列的首地址，即第 i+1 个星期字符串的首地址。

参考程序如下：

```
1    #include <stdio.h>
2    #include <string.h>
3    #define WEEKDAYS 7                    /* 每星期天数 */
```

```
#define MAX_STR_LEN 10                    /* 字符串最大长度 */
int main()
{
    int i, pos;
    int findFlag = 0;                     /*  置找到标志为假 */
    char x[MAX_STR_LEN];
    char weekDay[][MAX_STR_LEN] = {"Sunday", "Monday", "Tuesday",
                                   "Wednesday", "Thursday", "Friday",
                                   "Saturday"};
    printf("Please enter a string:");
    scanf("%s", x);                       /*  输入待查找的字符串 */
    for (i=0; i<WEEKDAYS && !findFlag; i++)
    {
        if (strcmp(x, weekDay[i]) == 0)
        {
            pos = i;                      /* 记录找到的位置 */
            findFlag = 1;                 /* 若找到,则置找到标志为真,退出循环 */
        }
    }
    if (findFlag)                         /* 找到标志为真,说明找到 */
        printf("%s is %d\n", x, pos);
    else                                  /* 找到标志为假,说明未找到 */
        printf("Not found! \n");
    return 0;
}
```

程序的两次测试结果如下：

① Please enter a string: Thursday↙
Thursday is 4

② Please enter a string: Thirsday↙
Not found!

1.10 习题11解答

11.1 请先分析说明表达式(*p)++和*p++的不同含义，然后写出下面程序的运行结果。

```
#include <stdio.h>
int main()
{
    int a[]={1,2,3,4,5};
    int *p=a;
```

```
6        printf("%d,",*p);
7            printf("%d,",*(++p));
8            printf("%d,",(*p)++);
9            printf("%d,",*p);
10           printf("%d,",*p--);
11           printf("%d,",--(*p));
12           printf("%d\n",*p);
13       return 0;
14   }
```

参考答案:表达式(*p)++和*p++具有不同的含义,(*p)++指的是先取出 p 指向的存储单元中的内容,然后将取出的数值加 1,而 p 仍然指向原来的存储单元。*p++则指的是先取出 p 指向的存储单元中的内容,然后将 p 值加 1,此时 p 不再指向原来的存储单元。

程序的运行结果为:1,2,2,3,3,0,0

11.2 通过上机运行程序并观察运行结果,分析下面程序错误的原因并改正之。

(1) 下面程序希望得到的运行结果如下:

```
Total string numbers = 3
How are you
```

```
1    #include <stdio.h>
2    void Print(char *arr[], int len);
3    int main()
4    {
5        char *pArray[] = {"How","are","you"};
6        int num = sizeof(pArray) / sizeof(char);
7        printf("Total string numbers = %d\n", num);
8        Print(pArray, num);
9        return 0;
10   }
11   void Print(char *arr[], int len)
12   {
13       int i;
14       for (i=0; i<len; i++)
15       {
16           printf("%s ", arr[i]);
17       }
18       printf("\n");
19   }
```

参考答案:程序第 6 行语句应修改为:

```
int num = sizeof(pArray) / sizeof(char*);
```

这是因为指针数组的基类型是 char * ,而非 char。

(2) 下面程序从键盘输入 5 个整数,然后将其输出到屏幕上。

```
1    #include <stdio.h>
2    void InputArray(int *pa, int n);
3    void OutputArray(int *pa, int n);
4    int main()
5    {
6        int a[5];
7        printf("Input five numbers:");
8        InputArray(a, 5);
9        OutputArray(a, 5);
10       return 0;
11   }
12   void InputArray(int *pa, int n)
13   {
14       for (; pa<pa+n; pa++)
15       {
16           scanf("%d", pa);
17       }
18   }
19   void OutputArray(int *pa, int n)
20   {
21       for (; pa<pa+n; pa++)
22       {
23           printf("%4d", *pa);
24       }
25       printf("\n");
26   }
```

参考答案:该程序的主要问题出在第 14 行和第 21 行的语句上,应将程序第 12 ~ 26 行修改为:

```
12   void InputArray(int *pa, int n)
13   {
14       int *p = pa;
15       for (; p<pa+n; p++)
16       {
17           scanf("%d", p);
18       }
```

```
}
void OutputArray(int *pa, int n)
{
    int *p = pa;
    for (; p<pa+n; p++)
    {
        printf("%4d", *p);
    }
    printf("\n");
}
```

(3) 输入 m 个学生(最多为 30 人)n 门课程(最多为 5 门)的成绩,然后计算并打印每个学生各门课的总分和平均分。其中,m 和 n 的值由用户从键盘输入。

```
#include <stdio.h>
#define STUD 30              /* 最多可能的学生人数 */
#define COURSE 5             /* 最多可能的考试科目数 */
void Total(int *pScore, int sum[], float aver[], int m, int n);
void Print(int *pScore, int sum[], float aver[], int m, int n);
int main()
{
    int i, j, m, n, score[STUD][COURSE], sum[STUD];
    float aver[STUD];
    printf("How many students?");
    scanf("%d", &m);
    printf("How many courses?");
    scanf("%d", &n);
    printf("Input scores:\n");
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            scanf("%d", &score[i][j]);
        }
    }
    Total(*score, sum, aver, m, n);
    Print(*score, sum, aver, m, n);
    return 0;
}
```

```
26  void Total(int *pScore, int sum[], float aver[], int m, int n)
27  {
28      int i, j;
29      for (i=0; i<m; i++)
30      {
31          sum[i] = 0;
32          for (j=0; j<n; j++)
33          {
34              sum[i] = sum[i] + pScore[i*n+j];
35          }
36          aver[i] = (float) sum[i] / n;
37      }
38  }
39  void Print(int *pScore, int sum[], float aver[], int m, int n)
40  {
41      int i, j;
42      printf("Result:\n");
43      for (i=0; i<m; i++)
44      {
45          for (j=0; j<n; j++)
46          {
47              printf("%4d\t", pScore[i*n+j]);
48          }
49          printf("%5d\t%6.1f\n", sum[i], aver[i]);
50      }
51  }
```

参考答案:这个程序在 Code::Blocks 下的运行结果为:

```
How many students? 4↙
How many courses? 3↙
Input scores:
60 60 60↙
70 70 70↙
80 80 80↙
90 90 90↙
Result:
```

```
  60          60          60        180         60.0
2358744    2090008641              70       2092367455    697455808.0
  70          70          0         140         46.7
  65          80          80        225         75.0
```

从上面的错误运行结果可知，仅第 1 个学生的统计结果是正确的，其余各行的统计结果均有一些乱码，但总分和平均分又确实是按照这些乱码值计算的，这说明计算总分和平均分的程序没有错误，错误很可能存在于从主函数传给函数 Total( ) 的成绩值。经分析发现，在主函数中二维数组 score 被定义为 STUD 行、COURSE 列，这 STUD×COURSE 个数组元素值是按行连续存储在内存中的，而在函数 Total( ) 中，是通过 *(score + i * n + j) 来间接寻址每个数组元素的，即是按每行 n 列读取数组元素值的，如果 n 与 COURSE 的值相等，那么错误就不会发生，否则（这里 n 值为 3，COURSE 值为 5），由于数组 score 是按每行 COURSE 列分配的内存，而第 15～21 行程序仅为数组的每一行输入了前 n 个数据，后面的 COURSE−n 个数据未被初始化，导致其为乱码。此时，如果按每行 n 列从首地址开始连续读取数组元素值，那么就会导致第 39～51 行程序读出的数组元素值发生错位。

在函数 Total( ) 的入口处插入打印语句，分别按照每行 COURSE 列和 n 列，从首地址 score 开始打印存于内存中的数组元素，对比其结果即可验证上述错误分析结果。

```
1    void Total(int *pScore, int sum[], float aver[], int m, int n)
2    {
3        int i, j;
4        printf("COURSE column results:\n");  /* 按每行 COURSE 列打印数组元素 */
5        for (i=0; i<m; i++)
6        {
7            for (j=0; j<COURSE; j++)
8            {
9                printf("%4d\t", pScore[i*COURSE+j]);
10           }
11           printf("\n");
12       }
13       printf("n column results:\n");      /* 按每行 n 列打印数组元素 */
14       for (i=0; i<m; i++)
15       {
16           for (j=0; j<n; j++)
17           {
18               printf("%4d\t", pScore[i*n+j]);
19           }
20           printf("\n");
```

```
21        }
22        for (i=0; i<m; i++)
23        {
24            sum[i] = 0;
25            for (j=0; j<n; j++)
26            {
27                sum[i] = sum[i] + pScore[i*n+j];
28            }
29            aver[i] = (float) sum[i] /n;
30        }
31    }
```

此时,程序的运行结果如下:

```
How many students? 4↙
How many courses? 3↙
Input scores:
60 60 60↙
70 70 70↙
80 80 80↙
90 90 90↙
COURSE column results:
   60          60          60       2358744   2090009937
   70          70          70             0           65
   80          80          80           128      2358600
   90          90          90       2358620   2090107312
n column results:
   60          60          60
2358744   2090009937                     70
   70          70           0
   65          80          80
Result:
   60          60          60           180         60.0
2358744   2090008641                     70      2092367455 697455808.0
   70          70           0           140         46.7
   65          80          80           225         75.0
```

修正这个错误的第 1 种方法是将函数 Total( ) 和函数 Print( ) 中的 pScore[i * n+j] 改为 pScore[i * COURSE + j]。第 2 种方法是将程序改为如下形式:

```
#include <stdio.h>
#define STUD 30                    /* 最多可能的学生人数 */
#define COURSE 5                   /* 最多可能的考试科目数 */
void Input(int *pScore, int m, int n);
void Total(int *pScore, int sum[], float aver[], int m, int n);
void Print(int *pScore, int sum[], float aver[], int m, int n);
int main()
{
    int m, n, score[STUD][COURSE], sum[STUD];
    float aver[STUD];
    printf("How many students?");
    scanf("%d", &m);
    printf("How many courses?");
    scanf("%d", &n);
    Input(*score, m, n);
    Total(*score, sum, aver, m, n);
    Print(*score, sum, aver, m, n);
    return 0;
}
/* 函数功能:连续输入 m×n 个整数存入起始地址为 pScore 的连续内存单元中 */
void Input(int *pScore, int m, int n)
{
    int i, j;
    printf("Input scores:\n");
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            scanf("%d", &pScore[i*n+j]);
        }
    }
}
/* 函数功能:计算起始地址为 pScore 的连续内存中存储的 m×n 个数组元素值的平均值 */
void Total(int *pScore, int sum[], float aver[], int m, int n)
{
    int i, j;
    for (i=0; i<m; i++)
```

```
38      {
39          sum[i] = 0;
40          for (j=0; j<n; j++)
41          {
42              sum[i] = sum[i] + pScore[i*n+j];
43          }
44          aver[i] = (float) sum[i] / n;
45      }
46  }
47  /* 函数功能:打印起始地址为 pScore 的连续内存单元中存储的 m×n 个数组元素值 */
48  void Print(int *pScore, int sum[], float aver[], int m, int n)
49  {
50      int i, j;
51      printf("Result:\n");
52      for (i=0; i<m; i++)
53      {
54          for (j=0; j<n; j++)
55          {
56              printf("%4d\t", pScore[i*n+j]);
57          }
58          printf("%5d\t%6.1f\n", sum[i], aver[i]);
59      }
60  }
```

此时,程序的运行结果如下:

```
How many students? 4↙
How many courses? 3↙
Input scores:
60 60 60↙
70 70 70↙
80 80 80↙
90 90 90↙
Result:
   60    60    60   180   60.0
   70    70    70   210   70.0
   80    80    80   240   80.0
   90    90    90   270   90.0
```

11.3 从键盘任意输入一个整型表示的月份值,用指针数组编程输出该月份的英文表示,若输入的月份值不在 1~12 之间,则输出“Illegal month”。

参考答案:从键盘任意输入一个月份值 n,用指针数组编程实现输出该月份的英文表示,若 n 不在 1~12 之间,则输出“Illegal month”。

```
#include <stdio.h>
int main()
{
    int n;
    static char * monthName[]={"Illegal month", "January", "February",
                               "March", "April", "May", "June", "July",
                               "August", "September", "October",
                               "November", "December"};
    printf("Input month number:");
    scanf("%d", &n);
    if ((n <= 12) && (n >= 1))
        printf("month %d is %s\n", n, monthName[n]);/* 输出相应月份 */
    else
        printf("%s\n", monthName[0]);          /* 输出错误提示信息 */
    return 0;
}
```

程序的 3 次测试结果如下:

```
① Input month number:5↙
  month 5 is May
② Input month number:12↙
  month 12 is December
③ Input month number:13↙
  Illegal month
```

11.4 利用例 9.6 程序中的函数 Swap(),分别按如下函数原型编程计算并输出 $n \times n$ 阶矩阵的转置矩阵。其中,n 由用户从键盘输入。已知 n 值不超过 10。

```
void Transpose(int a[][N], int n);
void Transpose(int (*a)[N], int n);
void Transpose(int *a, int n);
```

参考程序 1:用二维数组作为函数参数,实现矩阵转置。

```
#include <stdio.h>
#define N 10
void Swap(int *x, int *y);
void Transpose(int a[][N], int n);
```

```
5   void InputMatrix(int a[][N], int n);
6   void PrintMatrix(int a[][N], int n);
7   int main()
8   {
9       int s[N][N], n;
10      printf("Input n:");
11      scanf("%d", &n);
12      InputMatrix(s, n);
13      Transpose(s, n);
14      printf("The transposed matrix is:\n");
15      PrintMatrix(s, n);
16      return 0;
17  }
18  /* 函数功能:交换两个整型数的值 */
19  void Swap(int *x, int *y)
20  {
21      int temp;
22      temp = *x;
23      *x = *y;
24      *y = temp;
25  }
26  /* 计算 n×n 矩阵的转置矩阵 */
27  void Transpose(int a[][N], int n)
28  {
29      int i, j;
30      for (i=0; i<n; i++)
31      {
32          for (j=i; j<n; j++)
33          {
34              Swap(&a[i][j], &a[j][i]);
35          }
36      }
37  }
38  /* 函数功能:输入 n×n 矩阵的值 */
39  void InputMatrix(int a[][N], int n)
40  {
41      int i, j;
```

```
42        printf("Input %d*%d matrix:\n", n, n);
43        for (i=0; i<n; i++)
44        {
45            for (j=0; j<n; j++)
46            {
47                scanf("%d", &a[i][j]);
48            }
49        }
50    }
51    /* 函数功能:输出 n×n 矩阵的值 */
52    void PrintMatrix(int a[][N], int n)
53    {
54        int i, j;
55        for (i=0; i<n; i++)
56        {
57            for (j=0; j<n; j++)
58            {
59                printf("%d\t", a[i][j]);
60            }
61            printf("\n");
62        }
63    }
```

参考程序 2:用指向一维数组的指针变量,即二维数组的行指针作为函数参数,实现矩阵转置。

```
1     #include <stdio.h>
2     #define N 10
3     void Swap(int *x, int *y);
4     void Transpose(int (*a)[N], int n);
5     void InputMatrix(int (*a)[N], int n);
6     void PrintMatrix(int (*a)[N], int n);
7     int main()
8     {
9         int s[N][N], n;
10        printf("Input n:");
11        scanf("%d", &n);
12        InputMatrix(s, n);
13        Transpose(s, n);
14        printf("The transposed matrix is:\n");
```

```
15      PrintMatrix(s, n);
16      return 0;
17  }
18  /* 函数功能:交换两个整型数的值 */
19  void Swap(int *x, int *y)
20  {
21      int temp;
22      temp = *x;
23      *x = *y;
24      *y = temp;
25  }
26  /* 函数功能:计算 n×n 矩阵的转置矩阵 */
27  void Transpose(int (*a)[N], int n)
28  {
29      int i, j;
30      for (i=0; i<n; i++)
31      {
32          for (j=i; j<n; j++)
33          {
34              Swap(*(a+i)+j, *(a+j)+i);
35          }
36      }
37  }
38  /* 函数功能:输入 n×n 矩阵的值 */
39  void InputMatrix(int (*a)[N], int n)
40  {
41      int i, j;
42      printf("Input %d*%d matrix:\n", n, n);
43      for (i=0; i<n; i++)
44      {
45          for (j=0; j<n; j++)
46          {
47              scanf("%d", *(a+i)+j);
48          }
49      }
50  }
51  /* 函数功能:输出 n×n 矩阵的值 */
```

```
52  void PrintMatrix(int (*a)[N], int n)
53  {
54      int i, j;
55      for (i=0; i<n; i++)
56      {
57          for (j=0; j<n; j++)
58          {
59              printf("%d\t", *(*(a+i)+j));
60          }
61          printf("\n");
62      }
63  }
```

参考程序 3:用二维数组的列指针作为函数实参,实现矩阵转置。

```
1   #include <stdio.h>
2   #define N 10
3   void Swap(int *x, int *y);
4   void Transpose(int *a, int n);
5   void InputMatrix(int *a, int n);
6   void PrintMatrix(int *a, int n);
7   int main()
8   {
9       int s[N][N], n;
10      printf("Input n:");
11      scanf("%d", &n);
12      InputMatrix(*s, n);
13      Transpose(*s, n);
14      printf("The transposed matrix is:\n");
15      PrintMatrix(*s, n);
16      return 0;
17  }
18  /* 函数功能:交换两个整型数的值 */
19  void Swap(int *x, int *y)
20  {
21      int temp;
22      temp = *x;
23      *x = *y;
```

```
    *y = temp;
}
/* 函数功能:计算 n×n 矩阵的转置矩阵 */
void Transpose(int *a, int n)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        for (j=i; j<n; j++)
        {
            Swap(&a[i*n+j], &a[j*n+i]);
        }
    }
}
/* 函数功能:输入 n×n 矩阵的值 */
void InputMatrix(int *a, int n)
{
    int i, j;
    printf("Input %d*%d matrix:\n", n, n);
    for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            scanf("%d", &a[i*n+j]);
        }
    }
}
/* 函数功能:输出 n×n 矩阵的值 */
void PrintMatrix(int *a, int n)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            printf("%d\t", a[i*n+j]);
        }
```

```
61            printf("\n");
62        }
63    }
```

程序的运行结果为：

```
Input n:3↙
Input 3*3 matrix:
1 2 3↙
4 5 6↙
7 8 9↙
The transposed matrix is:
1    4    7
2    5    8
3    6    9
```

11.5 在习题 11.4 的基础上，分别按如下函数原型编程计算并输出 $m \times n$ 阶矩阵的转置矩阵。其中，m 和 n 的值由用户从键盘输入。已知 m 和 n 的值都不超过 10。

```
void Transpose(int a[][N], int at[][M], int m, int n);
void Transpose(int (*a)[N], int (*at)[M], int m, int n);
void Transpose(int *a, int *at, int m, int n);
```

编写一个能对任意 $m \times n$ 阶矩阵进行转置运算的函数 Transpose()。

参考程序 1：用二维数组作为函数参数，实现矩阵转置。

```
1   #include <stdio.h>
2   #define M 10
3   #define N 10
4   void Transpose(int a[][N], int at[][M], int m, int n);
5   void InputMatrix(int a[][N], int m, int n);
6   void PrintMatrix(int at[][M], int n, int m);
7   int main()
8   {
9       int s[M][N], st[N][M], m, n;
10      printf("Input m, n:");
11      scanf("%d,%d", &m, &n);
12      InputMatrix(s, m, n);
13      Transpose(s, st, m, n);
14      printf("The transposed matrix is:\n");
15      PrintMatrix(st, n, m);
16      return 0;
17  }
18  /* 函数功能:计算 m×n 矩阵 a 的转置矩阵 at */
```

```
void Transpose(int a[][N], int at[][M], int m, int n)
{
    int i, j;
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            at[j][i] = a[i][j];
        }
    }
}
/* 函数功能:输入 m×n 矩阵 a 的值 */
void InputMatrix(int a[][N], int m, int n)
{
    int i, j;
    printf("Input %d*%d matrix:\n", m, n);
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}
/* 函数功能:输出 n×m 矩阵 at 的值 */
void PrintMatrix(int at[][M], int n, int m)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        for (j=0; j<m; j++)
        {
            printf("%d\t", at[i][j]);
        }
        printf("\n");
    }
}
```

参考程序 2:用指向一维数组的指针变量,即二维数组的行指针作为函数参数,实现矩阵转置。

```
#include <stdio.h>
#define M 10
#define N 10
void Transpose(int (*a)[N], int (*at)[M], int m, int n);
void InputMatrix(int (*a)[N], int m, int n);
void PrintMatrix(int (*at)[M], int n, int m);
int main()
{
    int s[M][N], st[N][M], m, n;
    printf("Input m, n:");
    scanf("%d,%d", &m, &n);
    InputMatrix(s, m, n);
    Transpose(s, st, m, n);
    printf("The transposed matrix is:\n");
    PrintMatrix(st, n, m);
    return 0;
}
/* 函数功能:计算 m×n 矩阵 a 的转置矩阵 at */
void Transpose(int (*a)[N], int (*at)[M], int m, int n)
{
    int i, j;
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            *(*(at+j)+i) = *(*(a+i)+j);
        }
    }
}
/* 函数功能:输入 m×n 矩阵 a 的值 */
void InputMatrix(int (*a)[N], int m, int n)
{
    int i, j;
    printf("Input %d*%d matrix:\n", m, n);
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
```

```
        {
            scanf("%d", *(a+i)+j);
        }
    }
}
/* 函数功能:输出 n×m 矩阵 at 的值 */
void PrintMatrix(int (*at)[M], int n, int m)
{
    int i, j;
    for (i=0; i<n; i++)
    {
        for (j=0; j<m; j++)
        {
            printf("%d\t", *(*(at+i)+j));
        }
        printf("\n");
    }
}
```

参考程序 3:用二维数组的列指针作为函数实参,实现矩阵转置。

```
#include <stdio.h>
#define M 10
#define N 10
void Transpose(int *a, int *at, int m, int n);
void InputMatrix(int *a, int m, int n);
void PrintMatrix(int *at, int n, int m);
int main()
{
    int s[M][N], st[N][M], m, n;
    printf("Input m, n:");
    scanf("%d,%d", &m, &n);
    InputMatrix(*s, m, n);
    Transpose(*s, *st, m, n);
    printf("The transposed matrix is:\n");
    PrintMatrix(*st, n, m);
    return 0;
}
/* 函数功能:计算 m×n 矩阵 a 的转置矩阵 at */
```

```
19  void Transpose(int *a, int *at, int m, int n)
20  {
21      int i, j;
22      for (i=0; i<m; i++)
23      {
24          for (j=0; j<n; j++)
25          {
26              at[j*m+i] = a[i*n+j];
27          }
28      }
29  }
30  /* 函数功能:输入 m×n 矩阵 a 的值 */
31  void InputMatrix(int *a, int m, int n)
32  {
33      int i, j;
34      printf("Input %d*%d matrix: \n", m, n);
35      for (i=0; i<m; i++)
36      {
37          for (j=0; j<n; j++)
38          {
39              scanf("%d", &a[i*n+j]);
40          }
41      }
42  }
43  /* 函数功能:输出 n×m 矩阵 at 的值 */
44  void PrintMatrix(int *at, int n, int m)
45  {
46      int i, j;
47      for (i=0; i<n; i++)
48      {
49          for (j=0; j<m; j++)
50          {
51              printf("%d\t", at[i*m+j]);
52          }
53          printf("\n");
54      }
55  }
```

程序的运行结果为：

```
Input m, n:3,4↙
Input 3*4 matrix:
1 2 3 4↙
5 6 7 8↙
9 10 11 12↙
The transposed matrix is:
1    5    9
2    6    10
3    7    11
4    8    12
```

11.6 参考习题 11.5，按如下函数原型编程从键盘输入一个 m 行 n 列的二维数组，然后计算数组中元素的最大值及其所在的行列下标值。其中，m 和 n 的值由用户从键盘输入。已知 m 和 n 的值都不超过 10。

```
void InputArray(int *p, int m, int n);
int FindMax(int *p, int m, int n, int *pRow, int *pCol);
```

参考程序如下：

```
#include <stdio.h>
#define M 10
#define N 10
void InputMatrix(int *p, int m, int n);
int FindMax(int *p, int m, int n, int *pRow, int *pCol);
int main()
{
    int a[M][N], m, n, row, col, max;
    printf("Input m, n:");
    scanf("%d,%d", &m, &n);
    InputMatrix(*a, m, n);
    max = FindMax(*a, m, n, &row, &col);
    printf("max = %d, row = %d, col = %d\n", max, row, col);
    return 0;
}
/* 函数功能:输入 m×n 矩阵的值 */
void InputMatrix(int *p, int m, int n)
{
    int i, j;
    printf("Input %d*%d array:\n", m, n);
```

```
21        for (i=0; i<m; i++)
22        {
23            for (j=0; j<n; j++)
24            {
25                scanf("%d", &p[i*n+j]);
26            }
27        }
28    }
29    /* 函数功能:在 m×n 矩阵中查找最大值及其所在的行列号 */
30    int FindMax(int *p, int m, int n, int *pRow, int *pCol)
31    {
32        int i, j, max = p[0];
33        *pRow = 0;
34        *pCol = 0;
35        for (i=0; i<m; i++)
36        {
37            for (j=0; j<n; j++)
38            {
39                if (p[i*n+j] > max)
40                {
41                    max = p[i*n+j];
42                    *pRow = i;          /* 记录行下标 */
43                    *pCol = j;          /* 记录列下标 */
44                }
45            }
46        }
47        return max;
48    }
```

程序的运行结果如下:

```
Input n:3,4↙
1 2 3 4↙
5 6 7 8↙
9 0 -1 -2↙
max = 9, row = 2, col = 0
```

11.7 (选做)参考习题 11.6,用动态数组编程输入任意 m 个班学生(每班 n 个学生)的某门课的成绩,计算最高分,并指出具有该最高分的学生是第几个班的第几个学生。其中,m 和 n 的值由用户从键盘任意输入(不限定 m 和 n 的上限值)。

参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
void InputScore(int *p, int m, int n);
int FindMax(int *p, int m, int n, int *pRow, int *pCol);
int main()
{
    int *pScore, m, n, maxScore, row, col;
    printf("Input array size m,n:");
    scanf("%d,%d", &m, &n);
    pScore = (int *)calloc(m*n, sizeof (int)); /* 申请动态内存 */
    if (pScore == NULL)
    {
        printf("No enough memory! \n");
        exit(0);
    }
    InputScore(pScore, m, n);
    maxScore = FindMax(pScore, m, n, &row, &col);
    printf("maxScore = %d, class = %d, number = %d\n", maxScore, row+
            1, col+1);
    free(pScore);                    /* 释放动态内存 */
    return 0;
}
/* 函数功能:输入 m 行 n 列二维数组的值 */
void InputScore(int *p, int m, int n)
{
    int i, j;
    printf("Input %d*%d array: \n", m, n);
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            scanf("%d", &p[i*n+j]);
        }
    }
}
/* 函数功能:计算任意 m 行 n 列二维数组中元素的最大值,并指出其所在行列下标值 */
int FindMax(int *p, int m, int n, int *pRow, int *pCol)
```

```
{
    int i, j, max = p[0];
    *pRow = 0;
    *pCol = 0;
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            if (p[i*n+j] > max)
            {
                max = p[i*n+j];
                *pRow = i;                    /* 记录行下标 */
                *pCol = j;                    /* 记录列下标 */
            }
        }
    }
    return max;
}
```

程序的运行结果如下：

```
Input array size m,n:3,4↙
Input scores:
80  82  63  74↙
60  81  75  68↙
87  91  78  92↙
maxScore = 92, class = 3, number = 4
```

1.11 习题 12 解答

12.1 设某大学有下列登记表，采用最佳方式对它进行类型定义。

姓名	性别	出生日期			职业状况		
		年	月	日	所在学院	职称	职务

参考答案：将某大学职工个人信息定义为如下的结构体类型：

```
struct date                                   /* 定义日期结构体类型 */
{
    int year;                                 /* 年 */
    int month;                                /* 月 */
    int day;                                  /* 日 */
```

```
};
struct professionalState                    /* 定义职业结构体类型 */
{
    char college[80];                       /* 所在学院 */
    char professionalTitle[20];             /* 职称 */
    char duty[20];                          /* 职务 */
};
struct person                               /* 定义职工个人信息结构体类型 */
{
    char name[20];                          /* 姓名 */
    char sex;                               /* 性别 */
    struct date birthday;                   /* 出生日期 */
    struct professionalState occupation;    /* 职业状况 */
};
```

12.2 请定义一个时钟结构体类型,它包含“时、分、秒”3 个成员,然后将第 7 章习题 7.2 中用全局变量编写的时钟模拟显示程序改成用结构体指针变量作函数参数重新编写。

参考程序如下:

```
#include <stdio.h>
typedef struct clock
{
    int hour;
    int minute;
    int second;
}CLOCK;
/* 函数功能:时、分、秒时间的更新 */
void Update(CLOCK *t)
{
    t->second++;
    if (t->second == 60)/* 若 second 值为 60,表示已过一分钟,则 minute 加 1 */
    {
        t->second = 0;
        t->minute++;
    }
    if (t->minute == 60)/* 若 minute 值为 60,表示已过一小时,则 hour 加 1 */
    {
        t->minute = 0;
        t->hour++;
    }
    if (t->hour == 24) /* 若 hour 值为 24,则 hour 从 0 开始计时 */
```

```
23        {
24            t->hour = 0;
25        }
26    }
27    /* 函数功能:时、分、秒时间的显示 */
28    void Display(CLOCK *t)
29    {
30        printf("%2d:%2d:%2d\r", t->hour, t->minute, t->second);
31    }
32    /* 函数功能:模拟延迟 1 s 的时间 */
33    void Delay(void)
34    {
35        long t;
36        for (t=0; t<50000000; t++)
37        {
38            /* 循环体为空语句的循环,起延时作用 */
39        }
40    }
41    int main()
42    {
43        long i;
44        CLOCK myclock;
45        myclock.hour = myclock.minute = myclock.second = 0;
46        for (i=0; i<100000; i++)          /* 利用循环,控制时钟运行的时间 */
47        {
48            Update(&myclock);             /* 时钟值更新 */
49            Display(&myclock);            /* 时间显示 */
50            Delay();                      /* 模拟延时 1s */
51        }
52        return 0;
53    }
```

12.3 请重新编写习题 12.2 程序,将其中的函数 Update()改用整除和求余运算来实现时钟值的更新。

参考程序 1 如下:

```
1     #include <stdio.h>
2     typedef struct clock
3     {
4         int hour;
5         int minute;
```

```
6       int second;
7   }CLOCK;
8   /* 函数功能:时、分、秒时间的更新 */
9   void Update(CLOCK *t)
10  {
11      static long m = 1;
12      t->hour = m / 3600;
13      t->minute = (m - 3600 * t->hour) / 60;
14      t->second = m %60;
15      m++;
16      if (t->hour == 24)m = 1;
17  }
18  /* 函数功能:时、分、秒时间的显示 */
19  void Display(CLOCK *t)
20  {
21      printf("%2d:%2d:%2d\r", t->hour, t->minute, t->second);
22  }
23  /* 函数功能:模拟延迟 1s 的时间 */
24  void Delay(void)
25  {
26      long t;
27      for (t=0; t<50000000; t++)
28      {
29          /* 循环体为空语句的循环,起延时作用 */
30      }
31  }
32  int main()
33  {
34      long i;
35      CLOCK myclock;
36      myclock.hour = myclock.minute = myclock.second = 0;
37      for (i=0; i<100000; i++)          /* 利用循环,控制时钟运行的时间 */
38      {
39          Update(&myclock);             /* 时钟值更新 */
40          Display(&myclock);            /* 时间显示 */
41          Delay();                      /* 模拟延时 1s */
42      }
43      return 0;
44  }
```

参考程序 2 如下：

```
#include <stdio.h>
typedef struct clock
{
    int hour;
    int minute;
    int second;
}CLOCK;
/* 函数功能:时、分、秒时间的更新 */
void Update(CLOCK *t)
{
    static long m = 1;
    t->second = m % 60;
    t->minute = (m / 60) % 60;
    t->hour = (m / 3600) % 24;
    m++;
    if (t->hour == 24)m = 1;
}
/* 函数功能:时、分、秒时间的显示 */
void Display(CLOCK *t)
{
    printf("%2d:%2d:%2d\r", t->hour, t->minutes, t->second);
}
/* 函数功能:模拟延迟 1s 的时间 */
void Delay(void)
{
    long t;
    for (t=0; t<50000000; t++)
    {
        /* 循环体为空语句的循环,起延时作用 */
    }
}
int main()
{
    long i;
    CLOCK myclock;
    myclock.hour = myclock.minute = myclock.second = 0;
    for (i=0; i<100000; i++)          /* 利用循环,控制时钟运行的时间 */
    {
```

```
39          Update(&myclock);              /* 时钟值更新 */
40          Display(&myclock);             /* 时间显示 */
41          Delay();                       /* 模拟延时 1s */
42      }
43      return 0;
44  }
```

12.4 编程统计候选人的得票数。设有 3 个候选人 zhang、li、wang(候选人姓名不区分大小写),10 个选民,选民每次输入一个得票的候选人的名字,若选民输错候选人姓名,则按废票处理。选民投票结束后,程序自动显示各候选人的得票结果和废票信息。要求用结构体数组 candidate 表示 3 个候选人的姓名和得票结果。

参考程序 1 如下:

```
1   #include <stdio.h>
2   #include <string.h>
3   #define NUM_ELECTORATE 10
4   #define NUM_CANDIDATE 3
5   struct candidate
6   {
7       char name[20];
8       int count;
9   }candidate[3] = {"li",0, "zhang",0, "wang",0};
10  int main()
11  {
12      int i, j, flag = 1, wrong = 0;
13      char name[20];
14      for (i=1; i<=NUM_ELECTORATE; i++)
15      {
16          printf("Input vote %d:", i);
17          scanf("%s", name);
18          strlwr(name);              /* 将 name 中的字符全部转换为小写字母 */
19          flag = 1;
20          for (j=0; j<NUM_CANDIDATE; j++)
21          {
22              if (strcmp(name, candidate[j].name) == 0)
23              {
24                  candidate[j].count++;
25                  flag = 0;
26              }
27          }
28          if (flag)
```

```
29          {
30              wrong++; /* 废票计数 */
31              flag = 0;
32          }
33      }
34      printf("Election results:\n");
35      for (i=0; i<NUM_CANDIDATE; i++)
36      {
37           printf("%8s:%d\n", candidate[i].name, candidate[i].count);
38      }
39      printf("Wrong election:%d\n", wrong);
40      return 0;
41  }
```

参考程序 2 如下：

```
1   #include <stdio.h>
2   #include <string.h>
3   #define NUM_ELECTORATE 10
4   #define NUM_CANDIDATE 3
5   struct candidate
6   {
7       char name[20];
8       int count;
9   }candidate[3] = {"li",0, "zhang",0, "wang",0};
10  int Election(struct candidate candidate[]);
11  int main()
12  {
13      int i, wrong;
14      wrong = Election(candidate);
15      printf("Election results:\n");
16      for (i=0; i<NUM_CANDIDATE; i++)
17      {
18          printf("%8s:%d\n", candidate[i].name, candidate[i].count);
19      }
20      printf("Wrong election:%d\n", wrong);
21      return 0;
22  }
23  int Election(struct candidate candidate[])
24  {
25      int i, j, flag = 1, wrong = 0;
```

```
26        char name[20];
27        for (i=1; i<=NUM_ELECTORATE; i++)
28        {
29            printf("Input vote %d:", i);
30            scanf("%s", name);
31            strlwr(name);              /* 将 name 中的字符全部转换为小写字母 */
32            flag = 1;
33            for (j=0; j<NUM_CANDIDATE; j++)
34            {
35                if (strcmp(name, candidate[j].name) == 0)
36                {
37                    candidate[j].count++;
38                    flag = 0;
39                }
40            }
41            if (flag)
42            {
43                wrong++; /* 废票计数 */
44                flag = 0;
45            }
46        }
47        return wrong;
48    }
```

程序的运行结果如下：

```
Input vote 1:li↙
Input vote 2:li↙
Input vote 3:Li↙
Input vote 4:wang↙
Input vote 5:zhang↙
Input vote 6:Wang↙
Input vote 7:Zhang↙
Input vote 8:wang↙
Input vote 9:li↙
Input vote 10:lii↙
Election results:
     li:4
  zhang:2
   wang:3
Wrong election:1
```

12.5 编程模拟洗牌和发牌过程。一副扑克有52张牌,分为4种花色(Suit):黑桃(Spades)、红桃(Hearts)、草花(Clubs)、方块(Diamonds)。每种花色又有13张牌面(Face):A,2,3,4,5,6,7,8,9,10,Jack,Queen,King。要求用结构体数组card表示52张牌,每张牌包括花色和牌面两个字符型数组类型的数据成员。

参考程序如下:

```
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <stdlib.h>
typedef struct card
{
    char suit[10];
    char face[10];
}CARD;
void Deal(CARD *wCard);
void Shuffle(CARD *wCard);
void FillCard(CARD wCard[], char *wFace[], char *wSuit[]);
int main()
{
    char *suit[] = {"Spades","Hearts","Clubs","Diamonds"};
    char *face[] = {"A","2","3","4","5","6","7","8","9","10",
                    "Jack","Queen","King"};
    CARD card[52];
    srand (time(NULL));
    FillCard(card, face, suit);
    Shuffle(card);
    Deal(card);
    return 0;
}
/* 函数功能:花色按黑桃、红桃、草花、方块的顺序,面值按A~K的顺序,排列52张牌 */
void FillCard(CARD wCard[], char *wFace[], char *wSuit[])
{
    int i;
    for (i=0; i<52; i++)
    {
        strcpy(wCard[i].suit, wSuit[i/13]);
        strcpy(wCard[i].face, wFace[i%13]);
    }
```

```
34  }
35  /* 函数功能:将 52 张牌的顺序打乱以模拟洗牌过程 */
36  void Shuffle(CARD *wCard)
37  {
38      int i, j;
39      CARD temp;
40      for (i=0; i<52; i++)      /* 每次循环交换一次当前牌与随机数指示的牌 */
41      {
42          j = rand() % 52;          /* 每次循环产生一个 0~51 的随机数 */
43          temp = wCard[i];
44          wCard[i] = wCard[j];
45          wCard[j] = temp;
46      }
47  }
48  /* 函数功能:输出每张牌的花色和面值以模拟发牌过程 */
49  void Deal(CARD *wCard)
50  {
51      int i;
52      for (i=0; i<52; i++)
53      {
54      printf("%9s%9s%c",wCard[i].suit,wCard[i].face,i%2==0?'\n':'\t');
55      }
56      printf("\n");
57  }
```

程序的 1 次测试结果如下(注意:每次的输出结果都是不一样的):

```
   Spades          A
   Hearts      Queen       Hearts       King
   Spades         10       Spades          9
    Clubs          2     Diamonds          2
   Spades       King       Hearts          8
    Clubs       King     Diamonds          3
    Clubs          A       Spades          8
   Hearts          6       Hearts          9
   Spades          4       Spades      Queen
    Clubs          8       Hearts          A
 Diamonds       Jack       Spades          5
 Diamonds          4       Spades          7
    Clubs          9        Clubs      Queen
   Spades          2       Hearts          7
 Diamonds      Queen        Clubs          5
 Diamonds          8     Diamonds          6
```

```
Diamonds          7      Hearts          5
  Spades          3      Hearts       Jack
  Spades       Jack      Hearts          2
Diamonds       King       Clubs         10
  Hearts         10      Hearts          4
   Clubs          6    Diamonds          5
  Hearts          3       Clubs          7
Diamonds          A    Diamonds         10
  Spades          6       Clubs       Jack
   Clubs          4       Clubs          3
Diamonds          9
```

12.6 （选做）编程用链表方法实现堆栈数据结构。**堆栈（Stack）**是指这样一段内存，它可以理解为一个筒结构，先放进筒中的数据被后放进筒中的数据“压住”，只有后放进筒中的数据都取出后，先放进去的数据才能被取出，称为“**后进先出**”。堆栈的长度可随意增加。堆栈结构可用链表实现。设计一个链表结构需包含两个成员：一个存放数据，一个为指向下一个节点的指针。当每次有一个新数据要放入堆栈时，称为“**压入堆栈**”，这时动态建立一个链表的节点，并连接到链表的结尾；当每次从堆栈中取出一个数据时，称为“**弹出堆栈**”，这意味着从链表的最后一个节点中取出该节点的数据成员，同时删除该节点，释放该节点所占的内存。

参考答案：堆栈可看做一个特殊的“链表”，其特殊之处在于：不能在任意两个节点之间插入节点，每个新节点只能插入“链表”的尾部；也不能从“链表”中间删除节点，每次只能从“链表”的尾部删除最后一个节点，即每次执行压入堆栈的操作，就意味着在“链表”的尾部增加一个节点；而每次执行弹出堆栈的操作，则意味着读出“链表”尾部节点的数据，且将该节点删除。因此，需要编写两个函数：压入堆栈 * PushStack(int num) 和弹出堆栈 int PopStack(void)。参考程序如下：

```
1    #include<stdio.h>
2    #include<stdlib.h>
3    typedef struct stack
4    {
5        int data;
6        struct stack *next;
7    }STACK;
8    STACK *head, *pr;
9    int nodeNum = 0;  /* 堆栈节点数寄存器 */
10   STACK *CreateNode(int num);
11   STACK *PushStack(int num);
12   int PopStack(void);
13   int main()
14   {
15       int pushNum[5]={111,222,333,444,555}, popNum[5], i;
```

```
    for (i=0;i<5;i++)
    {
        PushStack(pushNum[i]);
        printf("Push %dth Data:%d\n", i+1, pushNum[i]);
    }
    for (i=0; i<5; i++)
    {
        popNum[i] = PopStack();
        printf("Pop %dth Data:%d\n", 5-i, popNum[i]);
    }
    return 0;
}
/* 函数功能:生成一个新的节点,并为该节点赋初值,返回指向新的节点的指针 */
STACK *CreateNode(int num)
{
    STACK *p;
    p = (STACK *)malloc(sizeof(STACK));
    if(p == NULL)
    {
        printf("No enough memory! \n");
        exit(0);
    }
    p->next = NULL; /* 为新建的节点指针域赋空指针 */
    p->data = num;  /* 为新建的节点数据区赋值 */
    return p;
}
/* 函数功能:将整型变量 num 的值压入堆栈,返回指向链表新节点的指针 */
STACK *PushStack(int num)
{
    if(nodeNum==0) /* 若为首节点,则保留该节点地址在 head 中 */
    {
        head = CreateNode(num);
        pr = head;
        nodeNum++; /* 堆栈节点数寄存器加 1 */
    }
    else   /* 若不是首节点,则将新建节点连接到链表的结尾处 */
    {
        pr->next = CreateNode(num);
        pr = pr->next;
```

```
55              nodeNum++; /* 堆栈节点数寄存器加 1 */
56          }
57          return pr;
58      }
59      /* 函数功能:将当前栈顶的数据弹出堆栈,返回从堆栈中弹出的数据 */
60      int PopStack(void)
61      {
62          STACK *p = head;
63          int result;
64          for (;;)
65          {
66              if (p->next == NULL) /* 查找最后一个节点 */
67              {
68                  break;
69              }
70              else
71              {
72                  pr = p;             /* 记录最后一个节点的前一个节点的地址 */
73                  p = p->next;
74                  nodeNum--;          /* 堆栈节点数寄存器减 1 */
75              }
76          }
77          pr->next = NULL;            /* 将末节点的前一个节点置成末节点 */
78          result = p->data;
79          free(p);
80          return result;
81      }
```

程序的运行结果如下:

```
Push 1th Data:111
Push 2th Data:222
Push 3th Data:333
Push 4th Data:444
Push 5th Data:555
Pop 5th Data:555
Pop 4th Data:444
Pop 3th Data:333
Pop 2th Data:222
Pop 1th Data:111
```

1.12 习题 13 解答

13.1 用文件操作函数编程模拟 DOS 下的 type 命令，即在 DOS 状态下输入如下命令，把文件内容以 ASCII 码字符方式显示到屏幕上。

type 文件名↙

参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
int type(const char *fileName);
int main(int argc, char *argv[])
{
    int i;
    if (argc < 2)                /* 没有参数时提示语法错误 */
    {
        printf("The syntax of the command is incorrect.\n");
        exit(0);
    }
    for (i=1; i<argc; i++)/* 把除 argv[0]外的所有参数代表的文件内容逐一输出 */
    {
        if (type(argv[i]) == 0)
        printf("No such a file: %s\n", argv[i]);
    }
    return 0;
}
/* 函数功能:把文件 fileName 的内容输出到屏幕,函数返回非 0 值时表示成功,否则表示出错 */
int type(const char *fileName)
{
    int val = 1;          /* 成功操作时的返回值为 1 */
    FILE *fp;
    char ch;
    if ((fp = fopen(fileName,"r")) == NULL)/* 判断文件是否成功打开 */
    {
        val = 0;                    /* 打开文件失败时的返回值为 0 */
    }
    else
    {
        printf("The contents of file %s:\n", fileName);
        while ((ch = fgetc(fp)) != EOF)  /* 从文件读取字符直到文件尾 */
```

```
33              {
34                  fputc(ch, stdout);              /* 在显示器上显示字符 */
35              }
36              fclose(fp);
37          }
38          return val;                             /* 返回 val 的值 */
39      }
```

这里,在 Code::Blocks 中设置命令行参数的方法为:选择 Project 菜单中的 Set programs' arguments 命令,具体如图 1-11 所示。

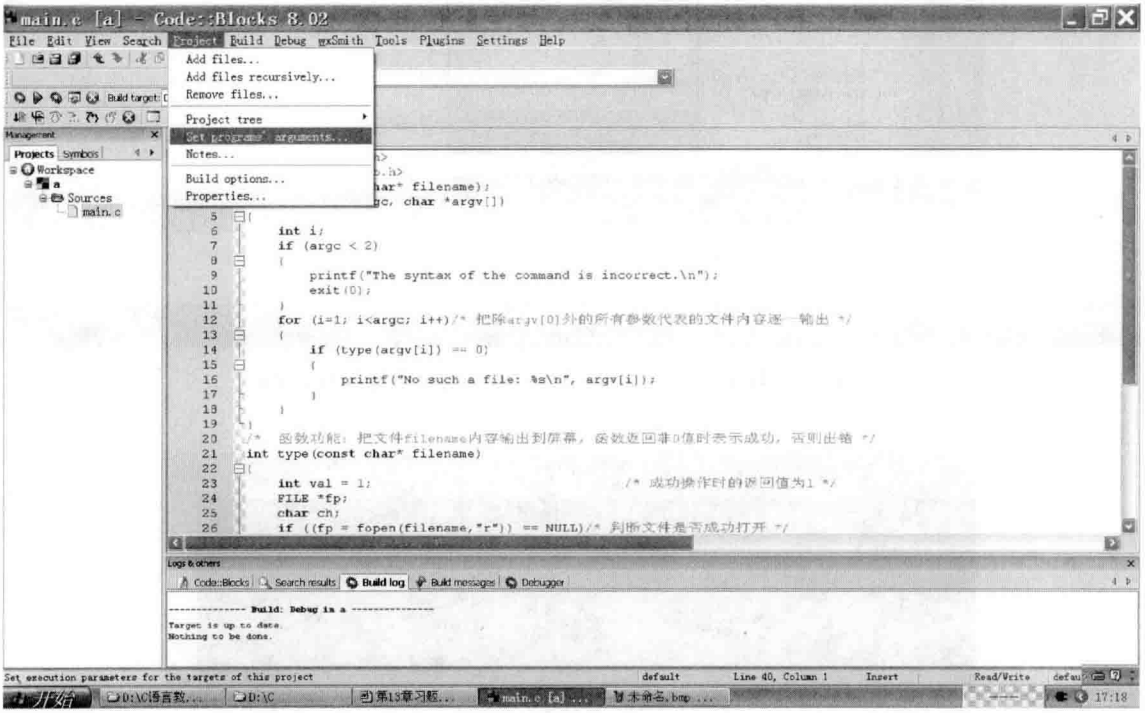


图 1-11 设置命令行参数的方法

如图 1-12 所示,在 Program arguments 文本框中输入命令行参数:d:\\c\\score.txt。注意:在命令行参数中应该给出要显示的文件所在的完整路径。

由于命令行参数是 d:\\c\\score.txt,因此,运行本例程序后将显示文件 d:\\c\\score.txt 的内容,其运行结果如图 1-13 所示。

13.2 修改例 12.3 程序,利用结构体数组计算每个学生的 4 门课程的平均分,将学生的各科成绩及平均分输出到文件 score.txt 中。

参考程序如下:

```
1    #include <stdio.h>
2    #include <stdlib.h>
3    typedef struct date
4    {
```

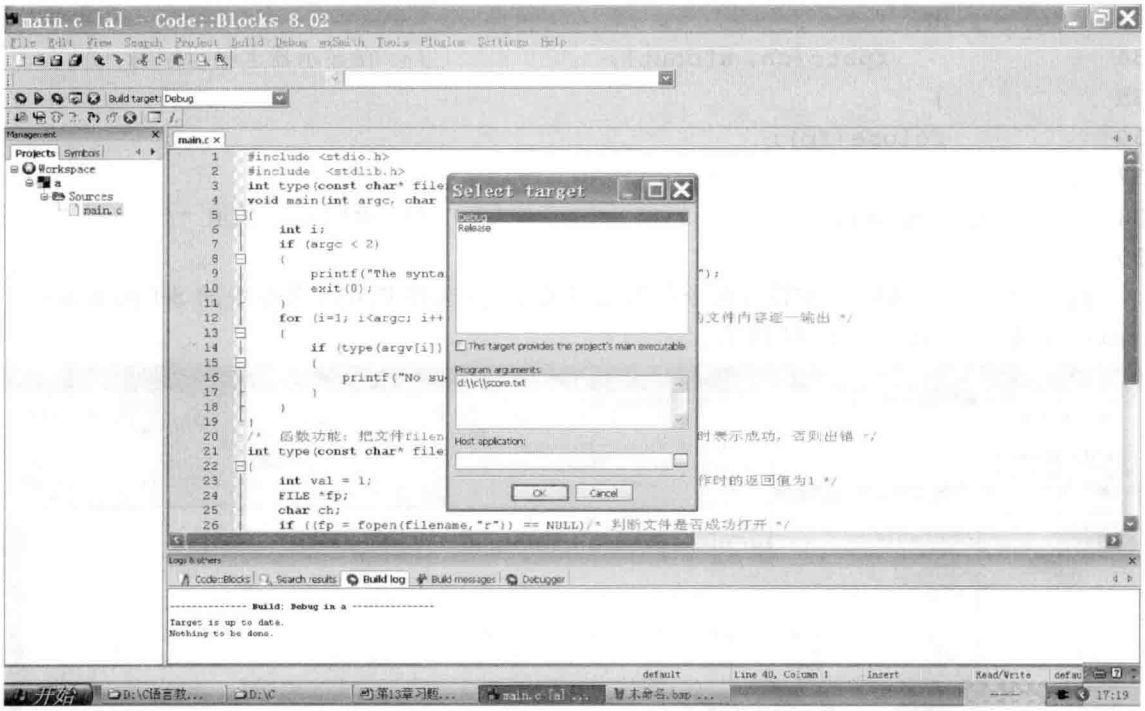


图 1-12 在 Program arguments 文本框中输入命令行参数的方法

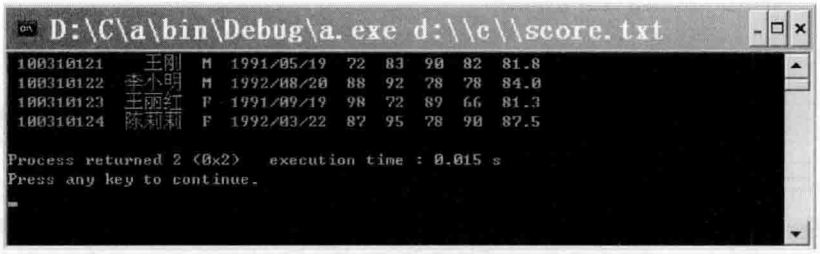


图 1-13 本例程序在输入命令行参数 d:\\c\\score.txt 时输出的结果

```
5       int year;
6       int month;
7       int day;
8   }DATE;
9   typedef struct student
10  {
11      long studentID;                    /* 学号 */
12      char studentName[20];              /* 姓名 */
13      char studentSex;                   /* 性别 */
14      DATE birthday;                     /* 出生日期 */
```

```
15      int score[4];                          /* 4门课程的成绩 */
16  }STUDENT;
17  int main()
18  {
19      FILE *fp;
20      int i, j, sum[30];
21      STUDENT stu[30] = {{100310121,"王刚",'M',{1991,5,19},{72,83,90,82}},
22                         {100310122,"李小明",'M',{1992,8,20},{88,92,78,78}},
23                         {100310123,"王丽红",'F',{1991,9,19},{98,72,89,66}},
24                         {100310124,"陈莉莉",'F',{1992,3,22},{87,95,78,90}}
25                        };
26      if ((fp = fopen("score.txt","w")) == NULL) /* 以写方式打开文本文件 */
27      {
28          printf("Failure to open score.txt! \n");
29          exit(0);
30      }
31      for (i=0; i<4; i++)
32      {
33          sum[i] = 0;
34          for (j=0; j<4; j++)
35          {
36              sum[i] = sum[i] + stu[i].score[j];
37          }
38          fprintf(fp, "%10ld%8s%3c%6d/%02d/%02d%4d%4d%4d%4d%6.1f\n",
39                  stu[i].studentID,
40                  stu[i].studentName,
41                  stu[i].studentSex,
42                  stu[i].birthday.year,
43                  stu[i].birthday.month,
44                  stu[i].birthday.day,
45                  stu[i].score[0],
46                  stu[i].score[1],
47                  stu[i].score[2],
48                  stu[i].score[3],
49                  sum[i]/4.0);
50      }
51      fclose(fp);
52      return 0;
53  }
```

程序的运行结果如下：

```
100310121  王刚 M 1991/05/19 72 83 90 82 81.8
100310122 李小明 M 1992/08/20 88 92 78 78 84.0
100310123 王丽红 F 1991/09/19 98 72 89 66 81.3
100310124 陈莉莉 F 1992/03/22 87 95 78 90 87.5
```

13.3 在例 13.7 建立的文件基础上，打开文件，顺序查找某个学号。

参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#define N 30
typedef struct date
{
    int year;
    int month;
    int day;
}DATE;
typedef struct student
{
    long studentID;                 /* 学号 */
    char studentName[10];           /* 姓名 */
    char studentSex;                /* 性别 */
    DATE birthday;                  /* 出生日期 */
    int score[4];                   /* 4 门课程的成绩 */
    float aver;                     /* 4 门课程的平均分 */
}STUDENT;
int SearchNuminFile(char fileName[], long key);
int main()
{
    int n;
    long key;
    printf("Input the searching ID:");
    scanf("%d", &key);
    n = SearchNuminFile("student.txt", key);
    if (n != -1)
        printf("Record number is %d.\n", n);
    else
        printf("Not found.\n");
    return 0;
}
```

```
/* 从文件 fileName 中查找并输出学号为 key 的记录信息,若找到,返回记录号;否则返回-1 */
int SearchNuminFile(char fileName[], long key)
{
    FILE *fp;
    STUDENT stu[N];
    int i, j;
    if ((fp = fopen(fileName, "r")) == NULL) /* 以读方式打开文本文件 */
    {
        printf("Failure to open %s! \n", fileName);
        exit(0);
    }
    for (i=0; !feof(fp); i++)
    {
        fread(&stu[i], sizeof(STUDENT), 1, fp); /* 按数据块读文件 */
        if (key == stu[i].studentID)
        {
            printf ("%10ld%8s%3c%6d/%02d/%02d", stu[i].studentID,
                                                stu[i].studentName,
                                                stu[i].studentSex,
                                                stu[i].birthday.year,
                                                stu[i].birthday.month,
                                                stu[i].birthday.day);
            for (j=0; j<4; j++)
            {
                printf("%4d", stu[i].score[j]);
            }
            printf("%6.1f\n", stu[i].aver);
            fclose(fp);
            return i+1;                         /* 若找到,则返回记录号 */
        }
    }
    fclose(fp);
    return -1;                                  /* 若未找到,则返回-1 */
}
```

程序的两次测试结果如下:

① Input the searching ID: 100310122↙
100310122 **李小明** M 1992/08/20 88 92 78 78 84.0
Record number is 1.

② Input the searching ID: 100310120↙

```
Not found.
```

13.4 复制文件。根据程序提示从键盘输入一个已存在的文本文件的完整文件名,再输入一个新文本文件的完整文件名,然后将已存在的文本文件中的内容全部复制到新文本文件中。利用文本编辑软件查看文件内容,验证程序执行结果。

参考程序如下:

```
#include <stdio.h>
#define N 80
int CopyFile(const char *srcName, const char *dstName);
int main()
{
    char srcFilename[N];                    /* 源文件名 */
    char dstFilename[N];                    /* 目标文件名 */
    printf("Input source filename:");
    scanf("%s", srcFilename);               /* 输入源文件名 */
    printf("Input destination filename:");
    scanf("%s", dstFilename);               /* 输入目标文件名 */
    if (CopyFile(srcFilename, dstFilename))/* 文件复制 */
        printf("Copy succeed! \n");
    else
        printf("Copy failed! \n");
    return 0;
}
/* 函数功能:把 srcName 文件内容复制到 dstName 文件中,返回非 0 值表示复制成功 */
int CopyFile(const char *srcName, const char *dstName)
{
    FILE *fpSrc = NULL, *fpDst = NULL;
    int ch, rval = 1;
    if ((fpSrc = fopen(srcName,"r")) == NULL) goto ERROR;
    if ((fpDst = fopen(dstName,"w")) == NULL) goto ERROR;
    while ((ch=fgetc(fpSrc)) != EOF) /* 复制文件 */
    {
        if (fputc(ch, fpDst) == EOF) goto ERROR;
    }
    fflush(fpDst);                          /* 确保存盘 */
    goto EXIT;
ERROR:
    rval = 0;
EXIT:
    if (fpSrc != NULL)fclose(fpSrc);
```

```
35        if (fpDst != NULL)fclose(fpDst);
36        return rval;
37    }
```

程序的两次测试结果如下：

① Input source filename:a.txt↙(假设a.txt 文件存在)
Input destination filename:b.txt↙
Copy succeed!

② Input source filename:a.txt↙(假设a.txt 文件不存在)
Input destination filename:b.txt↙
Copy failed!

13.5 文件追加。根据提示从键盘输入一个已存在的文本文件的完整文件名，再输入另一个已存在的文本文件的完整文件名，然后将第一个文本文件的内容追加到第二个文本文件的原内容之后。利用文本编辑软件查看文件内容，验证程序执行结果。

参考程序如下：

```
1     #include <stdio.h>
2     #define N 80
3     int AppendFile(const char *srcName, const char *dstName);
4     int main()
5     {
6         char srcFilename[N];                    /* 源文件名 */
7         char dstFilename[N];                    /* 目标文件名 */
8         printf("Input source filename:");
9         scanf("%s", srcFilename);               /* 输入源文件名 */
10        printf("Input destination filename:");
11        scanf("%s", dstFilename);               /* 输入目标文件名 */
12        if (AppendFile(srcFilename, dstFilename))/* 文件追加 */
13            printf("Append succeed! \n");
14        else
15            printf("Append failed! \n");
16        return 0;
17    }
18    /* 函数功能:把 srcName 文件内容复制到 dstName 文件中,返回非 0 值表示复制成功 */
19    int AppendFile(const char *srcName, const char *dstName)
20    {
21        FILE *fpSrc = NULL, *fpDst = NULL;
22        int ch, rval = 1;
23        if ((fpSrc = fopen(srcName,"r")) == NULL)goto ERROR;
24        if ((fpDst = fopen(dstName,"a")) == NULL) goto ERROR;
25        while ((ch=fgetc(fpSrc)) != EOF)/* 文件追加 */
```

```
26          {
27              if (fputc(ch, fpDst) == EOF) goto ERROR;
28          }
29          fflush(fpDst); /* 确保存盘 */
30          goto EXIT;
31      ERROR:
32          rval = 0;
33      EXIT:
34          if (fpSrc ! = NULL)fclose(fpSrc);
35          if (fpDst ! = NULL)fclose(fpDst);
36          return rval;
37      }
```

程序的两次测试结果如下：

① Input source filename:a.txt↙(假设a.txt 文件存在)
Input destination filename:b.txt↙
Append succeed!

② Input source filename:a.txt↙(假设a.txt 文件不存在)
Input destination filename:b.txt↙
Append failed!

第2章 实验指导

2.1 集成开发环境简介

2.1.1 Visual C++ 6.0 集成开发环境的使用与调试方法简介

1. C 语言程序的上机步骤

编好一个 C 语言程序后，如何上机运行呢？如图 2-1 所示，一般要经过以下几个步骤。

(1) 编辑源程序。按照 C 语言语法规则编写源程序，并保存在扩展名为“.c”的文件中。

(2) 编译。源程序在计算机内部用相应的 ASCII 码表示，而计算机的 CPU 只能执行二进制的机器指令，因此需要经过编译，将源程序转换为由机器指令组成的目标程序。

(3) 链接。在源程序中，输入、输出以及常用函数运算通常不是由用户编写的，而是直接调用系统函数库中的库函数。因此，必须把库函数的处理过程链接到经过编译生成的目标程序中，以生成可执行程序。

(4) 运行程序。在程序设计的过程中，不可避免地会存在一些错误。因此，通过调试(Debug)来查错、改错的技术就显得特别重要。图 2-1 中虚线表示当某一步骤出现错误时的修改路线。无论是出现编译错误、链接错误，还是运行时错误，都需要修改源程序，并对它重新编译、链接和运行，直至将程序调试正确为止。

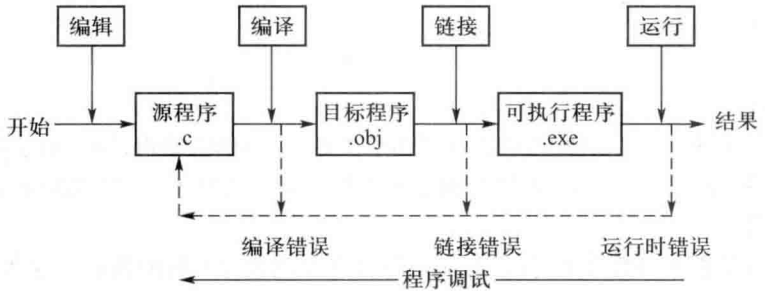


图 2-1 C 语言程序的上机步骤

2. C 语言程序中的错误分类及调试方法

下面通过例子介绍 C 语言程序中的错误分类及调试方法。

【例 2.1】下面的程序含有错误，试通过分析和调试程序，发现并改正其中的错误，使其正确实现求数组 a 中最小值的功能。

```
1    #include <stdio.h>
2
```

```
3    #define ARRSIZE 5
4    int FindMin(int a[], int n);
5
6    int main()
7    {
8        int min;
9        int a[ARRSIZE] = {5,4,3,2,1};
10
11       min = FindMin(a, ARRSIZE);
12       printf("min=%d\n", min);
13       return 0;
14   }
15   int FindMi(int a[], n)
16   {
17       int min, j;
18
19       for (j=1; j<n; j++)
20       {
21           if (a[j] < min)
22           {
23               min = a[j];
24           }
25       }
26       return min;
27   }
```

(1) 编译错误

编译错误是由于违反了 C 语言的语法规则而产生的,又称语法错误,如保留字输入错误、括号不匹配、语句缺少分号等。例 2.1 中的第 14 行由于形参缺少类型说明符 int,导致编译时出错。

(2) 链接错误

链接错误通常是由于函数名书写错误、缺少包含文件或包含文件的路径错误等原因导致的。例 2.1 中的第 10 行函数调用语句中的函数名 FindMin 与其想调用的第 14 行的函数名 FindMi 不匹配,导致链接时出错。

对于编译或链接错误,编程环境会提供出错位置(行号)以及错误类型等出错提示信息。可以根据这些提示信息编辑出错行,改错后重新编译。

注意:由于出错的情况繁多而且各种错误互有关联,有时编程环境提示的出错行并不是真正出错的行。如果在提示出错的行中找不到错误,应当到前面的行查找。有时显示出的多条出错信息可能只有一两个错误。例如,对所用的变量未定义,编译时就会对所有包含该变量的语句提示出错信息。只要加上一个变量定义,所有错误都消除了。因此,应当从上到下逐一改正错误,每纠正一个错误后,都应该重新编译、链接一次,然后根据新的出错信息继续纠正错误。

(3) 运行时错误

有些程序通过了编译、链接,能够运行,但得到的运行结果却与预想的结果不一致,如死循环、计算值不正确、出现内存访问冲突等,这类错误是运行时错误。

例 2.1 中除了第 14 行的编译、链接错误外,还存在缺少在第 18 行语句之前,将变量 min 初始化为 a[0]的错误,编译器无法检测到这种运行时错误。

导致运行时错误的原因:一部分是程序书写错误,如应该使用变量 x 的地方使用了变量 y,虽然没有语法错误,但意思完全错了;另一部分可能是程序的算法不正确,解题思路不对。还有一些程序,计算结果有时正确,有时不正确,这往往是编程时对各种情况考虑不周所致。运行时错误通常比较隐蔽,难以发现。通常需要检查程序的算法、逻辑以及执行顺序是否正确等。利用编程环境的调试工具跟踪程序的执行,可以有效地辅助用户找出程序中的真正错误。

为了提高调试效率,特别是当程序规模比较大时,可以采用模块调试方法,尽量缩小查错范围。先把程序分成几大模块,并在每一模块的结束位置手工计算该模块的期望结果,然后在程序中放入若干打印语句,或用调试方式运行程序到每一个模块结束,检查程序运行的实际结果与期望结果是否一致,从而确定各模块是否正确。对于出错的模块,可逐条仔细检查各语句,找出错误所在。

3. 调试手段

(1) 断点

断点(Breakpoint)可以让程序的执行中断在需要的地方,方便检查、分析程序。可以在一个程序中设置多个断点,这丝毫不会影响程序的正常执行。但当以调试状态运行程序时,每次运行到断点所在的代码行时,程序就暂停,这时可以用下面介绍的方法控制和查看程序的状态,检查程序语句是否存在错误。

(2) 单步跟踪

当程序在断点处暂停时,就进入了**单步跟踪**(Tracker)状态。通过单步跟踪,可以逐条语句或逐个函数地执行程序,每执行完一条语句或一个函数,程序就暂停,因此可逐条语句或逐个函数地检查它们的执行结果。

断点所在行的代码是下一行要被执行的代码,叫做当前代码行。此时对程序的单步跟踪执行有 6 个选择。

① **单步执行**(Step Over):执行一行代码,然后暂停,如果存在函数调用语句,则把整个函数视为单步一次执行(即不会在该函数中的任何语句处暂停)。该方式常用在模块(通常由一个或多个函数构成)调试时期,可以通过观察变量在模块执行前后的变化情况来确定该模块中是否存在错误,也可以用来跳过已测试完毕的模块。

② **单步进入**(Step Into):执行一行代码,如果此行中有函数调用语句,则进入当前代码行所调用的函数内部,在该函数的第一行代码处暂停,也就是跟踪到函数内部;如果此行中没有函数调用,其作用与单步执行等价。该方式可以跟踪程序的每步执行过程,优点是容易直接定位错误,缺点是调试速度较慢。因此,一般首先采用模块调试法尽量缩小错误范围,然后使用单步执行和断点快速跳过没有错误的部分,最后才使用该方式来逐步跟踪以找出错误。

单步进入一般只能进入有源代码的函数,如用户自己编写的函数。有的编译器提供了库函数的代码,可以跟踪到库函数中执行。如果库函数没有源代码,就不能跟踪进入,此时有的调试器会以汇编代码的方式单步执行函数,有的调试器则忽略函数调用。

③ **运行出函数**(Step Out):继续运行程序,当遇到断点或返回函数调用者时暂停。当只想调试函数中的一部分代码,并在调试完后快速跳出该函数时,可以使用这种方式。

④ **运行到光标所在行**(Run to Cursor):将光标定位在某行代码上并调用这个命令,程序会执行到断点或光标定位的那行代码暂停。如果想重点观察某一行(或多行)代码,而且不想从第一行启动,也不想设置断点时,可以采用这种方式。这种方式比较灵活,可以一次执行一行,也可以一次执行多行;可以直接跳过函数,也可以进入函数内部。

⑤ **继续运行**(Continue):继续运行程序,当遇到断点时暂停。如果已经单步执行了感兴趣的代码行,则可以使用该方式,使程序继续执行,直到遇到下一个断点暂停。

⑥ **停止调试**(Stop):程序运行终止,回到编辑状态。

(3) 监视窗

当程序暂停时,除了可以控制它的执行,还可以通过监视窗(Watch)来查看和修改各个变量的值。综合使用监视窗和单步跟踪功能,是找出程序中隐蔽错误的最简单的方法,因为每运行一行,就可以查看程序究竟做了些什么,变量的值发生了怎样的变化,它们是否按设计者的意图变化。若程序是按要求正常工作的,则这一行就算调试通过了,否则也就找到了错误所在。当程序的运行结果不正确时,采用这种方法显示出可能出错的局部和全局变量的值,可以有效地帮助用户检查究竟哪里的程序计算有误。

(4) 其他调试手段

大多数错误都可以通过上述3种调试手段来检查,其他手段如内存镜像、函数调用栈、assert宏、fprintf函数等,有时也很有用,感兴趣的读者可进一步查阅相关文档。

4. Visual C++ 6.0 的使用与调试

微软公司出品的Visual C++是Windows平台上最流行的C/C++集成开发环境之一。它不仅支持C++语言,也兼容C语言。本节介绍如何在Visual C++ 6.0(以下简称VC)中开发和调试C语言程序。

(1) 创建C语言程序源文件

① 进入VC环境,选择File|New命令,创建C语言程序源文件,如图2-2所示。

② 在打开的New对话框中,选择Files选项卡中的C++ Source File选项,在File文本框中输入文件名,并选择保存路径,如图2-3所示。注意:文件名一定要以".c"作为扩展名,否则系统将按扩展名".cpp"保存。

③ 单击OK按钮,进入程序编辑界面,如图2-4所示。

VC编辑器是标准的Windows风格程序,基本的光标移动、插入、删除、复制和粘贴等功能的操作方法都与普通的文本编辑程序一致。此外,还有许多专门为编写代码而开发的功能。例如:

- 关键字加亮显示。
- 根据输入的代码,智能判断应该缩进还是反缩进。
- 按快捷键Alt+F8自动调整选定文本的代码风格。
- 输入调用函数的代码时,自动提示函数参数。
- 按快捷键Ctrl+]自动寻找配套的括号。
- 按快捷键Ctrl+F2定义书签,按F2键在书签间移动光标。
- 鼠标和键盘配合使用各种功能,大大加快编码速度。

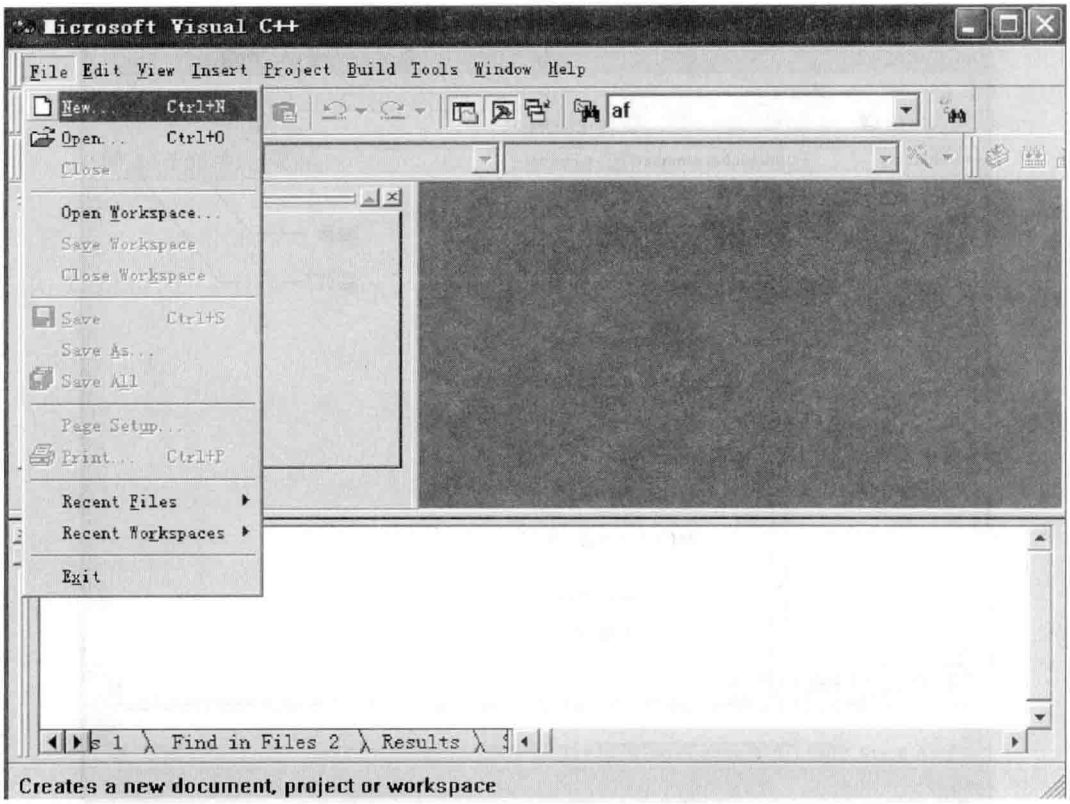


图 2-2 在 VC 中建立 C 语言源程序文件

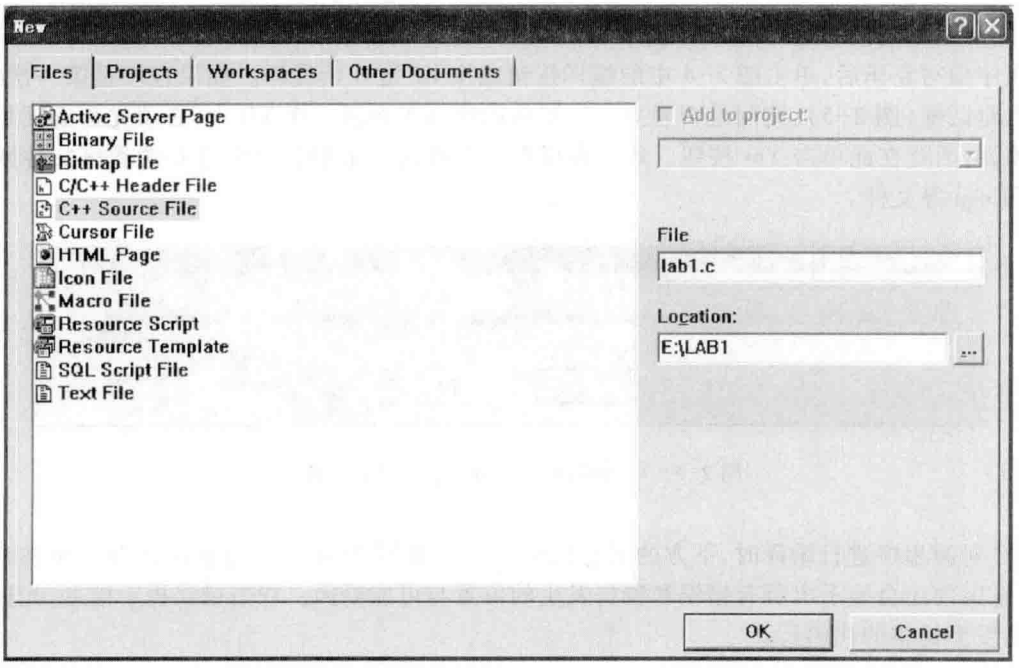


图 2-3 在 VC 中选择文件类型和保存路径

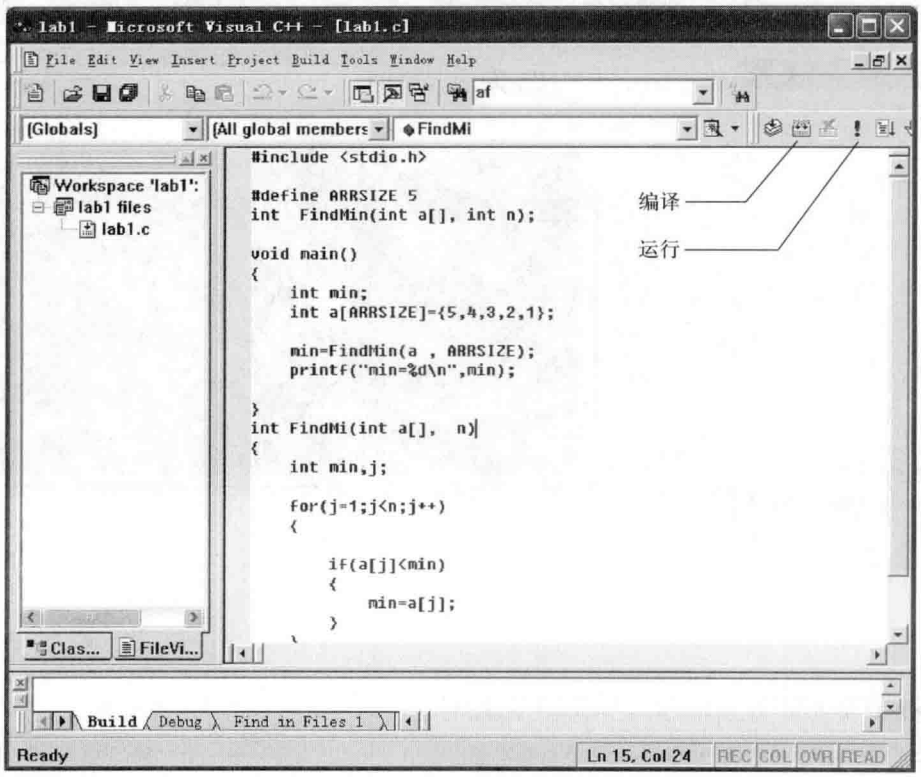


图 2-4 在 VC 中编辑 C 语言程序

(2) 编译、链接和运行 C 语言程序

程序编写完毕后,单击图 2-4 中的编译按钮或按 F7 键开始编译。如果程序是第一次编译,会弹出对话框(图 2-5),询问是否建立一个默认的项目工作区。在 VC 中,必须先建立项目才能开始编译,因此在此单击 Yes 按钮。然后在保存.c 文件的目录中会生成与 C 源程序文件同名的.dsw 和.dsp 等文件。

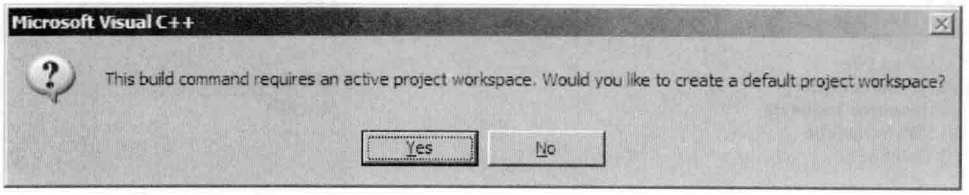


图 2-5 VC 询问是否建立默认的项目工作区

VC 对源程序进行编译时,下方的消息区中会显示如图 2-6 所示的编译报告。如果编译出错,消息区内还会显示出所有错误和警告发生的位置与可能原因。双击错误提示信息,光标立刻跳转到发生错误的代码行。

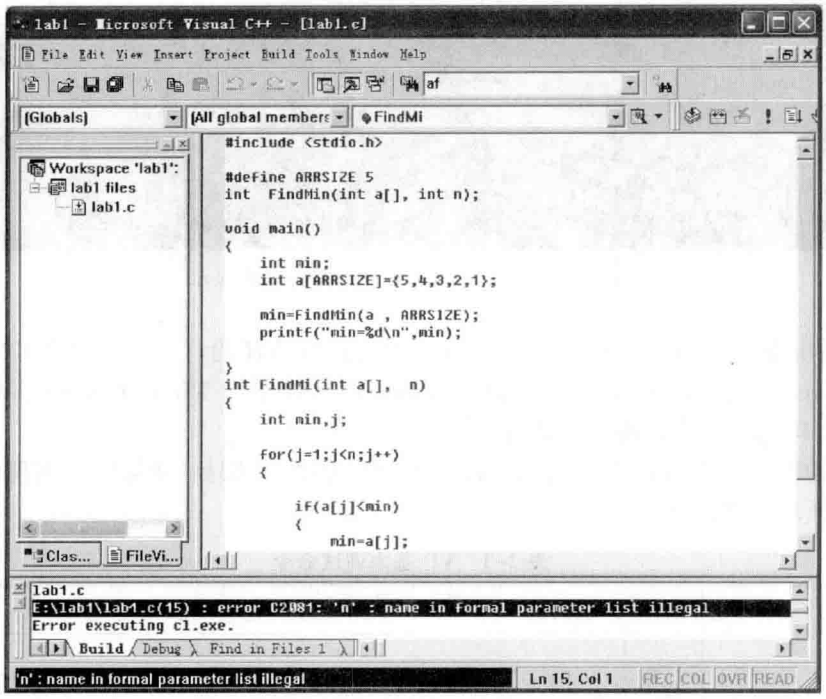


图 2-6 VC 中显示的编译错误提示信息

可以根据提示信息修改程序，如将第 15 行语句修改为：

```
int FindMin(int a[], int n)
```

然后重新编译、链接，如果源程序中没有语法错误，则给出编译结果，如图 2-7 所示。

图 2-7 VC 中显示没有语法错误的编译信息

如果编译和链接都没有错误，程序就可以运行了。单击图 2-4 中的运行按钮或按快捷键 Ctrl+F5，程序将在一个新打开的 DOS 窗口中运行并显示结果，如图 2-8 所示。在程序运行结果的后面会显示一行提示信息“Press any key to continue”，这是 VC 自动加上的，并不是程序的输出。看到此条提示时，说明程序已经运行完毕，按任意键关闭窗口即可。

⚠ 注意：若想创建并编译执行第二个程序，必须选择 File | Close Workspace 命令关闭前一个程序的工作区，然后通过新的编译、链接，产生第二个程序的工作区，否则运行的将一直是前一个程序。

（3）调试 C 语言程序

由图 2-8 可知，尽管该程序是可运行的，但是运行结果错误，并不是期望的值 1。为了进一步定位与修正这种运行时错误，需要对其进行调试。

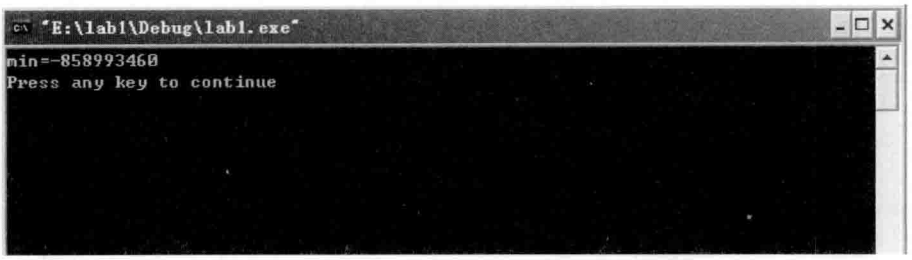


图 2-8 VC 中 C 语言程序的运行结果显示

在默认情况下,VC 中的程序都是采用调试方式进行编译的。关闭调试方式的方法是选择 Build|Set Active Configuration 命令,在弹出的对话框中选择带有 Win32 Release 字样的配置。如果选择 Win32 Debug,则会切换回调试方式。

VC 常用的调试命令如表 2-1 所示。每行中的菜单命令、图标、快捷键所实现的操作完全等价,采用其中任意一种即可。

表 2-1 VC 基本调试命令

菜单命令	图标	快捷键	说明
Go		F5	开始或继续在调试状态下运行程序
Run to Cursor		Ctrl+F10	运行到光标所在行
Insert/Remove Breakpoint		F9	插入或删除断点
Step Into		F11	单步进入,遇到函数则跟踪进入其内部
Step Over		F10	单步执行
Step Out		Shift+F11	跳出当前函数
Stop Debugging		Shift+F5	停止调试程序

① 让程序执行到中途暂停,以便观察阶段性结果。

若希望程序执行到第一条可执行语句(即第 9 行)时暂停,可采用以下任意一种方法。

方法一:使程序执行到光标所在的行暂停。

a. 在第 9 行上单击,定位光标。

b. 按快捷键 Ctrl+F10,程序执行到第 9 行就会暂停,进入跟踪状态,如图 2-9 所示。暂停的语句前会出现一个黄色的小箭头,表示该语句是下一条要执行的语句。

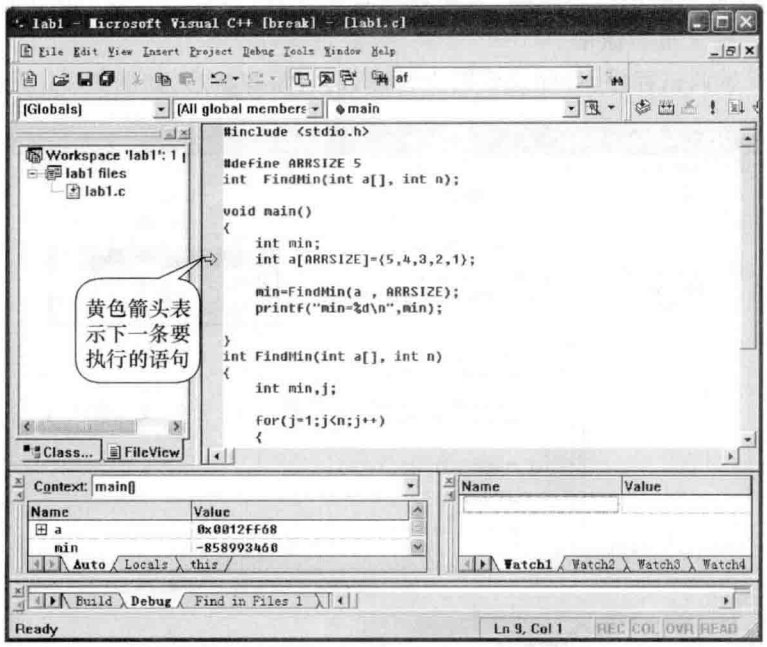


图 2-9 VC 中使程序执行到光标所在的行暂停

方法二:在需要暂停的行上设置断点。

a. 把光标定位在第 9 行,按 F9 键,这行代码前会出现一个红色的大圆点,标志着设置断点成功,如图 2-10 所示。

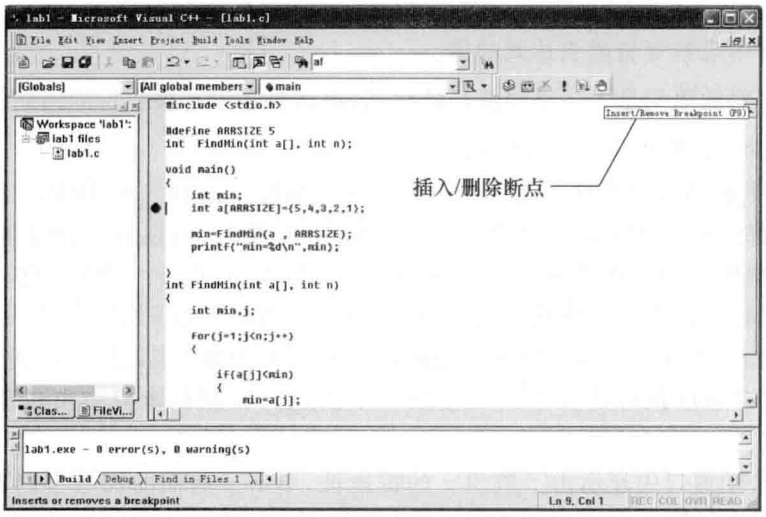


图 2-10 VC 中在需要暂停的行上设置断点

⚠注意:可以根据需要设置任意个数的断点。单击图 2-10 中的插入/删除断点按钮和按 F9 键完全等价,都可切换光标所在行的断点状态。如果断点已经设置,再在断点行按快捷键 Ctrl

+F9,红色圆点会变成空心的红色圆圈,表明该断点还存在,但是已失效(相当于没有)。再按快捷键 Ctrl+F9,断点则又重新激活。

b. 按 F5 键开始调试程序,程序遇到断点暂停,进入跟踪状态,如图 2-11 所示。

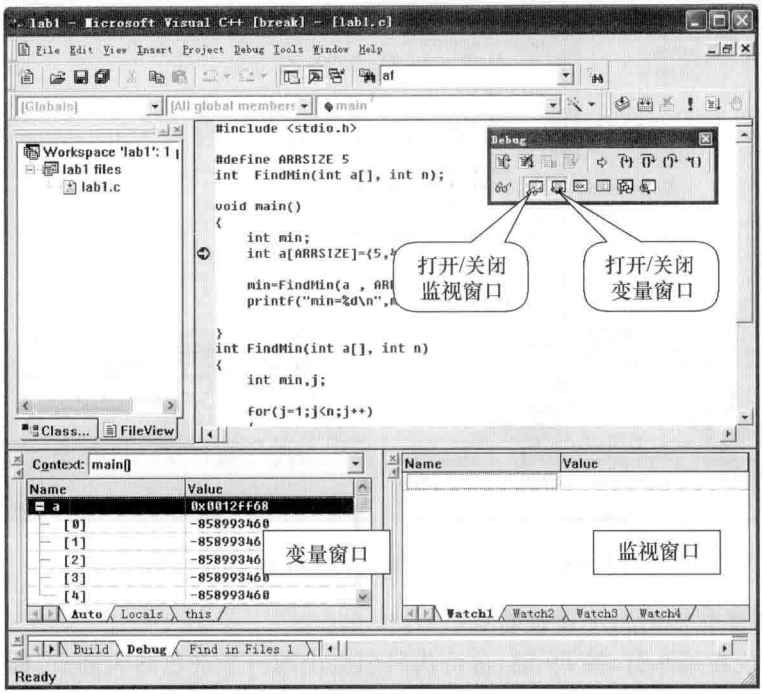


图 2-11 VC 中在需要暂停的行上设置断点的调试状态

② 在监视窗中观察变量或表达式的值。

注意:暂停的语句是下一条要执行的语句,也就是还没有被执行的语句。那么现在数组 a 有值吗?是多少?在监视窗中看看便知。

如图 2-11 所示,VC 中有两个监视窗:变量窗口和监视窗口。在 Debug 工具栏中可设置打开/关闭变量窗口和监视窗口。如果未出现 Debug 工具栏,可以在其他工具栏的任意位置右击,在弹出的快捷菜单中选择 Debug 命令即可。变量窗口有多个,其中,自动窗口(Auto)显示在当前代码行和前面代码行中使用的变量,还显示函数返回值(如果有的话)。局部变量窗口(Locals)显示对于当前上下文(通常是当前正在执行的函数)来说位于本地的变量,通过局部变量窗口可以在程序执行过程中给变量赋一个新值。在监视窗口中可以添加要监视其值的变量或表达式。

图 2-11 的自动窗口中显示出了数组 a 的首地址,单击 a 前面的加号可以看到所有元素的值。此时,每个元素的值均为-858 993 460,这是随机数,不过,如果在 VC 中用调试方式编译程序,这个值总是固定的。

③ 单步执行,在自动窗口中观察变量的值和函数返回值。

a. 按 F10 键,开始单步执行,如图 2-12 所示。黄色箭头马上指向下一条语句,而"int a[ARRSIZE]={5,4,3,2,1};"已经执行完毕了,通过自动窗口可以看到 a 中的元素值确实变成

了 5、4、3、2、1。变量 min 的值仍然为-858 993 460,因为它还没有被赋值。

把光标移动到要观察的变量处,VC 会自动显示该变量的值。图 2-12 中显示了 min 的值。

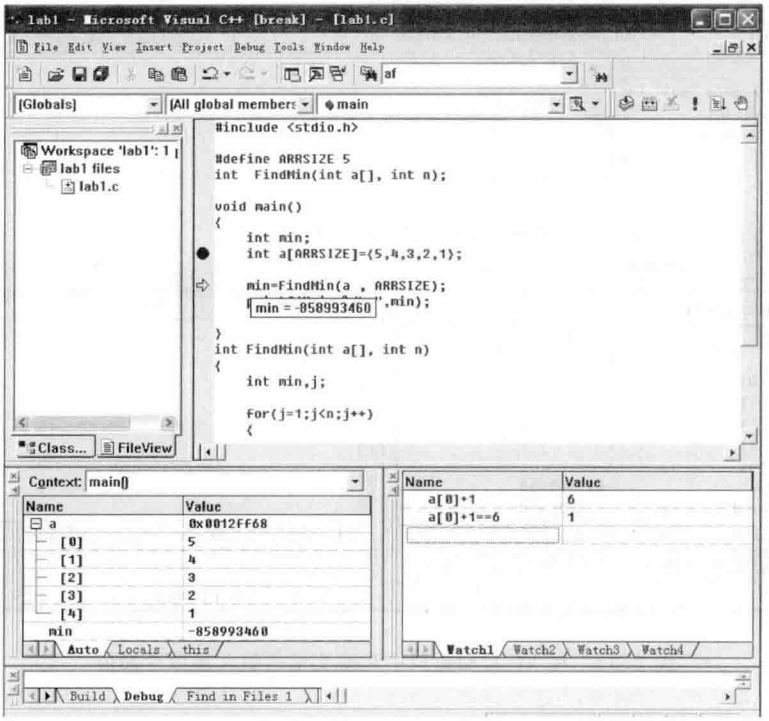


图 2-12 VC 中观察变量或表达式的值

在监视窗口内可检查更复杂的算术表达式或布尔表达式的值。例如,在第 1 行 name 列中输入 a[0]+1,在该行的 Value 列中就会显示该算术表达式的值 6;在第 2 行 name 列中输入 a[0]+1==6,在该行的 Value 列中就会显示该布尔表达式的值 1。

b. 按 F10 键,继续单步执行。黄色箭头马上指向下一条语句,如图 2-13 所示。而执行完 min=FindMin(a, ARRSIZE)语句后,却发现自动窗口中 min 的值仍然为-858 993 460,函数 FindMin 的返回值也是-858 993 460,而不是期望的值 1,可知此处存在错误。

④ 单步进入,跟踪函数调用。

FindMin(a, ARRSIZE)是函数调用语句,想要进一步分析为何执行完该语句得到的结果是错误的,则须跟踪进入 FindMin 函数。

a. 按 F10 键是不会进入函数内部单步跟踪的。此时,可按快捷键 Shift+F5 停止调试程序。

b. 重新开始调试这个程序,这次重点调试 FindMin 函数。首先将光标定位在第 9 行前,按 F9 键,删除第 9 行中设置的断点(如果设置的话),然后将光标定位在需要重点分析的第 11 行 min=FindMin(a, ARRSIZE)语句上,按快捷键 Ctrl+F10 运行到光标所在的行,程序暂停在第 11 行,如图 2-14 所示。

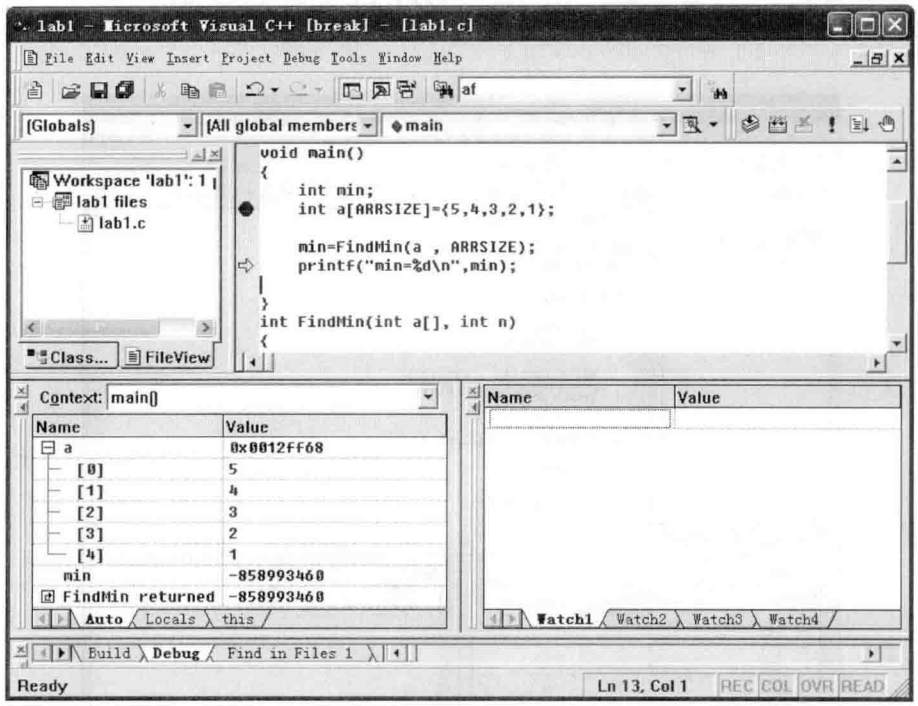


图 2-13 在 VC 自动窗口中观察变量的值和函数返回值

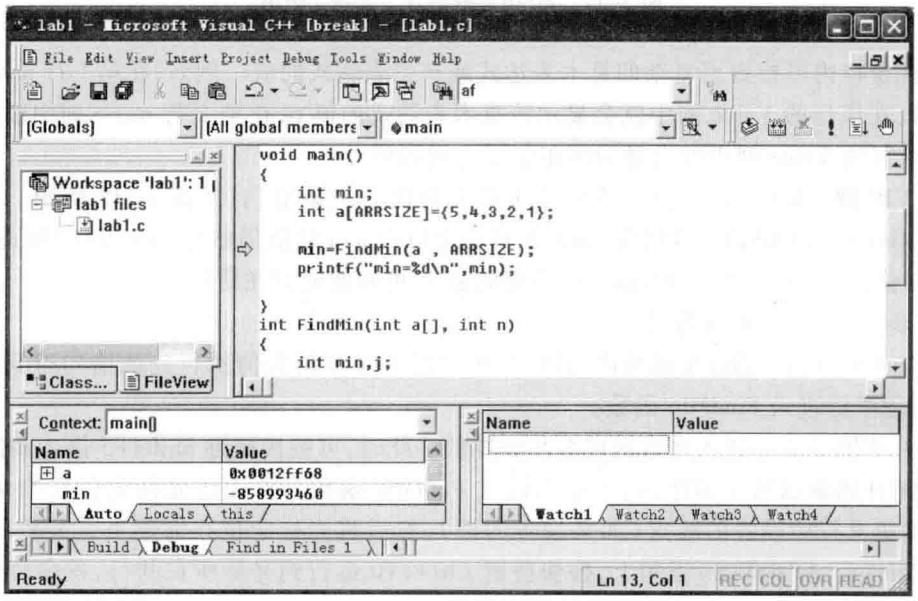


图 2-14 VC 中在需要重点跟踪的函数调用语句处暂停

c. 按 F11 键单步进入,黄色箭头暂停在函数 FindMin 上,如图 2-15 所示。

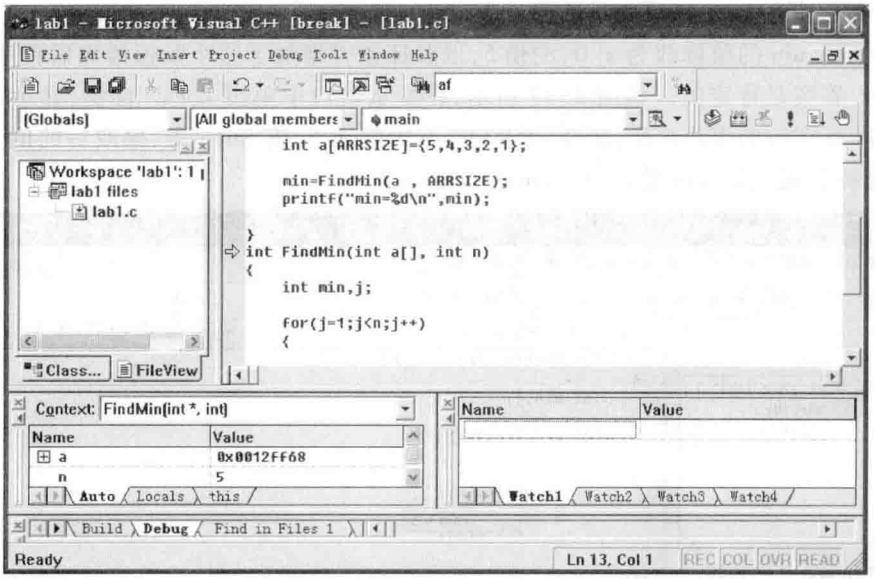


图 2-15 VC 中单步进入跟踪函数调用

⚠ 注意：以上步骤采用的是模块化调试方法，即首先定位出可疑函数，然后跟踪至函数内部。对于这个规模较小的程序而言，也可以在第一次调试时，当黄色箭头停在第 11 行函数调用语句处时，不按 F10 键，而按 F11 键，直接进入函数内部跟踪。

d. 现在，可以在 FindMin 函数中调试程序了。按 F10 键单步跟踪，如图 2-16 所示，当跟踪至第 20 行 if(a[j]<min)时，自动窗口中 j 的值为 1，a[j]的值为 4 是正确的，但是 min 的值却为 -858 993 460，不是期望的结果。此时希望 if(a[j]<min)判定的是 a[1]和 a[0]的大小关系，因此可以发现导致错误的原因可能是缺少在第 18 行 for 循环之前将 min 赋初值为 a[0]。

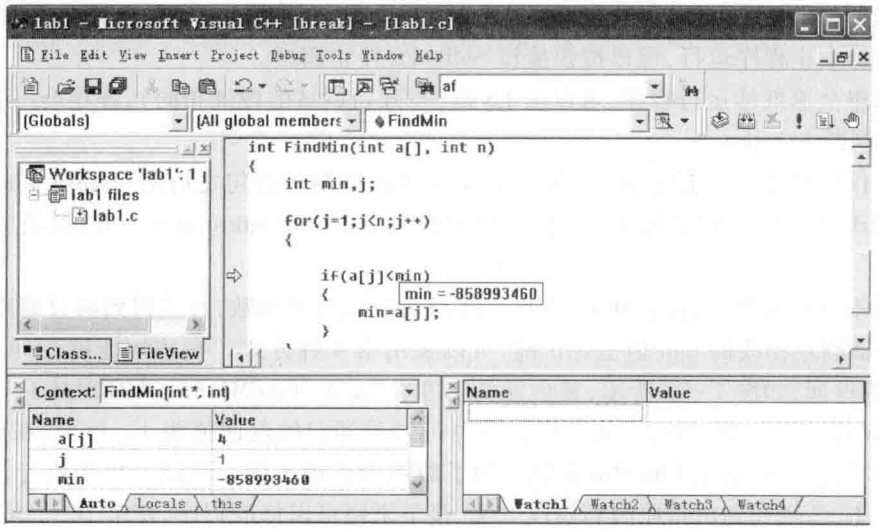


图 2-16 VC 中跟踪到函数内部调试

⑤ 在局部变量窗口中修改变量的值。

此时希望将 min 的值修改为 a[0]的值 5,以验证是否增加了这个赋值语句后程序就正确了,可不可以呢? 答案是肯定的。如图 2-17 所示,在变量窗口中单击 Locals 标签,显示局部变量窗口。单击 min 所在行的 Value 列,选择-858 993 460,输入 5,按 Enter 键,修改后的值变为红色显示,此时把光标移到变量 min 处,可见 min 的值已变为 5。

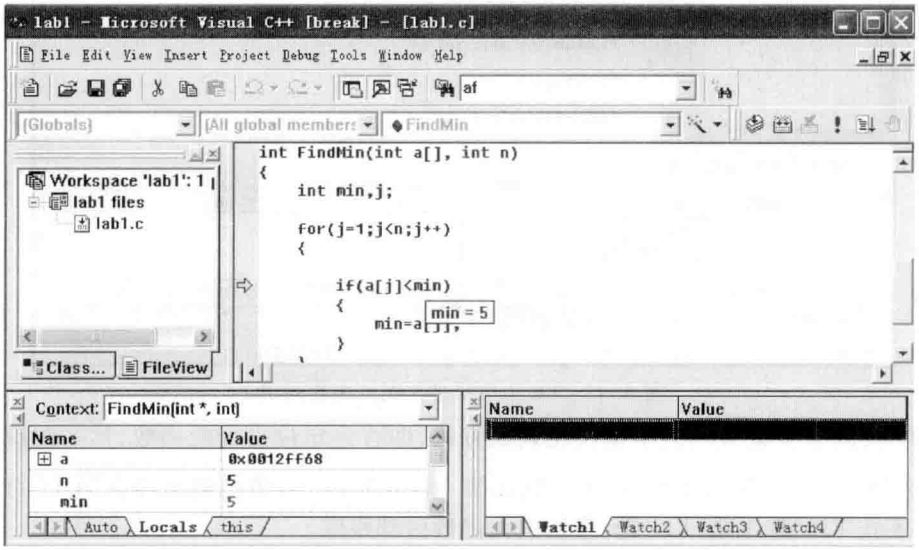


图 2-17 在 VC 局部变量窗口中修改变量的值

⑥ 控制调试的“步伐”。

本例中只执行 5 次循环,用户还可以有足够的耐心等待循环结束,如果程序中有成千上万次循环,仍然按这种单步方式一步步地执行下去是不现实的。因此,可以根据需要控制调试的“步伐”。

a. 如果想中止程序运行,可以按快捷键 Shift+F5 停止调试。

b. 如果想全速继续运行程序,可以按 F5 键(与开始调试的按键相同),程序会一直运行到结束或再次遇到断点为止。

c. 如果仅仅是想完成这个循环,那么把光标移动到循环语句之后的“return min;”这一行,按快捷键 Ctrl+F10 运行到光标所在的行,则黄色箭头停到“return min;”处,所有循环都已经完成。

d. 如果不想逐条跟踪,按快捷键 Shift+F11 就可以运行出函数,直接回到函数调用处。

为了验证刚才修改的 min 值是否正确,可以采用第 4 种方式。在按快捷键 Shift+F11 后,从 FindMin 函数返回,如图 2-18 所示,黄色箭头仍停在“min=FindMin(a, ARRSIZE);”处,但这时 FindMin 函数的调用已经结束,自动变量窗口中可以看到它的返回值为 1。可见,在给 min 赋初始值 5(a[0]的值)后,函数 FindMin 正确返回了数组中的最小值。再向下单步执行,min 值被正确输出。至此,已经找到出错原因和修改方法,接下来便可以修改程序,在第 18 行语句之前增加语句“min=a[0];”,重新编译、链接、运行程序,即可得到正确结果。

⚠ 注意:程序中的错误不同,调试步骤可能也不尽相同,需要灵活掌握。

（4）main 函数带参数的 C 语言程序的调试

以上调试的程序中 main 函数是无参数的，下面用例 2.2 介绍如何在 VC 中调试 main 函数带参数的程序。

【例 2.2】main 函数带参数的程序实例。

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    int a = 0, n, m, i;
    n = atoi(argv[1]);
    m = atoi(argv[2]);
    i = atoi(argv[3]);
    printf("%d\n", n);
    printf("%d\n", m);
    printf("%d\n", i);
    return 0;
}
```

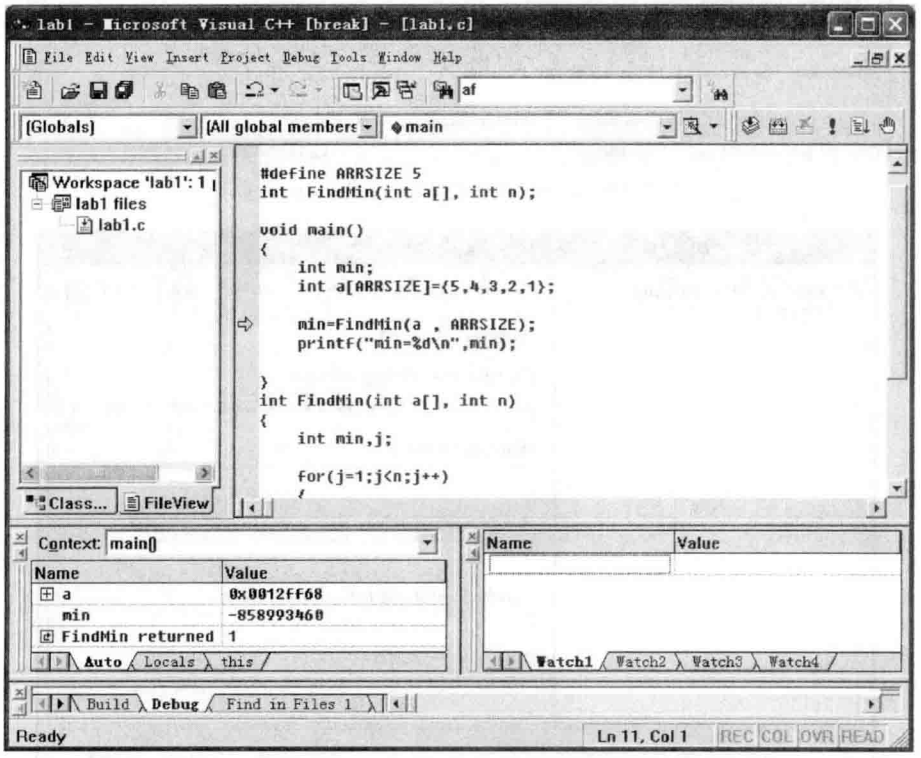


图 2-18 在自动变量窗口中查看函数的返回值

main 函数首部中的 argc 表示命令行参数的个数，命令行参数以字符串的形式存储在 argv 中。由于 main 函数不能被其他函数调用，因此命令行参数不可能在程序内部取得实际值。那么，在何处把实参赋给 main 函数呢？

在 VC 中，首先选择 Project | Settings 命令，如图 2-19 所示。然后，在弹出的对话框中选择 Debug 选项卡，在 Program arguments 文本框中输入各个参数，如图 2-20 所示，输入 3 个参数的值分别为 1、2、3，单击 OK 按钮，调试就和普通程序一样没有差别了，命令行参数会随着程序的调试运行而被自动输入。

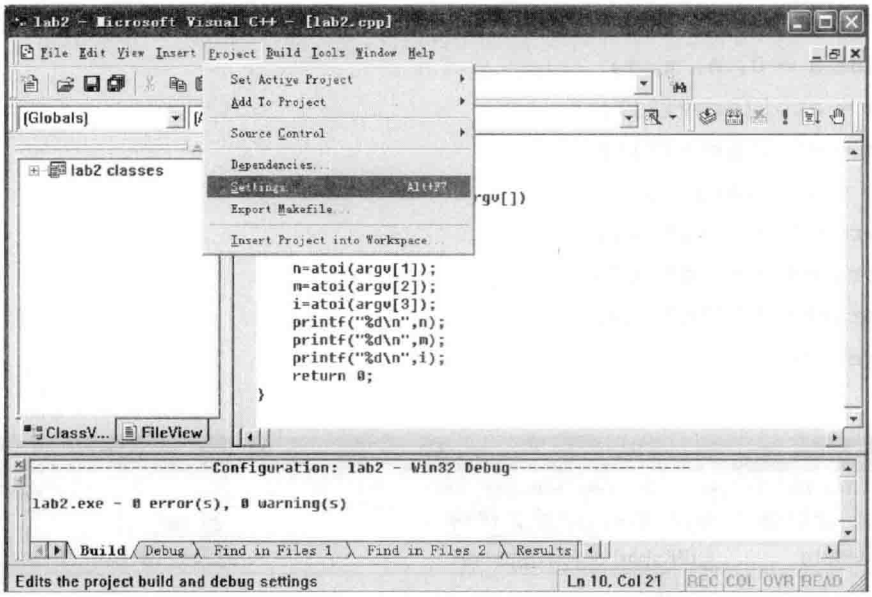


图 2-19 在 VC 中选择 Project | Settings 命令

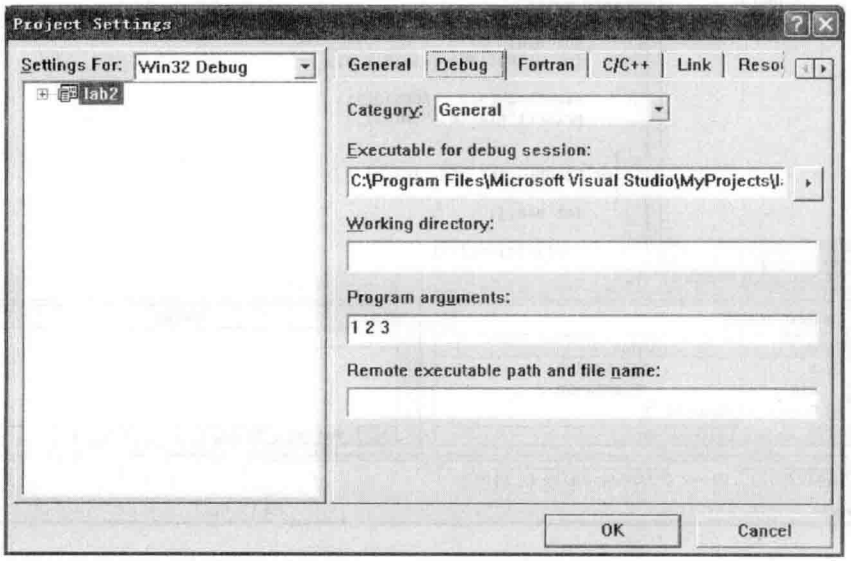


图 2-20 在 VC 中输入 main 函数命令行参数的值

2.1.2 Code::Blocks 集成开发环境的使用与调试方法简介

Code::Blocks(有时也简写成 CodeBlocks)是一个开放源码的跨平台 C/C++集成开发环境(Integrated Development Environment,IDE),由纯粹的 C/C++语言基于著名的图形界面库 wxWidgets 开发。与使用 Java 语言开发的集成开发环境,如 Eclipse、NetBeans 等相比,其运行速度要快得多,而且由于其开源性质,用户也省去了购买微软公司开发的庞大的 Visual Studio 的高昂费用。

Code::Blocks 支持 GCC、Visual C++、Inter C++等 20 多种编译器,本书以开源的 GCC 编译器作为示例,与之配对的调试器为 GDB。Code::Blocks 还支持插件,这种方式让其具备良好的可扩展性。同时,Code::Blocks 提供包括中文在内的近40 种语言显示方式。本书使用英文版本的 Code::Blocks,感兴趣的读者可以选择中文版本。

Code::Blocks 还提供了许多工程模板,包括控制台应用、DirectX 应用、动态链接库、FLTK 应用等,本书中一般使用控制台应用模板,即编写可在控制台中运行的应用程序。

另外,Code::Blocks 还支持语法彩色醒目显示、代码自动完成等许多实用的代码编辑功能,帮助用户方便、快捷地编辑 C/C++源代码。

正是基于上述优点,本书选择 Code::Blocks 作为集成开发环境介绍 C/C++程序的编辑、调试等内容。

1. Code::Blocks 的安装

目前,最新的 Code::Blocks 版本是13.12(2013 年 12 月份发布的版本),其安装版本可以从 Code::Blocks 网站(http://www.codeblocks.org)下载。该网站提供了 Windows、Linux(多种发行版)及 Mac OS X 等系统的安装文件或源文件。本书以 Windows 为例进行讲解,其他操作系统与之类似。因此,首先需要下载 Windows 版本的 Code::Blocks,可以使用以下网址直接下载:http://prdownload.berlios.de/codeblocks/codeblocks-13.12mingw-setup.exe。

Code::Blocks 支持多种编译器,但本书主要使用 Windows 下的 MinGW G++编译器,因此需要安装 MinGW。当然,前面下载的安装程序已经自带完整的 MinGW 环境,无须额外安装。

下面详细介绍 Code::Blocks 的安装步骤。双击下载的文件就可以开始安装了,主要需要注意两点。

(1) 如图 2-21 所示,选择 Full:All plugins, all tools, just everything 选项进行安装,避免一些插件没有被安装。

(2) 安装目录最好不要带有空格或汉字。这一点并不是 Code::Blocks 的限制,而是因为 MinGW 中的一些命令行工具对中文目录或带空格的目录的支持有问题。因此,安装在某分区根目录下的 CodeBlocks 目录即可,本书用例安装在 C:\CodeBlocks 中。

安装结束后,双击桌面上的 Code::Blocks 启动图标 ,或运行开始菜单中相应的程序启动 Code::Blocks。启动时,如果能看到如图 2-22 所示的启动界面,就说明安装成功了。

2. Code::Blocks 的基本配置

任何一款优秀的集成开发环境都允许用户不断地根据自己的习惯进行配置,以满足用户的

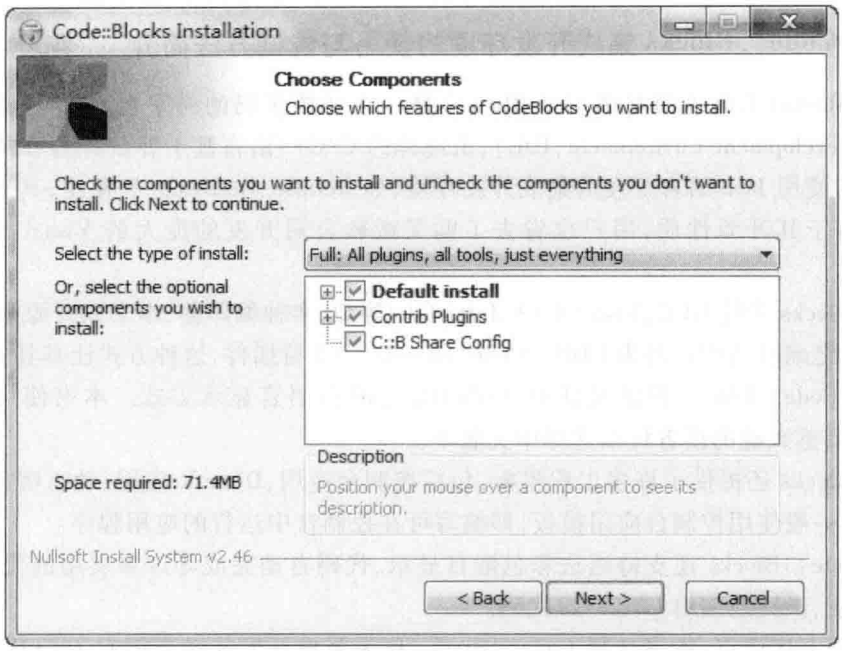


图 2-21 完整安装选项

需要。Code::Blocks可配置的内容很多,本书不可能面面俱到,因此只对一些基本的配置加以介绍。有关其他的选项,读者可以自己逐渐摸索。

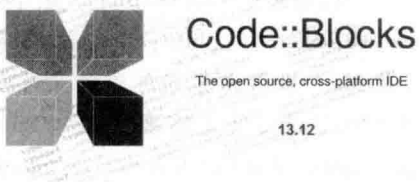


图 2-22 Code::Blocks 启动界面

(1) 配置 G++编译器及调试器

首先进入 C:\CodeBlocks\MinGW\bin 目录中,检查以下文件是否完整。

① mingw32-gcc.exe:C 的编译器。

② mingw32-g++.exe:C++的编译器。

③ ar.exe:静态库的连接器。

④ gdb.exe:调试器。

⑤ windres.exe:Windows 下的资源文件编译器。

⑥ mingw32-make.exe:制作程序。

如果以上文件都没有问题,选择 Settings|Compiler 命令,在打开的窗口中,选择 Toolchain executables(工具链执行程序)选项卡,对照图 2-23,检查配置是否正确无误。如果没有问题,现在就可以使用 Code::Blocks 来编写 C/C++代码了。

在完整安装的情况下,以上编译器的配置通常不会有问题。如果使用过程中出现不当操作,也许会由于编译器配置不正确,出现程序不能正常编译的情况,这时需要将编译器恢复至默认配置。具体操作很简单,单击图 2-23 中右上角的 Reset defaults 按钮,然后确认即可。

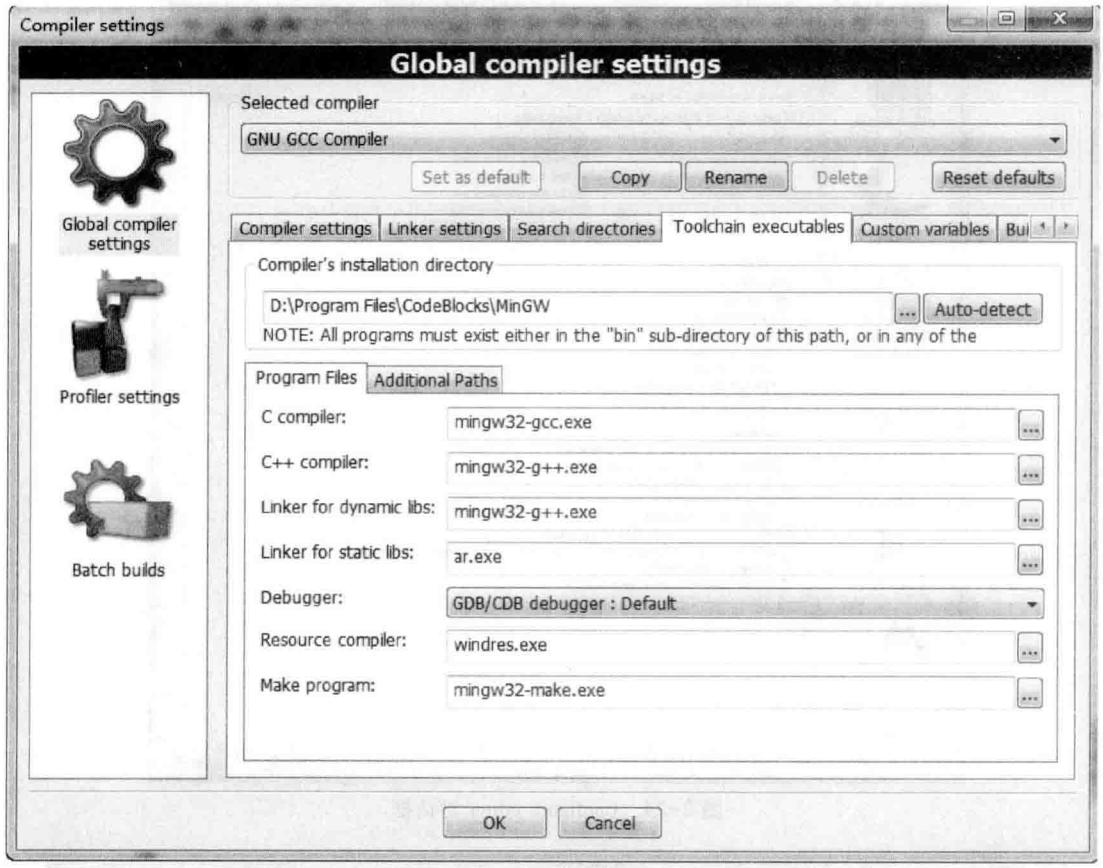


图 2-23 Toolchain executables 选项卡

(2) 配置编辑选项

若对 Code::Blocks 默认的字体和字号不满意,可以自行修改。具体操作是:选择 Settings | Editor(编辑器)命令,在打开的如图 2-24 所示的对话框中,单击 Choose 按钮即可根据用户个人的喜好选择合适的字体、字号等编辑选项。

3. 创建控制台应用程序

Code::Blocks 支持创建多种类型的程序,如动态链接库、图形界面应用程序等。本书介绍的程序多运行于控制台①,控制台是最基本的应用程序运行模式。

首先创建控制台应用程序。如图 2-25 所示,选择菜单栏中的 File | New | Project 命令,或者在 Start here 页面中单击链接 Create a new project。

之后出现新建项目的对话框,如图 2-26 所示。

① 控制台(Console),又称字符终端,是类似 DOS 的界面,只能显示字符信息。

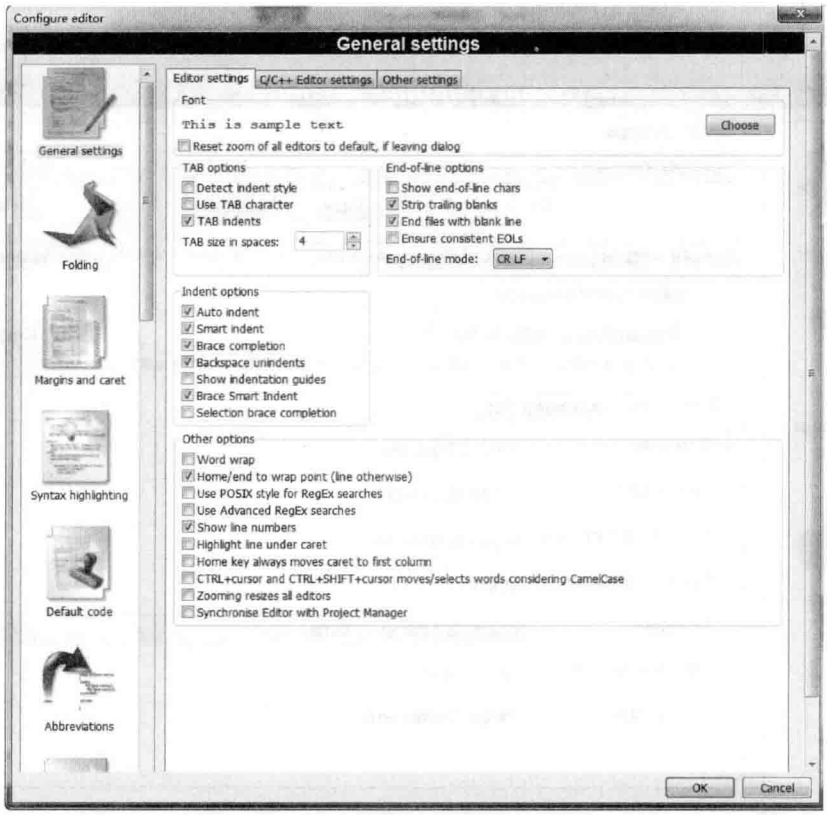


图 2-24 Configure editor 对话框

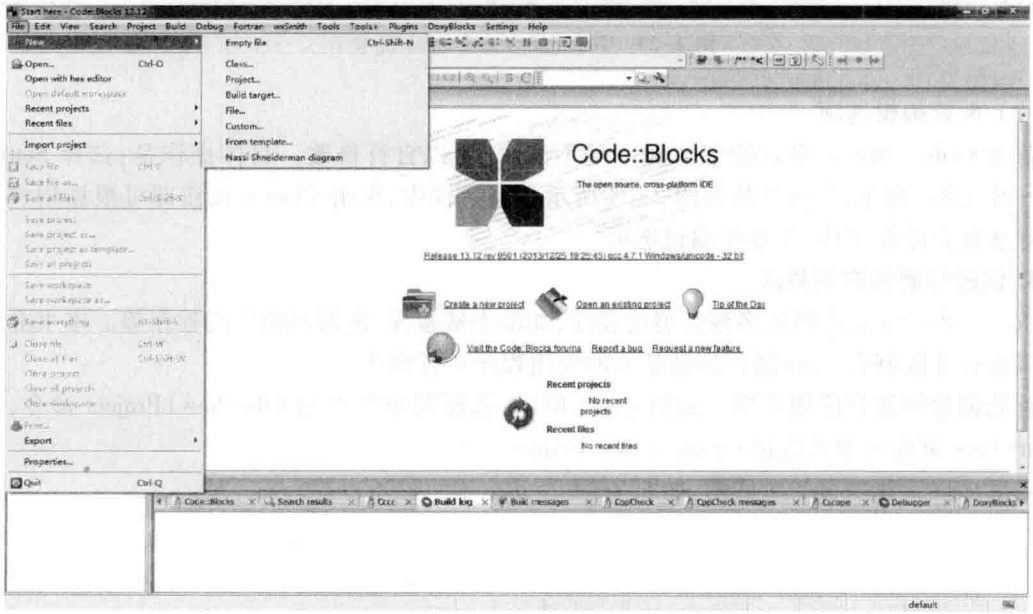


图 2-25 创建新的应用程序

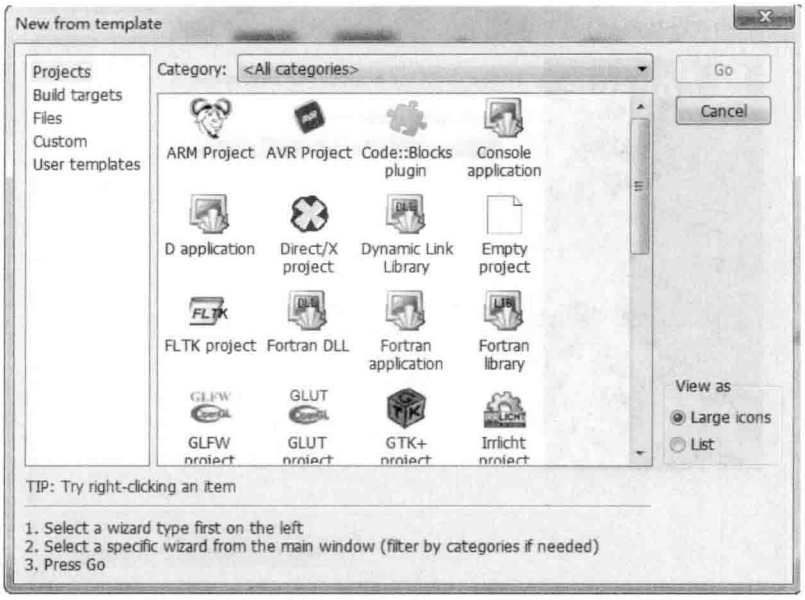


图 2-26 新项目类型选择对话框

在图 2-26 中,选中 Console application(控制台应用程序)选项,单击 Go 按钮,打开创建控制台应用程序的向导。

向导第一步相当于一个欢迎界面,如图 2-27 所示,单击 Next 按钮。

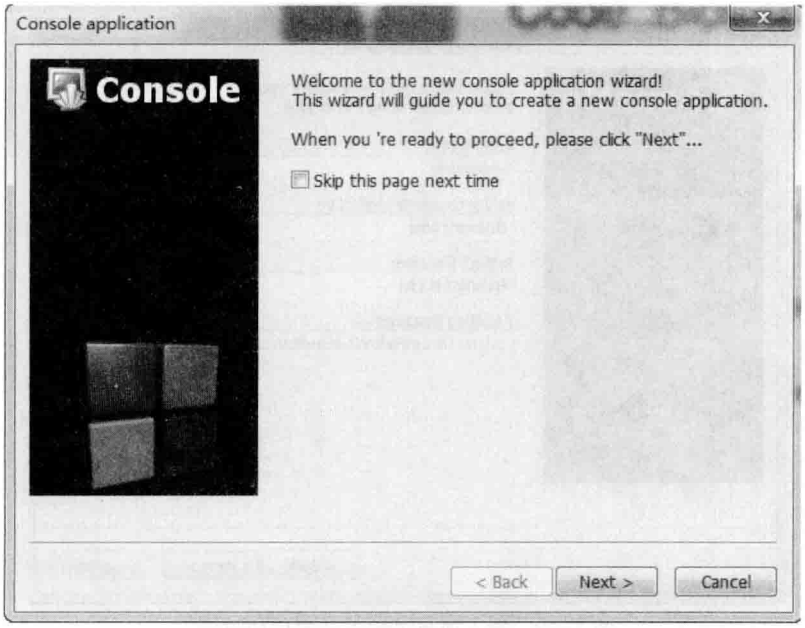


图 2-27 创建控制台应用程序的欢迎界面

在 Please make a selection 列表框中选择 C,创建 C 语言程序,如图 2-28 所示,单击 Next 按钮。

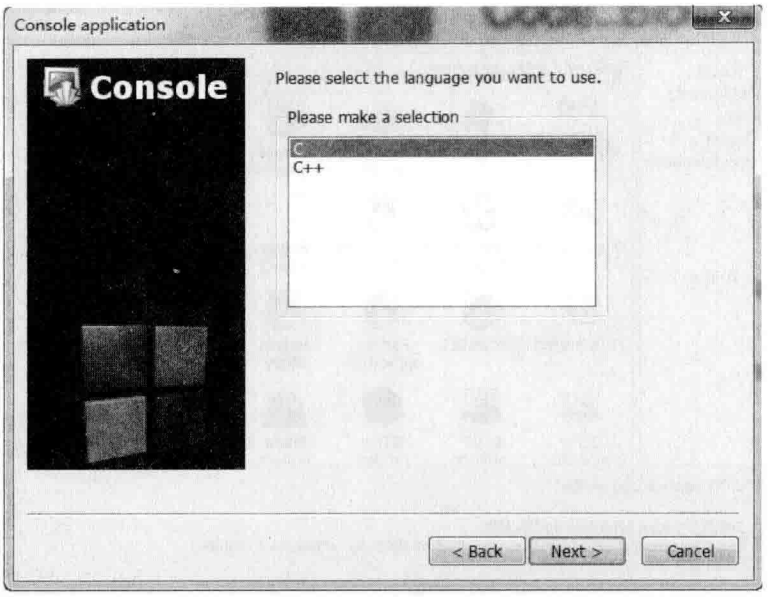


图 2-28 选择编程语言类型

在 Project title 文本框中输入项目名称,本项目将创建在以此命名的文件夹中,在此输入 HelloWorld,如图 2-29 所示。其他选项保持默认值即可。Folder to create project in 指的是项目创建于哪个文件夹中。单击 Next 按钮。

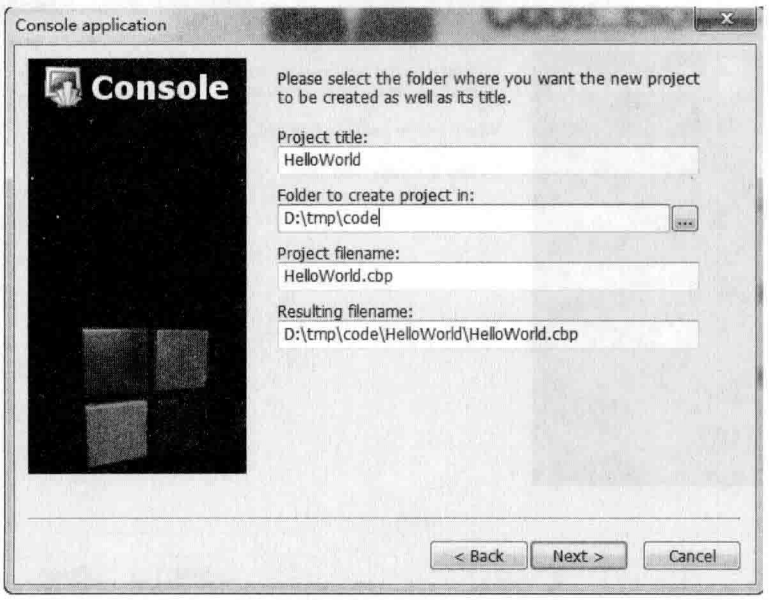


图 2-29 输入项目名称以及创建的位置

如图 2-30 所示,选择编译器为 GNU GCC Compiler(默认选项),其他选项保持默认值即可。单击 Finish 按钮,结束向导。

图 2-30 选择编译器类型

此时,Code::Blocks 左侧出现项目浏览界面,在 HelloWorld 项目下的 Sources 中找到 main.c,双击开始编辑。可以发现,Code::Blocks 已经生成了一个最简单的 Hello World 程序,如图 2-31 所示。

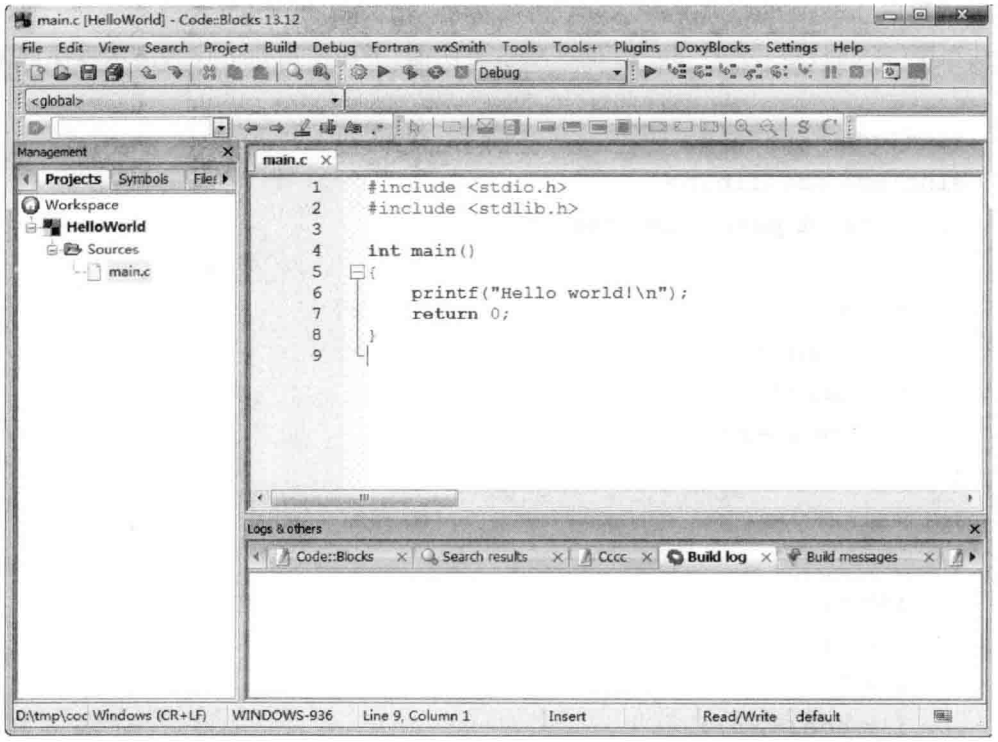


图 2-31 Code::Blocks 代码编辑界面

4. 运行控制台应用程序

选择 Build|Build and run 命令，或者按 F9 键，构建应用程序并自动运行，出现如图 2-32 所示的结果，说明 Code::Blocks 配置正确，可以开始激动人心的编程之旅了。

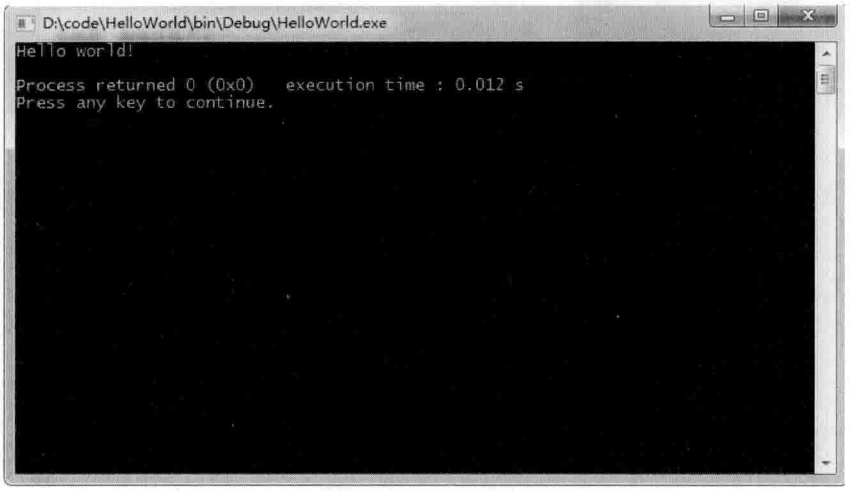


图 2-32 程序运行结果

在 Code::Blocks 中运行控制台应用程序时，控制台窗口会自动停止，并且显示程序执行所用的时间。

5. 调试程序

调试程序(Debug)是 Code::Blocks 的一项重要功能，下面使用如下代码演示程序的基本调试方法。

```
#include <stdio.h>
#include <stdlib.h>
int add(int para1, int para2)
{
    int a, b;
    a = para1;
    b = para2;
    return a + b;
}
int main()
{
    int i;
    i = 1;
    i = 10;
    i = add(3, 4);
    return 0;
```

```
}
```

(1) 设置断点

断点(Breakpoint)设置是调试器的基本功能之一,可以让程序中断在需要的地方,从而方便分析。在 Code::Blocks 中设置断点的方法很简单,在代码行号的右侧空白处单击,或在光标所在行按 F5 键,出现红色圆点后,即表示在该行成功设置断点。如图 2-33 所示,在程序的第 18 行设置了断点。单击红色圆点则可取消断点。

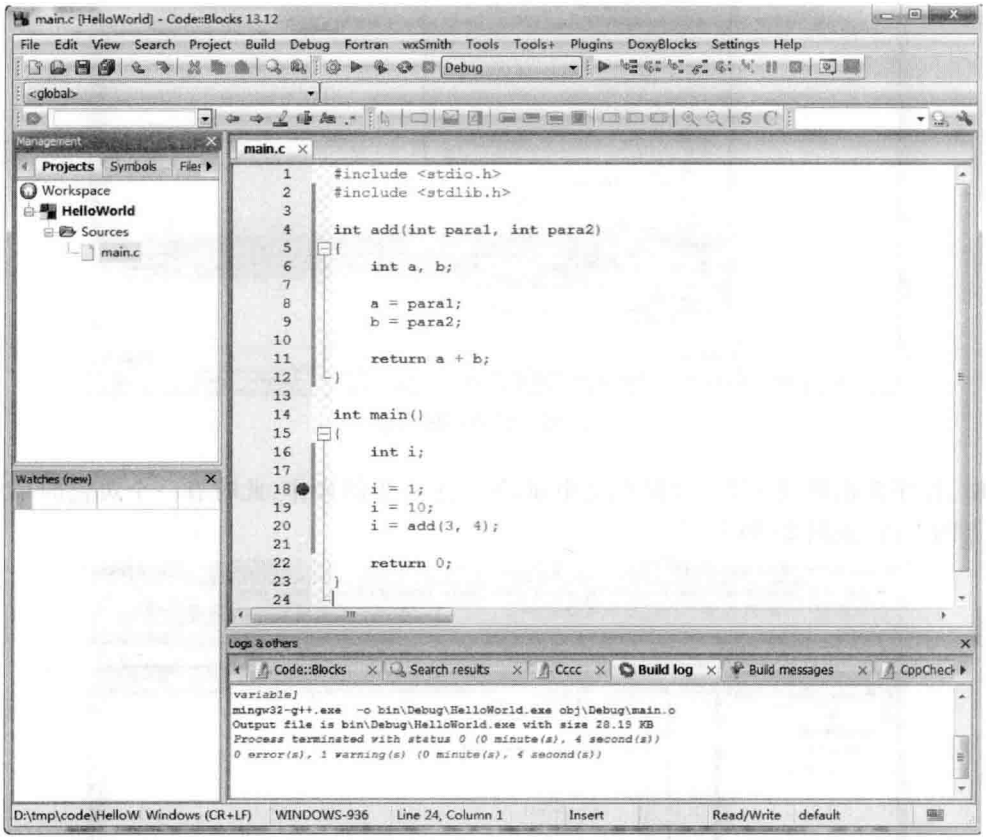


图 2-33 设置断点

(2) 开始调试

设置断点后,就可以开始调试操作了,注意此时 Build target 应为 Debug(默认选项),若为 Release,则需要改为 Debug,修改方法如图 2-34 所示。

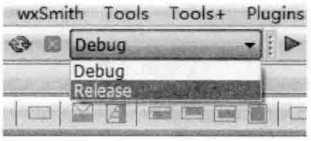


图 2-34 设置编译方式

选择 Debug|Start 命令(图 2-35),或按 F8 键开始调试。

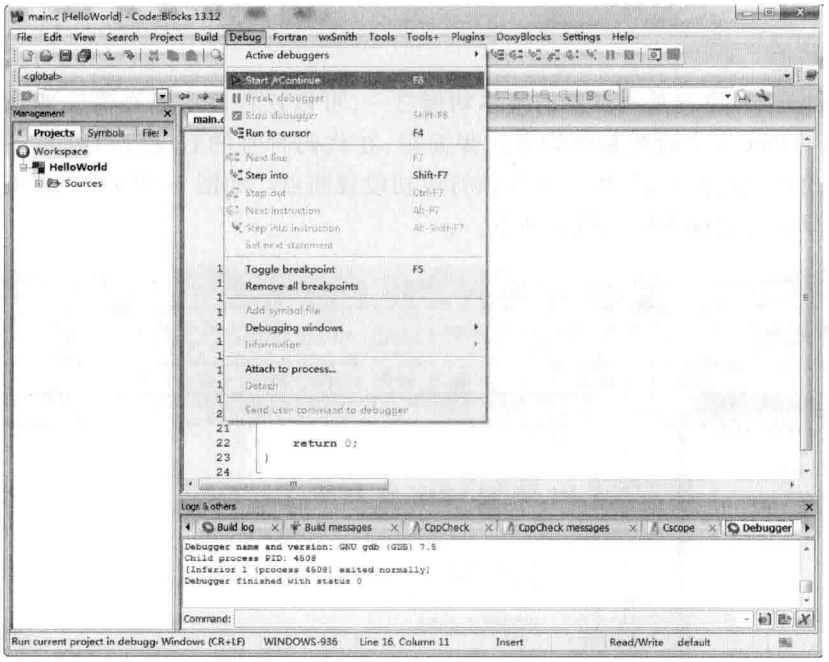


图 2-35 启动调试功能

此时，程序会在遇到的第一个断点处中断，等待进一步的操作，此时有一个黄色的小三角在红色断点圆点内，如图 2-36 所示。

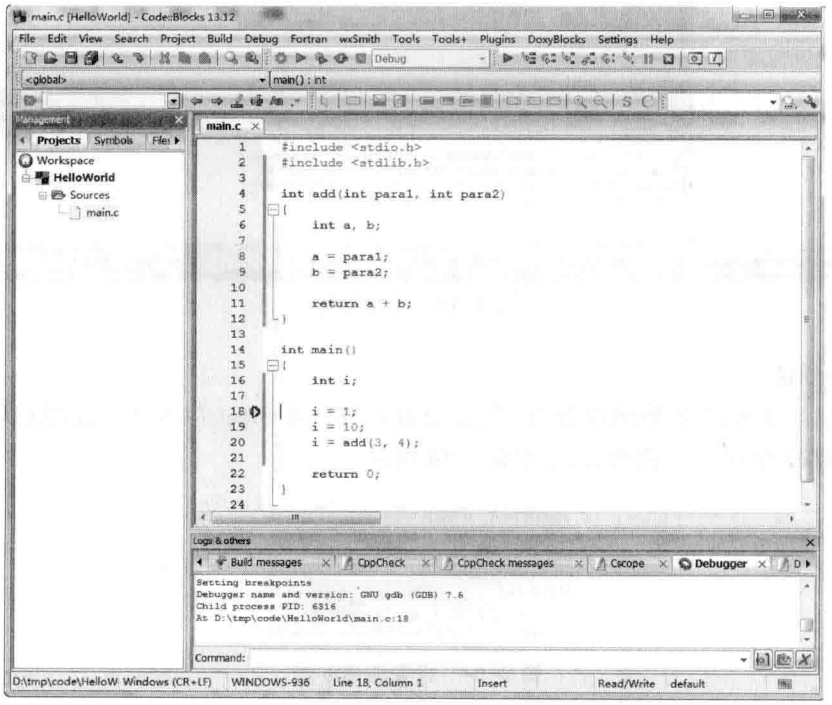


图 2-36 调试程序暂停

(3) 观察变量

程序在断点处中断后,用户更关心此时各个变量的值是否是其预想的,因此需要观察各个变量的值。单击如图 2-37 所示的工具条按钮后,选择 Watches 选项。

此时会出现如图 2-38 所示的变量观察窗口,其中显示各个局部变量(这里是变量 i)当前的值。

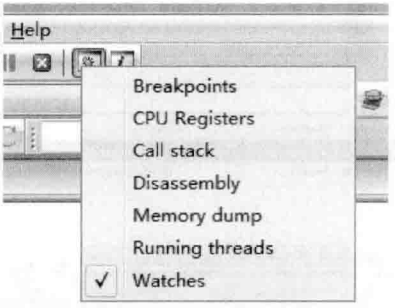


图 2-37 启动变量观察窗口

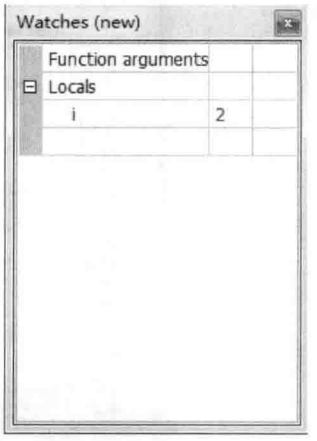


图 2-38 变量观察窗口

i 的值为 2,这是一个垃圾数,也叫不能测数。使用不同的编译器或计算机可能得到不同的值。在本程序中,要给它一个确定的数,就是把 1 赋值给它。按 F7 键或单击 Next line 按钮,开始单步执行。黄色箭头马上指向下一条语句"i = 10;"处,而"i = 1;"已经执行完毕了,通过监视窗可以看到 i 的值确实变成了 1。继续按 F7 键,黄色箭头随之逐条下行,i 的值也随之而变。光标移到"i = add(3, 4);"这一行时,此时再按 F7 键,i 的值变为 7,说明 add()函数返回了 7,黄色箭头停到"return 0;"处。

add()函数是如何返回 7 这个结果的呢? 按 F7 键是不进入函数内部单步跟踪的。为了对函数内部语句的执行情况进行跟踪,当黄色箭头停在调用它的语句处时,按快捷键 Shift+F7(或单击 Step into 按钮),黄色箭头暂停在函数 add()内,如图 2-39 所示。

图 2-39 中的 Call stack 是函数调用栈窗口,从中可以看出 main()函数调用了 add()函数,两个参数分别是 3 和 4。在其中的任意一行上右击,在弹出的快捷菜单中选择 Switch to this frame 命令,可把运行环境切换到函数的该次调用,进而查看该次调用时各个变量和参数的值。函数调用栈一般不会自动显示,需要选择调试工具条中 Debugging windows 按钮的 Call stack 选项才会出现。

现在,可以按 F7 键或快捷键 Shift+F7 在函数 add()中进行调试。监视窗中已经显示了 para1 和 para2 两个参数,观察它们的值,直观体会一下函数参数是如何对应传递的。然后按 F7 键或快捷键 Shift+F7,一步步观察函数的执行过程,直到函数返回。

add()函数返回后,停在主函数的"return 0;"处。再向下执行,程序将正常退出。

Memory dump(内存镜像)的功能也很重要,通过这项功能,能够直接观察变量在内存中的存储方式。程序暂停时,选择 Debugging windows | Memory dump 命令,打开 Memory 对话框,在

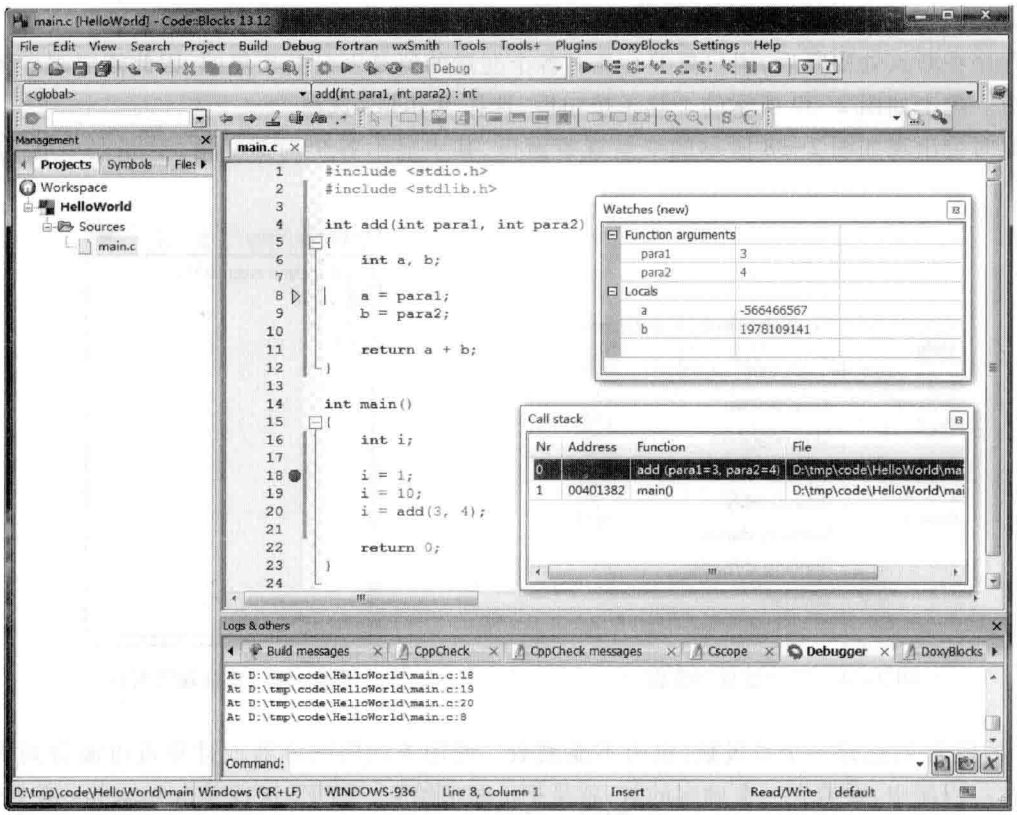


图 2-39 程序进入函数执行

Address 文本框中输入“&i”，即变量 i 的地址，此时也可以输入数组名，表示数组的首地址，按 Enter 键，则变量 i 的内存信息便按字节展现出来，如图 2-40 所示。

可见，此时变量 i 的值是 10，由于 i 是整数，在内存中占 4 个字节，由低位到高位分别是 0a 00 00 00。

(4) 其他调试工具

以下是调试工具条，分别对其中的按钮加以介绍。

从左至右各按钮的功能分别为：

① Debug/Continue：开始和继续调试，也就是之前使用的开始调试功能。另外，若程序中断在某个断点处，单击该按钮后，程序会继续执行，直到遇到下一个断点或程序执行结束。

② Run to cursor：执行程序，并且在光标所在行中断，当不想设置断点，却又想在某处中断时，可以将光标移动到想要中断的那行代码上，然后单击此按钮。

③ Next line：下一行，执行一行代码，然后在下一行中断，即使本行含有函数调用，也不会进入函数执行，而是直接跳过函数。这是最常用的功能。

④ Step into：步入，与下一行功能相对，此功能会进入函数执行。如果对函数中的程序感兴趣，可以使用此功能。

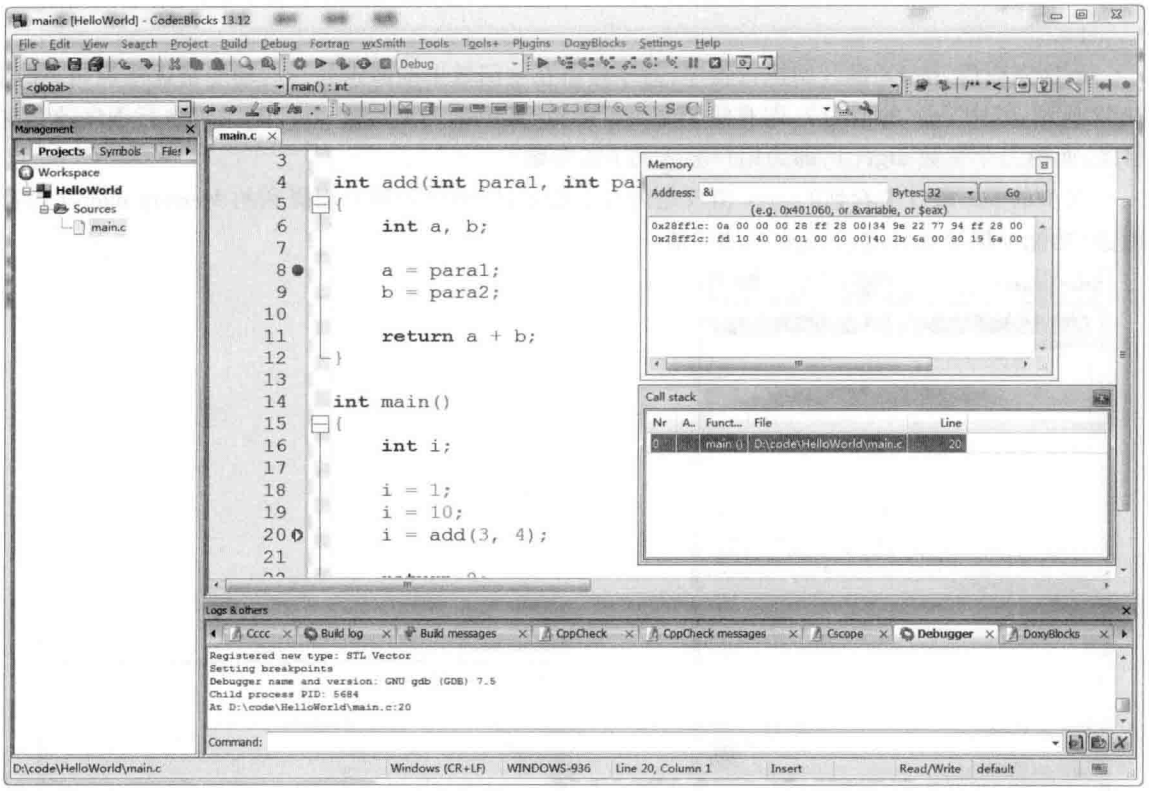


图 2-40 Memory 对话框

⑤ Step out:跳出,当想跳出正在执行的函数时,可以使用此功能。

⑥ Next instruction:下一条指令,即下一条汇编语言指令,此功能不常用。

⑦ Step into instruction:运行下一条汇编命令,如果下一条汇编命令是汇编函数调用,则步入该函数。

⑧ Break debugger:暂停调试。暂停调试程序用于检查调试程序的状态。由于程序中有断点,在断点处已自动暂停,按钮为灰色,不可点击。

⑨ Stop debugger:中止调试,如果找到错误,或者不想继续调试,可以使用此功能。

(5) 命令行参数程序调试

有一些程序在运行时需要通过命令行方式将参数传递给程序,如何在 Code::Blocks 中调试带有命令行参数的程序呢? 首先选择 Project|Set programs' arguments 命令,打开命令行参数设置对话框,设置项目运行所需的命令行参数,如图 2-41 所示,在 Program arguments 文本框中输入一个参数 test。

在 main()函数的形参列表中添加 argc、argv 参数。一般可写为:

```
int main(intargc, char* argv[])
{
```

```
    // main function code
}
```

添加完成后，按 F8 键开始执行程序，程序停留在主函数中的断点处。启动变量观察窗口，如图 2-42 所示，其中 argc 的值为 2，即有两个命令行参数，其中第一个参数 argv[0]为可执行程序的完整目录，而第二个参数 argv[1]即为用户输入的 test 参数。

在 Watches 窗口中看到的 argv 值仅是内存中的地址，可以利用上面提到的 Memory dump（内存镜像）功能根据该地址查看 argv 具体的值。

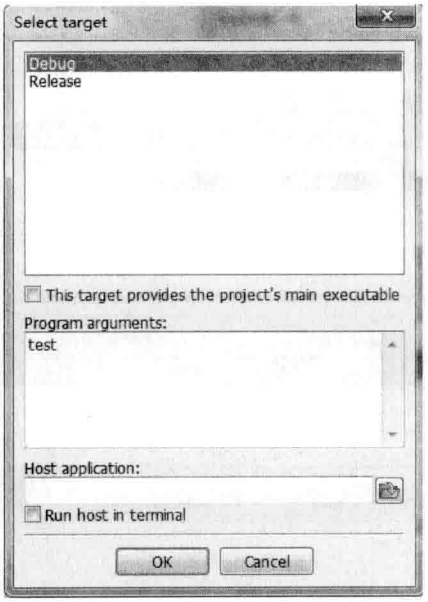


图 2-41 设置命令行参数

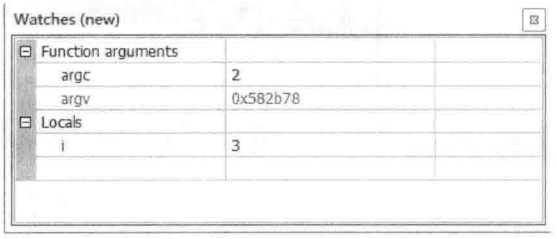


图 2-42 变量观察窗口

按照之前的步骤打开 Memory 窗口，输入 argv[0]，按 Enter 键或单击 Go 按钮，即可显示内存中的二进制代码（左侧）以及对应的字符（右侧），如图 2-43 所示。可以看到该值是程序所在的完整路径（Memory dump 默认显示 32 字节的内存信息，如果地址太长，可能显示不全。这时可以选择显示更多的字节以查看完整的路径，如选择显示 64 字节）。

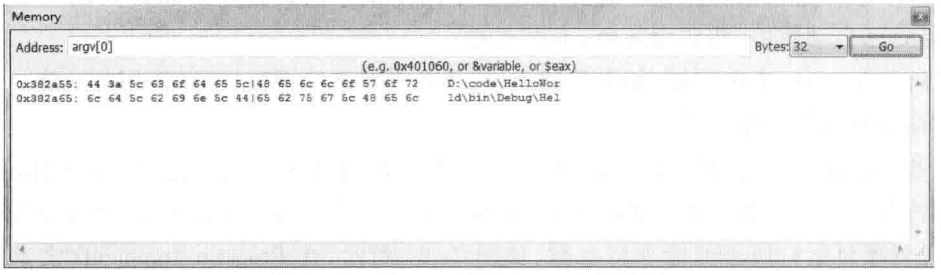


图 2-43 Memory 窗口中 argv[0]的值

相应地，输入 argv[1]可以显示第二个参数，即设置的 test 值，如图 2-44 所示。

输入 argv[2]会显示“Cannot access memory at address 0x0”错误信息，如图 2-45 所示。这是因为程序仅包含两个参数，即 argc 值为 2。

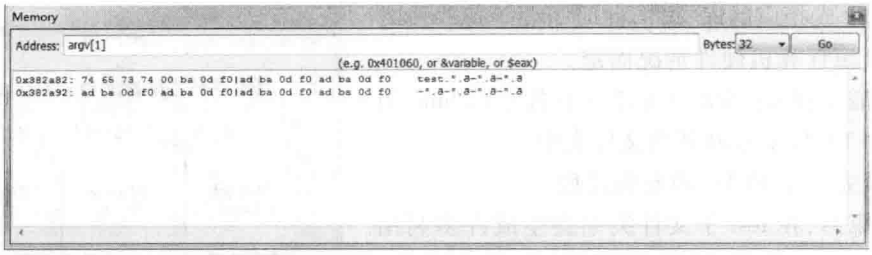


图 2-44 Memory 窗口中 argv[1]的值

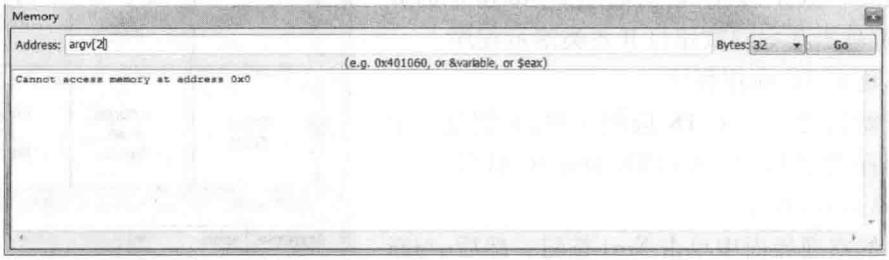


图 2-45 试图访问 argv[2]时 Memory 窗口中显示的错误信息

6. 使用 Code::Blocks 开发图形用户界面

图形用户界面(Graphical User Interface,GUI)是指采用图形方式显示的计算机操作用户界面,如 Windows 就是典型的图形用户界面。与前面介绍的控制台应用程序(又称命令行用户界面)相比,图形用户界面显示内容更丰富,操作更直观、方便。

下面介绍如何使用 Code::Blocks 来编写具有图形用户界面的应用程序。一般地,图形用户界面是操作系统提供的功能,用户可以直接调用操作系统提供的应用程序接口(Application Programming Interface,API)来编写具有图形用户界面的应用程序。然而,由于不同的操作系统提供的 API 不同,而且这些 API 往往功能比较基础、简单,因此,直接调用操作系统提供的 API 编写 GUI 程序是非常复杂的,而且这些程序不具有平台可移植性,往往只能应用于单一的平台。

为了简化 GUI 程序的编写过程,同时使得程序具有较好的可移植性,人们开发了许多功能强大的图形库,如 QT、xwWidgets 等。然而,这些图形库往往过于庞大,不适合初学者使用,因此本书以一个轻量级的图形库 FLTK(The Fast Light Tool Kit)①为例,介绍使用 Code::Blocks 进行图形开发的一般过程。

虽然称作轻量级的图形库,但是 FLTK 的功能并不弱。除了能够实现基本的 GUI 功能之外,还具有跨平台、内置 OpenGL、速度快、尺寸小等优点,适合初学者使用。下面就如何安装 FLTK、如何创建 FLTK 应用程序等加以简单介绍,更详细的使用方法读者可参阅其官方网站。

(1) FLTK 的安装

首先需要在 FLTK 的官方网站下载最新版本的 FLTK 源代码,这里以使用 1.3.x 版本为例。将下载的压缩文件解压到一个目录中,如 C:\fltk。

打开 Windows 的命令行界面,执行命令 cd C:\fltk 进入解压后的文件夹,并执行 make 命

① 可以从网址 http://www.fltk.org 下载。

令①,以编译生成相应的库文件,此时需要等待一段时间,时间长短视计算机硬件情况而定。

接下来需要在 C:\fltk 中新建一个名为 include 的文件夹,并将 FL 目录复制到该文件夹中。

至此,就完成了 FLTK 的安装过程。

安装结束后,在 test 子文件夹中会生成许多利用 FLTK 编写的演示程序,以展示 FLTK 的强大功能。运行其中的 demo.exe 程序,启动如图 2-46 所示的界面,可以通过单击其中的按钮打开各类演示程序。

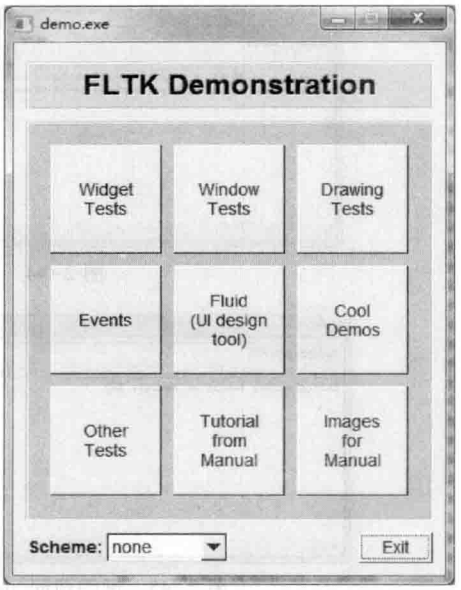


图 2-46 FLTK 的演示程序

(2) 创建 FLTK 应用程序

下面开始创建一个 FLTK 应用程序,在创建一个新的应用程序类型时,选择 FLTK project,如图 2-47 所示,然后单击 Go 按钮。

在弹出的欢迎界面中单击 Next 按钮。然后,与新建控制台应用程序一样,输入应用程序的名称以及创建的位置,单击 Next 按钮。

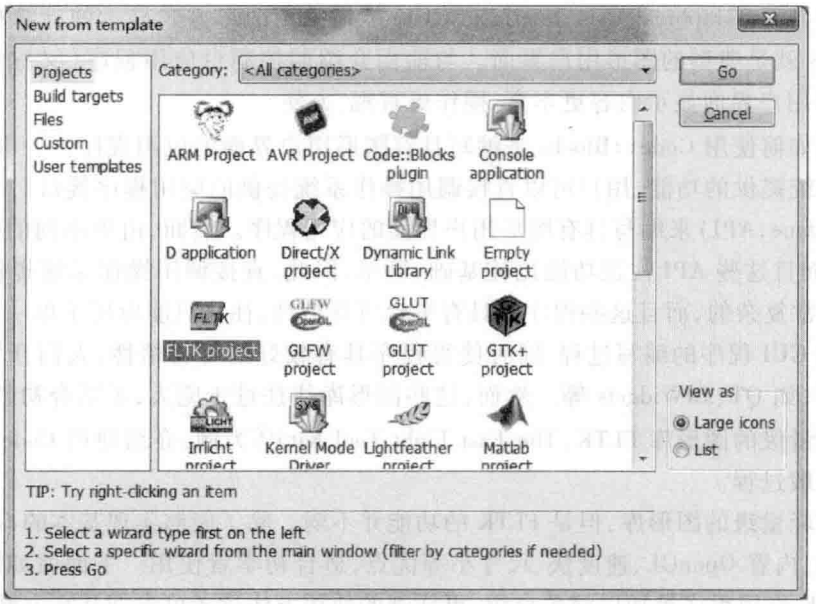


图 2-47 选择 FLTK project 应用程序类型

此时,出现如图 2-48 所示的对话框,输入 FLTK 的安装目录,如 C:\fltk。

单击 Next 按钮,出现如图 2-49 所示的对话框,选择生成 FLTK 应用程序的类型,这里选择 Simple main() example。

① 若找不到 make 命令,需要安装 MSYS 系统,可从以下网址下载:http://downloads.sourceforge.net/mingw/MSYS-1.0.11.exe。

单击 Next 按钮，出现编译器类型选择对话框，保持默认选项，单击 Finish 按钮，此时生成一个 FLTK 的项目。打开 main.cpp 文件，显示源程序，其功能是生成一个显示 Hello World 字符串的窗体程序。

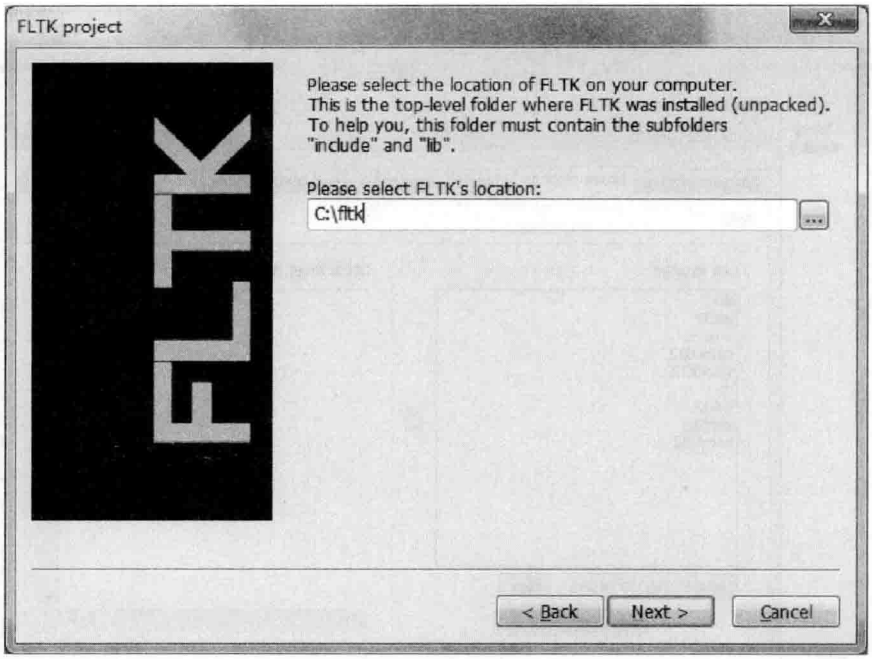


图 2-48 输入 FLTK 的安装目录

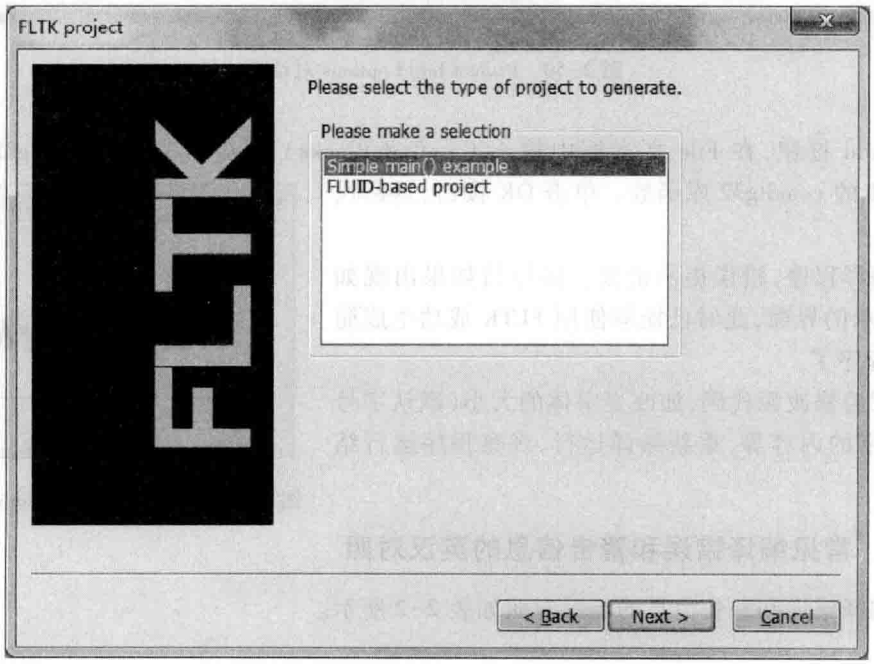


图 2-49 选择 FLTK 应用程序类型

此时，编译此程序会出现错误，提示找不到相应的库函数。下面来看一下如何让 Code::Blocks 找到相应的库函数。

选择 Project | Build options 命令，打开如图 2-50 所示的对话框，并选择 Linker settings 选项卡。

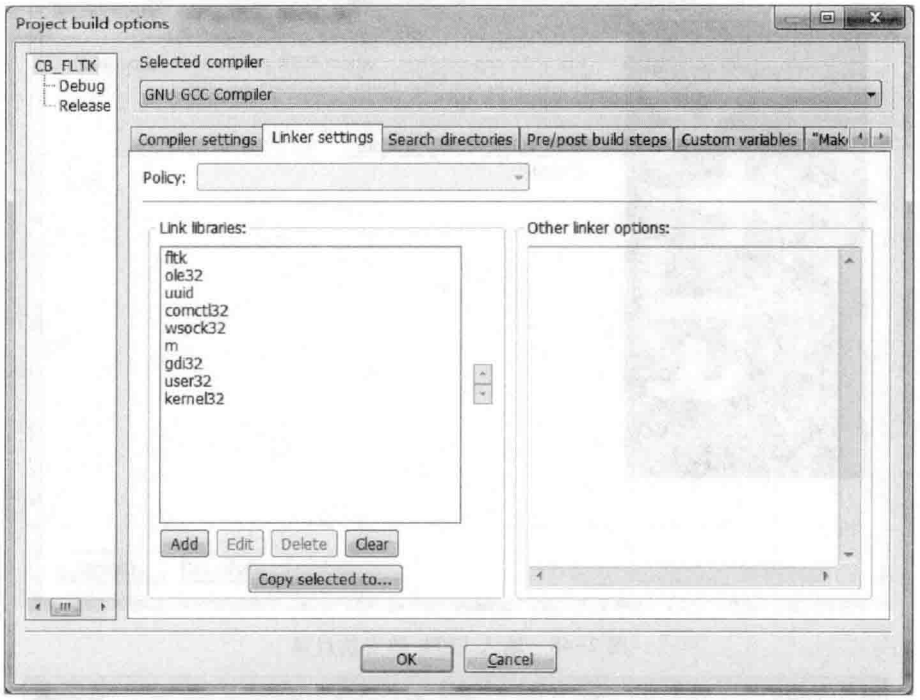


图 2-50 Project build options 对话框

单击 Add 按钮，在 File 文本框中输入 C:\CodeBlocks\MinGW\lib\libcomdlg32.a，即增加 MinGW 自带的 comdlg32 库函数。单击 OK 按钮，返回代码编辑器。

重新编译程序，错误提示消失。运行后如果出现如图 2-51 所示的界面，此时已能够使用 FLTK 成功生成简单的 GUI 程序了。

此时试着修改源代码，如改变字体的大小(默认字号是 36)、文字的内容等，重新编译运行，观察程序运行结果的变化。

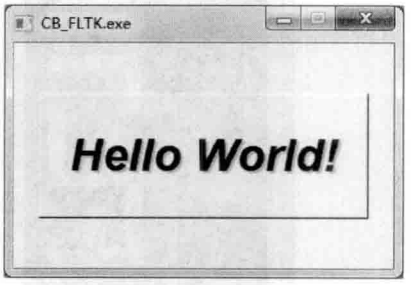


图 2-51 FLTK 的 Hello World 程序界面

2.1.3 常见编译错误和警告信息的英汉对照

常见编译错误和警告信息的英汉对照如表 2-2 所示。

表 2-2 常见编译错误和警告信息的英汉对照

常见编译错误的英文提示信息	中文含义	备注
Ambiguous symbol 'xxx'	具有二义性的符号“xxx”	两个或多个结构体的某一域名相同,在变量或表达式中引用该域名而未带结构体名时,会产生二义性,此时需要在引用时加上结构体名
Argument list syntax error	参数表出现语法错误	函数调用的参数间要以逗号隔开,并以一个右括号结束,否则出错
Array bounds missing]	数组的定界符]丢失	在源程序中定义了一个数组,但此数组没有以右方括号结束
Array size too large	数组太大	定义的数组太大,超过了可用内存空间
assignment of read-only location	对只读的内存空间进行赋值操作	
assignment from incompatible pointer type	不兼容的指针类型赋值	
Bad file name format in include directive	文件包含指令中文件名格式不正确	必须用引号或尖括号将包含文件名括起来,否则会出错
Call of non-function	调用了未定义的函数	通常是由于不正确的函数声明或拼错函数名而造成的
Cannot modify a const object	不能修改一个常量对象	通常是因为对定义为常量的对象进行赋值等非法操作而引起的
case outside of switch	case 语句出现在 switch 语句之外	通常是由于括号不匹配造成的
case statement missing:	case 语句漏掉冒号	case 语句必须包含一个以冒号结束的常量表达式,如果漏了冒号或在冒号前多了其他符号,则会出错
comparison of unsigned expression<0 is always false	对无符号表达式小于 0 的比较将永远为假	无符号表达式的值不可能小于 0,因此,若将其作为 if 后的判断条件时,将导致其分支中的语句为死语句
conflicting types for 'Fact'	Fact( )函数的函数原型冲突	通常是由于函数调用语句中实参的类型和数量与函数原型不匹配所致

续表

常见编译错误的英文提示信息	中文含义	备注
Constant expression required	需要常量表达式	数组的大小必须是常量，通常是由于#define 定义的常量拼写错误造成的
conversion from ' const double ' to ' const int ', possible loss of data	将 double 型数据赋值给 int 型 const 常量有可能出现截断误差	
Declaration syntax error	声明出现语法错误	通常是因为变量声明时丢失了某些符号或输入多余的符号而引起的，也可能是函数定义时在函数头部的行末输入了不该输入的分号引起的
default outside of switch	default 语句在 switch 语句外出现	通常是由于括号不匹配引起的
different types for formal and actual parameter	函数的实参类型与形参类型不一致	
Division by zero	除数为 0	
Do statement must have while	do 语句中必须有关键字 while	
Do while statement missing;	do-while 语句中缺少分号	
Duplicate case	case 情况不唯一	switch 语句的每个 case 必须有一个唯一的常量表达式值，否则将导致此类错误发生
enum syntax error	enum 语法错误	通常是由于 enum 说明的标识符表的格式错误引起的
Enumeration constant syntax error	枚举常量语法错误	通常是由于赋值给 enum 常量的表达式值不为常量引起的
Error Writing output file	写输出文件错误	通常是由于磁盘空间已满，无法进行写入操作而造成的

续表

常见编译错误的英文提示信息	中文含义	备注
Expression syntax error	表达式语法错误	通常是由于出现两个连续的操作符、括号不匹配或缺少括号、前一语句漏掉了分号引起的
Extra parameter in call to 'xxx'	调用“xxx”函数时出现了多余参数	通常是由于调用函数时实际参数个数多于函数定义中的参数个数所引起的
File name too long	文件名太长	通常是由于#include 指令给出的文件名太长引起的，DOS 下的文件名长度通常不能超过 64 个字符
for statement missing;	for 语句缺少分号	通常是因为 for 语句中某个表达式后的分号被误写成逗号导致的
Function call missing)	函数调用缺少右括号	通常是由于函数调用的参数表漏掉了右括号或括号不匹配造成的
function returns address of local variable	返回了局部变量的地址	
goto statement missing label	goto 语句缺少标号	
if statement missing(	if 语句缺少左括号	
if statement missing)	if 语句缺少右括号	
illegal else without matching if	非法的 else，没有能与之直接配对的 if	有可能是由于前面邻近的 if 分支的语句漏掉了花括号所致
incompatible types	不兼容的类型转换	
Invalid l-value in assignment	在赋值中出现无效的左值	通常是由于左操作数是常量而不是变量所致
left operand must be l-value	左操作数必须是左值	通常是由于左操作数是常量而不是变量所致
local variable 'b' used without having been initialized	局部变量 b 未被初始化就使用了	

续表

常见编译错误的英文提示信息	中文含义	备注
l-value specifies const object	赋值操作中的左值被指定为了 const 对象	
Mismatch number of parameters in definition	定义中参数个数不匹配	
Misplaced else	else 位置错	通常是由于 if 表达式右括号后面多了一个分号所致
not all control paths return a value	并非所有的控制分支都有返回值	
Out of memory	内存不够	
Pointer required on left side of	操作符左边必须是一个指针	
Redeclaration of 'xxx'	“xxx”重定义	通常是由于该标识符在不同位置被重复定义导致的,如函数的形参在函数内又被定义为局部变量
returning address of local variable or temporary	返回了局部变量的地址	
Size of structure or array not known	结构或数组大小未知	
Statement missing;	语句缺少分号	通常是由于前一行语句缺少分号导致的
Structure size too large	结构太大	通常是由于定义结构体类型时使用了本结构体类型来定义域名的类型所引起的
Subscription missing]	下标缺少右方括号	
Too few parameter in call to 'xxx'	调用“xxx”时参数太少	
Type mismatch in parameter 'yyy'in call to 'xxx'	调用“xxx”时参数“yyy”的数据类型不匹配	

续表

常见编译错误的英文提示信息	中文含义	备注
Type mismatch in redeclaration of ' xxx '	重定义“xxx”类型不匹配	通常是由于缺少函数原型导致了函数声明与函数定义的类型不匹配
truncation from ' const double ' to ' float '	将 double 型常量赋值给 float 型常量时将发生数据截断错误	有可能产生舍入误差
Undefined symbol ' xxx '	标识符“xxx”没有定义	通常是由于字母拼写错误使得已定义的标识符和实际使用的标识符不一致导致的，如数字 1 和小写字母 l 混淆、字母大小写混淆等
Unable to create output file ' xxx '	不能创建输出文件“xxx”	通常由于磁盘空间已满，不能创建输出文件所致
Unable to open include file ' xxx.xxx '	不能打开包含文件“xxx.xxx”	通常是由于当前路径下不存在此文件导致的
Unable to open input file ' xxx.xxx '	不能打开输入文件“xxx.xxx”	通常是由于当前路径下不存在此文件或该文件已损坏导致的
Undefined structure ' xxx '	结构“xxx”未定义	
Untermimated character constant	未终结的字符常量	通常是由于书写字符常量时丢失单引号导致的
Unterminated string	未终结的字符串	通常是由于书写字符串时丢失双引号导致的
while statement missing(	while 语句漏掉左括号	
while statement missing)	while 语句漏掉右括号	
Wrong number of arguments in of ' xxx '	调用“xxx”时参数个数错误	

2.2 实验题目与解答

2.2.1 第 2 章实验题：熟悉上机实验环境和不同类型变量的定义与赋值

熟悉 Visual C++ 6.0 和 Code::Blocks 集成开发环境，并参考例 2.1b 和例 2.2 程序，在每一行输出变量的值以后，再输出该变量所占内存的字节数。在不同的集成开发环境下运行程序，观察程序的运行结果。

实验目的：熟悉 Visual C++ 6.0 和 Code::Blocks 集成开发环境以及不同类型的变量的定义和赋值。

参考程序如下：

```
1    #include <stdio.h>
2    main()
3    {
4        int a = 1;
5        float b = 2.5;
6        char c = 'A';
7        printf("a=%d,sizeof(a)=%d\n",a,sizeof(a));
8        printf("b=%f,sizeof(b)=%d\n",b,sizeof(b));
9        printf("c=%c,sizeof(c)=%d\n",c,sizeof(c));
10       printf("End of program\n");
11   }
```

在 Visual C++ 6.0 或 Code::Blocks 下的程序运行结果为：

```
a=1,sizeof(a)=4
b=2.500000,sizeof(a)=4
c =A,sizeof(a)=1
End of program
```

2.2.2 第 3 章实验题：计算球的体积和表面积

参考例 3.4 和例 3.5 程序，分别使用宏定义和 const 常量定义 π 的值，编程计算并输出球的体积和表面积，球的半径 r 的值由用户从键盘输入。

实验目的：熟悉简单的算术运算、宏定义和 const 常量的使用。

参考答案：球的表面积计算公式为 $S=4\pi R^2$，球的体积计算公式为 $V=\frac{4}{3}\pi R^3$。

方法 1：使用宏定义定义 π 的值。参考程序如下：

```
1    #include <math.h>
2    #include <stdio.h>
3    #define PI 3.14159        /* 定义宏常量 PI */
```

```
main()
{
    double r,surface,volume;
    printf("Input r:");
    scanf("%lf",&r);
    surface = 4 * PI * pow(r,2);
    volume = 4.0/3.0 * PI * pow(r,3);
    printf("surface=%lf\n",surface);
    printf("volume=%lf\n",volume);
}
```

或者

```
#include <stdio.h>
#define PI 3.14159        /* 定义宏常量 PI */
main()
{
    double r,surface,volume;
    printf("Input r:");
    scanf("%lf",&r);
    surface = 4 * PI * r * r;
    volume = 4.0/3.0 * PI * r * r * r;
    printf("surface=%lf\n",surface);
    printf("volume=%lf\n",volume);
}
```

方法 2:使用 const 常量定义 π 的值。参考程序如下:

```
#include <math.h>
#include <stdio.h>
const double pi=3.14159;        /* 定义双精度实型的 const 常量 pi */
main()
{
    double r,surface,volume;
    printf("Input r:");
    scanf("%lf",&r);
    surface = 4 * pi * pow(r,2);
    volume = 4.0/3.0 * pi * pow(r,3);
    printf("surface=%lf\n",surface);
    printf("volume=%lf\n",volume);
}
```

或者

```
#include <stdio.h>
const double pi=3.14159;        /* 定义双精度实型的 const 常量 pi */
main()
{
    double r,surface,volume;
    printf("Input r:");
    scanf("%lf",&r);
    surface = 4 * pi * r * r;
    volume = 4.0/3.0 * pi * r * r * r;
    printf("surface=%lf\n",surface);
    printf("volume=%lf\n",volume);
}
```

程序的运行结果为：

```
Input r:5↙
surface=314.159000
volume=523.598333
```

2.2.3 第 4 章实验题：大小写英文字母转换

参考例 4.2 程序，编程从键盘输入一个小写英文字母，将其转换为大写英文字母，将转换后的大写英文字母及其十进制的 ASCII 码值显示到屏幕上。

实验目的：熟悉字符与字符的 ASCII 码值的输入/输出方法。

参考程序如下：

```
#include <stdio.h>
main()
{
    char ch;
    printf("Press a key and then press Enter:");
    ch = getchar();
    ch = ch-32;
    printf("%c,%d\n",ch,ch);   /* 分别输出变量 ch 中的字符及其 ASCII 码值 */
}
```

程序的运行结果如下：

```
Press a key and then press Enter: a↙
A,65
```

2.2.4 第5章实验题：身高预测、体型判断

1. 身高预测

每个父母都关心自己孩子成人后的身高，据有关生理卫生知识与数理统计分析表明，影响小孩成人后的身高的因素包括遗传、饮食习惯与体育锻炼等。小孩成人后的身高与其父母的身高和自身的性别密切相关。

设 faHeight 为父亲的身高，moHeight 为母亲的身高，身高预测公式为：

男性成人时身高 = (faHeight+moHeight) × 0.54 cm

女性成人时身高 = (faHeight × 0.923+moHeight)/2 cm

此外，如果喜爱体育锻炼，那么身高可增加 2%；如果有良好的卫生饮食习惯，那么身高可增加 1.5%。

试编程从键盘输入用户的性别(用字符型变量 sex 存储，输入字符 F 表示女性，输入字符 M 表示男性)、父母身高(用实型变量存储，faHeight 为父亲的身高，moHeight 为母亲的身高)、是否喜爱体育锻炼(用字符型变量 sports 存储，输入字符 Y 表示喜爱，输入字符 N 表示不喜爱)、是否有良好的饮食习惯(用字符型变量 diet 存储，输入字符 Y 表示良好，输入字符 N 表示不好)等条件，利用给定公式和身高预测方法对身高进行预测。

实验目的：熟悉 if 语句、关系运算符和逻辑运算符以及不同类型数据的输入/输出方法。

参考程序如下：

```
#include <stdio.h>
main()
{
    char sex;              /* 孩子的性别 */
    char sports;           /* 是否喜欢体育运动 */
    char diet;             /* 是否有良好的饮食习惯 */
    float myHeight;        /* 孩子身高 */
    float faHeight;        /* 父亲身高 */
    float moHeight;        /* 母亲身高 */
    printf("Are you a boy(M) or a girl(F)?");
    scanf(" %c",&sex);     /* 在%c 前加一个空格,将存于缓冲区中的回车符读走 */
    printf("Please input your father's height(cm):");
    scanf("%f",&faHeight);
    printf("Please input your mother's height(cm):");
    scanf("%f",&moHeight);
    printf("Do you like sports(Y/N)?");
    scanf(" %c",&sports);/* %c 前加一个空格,读走缓冲区中的回车符 */
    printf("Do you have a good habit of diet(Y/N)?");
    scanf(" %c",&diet); /* 在%c 前加一个空格,读走缓冲区中的回车符 */
    if(sex == 'M'||sex == 'm')
```

```
21          myHeight=(faHeight+moHeight) * 0.54;
22      else
23          myHeight=(faHeight * 0.923+moHeight)/2.0;
24      if(sports == 'Y'||sports == 'y')
25          myHeight = myHeight * (1+0.02);
26      if(diet == 'Y'||diet == 'y')
27          myHeight = myHeight * (1+0.015);
28      printf("Your future height will be %.0f(cm) \n",myHeight);
29  }
```

程序的运行结果如下：

```
Are you a boy(M) or a girl(F)? F↙
Please input your father's height(cm):182↙
Please input your mother's height(cm):162↙
Do you like sports(Y/N)? N↙
Do you have a good habit of diet(Y/N)? Y↙
Your future height will be 167(cm)
```

2. 体型判断

医务工作者经广泛的调查和统计分析，根据身高与体重因素给出了以下按“体指数”进行体型判断的方法：

$$\text{体指数 } t = \text{体重 } w / (\text{身高 } h)^2 \quad (w \text{ 单位为 kg}, h \text{ 单位为 m})$$

当 $t<18$ 时，为低体重；

当 t 介于 18 和 25 之间时，为正常体重；

当 t 介于 25 和 27 之间时，为超重体重；

当 $t \geqslant 27$ 时，为肥胖。

分别用 if 语句和 if-else 语句编程，从键盘输入你的身高 h 和体重 w，根据上述给定的公式计算体指数 t，然后判断你的体重属于何种类型。

实验目的：熟悉 if 语句、if-else 语句、关系运算符和逻辑运算符以及程序测试方法。

参考程序 1 如下：

```
1   #include <stdio.h>
2   main()
3   {
4       float h,w,t;
5       printf("Please enter h,w:");
6       scanf("%f,%f",&h,&w);
7       t = w/(h * h);
8       if(t<18)
9           printf("t=%f\tLower weight!  \n",t);
```

```
10      if(t>=18&&t<25)
11          printf("t=%f\tStandard weight! \n",t);
12      if(t>=25&&t<27)
13          printf("t=%f\tHigher weight! \n",t);
14      if(t>=27)
15          printf("t=%f\tToo fat! \n",t);
16  }
```

参考程序 2 如下：

```
1   #include <stdio.h>
2   main()
3   {
4       float h,w,t;
5       printf("Please enter h,w:");
6       scanf("%f,%f",&h,&w);
7       t = w/(h * h);
8       if(t < 18)
9           printf("t=%f\tLower weight! \n",t);
10      else if(t < 25)
11          printf("t=%f\tStandard weight! \n",t);
12      else if(t < 27)
13          printf("t=%f\tHigher weight! \n",t);
14      else
15          printf("t=%f\tToo fat! \n",t);
16  }
```

程序的 4 次测试结果如下：

```
① Please enter h,w: 1.64,45↙
  t=16.731113 Lower weight!
② Please enter h,w: 1.64,60↙
  t=22.308151 Standard weight!
③ Please enter h,w: 1.64,70↙
  t=26.026175 Higher weight!
④ Please enter h,w: 1.64,75↙
  t=27.885187 Too fat!
```

2.2.5 第 6 章实验题：国王的许诺、小学生计算机辅助教学系统

1. 国王的许诺

相传国际象棋是古印度舍罕王的宰相达依尔发明的。舍罕王十分喜欢象棋，决定让宰相自己选择何种赏赐。这位聪明的宰相指着 8×8 共 64 格的象棋盘说："陛下，请您赏给我一些麦子

吧，就在棋盘的第1个格子中放1粒，第2格中放2粒，第3格中放4粒，以后每一格都比前一格增加一倍，依此放完棋盘上的64个格子，我就感恩不尽了。”舍罕王让人扛来一袋麦子，他要兑现他的许诺。请问：国王能兑现他的许诺吗？分别采用两种累加方法（直接计算累加的通项和利用前项计算后项）编程计算舍罕王共需要多少麦子赏赐他的宰相，这些麦子相当于多少立方米（已知 $1m^3$ 麦子约1.42e8粒）。

实验目的：熟悉循环语句、累加累乘算法、通项的构成规律。

参考答案：第1格放1粒，第2格放2粒，第3格放 $4=2^2$ 粒，…，第 i 格放 2^{i-1} 粒，因此总粒数为 $sum=1+2+2^2+2^3+\cdots+2^{63}$。这是一个典型的等比数列求和问题，由于循环次数已知，因此可用计数控制的循环来实现。

寻找累加项的构成规律是累加求和问题求解的关键。一般地，有两种方法：**一种是寻找一个通式来表示累加项；另一种是通过寻找前项与后项之间的联系，利用前项计算后项。**

方法1：采用第1种方法计算累加项，累加项的通式可以表示为：$term=2^{n-1}$，即 term=pow(2,n-1)，令 n 从1变化到64，从第1项开始计算累加和，执行64次累加运算：sum=sum+term，取sum初值为0。参考程序如下：

```
#include <math.h>
#include <stdio.h>
#define CONST 1.42e8          /* 定义符号常量 CONST 的值为 1.42e8 */
main()
{
    int n;
    double term,sum = 0;
    for(n=1; n<=64; n++)
    {
        term = pow(2,n-1);
        sum = sum+term;
    }
    printf("sum=%e\n",sum);              /* 打印总麦粒数 */
    printf("volum=%e\n",sum/CONST);      /* 打印折合的总麦粒体积数 */
}
```

方法2：采用第2种方法计算累加项，即通过前项计算后项的方法得到累加项，由于后项均为前项的2倍，于是可采用累乘的方法计算累加项的通项：term=term * 2，取term初值为1，从第2项开始计算累加项term，执行63次累加运算：sum=sum+term，由于事先已将第1项累加到了sum中，所以这里取sum初值为1。参考程序如下：

```
#include<math.h>
#include <stdio.h>
#define CONST 1.42e8          /* 定义符号常量 CONST 的值为 1.42e8 */
main()
{
```

```
6         int n;
7         double term = 1,sum = 1;
8         for(n=2; n<=64; n++)
9         {
10            term = term * 2;   /* 根据后项是前项的 2 倍计算累加项 */
11            sum = sum+term;
12        }
13        printf("sum=%e\n",sum);     /* 打印总麦粒数 */
14        printf("volum=%e\n",sum/CONST);     /* 打印折合的总麦粒体积数 */
15    }
```

程序的运行结果为：

```
sum=1.844674e+019
volum=1.299066e+011
```

这个数值如此庞大，是舍罕王绝对没有预料到的，它相当于全世界若干世纪的全部小麦数量。看来舍罕王失算了，他无法兑现自己的诺言。

2. 小学生计算机辅助教学系统

计算机在教育中的应用常被称为**计算机辅助教学**(Computer-Aided Instruction，CAI)。试编写一个程序来帮助小学生学习乘法。参考本章介绍的猜数游戏(例 6.7、例 6.11 和例 6.12)程序，按下列任务要求以循序渐进的方式编程。

任务 1：程序首先随机产生两个 1~10 之间的正整数，在屏幕上打印出问题，例如：

```
6 * 7 =?
```

然后让学生输入答案。程序检查学生输入的答案是否正确。若正确，则打印“Right!”，然后问下一个问题；否则打印“Wrong! Please try again.”，提示学生重做，**直到答对为止**。

任务 2：在任务 1 的基础上，当学生回答错误时，**最多给 3 次重做的机会**，3 次仍未做对，则显示“Wrong! You have tried three times! Test over!”，程序结束。

任务 3：在任务 1 的基础上，**连续做 10 道乘法运算题，不给机会重做**，若学生回答正确，则显示“Right!”，否则显示“Wrong!”。10 道题全部做完后，按每题 10 分统计并输出总分，同时为了记录学生能力提高的过程，再输出学生的回答正确率(即答对题数除以总题数得到的百分比)。

任务 4：在任务 3 的基础上，通过**计算机随机产生 10 道四则运算题**，两个操作数为 1~10 之间的随机数，运算类型为随机产生的加、减、乘、整除中的任意一种，不给机会重做，如果学生回答正确，则显示“Right!”，否则显示“Wrong!”。10 道题全部做完后，按每题 10 分统计总得分，然后**打印出总分和学生的回答正确率**。

任务 5：在任务 4 的基础上，为使学生通过反复练习熟练掌握所学内容，在学生完成 10 道运算题后，若回答正确率低于 75%，则重新做 10 道题，**直到回答正确率高于 75%时才退出程序**。

任务 6：开发一个 CAI 系统所要解决的另一个问题是学生疲劳的问题。消除学生疲劳的一种办法就是通过改变人机对话界面来吸引学生的注意力。在任务 5 的基础上，使用随机数产生函数产生一个 1~4 之间的随机数，配合使用 switch 语句和 printf( )函数调用，**为学生输入的每一**

个正确或者错误的答案输出不同的评价。

对于正确答案,可在以下 4 种提示信息中选择一个进行显示:

```
Very good!
Excellent!
Nice work!
Keep up the good work!
```

对于错误答案,可在以下 4 种提示信息中选择一个进行显示:

```
No. Please try again.
Wrong. Try once more.
Don't give up!
Not correct.Keep trying.
```

实验目的:熟悉计数控制的循环、条件控制的循环以及嵌套循环。

任务 1 的参考程序如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
main()
{
    int a,b,answer;
    int flag = 0;                    /* 置标志变量为假 */
    srand(time(NULL));
    a = rand() % 10 + 1;
    b = rand() % 10 + 1;
    do{
        printf("%d * %d=? \n", a, b);
        scanf("%d", &answer);
        if(answer == a * b)
        {
            printf("Right! \n");
            flag = 1;                /* 做对,则将标志变量置为真 */
        }
        else
        {
            printf("Wrong! Please try again.\n");
        }
        }while(flag ! = 1);          /* 循环,直到做对(标志变量为真)为止 */
}
```

程序的运行结果为:

```
3 * 9=?
21↙
Wrong! Please try again.
3 * 9=?
27↙
Right!
```

任务 2 的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
main()
{
    int a,b,answer;
    int flag = 0;                    /* 置标志变量为假 */
    int wrongTimes=0;
    srand(time(NULL));
    a = rand() % 10 + 1;
    b = rand() % 10 + 1;
    do{
        printf("%d * %d=? \n", a, b);
        scanf("%d", &answer);
        if(answer == a * b)
        {
           printf("Right! \n");
           flag = 1;                 /* 做对,则将标志变量置为真 */
        }
        else
        {
           wrongTimes++;
           if(wrongTimes<3)
              printf("Wrong! Please try again.\n");
           else
              printf("Wrong! You have tried three times! Test over!");
        }
    }while(flag !=1 && wrongTimes<3);/* 未做对且未超过 3 次时继续循环 */
}
```

程序的两次测试结果为：

① 6 * 10=?

6↙

Wrong! Please try again.

6 * 10=?

10↙

Wrong! Please try again.

6 * 10=?

80↙

Wrong! You have tried three times! Test over!

② 3 * 5=?

15↙

Right!

任务 3 的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
main()
{
    int a,b,answer;
    int i;
    int rightNumber = 0;
    srand(time(NULL));
    a = rand() % 10 + 1;
    b = rand() % 10 + 1;
    for(i=0; i<10; i++)
    {
        a = rand()%10+1;
        b = rand()%10+1;
        printf("%d * %d=? \n", a, b);
        scanf("%d", &answer);
        if(answer == a * b)
        {
            printf("Right! \n");
            rightNumber++;
        }
        else
```

```
24          {
25              printf("Wrong! \n");
26          }
27      }
28      printf("Total score is %d\n", rightNumber * 10);
29      printf("Rate of correctness is %d%% \n", rightNumber * 10);
30  }
```

程序的运行结果为：

```
1 * 8=?
8↙
Right!
2 * 2=?
4↙
Right!
3 * 1=?
3↙
Right!
5 * 8=?
40↙
Right!
6 * 6=?
36↙
Right!
10 * 9=?
90↙
Right!
6 * 7=?
41↙
Wrong!
2 * 10=?
20↙
Right!
6 * 4=?
24↙
Right!
2 * 5=?
10↙
```

```
Right!
Total score is 90
Rate of correctness is 90%
```

任务 4 的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
main()
{
    int a,b,userAnswer,systemAnswer,op,i,rightNumber=0;
    char opChar;
    srand(time(NULL));
    for(i=0; i<10; i++)
    {
        a = rand() % 10 + 1;
        b = rand() % 10 + 1;
        op = rand() % 4 + 1;
        switch(op)              /* 根据运算符的不同执行不同的运算 */
        {
            case 1:opChar = '+';
                   systemAnswer = a + b;
                   break;
            case 2:opChar = '-';
                   systemAnswer = a - b;
                   break;
            case 3:opChar = '*';
                   systemAnswer = a * b;
                   break;
            case 4:opChar = '/';
                   systemAnswer = a / b;
                   break;
            default:printf("Unknown operator!");
        }
        printf("%d%c%d=? \n", a, opChar, b);
        scanf("%d", &userAnswer);
        if(userAnswer == systemAnswer)
        {
            printf("Right! \n");
```

```
35              rightNumber++;
36          }
37          else
38          {
39              printf("Wrong! \n");
40          }
41      }
42      printf("Total score is %d\n", rightNumber * 10);
43      printf("Rate of correctness is %d%% \n", rightNumber * 10);
44  }
```

程序的运行结果为：

```
4-8=?
-4↙
Right!
6/9=?
1↙
Wrong!
5/6=?
0↙
Right!
7+7=?
14↙
Right!
7-2=?
5↙
Right!
1+3=?
4↙
Right!
10 * 8=?
80↙
Right!
8+1=?
9↙
Right!
2-1=?
1↙
Right!
```

```
6+10=?
16↙
Right!
Total score is 90
Rate of correctness is 90%
```

任务 5 的参考程序如下：

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
main()
{
    int a,b,userAnswer,systemAnswer,op,i,rightNumber=0,flag=0;
    char opChar;
    srand(time(NULL));
    do{
        for(i=0;i<10;i++)
        {
            a = rand() % 10 + 1;
            b = rand() % 10 + 1;
            op = rand() % 4 + 1;
            switch(op)    /* 根据运算符的不同执行不同的运算 */
            {
                case 1:opChar = '+';
                       systemAnswer = a + b;
                       break;
                case 2:opChar = '-';
                       systemAnswer = a - b;
                       break;
                case 3:opChar = '*';
                       systemAnswer = a * b;
                       break;
                case 4:opChar = '/';
                       systemAnswer = a/b;
                       break;
                default:printf("Unknown operator!");
            }
            printf("%d%c%d=? \n", a, opChar,b);
            scanf("%d", &userAnswer);
```

```
33              if(userAnswer==systemAnswer)
34              {
35                  printf("Right! \n");
36                  rightNumber++;
37              }
38              else
39              {
40                  printf("Wrong! \n");
41              }
42          }
43          printf("Total score is %d\n", rightNumber * 10);
44          printf("Rate of correctness is %d%% \n", rightNumber* 10);
45          if(rightNumber * 10 < 75)
46          {
47              printf("Once Again! \n");
48              rightNumber = 0;
49              flag = 1;
50          }
51      }while(flag);
52  }
```

程序的运行结果为：

```
4-8=?
4↙
Wrong!
6/9=?
1↙
Wrong!
5/6=?
1↙
Wrong!
7+7=?
17↙
Wrong!
7-2=?
2↙
Wrong!
1+3=?
4↙
```

```
Right!
10*8=?
10↙
Wrong!
8+1=?
10↙
Wrong!
2-1=?
1↙
Right!
6+10=?
60↙
Wrong!
Total score is 20
Rate of correctness is 20%
Once Again!
7+6=?
13↙
Right!
2+2=?
4↙
Right!
10*1=?
10↙
Right!
10-4=?
6↙
Right!
5*2=?
10↙
Right!
1+3=?
4↙
Right!
5*10=?
50↙
Right!
```

```
8-1=?
7↙
Right!
2+1=?
3↙
Right!
6-10=?
-4↙
Right!
Total score is 100
Rate of correctness is 100%
```

任务6的参考程序如下：

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
main()
{
    int a,b,userAnswer,systemAnswer,op,i,rightType,wrongType;
    char opChar;
    int rightNumber = 0,flag = 0;
    srand(time(NULL));
    do{
        for(i=0;i<10;i++)
        {
            a = rand() % 10 + 1;
            b = rand() % 10 + 1;
            op = rand() % 4 + 1;
            rightType=rand()%4+1;
            wrongType=rand()%4+1;
            switch(op)      /* 根据运算符的不同执行不同的运算 */
            {
                case 1:opChar = '+';
                       systemAnswer = a + b;
                       break;
                case 2:opChar = '-';
                       systemAnswer = a - b;
                       break;
                case 3:opChar = '*';
```

```
27                      systemAnswer = a * b;
28                      break;
29                  case 4:opChar = '/';
30                      systemAnswer = a / b;
31                      break;
32                  default:printf("Unknown operator!");
33              }
34              printf("%d%c%d=? \n",a,opChar,b);
35              scanf("%d",&userAnswer);
36              if(userAnswer==systemAnswer)
37              {
38                  switch(rightType)
39                  {
40                      case 1:printf("Very good! \n");
41                          break;
42                      case 2:printf("Excellent! \n");
43                          break;
44                      case 3:printf("Nice work! \n");
45                          break;
46                      case 4:printf("Keep up the good work! \n");
47                          break;
48                      default:printf("Wrong type!");
49                  }
50                  rightNumber++;
51              }
52              else
53              {
54                  switch(wrongType)
55                  {
56                      case 1:printf("No. Please try again.\n");
57                          break;
58                      case 2:printf("Wrong. Try once more.\n");
59                          break;
60                      case 3:printf("Don't give up! \n");
61                          break;
62                      case 4:printf("Not correct. Keep trying.\n");
63                          break;
64                      default:printf("Wrong type!");
```

```
                }
            }
        }
        printf("Total score is %d\n", rightNumber * 10);
        printf("Rate of correctness is %d %% \n", rightNumber * 10);
        if(rightNumber * 10 < 75)
        {
            printf("Once Again! \n");
            rightNumber = 0;
            flag = 1;
        }
    }while(flag);
}
```

程序的运行结果为：

```
4+8=?
10↙
Not correct. Keep trying.
7+7=?
14↙
Excellent!
2-7=?
5↙
Wrong. Try once more.
10*8=?
80↙
Nice work!
8/1=?
8↙
Very good!
6+10=?
16↙
Keep up the good work!
7*6=?
42↙
Excellent!
2+3=?
5↙
```

```
Nice work!
10*2=?
20↙
Keep up the good work!
10-4=?
6↙
Very good!
Total score is 80
Rate of correctness is 80%
```

2.2.6 第7章实验题：素数探求、小学生计算机辅助教学系统

1. 素数探求

素数(Prime Number)，又称为**质数**，它是不能被1和它本身以外的其他整数整除的正整数。按照这个定义，负数、0和1都不是素数，而17之所以是素数，是因为除了1和17以外，它不能被2~16之间的任何整数整除。

任务1：试商法是最简单的判断素数的方法。用 $i=2\sim m-1$ 之间的整数去试商，若存在某个 m 能被1与 m 本身以外的整数 i 整除(即余数为0)，则 m 不是素数，若上述范围内的所有整数都不能整除 m，则 m 是素数。采用试商法，分别用goto语句、break语句和采用设置标志变量并加强循环测试等3种方法编写素数判断函数IsPrime()，从键盘任意输入一个整数 m，判断 m 是否为素数，如果 m 是素数，则按"%d is a prime number\n"格式打印该数是素数，否则按"%d is not a prime number\n"格式打印该数不是素数。分析哪一种方法的可读性更好。

任务2：用数学的方法可以证明，不能被 $2\sim\sqrt{m}$(取整)之间的数整除的数，也必定不能被 $\sqrt{m}+1\sim m$ 之间的任何整数整除。因此，采用试商法只要测试 $2\sim\sqrt{m}$ 之间的整数不能整除 m 即可判断 m 是素数。根据这个性质，修改函数IsPrime( )，编程完成任务1。

任务3：从键盘任意输入一个整数 n，编程计算并输出 $1\sim n$ 之间的所有素数之和。

任务4：从键盘任意输入一个整数 m，若 m 不是素数，则计算并输出其所有的因子(不包括1)，例如，对于16，输出2、4、8；否则输出"No divisor! It is a prime number"。

任务5：如果一个正整数 m 的所有小于 m 的不同因子(包括1)加起来正好等于 m 本身，那么就称它为**完全数(Perfect Number)**。例如，6就是一个完全数，因为6=1+2+3。试编写一个判断完全数的函数IsPerfect()，然后判断从键盘输入的整数是否是完全数。

任务6：从键盘任意输入一个整数 m，若 m 不是素数，则对 m 进行质因数分解，并将 m 表示为质因数从小到大顺序排列的乘积形式输出，否则输出"It is a prime number"。例如，用户输入90时，程序输出90=2*3*3*5；用户输入13时，程序输出"It is a prime number"。

实验目的：熟悉函数设计、循环的控制方法以及程序测试方法，了解什么是边界条件测试，体会模块化程序设计和函数复用的好处和意义。

任务1的参考程序如下：

用goto语句编写的程序如下：

```
#include<stdio.h>
int IsPrime(int x);
int main()
{
    int m;
    printf("Input m:");
    scanf("%d", &m);
    if(IsPrime(m))/* 素数判定 */
        printf("%d is a prime number\n", m);
    else
        printf("%d is not a prime number\n", m);
    return 0;
}
/* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
int IsPrime(int x)
{
    int i;
    if(x <= 1) return  0;/* 负数、0 和 1 都不是素数 */
    for(i=2;i<=x-1;i++)
    {
        if(x%i==0) goto End;/* 若能被整除,则不是素数 */
    }
End:return i<=x-1 ? 0 : 1;
}
```

用 break 语句编写的程序如下:

```
#include <stdio.h>
int IsPrime(int x);
int main()
{
    int m;
    printf("Input m:");
    scanf("%d", &m);
    if(IsPrime(m))      /* 素数判定 */
        printf("%d is a prime number\n", m);
    else
        printf("%d is not a prime number\n", m);
```

```
    return 0;
}
/* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
int IsPrime(int x)
{
    int i;
    if(x <= 1) return 0;        /* 负数、0 和 1 都不是素数 */
    for(i=2;i<=x-1;i++)
    {
        if(x%i==0) break;       /* 若能被整除,则不是素数 */
    }
    return i<=x-1 ? 0 : 1;
}
```

不采用 goto 和 break 语句编写的程序如下:

```
#include <stdio.h>
int IsPrime(int x);
int main()
{
    int m;
    printf("Input m:");
    scanf("%d", &m);
    if(IsPrime(m))      /* 素数判定 */
        printf("%d is a prime number\n", m);
    else
        printf("%d is not a prime number\n", m);
    return 0;
}
/* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数,若返回 1,则表示是素数 */
int IsPrime(int x)
{
    int i;
    if(x <= 1) return 0;        /* 负数、0 和 1 都不是素数 */
    for(i=2;i<=x-1;i++)
    {
        if(x%i==0) return 0;    /* 若能被整除,则不是素数 */
    }
```

```
23        return 1;
24    }
```

采用设置标志变量并加强循环测试的方法编写程序如下：

```
1     #include <stdio.h>
2     int IsPrime(int x);
3     int main()
4     {
5         int m;
6         printf("Input m:");
7         scanf("%d", &m);
8         if(IsPrime(m))          /* 素数判定 */
9             printf("%d is a prime number\n", m);
10        else
11            printf("%d is not a prime number\n", m);
12        return 0;
13    }
14    /* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
15    int IsPrime(int x)
16    {
17        int i,flag = 1;
18        if(x <= 1) flag = 0;        /* 负数、0 和 1 都不是素数 */
19        for(i=2;i<=x-1&&flag;i++)
20        {
21            if(x%i == 0) flag = 0;        /* 若能被整除,则不是素数 */
22        }
23        return flag;
24    }
```

任务 2 的参考程序如下：

```
1     #include <stdio.h>
2     #include <math.h>
3     int IsPrime(int x);
4     int main()
5     {
6         int m;
7         printf("Input m:");
8         scanf("%d", &m);
```

```
9         if(IsPrime(m))     /* 素数判定 */
10            printf("%d is a prime number\n", m);
11        else
12            printf("%d is not a prime number\n", m);
13        return 0;
14    }
15    /* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
16    int IsPrime(int x)
17    {
18        int i,flag = 1;
19        int squareRoot=(int)sqrt(x);
20        if(x <= 1) flag = 0;        /* 负数、0 和 1 都不是素数 */
21        for(i=2;i<=squareRoot&&flag;i++)
22        {
23            if(x%i == 0) flag = 0;     /* 若能被整除,则不是素数 */
24        }
25        return flag;
26    }
```

程序的 5 次测试结果如下:

① 4↙

```
4 is not a prime number
```

② 7↙

```
7 is a prime number
```

③ 1↙

```
1 is not a prime number
```

④ 0↙

```
0 is not a prime number
```

⑤ -1↙

```
-1 is not a prime number
```

任务 3 的参考程序如下:

```
1     #include <stdio.h>
2     #include <math.h>
3     int IsPrime(int x);
4     int main()
5     {
6         int m, n, sum = 0;
7         printf("Input n:");
8         scanf("%d",&n);
```

```
        for(m=1; m<=n; m++)
        {
            if (IsPrime(m))     /* 素数判定 */
            {
                sum += m;
            }
        }
        printf("sum=%d\n",sum);
        return 0;
    }
    /* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
    int IsPrime(int x)
    {
        int i, flag = 1;
        int squareRoot=(int)sqrt(x);
        if(x <= 1) flag = 0;      /* 负数、0 和 1 都不是素数 */
        for(i=2;i<=squareRoot&&flag;i++)
        {
            if(x%i==0) flag = 0;       /* 若能被整除,则不是素数 */
        }
        return flag;
    }
```

程序的运行结果为:

```
Input n:8↙
sum =17
```

任务 4 的参考程序如下:

```
#include <stdio.h>
#include <math.h>
int IsPrime(int x);
int main()
{
    int m,i,isFirstFactor=1;
    printf("Input m:");
    scanf("%d", &m);
    if(IsPrime(m))                  /* 素数判定 */
    {
        printf("No divisor! It is a prime number! \n");
    }
```

```
13        else
14        {
15            for(i=2; i<fabs(m); i++)
16            {
17                if(m%i == 0)
18                {
19                    if(isFirstFactor==0) printf(",");
20                    printf("%d", i);
21                    isFirstFactor = 0;
22                }
23            }
24            printf("\n");
25        }
26        return 0;
27    }
28    /* 函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
29    int IsPrime(int x)
30    {
31        int i, flag = 1;
32        int squareRoot = (int)sqrt(x);
33        if(x <= 1) flag = 0;          /* 负数、0 和 1 都不是素数 */
34        for(i=2; i<=squareRoot&&flag; i++)
35        {
36            if(x%i==0) flag = 0;    /* 若能被整除,则不是素数 */
37        }
38        return flag;
39    }
```

程序的运行结果为:

① Input n:16↙

2,4,8

② Input n:17↙

No divisor! It is a prime number!

任务 5 的参考程序如下:

完全数又称为完美数或完数,是指这样一些特殊的自然数:它所有的真因子(即除了自身以外的约数)的和恰好等于它本身。注意:1 没有真因子,所以不是完全数。计算机已经证实,在 10^{300} 以下没有奇的完全数。

```
1    #include <stdio.h>
2    #include <math.h>
```

```
3    int IsPerfect(int x);
4    int main()
5    {
6        int m;
7        printf("Input m:");
8        scanf("%d", &m);
9        if(IsPerfect(m))
10           printf("%d is a perfect number\n", m);
11       else
12           printf("%d is not a perfect number\n", m);
13       return 0;
14   }
15   /* 函数功能:判断完全数,若函数返回 0,则代表不是完全数;若返回 1,则代表是完全数 */
16   int IsPerfect(int x)
17   {
18       int i;
19       int total = 0;          /* 1 没有真因子,不是完全数 */
20       for(i=1; i<x; i++)
21       {
22           if(x%i== 0)
23               total = total + i;
24       }
25       return total==x ? 1 : 0;
26   }
```

程序的两次测试结果为:

```
① Input m:28↙
  28 is a perfect number
② Input m:8↙
  8 is not a perfect number
```

思考题:试用函数 IsPerfect( )来编写一个程序判断并打印出 1~1 000 之间所有的完美数,并同时打印出每一个完美数的全部因子,以验证这个数确实是一个完美数。

任务 6 的参考程序如下:

```
1    #include <stdio.h>
2    #include <math.h>
3    int IsPrime(int x);
4    void OutputPrimeFactor(int x);
5    int main()
6    {
```

```
7       int m;
8       printf("Input m:");
9       scanf("%d", &m);
10      if(IsPrime(m))      /* 素数判定*/
11      {
12          printf("It is a prime number\n");
13      }
14      else
15      {
16          printf("%d=", m);
17          OutputPrimeFactor(m);      /* 输出 x 的质因数连乘 */
18      }
19      return 0;
20  }
21  /*  函数功能:判断 x 是否是素数,若函数返回 0,则表示不是素数;若返回 1,则表示是素数 */
22  int IsPrime(int x)
23  {
24      int i, flag = 1;
25      int squareRoot = (int)sqrt(x);
26      if(x <= 1) flag = 0;          /*  负数、0 和 1 都不是素数 */
27      for(i=2;i<=squareRoot&&flag;i++)
28      {
29          if(x%i== 0) flag = 0;     /*  若能被整除,则不是素数 */
30      }
31      return flag;
32  }
33  /*  函数功能:输出 x 的质因数连乘 */
34  void OutputPrimeFactor(int x)
35  {
36      int i;
37      for(i=2; i<x; i++)
38      {
39          if(x%i == 0)
40          {
41              printf("%d * ", i);
42              OutputPrimeFactor(x/i);  /*  递归调用该函数 */
```

```
            return;                          /* 不可以使用 break */
        }
    }
    printf("%d",x);          /* 输出最后一个因子(质因数,不能再分解) */
}
```

其中,函数 OutputPrimeFactor( )还可以按如下方式编写:

```
/* 函数功能:输出 x 的质因数连乘 */
void OutputPrimeFactor(int x)
{
    int i;
    if(! IsPrime(x))
    {
        for(i=2; i<x; i++)
        {
            if(x%i == 0)
            {
                printf("%d * ", i);
                OutputPrimeFactor(x/i);          /* 递归调用该函数 */
                return;                          /* 也可以使用 break */
            }
        }
    }
    else                                 /* 输出最后一个因子(质因数,不能再分解) */
    {
        printf("%d",x);
    }
}
```

程序的两次测试结果为:

① Input m:90↙

90=2 * 3 * 3 * 5

② Input m:13↙

It is a prime number

2. 小学生计算机辅助教学系统

改用模块化程序设计方法重新编程实现 2.2.5 小节的实验 2。

实验目的:熟悉函数设计、模块化程序设计方法。

任务 1 的参考程序 1 如下:

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <time.h>
int Calculate(int x, char op, int y);
int CreateRandomNumber();
int main()
{
    int a, b, answer;
    int flag = 0;       /* 置标志变量为假 */
    srand(time(NULL));
    a = CreateRandomNumber();
    b = CreateRandomNumber();
    do{
        printf("%d * %d=? \n", a, b);
        scanf("%d",&answer);
        if(answer==Calculate(a, '*', b))
        {
            printf("Right! \n");
            flag = 1;       /* 做对,则将标志变量置为真 */
        }
        else
        {
            printf("Wrong! Please try again.\n");
        }
    }while(flag ! = 1);     /* 循环,直到做对(标志变量为真)为止 */
    return 0;
}
/* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
int Calculate(int x, char op, int y)
{
    switch(op)
    {
        case '+': return x + y;
        case '-': return x - y;
        case '*': return x * y;
        case '/': return x / y;
        default: printf("Operator error! \n");return 0;
    }
}
/* 函数功能:生成一个 1~10 的随机整数 */
```

```
41    int CreateRandomNumber()
42    {
43        return rand() % 10 + 1;
44    }
```

任务 1 参考程序 2 如下：

```
1     #include <stdio.h>
2     #include <stdlib.h>
3     #include <time.h>
4     int Calculate(int x, char op, int y);
5     int CreateRandomNumber();
6     int main()
7     {
8         int a, b, answer;
9         int isFirstTime = 1;
10        srand(time(NULL));
11        a = CreateRandomNumber();
12        b = CreateRandomNumber();
13        do{
14              if (isFirstTime == 0)
15                  printf("Wrong! Please try again.\n");
16              printf("%d *%d =? \n", a, b);
17              scanf("%d", &answer);
18              isFirstTime = 0;
19        }while (answer ! = Calculate(a, '*', b));
20        printf("Right! \n");
21        return 0;
22    }
23    /* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
24    int Calculate(int x, char op, int y)
25    {
26        switch(op)
27        {
28            case '+': return x + y;
29            case '-': return x - y;
30            case '*': return x * y;
31            case '/': return x / y;
32            default: printf("Operator error! \n"); return 0;
33        }
```

```
34    }
35    /* 函数功能:生成一个 1~10 的随机整数 */
36    int CreateRandomNumber()
37    {
38        return rand() %  10 + 1;
39    }
```

程序的运行结果为:

```
3 * 9=?
21↙
Wrong! Please try again.
3 * 9=?
27↙
Right!
```

任务 2 的参考程序如下:

```
1     #include<stdio.h>
2     #include<stdlib.h>
3     #include<time.h>
4     int Calculate(int x, char op, int y);
5     int CreateRandomNumber();
6     int main()
7     {
8         int a, b, answer, wrongTimes = 0, flag = 0;
9         srand(time(NULL));
10        a = CreateRandomNumber();
11        b = CreateRandomNumber();
12        do{
13            printf("%d * %d=? \n", a, b);
14            scanf("%d", &answer);
15            if(answer==Calculate(a, '*', b))
16            {
17               printf("Right! \n");
18               flag=1;     /* 做对,则将标志变量置为真 */
19            }
20            else
21            {
22               wrongTimes++;
23               if (wrongTimes < 3)
24                   printf("Wrong! Please try again. \n");
```

```
25              else
26                  printf("Wrong! You have tried three times! Test over!");
27          }
28      }while(flag != 1&&wrongTimes< 3);/* 未做对且未超过 3 次时继续循环 */
29      return 0;
30  }
31  /* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
32  int Calculate(int x, char op, int y)
33  {
34      switch(op)
35      {
36          case '+': return x + y;
37          case '-': return x - y;
38          case '*': return x * y;
39          case '/': return x / y;
40          default: printf("Operator error! \n");return 0;
41      }
42  }
43  /* 函数功能:生成一个 1~10 的随机整数 */
44  int CreateRandomNumber()
45  {
46      return rand() % 10 + 1;
47  }
```

程序的两次测试结果为:

```
① 6 * 10=?
6↙
Wrong! Please try again.
6 * 10=?
10↙
Wrong! Please try again.
6 * 10=?
80↙
Wrong! You have tried three times! Test over!
② 3 * 5=?
15↙
Right!
```

任务 3 的参考程序如下：

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
int Calculate(int x, char op, int y);
int CreateRandomNumber();
int main()
{
    int a, b, answer, i, rightNumber = 0;
    srand(time(NULL));
    for(i=0; i<10; i++)
    {
        a = CreateRandomNumber();
        b = CreateRandomNumber();
        printf("%d * %d=? \n", a, b);
        scanf("%d", &answer);
        if(answer == Calculate(a, '*', b))
        {
            printf("Right! \n");
            rightNumber++;
        }
        else
        {
            printf("Wrong! \n");
        }
    }
    printf("Total score is %d\n", rightNumber * 10);
    printf("Rate of correctness is %d%% \n", rightNumber * 10);
    return 0;
}
/* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
int Calculate(int x, char op, int y)
{
    switch(op)
    {
        case '+': return x + y;
        case '-': return x - y;
```

```
37            case '*': return x * y;
38            case '/': return x / y;
39            default: printf("Operator error! \n");return 0;
40        }
41    }
42    /* 函数功能:生成一个 1~10 的随机整数 */
43    int CreateRandomNumber()
44    {
45        return rand() % 10 + 1;
46    }
```

程序的运行结果如下:

```
1 * 8=?
8↙
Right!
2 * 2=?
4↙
Right!
3 * 1=?
3↙
Right!
5 * 8=?
40↙
Right!
6 * 6=?
36↙
Right!
10 * 9=?
90↙
Right!
6 * 7=?
41↙
Wrong!
2 * 10=?
20↙
Right!
6 * 4=?
24↙
```

```
Right!
2 * 5=?
10↙
Right!
Total score is 90
Rate of correctness is 90%
```

任务 4 的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int Calculate(int x, char op, int y);
int CreateRandomNumber();
char CreateRandomOperator();
int main()
{
    int a, b, userAnswer, i, rightNumber = 0;
    char opChar;
    srand(time(NULL));
    for(i=0; i<10; i++)
    {
        a = CreateRandomNumber();
        b = CreateRandomNumber();
        opChar = CreateRandomOperator();
        printf("%d %c %d=? \n", a, opChar, b);
        scanf("%d",&userAnswer);
        if (userAnswer == Calculate(a, opChar, b))
        {
            printf("Right! \n");
            rightNumber++;
        }
        else
        {
            printf("Wrong! \n");
        }
    }
    printf("Total score is %d\n", rightNumber * 10);
    printf("Rate of correctness is %d%% \n", rightNumber * 10);
    return 0;
```

```
}
/* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
int Calculate(int x, char op, int y)
{
    switch(op)
    {
        case '+': return x + y;
        case '-': return x - y;
        case '*': return x * y;
        case '/': return x / y;
        default: printf("Operator error! \n"); return 0;
    }
}
/* 函数功能:生成一个 1~10 的随机整数 */
int CreateRandomNumber()
{
    return rand() % 10 + 1;
}
/* 函数功能:随机生成一个运算符号(+,-,* ,/) */
char CreateRandomOperator()
{
    int op;
    op = rand() % 4 + 1;
    switch(op)
    {
        case 1: return '+';
        case 2: return '-';
        case 3: return '*';
        case 4: return '/';
    }
    return 0;
}
```

程序的运行结果如下:

```
4 - 8=?
-4↙
Right!
6 / 9=?
1↙
```

```
Wrong!
5 / 6=?
0↙
Right!
7 + 7=?
14↙
Right!
7 - 2=?
5↙
Right!
1 + 3=?
4↙
Right!
10 * 8=?
80↙
Right!
8 + 1=?
9↙
Right!
2 - 1=?
1↙
Right!
6 + 10=?
16↙
Right!
Total score is 90
Rate of correctness is 90%
```

任务 5 的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int Calculate(int x, char op, int y);
int CreateRandomNumber();
char CreateRandomOperator();
int main()
{
    int a, b, userAnswer, i, rightNumber = 0,flag = 0;
    char opChar;
```

```
    srand(time(NULL));
    do{
        for (i=0; i<10; i++)
        {
            a = CreateRandomNumber();
            b = CreateRandomNumber();
            opChar = CreateRandomOperator();
            printf("%d %c %d = ? \n", a, opChar, b);
            scanf("%d", &userAnswer);
            if (userAnswer == Calculate(a, opChar, b))
            {
                printf("Right! \n");
                rightNumber++;
            }
            else
            {
                printf("Wrong! \n");
            }
        }
        printf("Total score is %d\n", rightNumber * 10);
        printf("Rate of correctness is %d%%\n", rightNumber * 10);
        if (rightNumber * 10 < 75)
        {
            printf("Once Again! \n");
            rightNumber = 0;
            flag = 1;
        }
    }while (flag);
    return 0;
}
/* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
int Calculate(int x, char op, int y)
{
    switch(op)
    {
            case '+': return x + y;
            case '-': return x - y;
            case '*': return x * y;
```

```
49              case '/': return x / y;
50              default: printf("Operator error! \n"); return 0;
51          }
52      }
53      /* 函数功能:生成一个 1~10 的随机整数 */
54      int CreateRandomNumber()
55      {
56          return rand() % 10 + 1;
57      }
58      /* 函数功能:随机生成一个运算符号(+,-,* ,/) */
59      char CreateRandomOperator()
60      {
61          int op;
62          op = rand() % 4 + 1;
63          switch (op)
64          {
65              case 1: return '+';
66              case 2: return '-';
67              case 3: return '*';
68              case 4: return '/';
69          }
70          return 0;
71      }
```

程序的运行结果为:

```
4 - 8 = ?
4↙
Wrong!
6 / 9 = ?
1↙
Wrong!
5 / 6 = ?
1↙
Wrong!
7 + 7 = ?
17↙
Wrong!
7 - 2 = ?
2↙
```

```
Wrong!
1 + 3 = ?
4↙
Right!
10 * 8 = ?
10↙
Wrong!
8 + 1 = ?
10↙
Wrong!
2 - 1 = ?
1↙
Right!
6 + 10 = ?
60↙
Wrong!
Total score is 20
Rate of correctness is 20%
Once Again!
7 + 6 = ?
13↙
Right!
2 + 2 = ?
4↙
Right!
10 * 1 = ?
10↙
Right!
10 - 4 = ?
6↙
Right!
5 * 2 = ?
10↙
Right!
1 + 3 = ?
4↙
Right!
```

```
5 * 10 = ?
50↙
Right!
8 - 1 = ?
7↙
Right!
2 + 1 = ?
3↙
Right!
6 - 10 = ?
-4↙
Right!
Total score is 100
Rate of correctness is 100%
```

任务 6 的参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int Calculate(int x, char op, int y);
int CreateRandomNumber();
char CreateRandomOperator();
void PrintRandomRightEvaluation();
void PrintRandomWrongEvaluation();
int main()
{
    int a, b, userAnswer, i, rightNumber = 0, flag = 0;
    char opChar;
    srand(time(NULL));
    do{
        for (i=0; i<10; i++)
        {
            a = CreateRandomNumber();
            b = CreateRandomNumber();
            opChar = CreateRandomOperator();
            printf("%d %c %d = ? \n", a, opChar, b);
            scanf("%d", &userAnswer);
            if (userAnswer == Calculate(a, opChar, b))
            {
```

```
24                  PrintRandomRightEvaluation();
25                  rightNumber++;
26              }
27              else
28              {
29                  PrintRandomWrongEvaluation();
30              }
31          }
32          printf("Total score is %d\n", rightNumber * 10);
33          printf("Rate of correctness is %d%% \n", rightNumber * 10);
34          if (rightNumber * 10 < 75)
35          {
36              printf("Once Again! \n");
37              rightNumber = 0;
38              flag = 1;
39          }
40      }while (flag);
41      return 0;
42  }
43  /* 函数功能:计算两个数(x,y)的四则运算(运算符由 op 确定),返回计算结果值 */
44  int Calculate(int x, char op, int y)
45  {
46      switch(op)
47      {
48          case '+': return x + y;
49          case '-': return x - y;
50          case '*': return x * y;
51          case '/': return x / y;
52          default: printf("Operator error! \n"); return 0;
53      }
54  }
55  /* 函数功能:生成一个 1~10 的随机整数 */
56  int CreateRandomNumber()
57  {
58      return rand() % 10 + 1;
59  }
60  /* 函数功能:随机生成一个运算符号(+,-,* ,/) */
61  char CreateRandomOperator()
```

```
62    {
63        int op;
64        op = rand() %  4 + 1;
65        switch (op)
66        {
67            case 1: return '+';
68            case 2: return '-';
69            case 3: return '*';
70            case 4: return '/';
71        }
72        return 0;
73    }
74    /* 函数功能:生成一个题目做对的随机提示 */
75    void PrintRandomRightEvaluation()
76    {
77        int i;
78        i = rand() %  4 + 1;
79        switch(i)
80        {
81            case 1:printf("Very good! \n");
82                   break;
83            case 2:printf("Excellent! \n");
84                   break;
85            case 3:printf("Nice work! \n");
86                   break;
87            case 4:printf("Keep up the good work! \n");
88                   break;
89            default:printf("Wrong type!");
90         }
91     }
92    /* 函数功能:生成一个题目做错的随机提示 */
93    void PrintRandomWrongEvaluation()
94    {
95        int i;
96        i = rand() %  4 + 1;
97        switch(i)
98        {
99             case 1:printf("No. Please try again.\n");
```

```
100                 break;
101           case 2:printf("Wrong. Try once more.\n");
102                 break;
103           case 3:printf("Don't give up! \n");
104                 break;
105           case 4:printf("Not correct. Keep trying.\n");
106                 break;
107           default:printf("Wrong type!");
108     }
109   }
```

程序的运行结果如下：

```
4 + 8 = ?
10↙
Not correct. Keep trying.
7 + 7 = ?
14↙
Excellent!
2 - 7 = ?
5↙
Wrong. Try once more.
10 * 8 = ?
80↙
Nice work!
8 / 1 = ?
8↙
Very good!
6 + 10 = ?
16↙
Keep up the good work!
7 * 6 = ?
42↙
Excellent!
2 + 3 = ?
5↙
Nice work!
10 * 2 = ?
20↙
Keep up the good work!
```

```
10 - 4 = ?
6↙
Very good!
Total score is 80
Rate of correctness is 80%
```

2.2.7 第 8 章实验题：学生成绩管理系统 V1.0

学生成绩管理系统 V1.0

某班有最多不超过 30 人(具体人数由键盘输入)参加某门课程的考试,参考例 8.5、例 8.9、例 8.10,用一维数组作函数参数编程实现如下学生成绩管理系统:

(1) 录入每个学生的学号和考试成绩。

(2) 计算课程的总分和平均分。

(3) 按成绩由高到低排出名次表。

(4) 按学号由小到大排出成绩表。

(5) 按学号查询学生排名及其考试成绩。

(6) 按优秀(90~100 分)、良好(80~89 分)、中等(70~79 分)、及格(60~69 分)、不及格(0~59 分)5 个类别,统计每个类别的人数以及所占的百分比。

(7) 输出每个学生的学号、考试成绩以及课程总分和平均分。

实验目的:熟悉一维数组作函数参数,排序、查找、统计分析等常用算法,模块化程序设计以及增量测试方法。

【思考题】如果要求程序运行后先显示如下菜单,并提示用户输入选项:

```
1. Input record
2. Caculate total and average score of course
3. Sort in descending order by score
4. Sort in ascending order by number
5. Search by number
6. Statistic analysis
7. List record
0. Exit
Please enter your choice:
```

然后根据用户输入的选项执行相应的操作,那么程序应该如何修改呢?

参考程序如下:

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #define STU_NUM 30        /* 最多的学生人数 */
4   int Menu(void);
5   void ReadScore(long num[], float score[], int n);
6   void AverSumofScore(float score[], int n);
```

```
7   void DeSortbyScore(long num[], float score[], int n);
8   void AsSortbyNum(long num[], float score[], int n);
9   void SearchbyNum(long num[], float score[], int n);
10  void StatisticAnalysis(float score[], int n);
11  void PrintScore(long num[], float score[], int n) ;
12  int main()
13  {
14      char itemSelected;
15      int n = 0;
16      float score[STU_NUM];
17      long num[STU_NUM];
18      printf("Input student number(n<%d):", STU_NUM);
19      scanf("%d", &n);
20      while (1)
21      {
22          itemSelected = Menu();/* 显示菜单,并读取用户输入 */
23          switch (itemSelected)
24          {
25              case 1:ReadScore(num, score, n);
26                     break;
27              case 2:AverSumofScore(score, n);
28                     break;
29              case 3:DeSortbyScore(num, score, n);
30                     printf("\nSort in descending order by score:\n");
31                     PrintScore(num, score, n);
32                     break;
33              case 4:AsSortbyNum(num, score, n);
34                     printf("\nSort in ascending order by number:\n");
35                     PrintScore(num, score, n);
36                     break;
37              case 5:SearchbyNum(num, score, n);
38                     break;
39              case 6:StatisticAnalysis(score, n);
40                     break;
41              case 7:PrintScore(num, score, n);
42                     break;
43              case 0:printf("End of program!\n");
44                     exit(0);
```

```
45                default:printf("Input error! \n");
46            }
47        }
48        return 0;
49    }
50    /* 函数功能:显示菜单并获得用户键盘输入的选项 */
51    int Menu(void)
52    {
53        int itemSelected;
54        printf("\nManagement for Students' scores\n");
55        printf(" 1.Input record\n");
56        printf(" 2.Caculate total and average score of course\n");
57        printf(" 3.Sort in descending order by score\n");
58        printf(" 4.Sort in ascending order by number\n");
59        printf(" 5.Search by number\n");
60        printf(" 6.Statistic analysis\n");
61        printf(" 7.List record\n");
62        printf(" 0.Exit\n");
63        printf("Please Input your choice:");
64        scanf("%d", &itemSelected);     /* 读入用户输入 */
65        return itemSelected;
66    }
67    /* 函数功能:输入 n 个学生的某门课成绩 */
68    void ReadScore(long num[], float score[], int n)
69    {
70        int i;
71        printf("Input student's ID, name and score:\n");
72        for (i=0; i<n; i++)
73        {
74            scanf("%ld %f", &num[i], &score[i]);
75        }
76    }
77    /* 函数功能:计算全班总分和平均分 */
78    void AverSumofScore(float score[], int n)
79    {
80        int i;
81        float sum = 0;
82        for (i=0; i<n; i++)
```

```
83      {
84          sum = sum + score[i];
85      }
86      printf("sum = %.0f, aver = %.2f\n", sum, n>0 ? sum/n : 0);
87  }
88  /* 函数功能:按选择法将数组 score 的元素值按从高到低排序 */
89  void DeSortbyScore(long num[], float score[], int n)
90  {
91      int i, j, k;
92      float temp1;
93      long temp2;
94      for (i=0; i<n-1; i++)
95      {
96          k = i;
97          for (j=i+1; j<n; j++)
98          {
99              if (score[j] > score[k]) k = j;
100         }
101         if (k != i)
102         {
103             /* 交换成绩 */
104             temp1 = score[k]; score[k] = score[i]; score[i] = temp1;
105             /* 交换学号 */
106             temp2 = num[k]; num[k] = num[i]; num[i] = temp2;
107         }
108     }
109 }
110 /* 函数功能:按选择法将数组 num 的元素值按从小到大排序 */
111 void AsSortbyNum(long num[], float score[], int n)
112 {
113     int i, j, k;
114     float temp1;
115     long temp2;
116     for (i=0; i<n-1; i++)
117     {
118         k = i;
119         for (j=i+1; j<n; j++)
120         {
```

```
121             if (num[j] < num[k]) k = j;
122         }
123         if (k ! = i)
124         {
125             /* 交换成绩 */
126             temp1 = score[k]; score[k] = score[i]; score[i] = temp1;
127             /* 交换学号 */
128             temp2 = num[k]; num[k] = num[i]; num[i] = temp2;
129         }
130     }
131 }
132 /* 函数功能:按学号查找学生成绩并显示查找结果 */
133 void SearchbyNum(long num[], float score[], int n)
134 {
135     long number;
136     int i;
137     printf("Input the number you want to search:");
138     scanf("%ld", &number);
139     for (i=0; i<n; i++)
140     {
141         if (num[i] == number)
142         {
143             printf("%ld\t%.0f\n", num[i], score[i]);
144             return;
145         }
146     }
147     printf("\nNot found! \n");
148 }
149 /* 函数功能:统计各分数段的学生人数及所占的百分比 */
150 void StatisticAnalysis(float score[], int n)
151 {
152     int i, total, t[6] = {0,0,0,0,0,0};
153     for (i=0; i<n; i++)
154     {
155         if (score[i]>=0 && score[i]<60)t[0]++;
156         else if (score[i]<70)          t[1]++;
157         else if (score[i]<80)          t[2]++;
158         else if (score[i]<90)          t[3]++;
```

```
159             else if (score[i]<100)          t[4]++;
160             else if (score[i] == 100)       t[5]++;
161         }
162         for (total=0, i=0; i<=5; i++)
163         {
164             total = total + t[i];
165         }
166         for (i=0; i<=5; i++)
167         {
168             if (i == 0)
169             {
170                 printf("<60\t%d\t%.2f%%\n", t[i], (float)t[i]/n*100);
171             }
172             else if (i == 5)
173             {
174                 printf("%d\t%d\t%.2f%%\n",(i+5)* 10, t[i],(float)t[i]/n*100);
175             }
176             else
177             {
178                 printf("%d-%d\t%d\t%.2f%%\n", (i+5)* 10, (i+5)* 10+9, t[i],
179                         (float)t[i]/n* 100);
180             }
181         }
182     }
183     /* 函数功能：打印学生成绩 */
184     void PrintScore(long num[], float score[], int n)
185     {
186         int i;
187         for (i=0; i<n; i++)
188         {
189             printf("%ld\t%.0f\n", num[i], score[i]);
190         }
191     }
```

2.2.8 第 9 章实验题：学生成绩管理系统 V2.0

学生成绩管理系统 V2.0

某班有最多不超过 30 人（具体人数由键盘输入）参加某门课程的考试，**参考例 9.9，用一维数组和函数指针作函数参数**编程实现如下菜单驱动的学生成绩管理系统：

(1) 录入每个学生的学号和考试成绩。

(2) 计算课程的总分和平均分。

(3) 按成绩由高到低排出名次表。

(4) **按成绩由低到高排出名次表。**

(5) 按学号由小到大排出成绩表。

(6) 按学号查询学生排名及其考试成绩。

(7) 按优秀(90~100 分)、良好(80~89 分)、中等(70~79 分)、及格(60~69 分)、不及格(0~59 分)5 个类别,统计每个类别的人数以及所占的百分比。

(8) 输出每个学生的学号、考试成绩以及课程总分和平均分。

要求程序运行后先显示如下菜单,并提示用户输入选项:

```
1. Input record
2. Caculate total and average score of course
3. Sort in descending order by score
4. Sort in ascending order by score
5. Sort in ascending order by number
6. Search by number
7. Statistic analysis
8. List record
0. Exit
Please enter your choice:
```

然后,根据用户输入的选项执行相应的操作。

实验目的:在第 8 章实验“学生成绩管理系统 V1.0”的基础上,通过增加任务要求,熟悉函数指针作函数参数、模块化程序设计以及增量测试方法。

参考程序如下:

```
#include <stdio.h>
#include <stdlib.h>
#define STU_NUM 30              /* 最多的学生人数 */
int Menu(void);
void ReadScore(long num[], float score[], int n);
void AverSumofScore(float score[], int n);
void SortbyScore(long num[], float score[], int n,
                 int (*compare)(float a, float b));
int Ascending(float a, float b);
int Descending(float a, float b);
void SwapFloat(float *x,float *y);
void SwapLong(long *x,long *y);
void AsSortbyNum(long num[], float score[], int n);
void SearchbyNum(long num[], float score[], int n);
```

```
15  void StatisticAnalysis(float score[], int n);
16  void PrintScore(long num[], float score[], int n) ;
17  int main()
18  {
19      char itemSelected;
20      int n = 0;
21      float score[STU_NUM];
22      long num[STU_NUM];
23      printf("Input student number(n<%d):", STU_NUM);
24      scanf("%d", &n);
25      while (1)
26      {
27          itemSelected = Menu();/* 显示菜单,并读取用户输入 */
28          switch (itemSelected)
29          {
30              case 1: ReadScore(num, score, n);
31                      break;
32              case 2: AverSumofScore(score, n);
33                      break;
34              case 3: SortbyScore(num, score, n, Descending);
35                      printf("\nSort in descending order by score:\n");
36                      PrintScore(num, score, n);
37                      break;
38              case 4: SortbyScore(num, score, n, Ascending);
39                      printf("\nSort in ascending order by score:\n");
40                      PrintScore(num, score, n);
41                      break;
42              case 5: AsSortbyNum(num, score, n);
43                      printf("\nSort in ascending order by number:\n");
44                      PrintScore(num, score, n);
45                      break;
46              case 6: SearchbyNum(num, score, n);
47                      break;
48              case 7: StatisticAnalysis(score, n);
49                      break;
50              case 8: PrintScore(num, score, n);
51                      break;
52              case 0: printf("End of program! \n");
```

```
53                      exit(0);
54                default:printf("Input error! \n");
55             }
56         }
57         return 0;
58     }
59     /* 函数功能:显示菜单并获得用户键盘输入的选项 */
60     int Menu(void)
61     {
62         int itemSelected;
63         printf(" Management for Students' scores \n");
64         printf(" 1.Input record \n");
65         printf(" 2.Caculate total and average score of course \n");
66         printf(" 3.Sort in descending order by score \n");
67         printf(" 4.Sort in ascending order by score \n");
68         printf(" 5.Sort in ascending order by number \n");
69         printf(" 6.Search by number \n");
70         printf(" 7.Statistic analysis \n");
71         printf(" 8.List record \n");
72         printf(" 0.Exit \n");
73         printf("Please Input your choice:");
74         scanf("%d", &itemSelected); /* 读入用户输入 */
75         return itemSelected;
76     }
77     /* 函数功能:输入 n 个学生的某门课成绩 */
78     void ReadScore(long num[], float score[], int n)
79     {
80         int i;
81         printf("Input student's ID and score: \n");
82         for (i=0; i<n; i++)
83         {
84             scanf("%ld %f", &num[i], &score[i]);
85         }
86     }
87     /* 函数功能:计算全班总分和平均分 */
88     void AverSumofScore(float score[], int n)
89     {
90         int i;
```

```
91      float sum = 0;
92      for (i=0; i<n; i++)
93      {
94          sum = sum + score[i];
95      }
96      printf("sum = %.0f, aver = %.2f\n", sum, n>0 ? sum/n : 0);
97  }
98  /* 函数功能:按选择法将数组 score 的元素值排序 */
99  void SortbyScore(long num[], float score[], int n,
100                   int (*compare)(float a, float b))
101  {
102     int i, j, k;
103     for (i=0; i<n-1; i++)
104     {
105         k = i;
106         for (j=i+1; j<n; j++)
107         {
108             if((*compare)(score[j], score[k])) k = j;
109         }
110         if (k != i)
111         {
112             SwapFloat(&score[k],&score[i]);/* 交换成绩 */
113             SwapLong(&num[k],&num[i]);/* 交换学号 */
114         }
115     }
116  }
117  /* 使数据按升序排序 */
118  int Ascending(float a, float b)
119  {
120     return a < b;      /* 这样比较决定了按升序排序,如果 a<b,则交换 */
121  }
122  /* 使数据按降序排序 */
123  int Descending(float a, float b)
124  {
125     return a > b;      /* 这样比较决定了按降序排序,如果 a>b,则交换 */
126  }
127  /* 交换两个单精度浮点型数据 */
128  void SwapFloat(float *x,float *y)
```

```
129    {
130        float temp;
131        temp = *x;
132        *x = *y;
133        *y = temp;
134    }
135    /* 交换两个长整型数据 */
136    void SwapLong(long *x,long *y)
137    {
138        long  temp;
139        temp = *x;
140        *x = *y;
141        *y = temp;
142    }
143    /* 函数功能:按选择法将数组 num 的元素值按从小到大排序 */
144    void AsSortbyNum(long num[], float score[], int n)
145    {
146        int i, j, k;
147        for (i=0; i<n-1; i++)
148        {
149            k = i;
150            for (j=i+1; j<n; j++)
151            {
152                if (num[j] < num[k]) k = j;
153            }
154            if (k ! = i)
155            {
156                SwapFloat(&score[k],&score[i]);/* 交换成绩 */
157                SwapLong(&num[k],&num[i]);/* 交换学号 */
158            }
159        }
160    }
161    /* 函数功能:按学号查找学生成绩并显示查找结果 */
162    void SearchbyNum(long num[], float score[], int n)
163    {
164        long number;
165        int i;
166        printf("Input the number you want to search:");
```

```
167         scanf("%ld", &number);
168         for (i=0; i<n; i++)
169         {
170             if (num[i] == number)
171             {
172                 printf("%ld\t%.0f\n", num[i], score[i]);
173                 return;
174             }
175         }
176         printf("\nNot found! \n");
177     }
178     /* 函数功能:统计各分数段的学生人数及所占的百分比 */
179     void StatisticAnalysis(float score[], int n)
180     {
181         int i, total, t[6] = {0,0,0,0,0,0};
182         for (i=0; i<n; i++)
183         {
184             if (score[i]>=0 && score[i]<60)t[0]++;
185             else if (score[i]<70)          t[1]++;
186             else if (score[i]<80)          t[2]++;
187             else if (score[i]<90)          t[3]++;
188             else if (score[i]<100)         t[4]++;
189             else if (score[i] == 100)      t[5]++;
190         }
191         for (total=0, i=0; i<=5; i++)
192         {
193             total = total + t[i];
194         }
195         for (i=0; i<=5; i++)
196         {
197             if (i == 0)printf("<60\t%d\t%.2f%%\n", t[i], (float)t[i]/n*100);
198             else if (i == 5)printf ("%d\t%d\t%.2f%%\n",(i+5)* 10,t[i],
199                                     (float)t[i]/n*100);
200             else printf ("%d-%d\t%d\t%.2f%%\n",(i+5)* 10, (i+5)* 10+9, t[i],
201                          (float)t[i]/n*100);
202         }
203     }
204     /* 函数功能: 打印学生成绩 */
```

```
205    void PrintScore(long num[], float score[], int n)
206    {
207        int i;
208        for (i=0; i<n; i++)
209        {
210            printf("%ld\t%.0f\n", num[i], score[i]);
211        }
212    }
```

2.2.9 第 10 章实验题:学生成绩管理系统 V3.0

学生成绩管理系统 V3.0

某班有最多不超过 30 人(具体人数由键盘输入)参加某门课程的考试,**参考例 10.4,用二维字符数组作函数参数**编程实现如下菜单驱动的学生成绩管理系统:

(1) 录入每个学生的学号、**姓名**和考试成绩。

(2) 计算课程的总分和平均分。

(3) 按成绩由高到低排出名次表。

(4) 按成绩由低到高排出名次表。

(5) 按学号由小到大排出成绩表。

(6) **按姓名的字典顺序排出成绩表。**

(7) 按学号查询学生排名及其考试成绩。

(8) **按姓名查询学生排名及其考试成绩。**

(9) 按优秀(90~100 分)、良好(80~89 分)、中等(70~79 分)、及格(60~69 分)、不及格(0~59 分)5 个类别,统计每个类别的人数以及所占的百分比。

(10) 输出每个学生的学号、**姓名**、考试成绩以及课程总分和平均分。

要求程序运行后先显示如下菜单,并提示用户输入选项:

```
1. Input record
2. Caculate total and average score of course
3. Sort in descending order by score
4. Sort in ascending order by score
5. Sort in ascending order by number
6. Sort in dictionary order by name
7. Search by number
8. Search by name
9. Statistic analysis
10. List record
0. Exit
Please enter your choice:
```

然后,根据用户输入的选项执行相应的操作。

实验目的：在第9章实验“学生成绩管理系统V2.0”的基础上，通过增加任务要求，熟悉二维字符数组作函数参数、字符串处理函数、字符串处理操作、模块化程序设计以及增量测试方法。

参考程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LEN 10          /* 字符串最大长度 */
#define STU_NUM 30          /* 最多的学生人数 */
int Menu(void);
void ReadScore(long num[], char name[][MAX_LEN], float score[], int n);
void AverSumofScore(float score[], int n);
void SortbyScore(long num[], char name[][MAX_LEN], float score[], int n,
                 int (*compare)(float a, float b));
int Ascending(float a, float b);
int Descending(float a, float b);
void SwapFloat(float *x,float *y);
void SwapLong(long *x,long *y);
void SwapChar(char x[],char y[]);
void AsSortbyNum(long num[], char name[][MAX_LEN], float score[], int n);
void SortbyName(long num[], char name[][MAX_LEN], float score[], int n);
void SearchbyNum(long num[], char name[][MAX_LEN], float score[], int n);
void SearchbyName(long num[], char name[][MAX_LEN], float score[], int n);
void StatisticAnalysis(float score[], int n);
void PrintScore(long num[], char name[][MAX_LEN], float score[], int n) ;
int main()
{
    char ch;
    int n = 0;
    float score[STU_NUM];
    long num[STU_NUM];
    char name[STU_NUM][MAX_LEN];
    printf("Input student number(n<%d):", STU_NUM);
    scanf("%d", &n);
    while (1)
    {
        ch = Menu();          /* 显示菜单,并读取用户输入 */
        switch (ch)
```

```
35          {
36              case 1: ReadScore(num, name, score, n);
37                      break;
38              case 2: AverSumofScore(score, n);
39                      break;
40              case 3: SortbyScore(num, name, score, n, Descending);
41                      printf("\nSort in descending order by score:\n");
42                      PrintScore(num, name, score, n);
43                      break;
44              case 4: SortbyScore(num, name, score, n, Ascending);
45                      printf("\nSort in ascending order by score:\n");
46                      PrintScore(num, name, score, n);
47                      break;
48              case 5: AsSortbyNum(num, name, score, n);
49                      printf("\nSort in ascending order by number:\n");
50                      PrintScore(num, name, score, n);
51                      break;
52              case 6: SortbyName(num, name, score, n);
53                      printf("\nSort in dictionary order by name:\n");
54                      PrintScore(num, name, score, n);
55                      break;
56              case 7: SearchbyNum(num, name, score, n);
57                      break;
58              case 8: SearchbyName(num, name, score, n);
59                      break;
60              case 9: StatisticAnalysis(score, n);
61                      break;
62              case 10: PrintScore(num, name, score, n);
63                       break;
64              case 0: printf("End of program! \n");
65                      exit(0);
66              default:printf("Input error! \n");
67          }
68      }
69      return 0;
70  }
71  /* 函数功能:显示菜单并获得用户键盘输入的选项 */
72  int Menu(void)
```

```
{
    int itemSelected;
    printf(" Management for Students' scores \n");
    printf(" 1.Input record \n");
    printf(" 2.Caculate total and average score of course \n");
    printf(" 3.Sort in descending order by score \n");
    printf(" 4.Sort in ascending order by score \n");
    printf(" 5.Sort in ascending order by number \n");
    printf(" 6.Sort in dictionary order by name \n");
    printf(" 7.Search by number \n");
    printf(" 8.Search by name \n");
    printf(" 9.Statistic analysis \n");
    printf(" 10.List record \n");
    printf(" 0.Exit \n");
    printf("Please Input your choice:");
    scanf("%d", &itemSelected); /* 读入用户输入 */
    return itemSelected;
}
/* 函数功能:输入 n 个学生的某门课成绩 */
void ReadScore(long num[], char name[][MAX_LEN], float score[], int n)
{
    int i;
    printf("Input student's ID, name and score: \n");
    for (i=0; i<n; i++)
    {
        scanf("%ld%s%f", &num[i], name[i], &score[i]);
    }
 }
 /* 函数功能:计算全班总分和平均分 */
 void AverSumofScore(float score[], int n)
 {
     int i;
     float sum = 0;
     for (i=0; i<n; i++)
     {
         sum = sum + score[i];
     }
     printf("sum = %.0f, aver = %.2f \n", sum, n>0 ? sum/n : 0);
```

```
111    }
112    /* 函数功能:按选择法将数组 score 的元素值排序 */
113    void SortbyScore(long num[], char name[][MAX_LEN], float score[],
114                     int n,int (*compare)(float a, float b))
115    {
116        int i, j, k;
117        for (i=0; i<n-1; i++)
118        {
119            k = i;
120            for (j=i+1; j<n; j++)
121            {
122                if((*compare)(score[j], score[k])) k = j;
123            }
124            if (k != i)
125            {
126                SwapFloat(&score[k], &score[i]);/* 交换成绩 */
127                SwapLong(&num[k], &num[i]);/* 交换学号 */
128                SwapChar(name[k], name[i]);/* 交换姓名 */
129            }
130        }
131    }
132    /* 使数据按升序排序 */
133    int Ascending(float a, float b)
134    {
135        return a < b;          /* 这样比较决定了按升序排序,如果 a<b,则交换 */
136    }
137    /* 使数据按降序排序 */
138    int Descending(float a, float b)
139    {
140        return a > b;          /* 这样比较决定了按降序排序,如果 a>b,则交换 */
141    }
142    /* 交换两个单精度浮点型数据*/
143    void SwapFloat(float *x,float *y)
144    {
145        float temp;
146        temp = *x;
147        *x = *y;
148        *y = temp;
```

```
149    }
150    /* 交换两个长整型数据*/
151    void SwapLong(long *x,long *y)
152    {
153        long temp;
154        temp = *x;
155        *x = *y;
156        *y = temp;
157    }
158    /* 交换两个字符串 */
159    void SwapChar(char x[],char y[])
160    {
161        chartemp[MAX_LEN];
162        strcpy(temp, x);
163        strcpy(x, y);
164        strcpy(y, temp);
165    }
166    /* 函数功能:按选择法将数组 num 的元素值按从小到大排序 */
167    void AsSortbyNum(long num[], char name[][MAX_LEN], float score[], int n)
168    {
169        int i, j, k;
170        for (i=0; i<n-1; i++)
171        {
172            k = i;
173            for (j=i+1; j<n; j++)
174            {
175                if (num[j] < num[k]) k = j;
176            }
177            if (k ! = i)
178            {
179                SwapFloat(&score[k],&score[i]);/* 交换成绩 */
180                SwapLong(&num[k],&num[i]);/* 交换学号 */
181                SwapChar(name[k],name[i]);/* 交换姓名 */
182            }
183        }
184    }
185    /* 函数功能:交换法实现字符串按字典顺序排序 */
186    void SortbyName(long num[], char name[][MAX_LEN], float score[], int n)
```

```
{
    int i, j;
    for (i=0; i<n-1; i++)
    {
        for (j = i+1; j<n; j++)
        {
            if (strcmp(name[j], name[i]) < 0)
            {
                SwapFloat(&score[i], &score[j]);/* 交换成绩 */
                SwapLong(&num[i], &num[j]);/* 交换学号 */
                SwapChar(name[i], name[j]);/* 交换姓名 */
            }
        }
    }
}
/* 函数功能:按学号查找学生成绩并显示查找结果 */
void SearchbyNum(long num[], char name[][MAX_LEN], float score[], int n)
{
    long number;
    int i;
    printf("Input the number you want to search:");
    scanf("%ld", &number);
    for (i=0; i<n; i++)
    {
        if (num[i] == number)
        {
            printf("%ld\t%s\t%.0f\n", num[i], name[i], score[i]);
            return;
        }
    }
    printf("\nNot found! \n");
}
/* 函数功能:按姓名查询学生排名及其考试成绩 */
void SearchbyName(long num[], char name[][MAX_LEN], float score[], int n)
{
    char x[MAX_LEN];
    int i;
    printf("Input the name you want to search:");
```

```
225         scanf("%s", x);
226         for (i=0; i<n; i++)
227         {
228             if (strcmp(name[i], x) == 0)
229             {
230                 printf("%ld\t%s\t%.0f\n", num[i], name[i], score[i]);
231                 return;
232             }
233         }
234         printf("\nNot found! \n");
235     }
236     /* 函数功能:统计各分数段的学生人数及所占的百分比 */
237     void StatisticAnalysis(float score[], int n)
238     {
239         int i, total, t[6] = {0,0,0,0,0,0};
240         for (i=0; i<n; i++)
241         {
242             if (score[i]>=0 && score[i]<60)t[0]++;
243             else if (score[i]<70)          t[1]++;
244             else if (score[i]<80)          t[2]++;
245             else if (score[i]<90)          t[3]++;
246             else if (score[i]<100)         t[4]++;
247             else if (score[i] == 100)      t[5]++;
248         }
249         for (total=0, i=0; i<=5; i++)
250         {
251             total = total + t[i];
252         }
253         for (i=0; i<=5; i++)
254         {
255             if (i == 0)printf("<60\t%d\t%.2f%%\n", t[i],(float)t[i]/n*100);
256             else if (i == 5)printf ("%d\t%d\t%.2f%%\n",(i+5)* 10, t[i],
257                                     (float)t[i]/n*100);
258             else printf("%d-%d\t%d\t%.2f%%\n", (i+5)*10, (i+5)* 10+9,
259                         t[i],(float)t[i]/n*100);
260         }
261     }
```

```
/* 函数功能：打印学生成绩 */
void PrintScore(long num[], char name[][MAX_LEN], float score[], int n)
{
    int i;
    for (i=0; i<n; i++)
    {
        printf("%ld\t%s\t%.0f\n", num[i], name[i], score[i]);
    }
}
```

2.2.10 第11章实验题：学生成绩管理系统 V4.0

学生成绩管理系统 V4.0

某班有最多不超过30人(具体人数由键盘输入)参加期末考试，**最多不超过6门(具体门数由键盘输入)**。**参考例8.12，用二维数组作函数参数**编程实现如下菜单驱动的学生成绩管理系统：

(1) 录入每个学生的学号、姓名和各科考试成绩。

(2) 计算**每门课程**的总分和平均分。

(3) **计算每个学生的总分和平均分**。

(4) 按**每个学生的总分**由高到低排出名次表。

(5) 按**每个学生的总分**由低到高排出名次表。

(6) 按学号由小到大排出成绩表。

(7) 按姓名的字典顺序排出成绩表。

(8) 按学号查询学生排名及其**各科**考试成绩。

(9) 按姓名查询学生排名及其**各科**考试成绩。

(10) 按优秀(90~100分)、良好(80~89分)、中等(70~79分)、及格(60~69分)、不及格(0~59分)5个类别，对每门课程分别统计每个类别的人数以及所占的百分比。

(11) 输出每个学生的学号、姓名、**各科**考试成绩、**总分、平均分以及每门课程的总分和平均分**。

要求程序运行后先显示如下菜单，并提示用户输入选项：

```
1. Input record
2. Caculate total and average score of every course
3. Caculate total and average score of every student
4. Sort in descending order by total score of every student
5. Sort in ascending order by total score of every student
6. Sort in ascending order by number
7. Sort in dictionary order by name
8. Search by number
9. Search by name
10. Statistic analysis for every course
```

```
    11. List record
    0. Exit
    Please enter your choice:
```

然后,根据用户输入的选项执行相应的操作。

实验目的:在第 10 章实验“学生成绩管理系统 V3.0”的基础上,通过增加任务要求,熟悉二维数组作函数参数、模块化程序设计以及增量测试方法。

【思考题】

(1) 参考例 11.3,用二维数组的列指针代替二维数组作函数参数,编程实现上述程序中的函数。

(2) 参考例 11.4,用指针数组代替二维字符数组作函数参数,编写“按姓名查询”函数。

(3) 参考例 11.6、例 11.7,用动态数组编程实现上述程序中的函数。

参考程序如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LEN 10                  /* 字符串最大长度 */
#define STU_NUM 30                  /* 最多的学生人数 */
#define COURSE_NUM 6                /* 最多的考试科目数 */
int Menu(void);
void ReadScore(long num[], char name[][MAX_LEN],
               float score[][COURSE_NUM], int n, int m);
void AverSumofEveryStudent(float score[][COURSE_NUM], int n, int m,
                           float sum[STU_NUM],float aver[STU_NUM]);
void AverSumofEveryCourse(float score[][COURSE_NUM], int n, int m);
void SortbyScore(long num[], char name[][MAX_LEN],
                 float score[][COURSE_NUM], float sum[], float aver[],
                 int n, int m, int (*compare)(float a, float b));
int Ascending(float a, float b);
int Descending(float a, float b);
void SwapFloat(float *x, float *y);
void SwapLong(long *x, long *y);
void SwapChar(char x[], char y[]);
void AsSortbyNum(long num[], char name[][MAX_LEN],
                 float score[][COURSE_NUM], float sum[], float aver[],
                 int n, int m);
void SortbyName(long num[], char name[][MAX_LEN],
                float score[][COURSE_NUM], float sum[], float aver[],
```

```
                    int n, int m);
void SearchbyNum(long num[], char name[][MAX_LEN],
                    float score[][COURSE_NUM], float sum[], float aver[],
                    int n, int m);
void SearchbyName(long num[], char name[][MAX_LEN],
                    float score[][COURSE_NUM], float sum[], float aver[],
                    int n, int m);
void StatisticAnalysis(float score[][COURSE_NUM], int n, int m);
void PrintScore(long num[], char name[][MAX_LEN],
                    float score[][COURSE_NUM], float sum[], float aver[],
                    int n, int m) ;
int main()
{
    char ch;
    int n = 0, m = 0;
    float score[STU_NUM][COURSE_NUM], sum[STU_NUM], aver[STU_NUM];
    long num[STU_NUM];
    char name[STU_NUM][MAX_LEN];
    printf("Input student number(n<%d):", STU_NUM);
    scanf("%d", &n);
    printf("Input course number(m<=%d):",COURSE_NUM);
    scanf("%d", &m);
    while (1)
    {
        ch = Menu();          /* 显示菜单,并读取用户输入 */
        switch (ch)
        {
            case 1:ReadScore(num, name, score, n, m);
                   break;
            case 2:AverSumofEveryCourse(score, n, m);
                   break;
            case 3:AverSumofEveryStudent(score, n, m, sum, aver);
                   break;
            case 4:SortbyScore(num,name,score,sum,aver,n,m,Descending);
                   printf("\nSort in descending order by score:\n");
                   PrintScore(num, name, score, sum, aver, n, m);
                   break;
            case 5:SortbyScore(num,name,score,sum,aver,n,m,Ascending);
```

```
64                      printf("\nSort in ascending order by score:\n");
65                      PrintScore(num, name, score, sum, aver, n, m);
66                      break;
67              case 6:AsSortbyNum(num, name, score, sum, aver, n, m);
68                      printf("\nSort in ascending order by number:\n");
69                      PrintScore(num, name, score, sum, aver, n, m);
70                      break;
71              case 7:SortbyName(num, name, score, sum, aver, n, m);
72                      printf("\nSort in dictionary order by name:\n");
73                      PrintScore(num, name, score, sum, aver, n, m);
74                      break;
75              case 8:SearchbyNum(num, name, score, sum, aver, n, m);
76                      break;
77              case 9:SearchbyName(num, name, score, sum, aver, n, m);
78                      break;
79              case 10:StatisticAnalysis(score, n, m);
80                       break;
81              case 11:PrintScore(num, name, score, sum, aver, n, m);
82                       break;
83              case 0: printf("End of program!");
84                       exit(0);
85              default:printf("Input error!");
86          }
87      }
88      return 0;
89  }
90  /* 函数功能:显示菜单并获得用户键盘输入的选项 */
91  int Menu(void)
92  {
93          int itemSelected;
94          printf("Management for Students' scores\n");
95          printf("1.Input record\n");
96          printf("2.Caculate total and average score of every course\n");
97          printf("3.Caculate total and average score of every student\n");
98          printf("4.Sort in descending order by score\n");
99          printf("5.Sort in ascending order by score\n");
100         printf("6.Sort in ascending order by number\n");
101         printf("7.Sort in dictionary order by name\n");
```

```
102         printf("8.Search by number\n");
103         printf("9.Search by name\n");
104         printf("10.Statistic analysis\n");
105         printf("11.List record\n");
106         printf("0.Exit\n");
107         printf("Please Input your choice:");
108         scanf("%d", &itemSelected);              /* 读入用户输入 */
109         return itemSelected;
110     }
111     /* 函数功能:输入 n 个学生的 m 门课成绩 */
112     void ReadScore(long num[], char name[][MAX_LEN],
113                    float score[][COURSE_NUM], int n, int m)
114     {
115         int i, j;
116         printf("Input student's ID, name and score:\n");
117         for (i=0; i<n; i++)
118         {
119             scanf("%ld%s", &num[i], name[i]);
120             for (j=0; j<m; j++)
121             {
122                 scanf("%f", &score[i][j]);
123             }
124         }
125     }
126     /* 函数功能:计算每个学生各门课程的总分和平均分 */
127     void AverSumofEveryStudent(float score[][COURSE_NUM], int n, int m,
128                                float sum[STU_NUM],float aver[STU_NUM])
129     {
130         int i, j;
131         for (i=0; i<n; i++)
132         {
133             sum[i] = 0;
134             for (j=0; j<m; j++)
135             {
136                 sum[i] = sum[i] + score[i][j];
137             }
138             aver[i] = m>0 ? sum[i] / m : -1;
139             printf("student %d:sum=%.0f, aver=%.0f\n",i+1,sum[i],aver[i]);
```

```
140         }
141     }
142     /* 函数功能:计算每门课程的总分和平均分 */
143     void AverSumofEveryCourse(float score[][COURSE_NUM], int n, int m)
144     {
145         int i, j;
146         float sum[COURSE_NUM], aver[COURSE_NUM];
147         for (j=0; j<m; j++)
148         {
149             sum[j] = 0;
150             for (i=0; i<n; i++)
151             {
152                 sum[j] = sum[j] + score[i][j];
153             }
154             aver[j] = n>0 ? sum[j] / n : -1;
155             printf("course %d:sum=%.0f, aver=%.0f\n",j+1,sum[j],aver[j]);
156         }
157     }
158     /* 函数功能:按选择法将数组 sum 的元素值排序 */
159     void SortbyScore(long num[], char name[][MAX_LEN],
160                      float score[][COURSE_NUM], float sum[], float aver[],
161                      int n, int m, int (*compare)(float a, float b))
162     {
163         int i, j, k, t;
164         for (i=0; i<n-1; i++)
165         {
166             k = i;
167             for (j=i+1; j<n; j++)
168             {
169                 if ((*compare)(sum[j], sum[k])) k = j;
170             }
171             if (k != i)
172             {
173                 for (t=0; t<m; t++)       /* 交换 m 门课程的成绩 */
174                 {
175                     SwapFloat(&score[k][t], &score[i][t]);
176                 }
177                 SwapFloat(&sum[k], &sum[i]);       /* 交换总分 */
```

```
178                 SwapFloat(&aver[k], &aver[i]);    /* 交换平均分 */
179                 SwapLong(&num[k], &num[i]);       /* 交换学号 */
180                 SwapChar(name[k], name[i]);       /* 交换姓名 */
181             }
182         }
183     }
184     /* 使数据按升序排序 */
185     int Ascending(float a, float b)
186     {
187         return a < b;       /* 这样比较决定了按升序排序,如果 a<b,则交换 */
188     }
189     /* 使数据按降序排序 */
190     int Descending(float a, float b)
191     {
192         return a > b;     /* 这样比较决定了按降序排序,如果 a>b,则交换 */
193     }
194     /* 交换两个单精度浮点型数据 */
195     void SwapFloat(float *x, float *y)
196     {
197         float temp;
198         temp =*x;
199         *x = *y;
200         *y = temp;
201     }
202     /* 交换两个长整型数据 */
203     void SwapLong(long *x, long *y)
204     {
205         long temp;
206         temp = *x;
207         *x = *y;
208         *y = temp;
209     }
210     /* 交换两个字符串 */
211     void SwapChar(char x[], char y[])
212     {
213         char temp[MAX_LEN];
214         strcpy(temp, x);
215         strcpy(x, y);
```

```
    strcpy(y, temp);
}
/* 函数功能:按选择法将数组 num 的元素值按从低到高排序 */
void AsSortbyNum(long num[], char name[][MAX_LEN],
                 float score[][COURSE_NUM], float sum[], float aver[],
                 int n, int m)
{
    int i, j, k, t;
    for (i=0; i<n-1; i++)
    {
        k = i;
        for (j=i+1; j<n; j++)
        {
            if (num[j] < num[k]) k = j;
        }
        if (k != i)
        {
            for (t=0; t<m; t++)     /* 交换 m 门课程的成绩 */
            {
                SwapFloat(&score[k][t], &score[i][t]);
            }
            SwapFloat(&sum[k], &sum[i]);        /* 交换总分 */
            SwapFloat(&aver[k], &aver[i]);      /* 交换平均分 */
            SwapLong(&num[k], &num[i]);         /* 交换学号 */
            SwapChar(name[k], name[i]);         /* 交换姓名 */
        }
    }
}
/* 函数功能:交换法实现字符串按字典顺序排序 */
void SortbyName(long num[], char name[][MAX_LEN], float score[][COURSE_NUM],
                float sum[], float aver[], int n, int m)
{
    int i, j, t;
    for (i=0; i<n-1; i++)
    {
        for (j = i+1; j<n; j++)
        {
            if (strcmp(name[j], name[i]) < 0)
```

```
254             {
255                 for (t=0; t<m; t++)      /* 交换 m 门课程的成绩 */
256                 {
257                     SwapFloat(&score[i][t], &score[j][t]);
258                 }
259                 SwapFloat(&sum[i], &sum[j]);      /* 交换总分 */
260                 SwapFloat(&aver[i], &aver[j]);    /* 交换平均分 */
261                 SwapLong(&num[i], &num[j]);       /* 交换学号 */
262                 SwapChar(name[i], name[j]);       /* 交换姓名 */
263             }
264         }
265     }
266 }
267 /* 函数功能:按学号查找学生成绩并显示查找结果 */
268 void SearchbyNum(long num[], char name[][MAX_LEN],
269                  float score[][COURSE_NUM], float sum[], float aver[],
270                  int n, int m)
271 {
272     long number;
273     int i, j;
274     printf("Input the number you want to search:");
275     scanf("%ld", &number);
276     for (i=0; i<n; i++)
277     {
278         if (num[i] == number)
279         {
280             printf("%ld\t%s\t", num[i], name[i]);
281             for (j=0; j<m; j++)
282             {
283                 printf("%.0f\t", score[i][j]);
284             }
285             printf("%.0f\t%.0f\n", sum[i], aver[i]);
286             return;
287         }
288     }
289     printf("\nNot found! \n");
290 }
291 /* 函数功能:按姓名查找学生成绩并显示查找结果 */
```

```
void SearchbyName(long num[], char name[][MAX_LEN],
                  float score[][COURSE_NUM], float sum[], float aver[],
                  int n, int m)
{
    char x[MAX_LEN];
    int i, j;
    printf("Input the name you want to search:");
    scanf("%s", x);
    for (i=0; i<n; i++)
    {
        if (strcmp(name[i], x) == 0)
        {
            printf("%ld\t%s\t", num[i], name[i]);
            for (j=0; j<m; j++)
            {
                printf("%.0f\t", score[i][j]);
            }
            printf("%.0f\t%.0f\n", sum[i], aver[i]);
            return;
        }
    }
    printf("\nNot found! \n");
}
/* 函数功能:统计各分数段的学生人数及所占的百分比 */
void StatisticAnalysis(float score[][COURSE_NUM], int n, int m)
{
    int i, j, total, t[6];
    for (j=0; j<m; j++)
    {
        printf("For course %d:\n", j+1);
        memset(t, 0, sizeof(t));      /* 将数组 t 的全部元素初始化为 0 */
        for (i=0; i<n; i++)
        {
            if(score[i][j]>=0 && score[i][j]<60)  t[0]++;
            else if (score[i][j]<70)              t[1]++;
            else if (score[i][j]<80)              t[2]++;
            else if (score[i][j]<90)              t[3]++;
            else if (score[i][j]<100)             t[4]++;
```

```
330             else if (score[i][j] == 100)            t[5]++;
331         }
332         for (total=0, i=0; i<=5; i++)
333         {
334             total = total + t[i];
335         }
336         for (i=0; i<=5; i++)
337         {
338             if (i == 0) printf("<60\t%d\t%.2f%%\n",t[i],(float)t[i]/n*100);
339             else if (i == 5) printf("%d\t%d\t%.2f%%\n",
340                                     (i+5)* 10,t[i],(float)t[i]/n*100);
341             else printf("%d-%d\t%d\t%.2f%%\n",
342                         (i+5)* 10, (i+5)* 10+9, t[i], (float)t[i]/n*100);
343         }
344     }
345 }
346 /* 函数功能: 打印学生成绩 */
347 void PrintScore(long num[], char name[][MAX_LEN],
348                 float score[][COURSE_NUM], float sum[],
349                 float aver[], int n, int m)
350 {
351     int i, j;
352     for (i=0; i<n; i++)
353     {
354         printf("%ld\t%s\t", num[i], name[i]);
355         for (j=0; j<m; j++)
356         {
357             printf("%.0f\t", score[i][j]);
358         }
359         printf("%.0f\t%.0f\n", sum[i], aver[i]);
360     }
361 }
```

2.2.11 第 12 章实验题:学生成绩管理系统 V5.0

某班有最多不超过 30 人(具体人数由键盘输入)参加期末考试,最多不超过 6 门(具体门数由键盘输入)。参考例 12.7,**定义结构体类型,用结构体数组作函数参数**,编程实现如下菜单驱动的学生成绩管理系统:

(1) 录入每个学生的学号、姓名和各科考试成绩。

(2) 计算每门课程的总分和平均分。

(3) 计算每个学生的总分和平均分。

(4) 按每个学生的总分由高到低排出名次表。

(5) 按每个学生的总分由低到高排出名次表。

(6) 按学号由小到大排出成绩表。

(7) 按姓名的字典顺序排出成绩表。

(8) 按学号查询学生排名及其考试成绩。

(9) 按姓名查询学生排名及其考试成绩。

(10) 按优秀(90~100)、良好(80~89)、中等(70~79)、及格(60~69)、不及格(0~59)5 个类别,对每门课程分别统计每个类别的人数以及所占的百分比。

(11) 输出每个学生的学号、姓名、各科考试成绩以及每门课程的总分和平均分。

要求程序运行后先显示如下菜单,并提示用户输入选项:

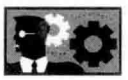
```
1. Input record
2. Caculate total and average score of every course
3. Caculate total and average score of every student
4. Sort in descending order by total score of every student
5. Sort in ascending order by total score of every student
6. Sort in ascending order by number
7. Sort in dictionary order by name
8. Search by number
9. Search by name
10. Statistic analysis for every course
11. List record
0. Exit
Please enter your choice:
```

然后,根据用户输入的选项执行相应的操作。

实验目的:在第 11 章实验“学生成绩管理系统 V4.0”的基础上,通过增加任务要求,熟悉结构体类型、结构体数组作函数参数、模块化程序设计方法,体会应用结构体类型代替普通的数组类型实现数据库管理的优越性。

【思考题】

(1) 参考例 12.9~例 12.11,用动态单向链表代替结构体数组,编程实现上述程序。



(2) 在(1)基础上,增加“添加记录”、“删除记录”和“插入记录”的功能,体会动态链表与结构体数组的不同点和优缺点。

参考程序如下:

```
1    #include <stdio.h>
2    #include <stdlib.h>
3    #include <string.h>
```

```
4    #define MAX_LEN 10                  /* 字符串最大长度 */
5    #define STU_NUM 30                  /* 最多的学生人数 */
6    #define COURSE_NUM 6                /* 最多的考试科目数 */
7    typedef struct student
8    {
9        long num;                       /* 每个学生的学号 */
10       char name[MAX_LEN];             /* 每个学生的姓名 */
11       float score[COURSE_NUM];        /* 每个学生 COURSE_NUM 门功课的成绩 */
12       float sum;                      /* 每个学生的总成绩 */
13       float aver;                     /* 每个学生的平均成绩 */
14   }STU;
15   int Menu(void);
16   void ReadScore(STU stu[], int n, int m);
17   void AverSumofEveryStudent(STU stu[], int n, int m);
18   void AverSumofEveryCourse(STU stu[], int n, int m);
19   void SortbyScore(STU stu[],int n,int m,int (*compare)(float a,float b));
20   int Ascending(float a, float b);
21   int Descending(float a, float b);
22   void SwapFloat(float *x, float *y);
23   void SwapLong(long *x, long *y);
24   void SwapChar(char x[], char y[]);
25   void AsSortbyNum(STU stu[], int n, int m);
26   void SortbyName(STU stu[], int n, int m);
27   void SearchbyNum(STU stu[], int n, int m);
28   void SearchbyName(STU stu[], int n, int m);
29   void StatisticAnalysis(STU stu[], int n, int m);
30   void PrintScore(STU stu[], int n, int m);
31   int main()
32   {
33       char ch;
34       int n = 0, m = 0;     /* 学生人数为 n,课程门数为 m */
35       STU stu[STU_NUM];
36       printf("Input student number(n<=%d):", STU_NUM);
37       scanf("%d", &n);
38       printf("Input course number(m<=%d):",COURSE_NUM);
39       scanf("%d", &m);
```

```
40      while (1)
41      {
42          ch = Menu();            /* 显示菜单,并读取用户输入 */
43          switch (ch)
44          {
45              case 1:ReadScore(stu, n, m);
46                     break;
47              case 2:AverSumofEveryCourse(stu, n, m);
48                     break;
49              case 3:AverSumofEveryStudent(stu, n, m);
50                     break;
51              case 4:SortbyScore(stu, n, m, Descending);
52                     printf("\nSort in descending order by score:\n");
53                     PrintScore(stu, n, m);
54                     break;
55              case 5:SortbyScore(stu, n, m, Ascending);
56                     printf("\nSort in ascending order by score:\n");
57                     PrintScore(stu, n, m);
58                     break;
59              case 6:AsSortbyNum(stu, n, m);
60                     printf("\nSort in ascending order by number:\n");
61                     PrintScore(stu, n, m);
62                     break;
63              case 7:SortbyName(stu, n, m);
64                     printf("\nSort in dictionary order by name:\n");
65                     PrintScore(stu, n, m);
66                     break;
67              case 8:SearchbyNum(stu, n, m);
68                     break;
69              case 9:SearchbyName(stu, n, m);
70                     break;
71              case 10:StatisticAnalysis(stu, n, m);
72                      break;
73              case 11:PrintScore(stu, n, m);
74                      break;
75              case 0: printf("End of program!");
```

```
76                      exit(0);
77              default:printf("Input error!");
78          }
79      }
80      return 0;
81  }
82  /* 函数功能:显示菜单并获得用户键盘输入的选项 */
83  int Menu(void)
84  {
85          int itemSelected;
86          printf("Management for Students' scores\n");
87          printf("1.Input record\n");
88          printf("2.Caculate total and average score of every course\n");
89          printf("3.Caculate total and average score of every student\n");
90          printf("4.Sort in descending order by score\n");
91          printf("5.Sort in ascending order by score\n");
92          printf("6.Sort in ascending order by number\n");
93          printf("7.Sort in dictionary order by name\n");
94          printf("8.Search by number\n");
95          printf("9.Search by name\n");
96          printf("10.Statistic analysis\n");
97          printf("11.List record\n");
98          printf("0.Exit\n");
99          printf("Please Input your choice:");
100         scanf("%d", &itemSelected);      /* 读入用户输入 */
101         return itemSelected;
102 }
103 /* 函数功能:输入 n 个学生的 m 门课程成绩 */
104 void ReadScore(STU stu[], int n, int m)
105 {
106     int i, j;
107     printf("Input student's ID, name and score:\n");
108     for (i=0; i<n; i++)
109     {
110         scanf("%ld%s", &stu[i].num, stu[i].name);
111         for (j=0; j<m; j++)
```

```
112         {
113             scanf("%f", &stu[i].score[j]);
114         }
115     }
116 }
117 /* 函数功能:计算每个学生各门课程的总分和平均分 */
118 void AverSumofEveryStudent(STU stu[], int n, int m)
119 {
120     int i, j;
121     for (i=0; i<n; i++)
122     {
123         stu[i].sum = 0;
124         for (j=0; j<m; j++)
125         {
126             stu[i].sum = stu[i].sum + stu[i].score[j];
127         }
128         stu[i].aver = m>0 ? stu[i].sum / m : -1;
129         printf("student %d: sum = %.0f, aver = %.0f\n",
130                 i+1,stu[i].sum, stu[i].aver);
131     }
132 }
133 /* 函数功能:计算每门课程的总分和平均分 */
134 void AverSumofEveryCourse(STU stu[], int n, int m)
135 {
136     int i, j;
137     float sum[COURSE_NUM], aver[COURSE_NUM];
138     for (j=0; j<m; j++)
139     {
140         sum[j] = 0;
141         for (i=0; i<n; i++)
142         {
143             sum[j] = sum[j] + stu[i].score[j];
144         }
145         aver[j] = n>0 ? sum[j] / n : -1;
146         printf("course %d:sum=%.0f, aver=%.0f\n", j+1,sum[j],aver[j]);
147     }
```

```
148 }
149 /* 函数功能:按选择法将数组 sum 的元素值排序 */
150 void SortbyScore(STU stu[], int n, int m, int (*compare)(float a, float b))
151 {
152     int i, j, k, t;
153     for (i=0; i<n-1; i++)
154     {
155         k = i;
156         for (j=i+1; j<n; j++)
157         {
158             if ((*compare)(stu[j].sum, stu[k].sum)) k = j;
159         }
160         if (k != i)
161         {
162             for (t=0; t<m; t++)              /* 交换 m 门课程的成绩 */
163             {
164                 SwapFloat(&stu[k].score[t], &stu[i].score[t]);
165             }
166             SwapFloat(&stu[k].sum, &stu[i].sum);     /* 交换总分 */
167             SwapFloat(&stu[k].aver, &stu[i].aver);   /* 交换平均分 */
168             SwapLong(&stu[k].num, &stu[i].num);      /* 交换学号 */
169             SwapChar(stu[k].name, stu[i].name);      /* 交换姓名 */
170         }
171     }
172 }
173 /* 使数据按升序排序 */
174 int Ascending(float a, float b)
175 {
176     return a < b;       /* 这样比较决定了按升序排序,如果 a<b,则交换 */
177 }
178 /* 使数据按降序排序 */
179 int Descending(float a, float b)
180 {
181     return a > b;     /* 这样比较决定了按降序排序,如果 a>b,则交换 */
182 }
183 /* 交换两个单精度浮点型数据 */
```

```
184  void SwapFloat(float *x, float *y)
185  {
186      float temp;
187      temp = *x;
188      *x = *y;
189      *y = temp;
190  }
191  /* 交换两个长整型数据 */
192  void SwapLong(long *x, long *y)
193  {
194      long temp;
195      temp = *x;
196      *x = *y;
197      *y = temp;
198  }
199  /* 交换两个字符串 */
200  void SwapChar(char x[], char y[])
201  {
202      char temp[MAX_LEN];
203      strcpy(temp, x);
204      strcpy(x, y);
205      strcpy(y, temp);
206  }
207  /* 函数功能:按选择法将数组 num 的元素值按从低到高排序 */
208  void AsSortbyNum(STUstu[], int n, int m)
209  {
210      int i, j, k, t;
211      for (i=0; i<n-1; i++)
212      {
213          k = i;
214          for (j=i+1; j<n; j++)
215          {
216              if (stu[j].num < stu[k].num) k = j;
217          }
218          if (k != i)
219          {
```

```
220                 for (t=0; t<m; t++)              /* 交换 m 门课程的成绩 */
221                 {
222                     SwapFloat(&stu[k].score[t], &stu[i].score[t]);
223                 }
224               SwapFloat(&stu[k].sum, &stu[i].sum);     /* 交换总分 */
225               SwapFloat(&stu[k].aver, &stu[i].aver);   /* 交换平均分 */
226               SwapLong(&stu[k].num, &stu[i].num);      /* 交换学号 */
227               SwapChar(stu[k].name, stu[i].name);      /* 交换姓名 */
228             }
229         }
230     }
231     /* 函数功能:交换法实现字符串按字典顺序排序 */
232     void SortbyName(STU stu[], int n, int m)
233     {
234         int i, j, t;
235         for (i=0; i<n-1; i++)
236         {
237             for (j = i+1; j<n; j++)
238             {
239                 if (strcmp(stu[j].name, stu[i].name) < 0)
240                 {
241                     for (t=0; t<m; t++)     /* 交换 m 门课程的成绩 */
242                     {
243                         SwapFloat(&stu[i].score[t], &stu[j].score[t]);
244                     }
245                     SwapFloat(&stu[i].sum, &stu[j].sum);   /* 交换总分 */
246                     SwapFloat(&stu[i].aver, &stu[j].aver);   /* 交换平均分 */
247                     SwapLong(&stu[i].num, &stu[j].num);    /* 交换学号 */
248                     SwapChar(stu[i].name, stu[j].name);    /* 交换姓名 */
249                 }
250             }
251         }
252     }
253     /* 函数功能:按学号查找学生成绩并显示查找结果 */
254     void SearchbyNum(STU stu[], int n, int m)
255     {
```

```
256     long number;
257     int i, j;
258     printf("Input the number you want to search:");
259     scanf("%ld", &number);
260     for (i=0; i<n; i++)
261     {
262         if (stu[i].num == number)
263         {
264             printf("%ld\t%s\t", stu[i].num, stu[i].name);
265             for (j=0; j<m; j++)
266             {
267                 printf("%.0f\t", stu[i].score[j]);
268             }
269             printf("%.0f\t%.0f\n", stu[i].sum, stu[i].aver);
270             return;
271         }
272     }
273     printf("\nNot found! \n");
274 }
275 /* 函数功能:按姓名查找学生成绩并显示查找结果 */
276 void SearchbyName(STU stu[], int n, int m)
277 {
278     char x[MAX_LEN];
279     int i, j;
280     printf("Input the name you want to search:");
281     scanf("%s", x);
282     for (i=0; i<n; i++)
283     {
284         if (strcmp(stu[i].name, x) == 0)
285         {
286             printf("%ld\t%s\t", stu[i].num, stu[i].name);
287             for (j=0; j<m; j++)
288             {
289                 printf("%.0f\t", stu[i].score[j]);
290             }
291             printf("%.0f\t%.0f\n", stu[i].sum, stu[i].aver);
```

```
292             return;
293         }
294     }
295     printf("\nNot found! \n");
296 }
297 /* 函数功能:统计各分数段的学生人数及所占的百分比 */
298 void StatisticAnalysis(STU stu[], int n, int m)
299 {
300     int i, j, total, t[6];
301     for (j=0; j<m; j++)
302     {
303         printf("For course %d:\n", j+1);
304         memset(t, 0, sizeof(t));    /* 将数组 t 的全部元素初始化为 0 */
305         for (i=0; i<n; i++)
306         {
307             if (stu[i].score[j]>=0 && stu[i].score[j]<60)  t[0]++;
308             else if (stu[i].score[j]<70)                   t[1]++;
309             else if (stu[i].score[j]<80)                   t[2]++;
310             else if (stu[i].score[j]<90)                   t[3]++;
311             else if (stu[i].score[j]<100)                  t[4]++;
312             else if (stu[i].score[j] == 100)               t[5]++;
313         }
314         for (total=0, i=0; i<=5; i++)
315         {
316             total = total + t[i];
317         }
318         for (i=0; i<=5; i++)
319         {
320             if (i==0) printf("<60\t%d\t%.2f%%\n",t[i],(float)t[i]/n*100);
321             else if (i==5) printf("%d\t%d\t%.2f%%\n",
322                                   (i+5)*10,t[i],(float)t[i]/n*100);
323             else printf("%d-%d\t%d\t%.2f%%\n",
324                         (i+5)*10, (i+5)* 10+9, t[i], (float)t[i]/n*100);
325         }
326     }
327 }
```

```
328    /* 函数功能：打印学生成绩 */
329    void PrintScore(STU stu[], int n, int m)
330    {
331        int i, j;
332        for (i=0; i<n; i++)
333        {
334            printf("%ld\t%s\t", stu[i].num, stu[i].name);
335            for (j=0; j<m; j++)
336            {
337                printf("%.0f\t", stu[i].score[j]);
338            }
339            printf("%.0f\t%.0f\n", stu[i].sum, stu[i].aver);
340        }
341    }
```

对比程序 V4.0 可知，改用结构体数组存储学生信息，将每个学生的姓名、学号、成绩（包括总分和平均分）也封装到结构体中后，函数的接口变简洁了，而且增减结构体成员时函数的接口参数不必修改。

2.2.12 第 13 章实验题：学生成绩管理系统 V6.0

某班有最多不超过 30 人（具体人数由键盘输入）参加期末考试，考试科目最多不超过 6 门（具体门数由键盘输入）课程。学生成绩管理系统是一个非常实用的程序，如果能够把用户输入的数据存盘，下次运行时读出，就更有用了。参考例 13.7，**增加文件读写的功能**，即编程实现如下菜单驱动的学生成绩管理系统：

（1）录入每个学生的学号、姓名和各科考试成绩。

（2）计算每门课程的总分和平均分。

（3）计算每个学生的总分和平均分。

（4）按每个学生的总分由高到低排出名次表。

（5）按每个学生的总分由低到高排出名次表。

（6）按学号由小到大排出成绩表。

（7）按姓名的字典顺序排出成绩表。

（8）按学号查询学生排名及其考试成绩。

（9）按姓名查询学生排名及其考试成绩。

（10）按优秀（90～100）、良好（80～89）、中等（70～79）、及格（60～69）、不及格（0～59）5 个类别，对每门课程分别统计每个类别的人数以及所占的百分比。

（11）输出每个学生的学号、姓名、各科考试成绩以及每门课程的总分和平均分。

（12）**将每个学生的记录信息写入文件。**

(13) 从文件中读出每个学生的记录信息并显示。

要求程序运行后先显示如下菜单,并提示用户输入选项:

```
1. Input record
2. Caculate total and average score of every course
3. Caculate total and average score of every student
4. Sort in descending order by total score of every student
5. Sort in ascending order by total score of every student
6. Sort in ascending order by number
7. Sort in dictionary order by name
8. Search by number
9. Search by name
10. Statistic analysis for every course
11. List record
12. Write to a file
13. Read from a file
0. Exit
Please enter your choice:
```

然后,根据用户输入的选项执行相应的操作。

实验目的:在第12章实验"学生成绩管理系统V5.0"的基础上,通过增加任务要求,熟悉文件的基本操作、模块化程序设计以及增量测试方法。

【思考题】

(1) 前面的实验中,没有要求程序检查用户输入数据的有效性。当输入的数据量较大时,输入不出错的可能性极小。那么,如何在程序中加入异常处理,检查用户输入数据的有效性,以保证程序的健壮性呢?

(2) 参考学生成绩管理系统,编写其他数据库管理程序,如通讯录管理系统、图书馆管理系统、学生宿舍管理系统、职工工资管理系统、职工人事档案管理系统、航空订票系统等。

本章实验题程序在第12章实验题程序的基础上增加了2个函数,参考程序如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LEN 10                    /* 字符串最大长度 */
#define STU_NUM 30                    /* 最多的学生人数 */
#define COURSE_NUM6                   /* 最多的考试科目数 */
typedef struct student
{
    long num;                         /* 每个学生的学号 */
```

```
10      char name[MAX_LEN];          /* 每个学生的姓名 */
11      float score[COURSE_NUM];     /* 每个学生 COURSE_NUM 门功课的成绩 */
12      float sum;                   /* 每个学生的总成绩 */
13      float aver;                  /* 每个学生的平均成绩 */
14  }STU;
15  int Menu(void);
16  void ReadScore(STU stu[], int n, int m);
17  void AverSumofEveryStudent(STU stu[], int n, int m);
18  void AverSumofEveryCourse(STU stu[], int n, int m);
19  void SortbyScore(STU stu[],int n,int m,int (*compare)(float a, float b));
20  int Ascending(float a, float b);
21  int Descending(float a, float b);
22  void SwapFloat(float *x, float *y);
23  void SwapLong(long *x, long *y);
24  void SwapChar(char x[], char y[]);
25  void AsSortbyNum(STU stu[], int n, int m);
26  void SortbyName(STU stu[], int n, int m);
27  void SearchbyNum(STU stu[], int n, int m);
28  void SearchbyName(STU stu[], int n, int m);
29  void StatisticAnalysis(STU stu[], int n, int m);
30  void PrintScore(STU stu[], int n, int m);
31  void WritetoFile(STU record[], int n, int m);
32  void ReadfromFile(STU record[], int *n,int *m);
33  int main()
34  {
35      char ch;
36      int n = 0, m = 0;
37      STU stu[STU_NUM];
38      printf("Input student number(n<%d):", STU_NUM);
39      scanf("%d", &n);
40      printf("Input course number(m<=%d):",COURSE_NUM);
41      scanf("%d", &m);
42      while (1)
43      {
44          ch = Menu();        /* 显示菜单,并读取用户输入 */
45          switch (ch)
```

```
46          {
47              case 1:ReadScore(stu, n, m);
48                     break;
49              case 2:AverSumofEveryCourse(stu, n, m);
50                     break;
51              case 3:AverSumofEveryStudent(stu, n, m);
52                     break;
53              case 4:SortbyScore(stu, n, m, Descending);
54                     printf("\nSort in descending order by score:\n");
55                     PrintScore(stu, n, m);
56                     break;
57              case 5:SortbyScore(stu, n, m, Ascending);
58                     printf("\nSort in ascending order by score:\n");
59                     PrintScore(stu, n, m);
60                     break;
61              case 6:AsSortbyNum(stu, n, m);
62                     printf("\nSort in ascending order by number:\n");
63                     PrintScore(stu, n, m);
64                     break;
65              case 7:SortbyName(stu, n, m);
66                     printf("\nSort in dictionary order by name:\n");
67                     PrintScore(stu, n, m);
68                     break;
69              case 8:SearchbyNum(stu, n, m);
70                     break;
71              case 9:SearchbyName(stu, n, m);
72                     break;
73              case 10:StatisticAnalysis(stu, n, m);
74                      break;
75              case 11:PrintScore(stu, n, m);
76                      break;
77              case 12:WritetoFile(stu, n, m);
78                     break;
79              case 13:ReadfromFile(stu, &n,&m);
80                     break;
81              case 0: printf("End of program! \n");
```

```
82                      exit(0);
83                  default:printf("Input error! \n");
84              }
85          }
86          return 0;
87      }
88      /* 函数功能:显示菜单并获得用户键盘输入的选项 */
89      int Menu(void)
90      {
91          int itemSelected;
92          printf("Management for Students' scores \n");
93          printf("1.Input record \n");
94          printf("2.Caculate total and average score of every course \n");
95          printf("3.Caculate total and average score of every student \n");
96          printf("4.Sort in descending order by score \n");
97          printf("5.Sort in ascending order by score \n");
98          printf("6.Sort in ascending order by number \n");
99          printf("7.Sort in dictionary order by name \n");
100         printf("8.Search by number \n");
101         printf("9.Search by name \n");
102         printf("10.Statistic analysis \n");
103         printf("11.List record \n");
104         printf("12.Write to a file \n");
105         printf("13.Read from a file \n");
106         printf("0.Exit \n");
107         printf("Please Input your choice:");
108         scanf("%d", &itemSelected);     /* 读入用户输入 */
109         return itemSelected;
110     }
111     /* 函数功能:输入 n 个学生的 m 门课程成绩 */
112     void ReadScore(STU stu[], int n, int m)
113     {
114         int i, j;
115         printf("Input student's ID, name and score: \n");
116         for (i=0; i<n; i++)
117         {
```

```
118             scanf("%ld%s", &stu[i].num, stu[i].name);
119             for (j=0; j<m; j++)
120             {
121                 scanf("%f", &stu[i].score[j]);
122             }
123         }
124     }
125     /* 函数功能:计算每个学生各门课程的总分和平均分 */
126     void AverSumofEveryStudent(STU stu[], int n, int m)
127     {
128         int i, j;
129         for (i=0; i<n; i++)
130         {
131             stu[i].sum = 0;
132             for (j=0; j<m; j++)
133             {
134                 stu[i].sum = stu[i].sum + stu[i].score[j];
135             }
136             stu[i].aver = m>0 ? stu[i].sum / m : -1;
137             printf("student %d: sum = %.0f, aver = %.0f\n",
138                    i+1, stu[i].sum, stu[i].aver);
139         }
140     }
141     /* 函数功能:计算每门课程的总分和平均分 */
142     void  AverSumofEveryCourse(STU stu[], int n, int m)
143     {
144         int i, j;
145         float sum[COURSE_NUM], aver[COURSE_NUM];
146         for (j=0; j<m; j++)
147         {
148             sum[j] = 0;
149             for (i=0; i<n; i++)
150             {
151                 sum[j] = sum[j] + stu[i].score[j];
152             }
153             aver[j] = n>0 ? sum[j] / n : -1;
```

```
154             printf("course %d:sum=%.0f, aver=%.0f\n",j+1,sum[j],aver[j]);
155         }
156     }
157     /* 函数功能:按选择法将数组 sum 的元素值排序 */
158     void SortbyScore(STU stu[],int n,int m,int (*compare)(float a,float b))
159     {
160         int i, j, k, t;
161         for (i=0; i<n-1; i++)
162         {
163             k = i;
164             for (j=i+1; j<n; j++)
165             {
166                 if ((*compare)(stu[j].sum, stu[k].sum)) k = j;
167             }
168             if (k ! = i)
169             {
170                 for (t=0; t<m; t++)              /* 交换 m 门课程的成绩 */
171                 {
172                     SwapFloat(&stu[k].score[t], &stu[i].score[t]);
173                 }
174                 SwapFloat(&stu[k].sum, &stu[i].sum);     /* 交换总分 */
175                 SwapFloat(&stu[k].aver, &stu[i].aver);   /* 交换平均分 */
176                 SwapLong(&stu[k].num, &stu[i].num);      /* 交换学号 */
177                 SwapChar(stu[k].name, stu[i].name);      /* 交换姓名 */
178             }
179         }
180     }
181     /* 使数据按升序排序 */
182     int Ascending(float a, float b)
183     {
184         return a < b;       /* 这样比较决定了按升序排序,如果 a<b,则交换 */
185     }
186     /* 使数据按降序排序 */
187     int Descending(float a, float b)
188     {
189         return a > b;       /* 这样比较决定了按降序排序,如果 a>b,则交换 */
```

```
}
/* 交换两个单精度浮点型数据 */
void SwapFloat(float *x, float *y)
{
    float temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
/* 交换两个长整型数据 */
void SwapLong(long *x, long *y)
{
    long temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
/* 交换两个字符串 */
void SwapChar(char x[], char y[])
{
    char temp[MAX_LEN];
    strcpy(temp, x);
    strcpy(x, y);
    strcpy(y, temp);
}
/* 函数功能:按选择法将数组 num 的元素值按从低到高排序 */
void AsSortbyNum(STU stu[], int n, int m)
{
    int i, j, k, t;
    for (i=0; i<n-1; i++)
    {
        k = i;
        for (j=i+1; j<n; j++)
        {
            if (stu[j].num < stu[k].num) k = j;
        }
```

```
226             if (k != i)
227             {
228                 for (t=0; t<m; t++)              /* 交换 m 门课程的成绩 */
229                 {
230                     SwapFloat(&stu[k].score[t], &stu[i].score[t]);
231                 }
232                 SwapFloat(&stu[k].sum, &stu[i].sum);     /* 交换总分 */
233                 SwapFloat(&stu[k].aver, &stu[i].aver);   /* 交换平均分 */
234                 SwapLong(&stu[k].num, &stu[i].num);      /* 交换学号 */
235                 SwapChar(stu[k].name, stu[i].name);      /* 交换姓名 */
236             }
237         }
238     }
239     /* 函数功能:交换法实现字符串按字典顺序排序 */
240     void SortbyName(STU stu[], int n, int m)
241     {
242         int i, j, t;
243         for (i=0; i<n-1; i++)
244         {
245             for (j = i+1; j<n; j++)
246             {
247                 if (strcmp(stu[j].name, stu[i].name) < 0)
248                 {
249                     for (t=0; t<m; t++)  /* 交换 m 门课程的成绩 */
250                     {
251                         SwapFloat(&stu[i].score[t], &stu[j].score[t]);
252                     }
253                     SwapFloat(&stu[i].sum, &stu[j].sum);  /* 交换总分 */
254                     SwapFloat(&stu[i].aver, &stu[j].aver);/* 交换平均分 */
255                     SwapLong(&stu[i].num, &stu[j].num);   /* 交换学号 */
256                     SwapChar(stu[i].name, stu[j].name);   /* 交换姓名 */
257                 }
258             }
259         }
260     }
261     /* 函数功能:按学号查找学生成绩并显示查找结果 */
```

```
void SearchbyNum(STU stu[], int n, int m)
{
    long number;
    int i, j;
    printf("Input the number you want to search:");
    scanf("%ld", &number);
    for (i=0; i<n; i++)
    {
        if (stu[i].num == number)
        {
            printf("%ld\t%s\t", stu[i].num, stu[i].name);
            for (j=0; j<m; j++)
            {
                printf("%.0f\t", stu[i].score[j]);
            }
            printf("%.0f\t%.0f\n", stu[i].sum, stu[i].aver);
            return;
        }
    }
    printf("\nNot found! \n");
}
/* 函数功能:按姓名查找学生成绩并显示查找结果 */
void SearchbyName(STU stu[], int n, int m)
{
    char x[MAX_LEN];
    int i, j;
    printf("Input the name you want to search:");
    scanf("%s", x);
    for (i=0; i<n; i++)
    {
        if (strcmp(stu[i].name, x) == 0)
        {
            printf("%ld\t%s\t", stu[i].num, stu[i].name);
            for (j=0; j<m; j++)
            {
                printf("%.0f\t", stu[i].score[j]);
```

```
298             }
299             printf("%.0f\t%.0f\n", stu[i].sum, stu[i].aver);
300             return;
301         }
302     }
303     printf("\nNot found! \n");
304 }
305 /* 函数功能:统计各分数段的学生人数及所占的百分比 */
306 void StatisticAnalysis(STU stu[], int n, int m)
307 {
308     int i, j, total, t[6];
309     for (j=0; j<m; j++)
310     {
311         printf("For course %d: \n", j+1);
312         memset(t, 0, sizeof(t));        /* 将数组 t 的全部元素初始化为 0 */
313         for (i=0; i<n; i++)
314         {
315             if (stu[i].score[j]>=0 && stu[i].score[j]<60)  t[0]++;
316             else if (stu[i].score[j]<70)                   t[1]++;
317             else if (stu[i].score[j]<80)                   t[2]++;
318             else if (stu[i].score[j]<90)                   t[3]++;
319             else if (stu[i].score[j]<100)                  t[4]++;
320             else if (stu[i].score[j] == 100)               t[5]++;
321         }
322         for (total=0, i=0; i<=5; i++)
323         {
324             total = total + t[i];
325         }
326         for (i=0; i<=5; i++)
327         {
328             if (i==0) printf("<60\t%d\t%.2f%%\n",t[i],(float)t[i]/n*100);
329             else if (i==5) printf("%d\t%d\t%.2f%%\n",
330                                   (i+5)*10,t[i],(float)t[i]/n*100);
331             else printf("%d-%d\t%d\t%.2f%%\n",
332                         (i+5)*10, (i+5)*10+9, t[i], (float)t[i]/n*100);
333         }
```

```
334         }
335     }
336     /* 函数功能：打印学生成绩 */
337     void PrintScore(STU stu[], int n, int m)
338     {
339         int i, j;
340         for (i=0; i<n; i++)
341         {
342             printf("%ld\t%s\t", stu[i].num, stu[i].name);
343             for (j=0; j<m; j++)
344             {
345                 printf("%.0f\t", stu[i].score[j]);
346             }
347             printf("%.0f\t%.0f\n", stu[i].sum, stu[i].aver);
348         }
349     }
350     /* 输出 n 个学生的学号、姓名及 m 门课程的成绩到文件 student.txt 中 */
351     void WritetoFile(STU stu[], int n, int m)
352     {
353         FILE *fp;
354         int i, j;
355         fprintf(fp,"%d\t%d\n",n,m);/* 将学生人数和课程门数写入文件*/
356         if ((fp = fopen("student.txt","w")) == NULL)
357         {
358             printf("Failure to open score.txt! \n");
359             exit(0);
360         }
361         for (i=0; i<n; i++)
362         {
363             fprintf(fp, "%10ld%10s", stu[i].num, stu[i].name);
364             for (j=0; j<m; j++)
365             {
366                 fprintf(fp, "%10.0f", stu[i].score[j]);
367           }
368             fprintf(fp, "%10.0f%10.0f\n", stu[i].sum, stu[i].aver);
369         }
```

```
    fclose(fp);
}
/* 从文件中读取学生的学号、姓名及成绩等信息写入到结构体数组 stu 中 */
void ReadfromFile(STU stu[], int *n,int *m)
{
    FILE *fp;
    int i, j;
    if ((fp = fopen("student.txt","r")) == NULL)
    {
        printf("Failure to open score.txt! \n");
        exit(0);
    }
    fscanf(fp,"%d\t%d",n,m);        /* 从文件中读出学生人数和课程门数*/
    for (i=0; i<*n; i++)            /* 学生人数保存在 n 指向的存储单元*/
    {
        fscanf(fp, "%10ld", &stu[i].num);
        fscanf(fp, "%10s", stu[i].name);
        for (j=0; j<*m; j++)        /* 课程门数保存在 m 指向的存储单元*/
        {
            fscanf(fp, "%10f", &stu[i].score[j]);  /* 不能用%10.0f */
        }
        fscanf(fp, "%10f%10f", &stu[i].sum, &stu[i].aver);  /* 不能用%10.0f */
    }
    fclose(fp);
}
```

2.3 基于 C/S 结构的 C 语言编程题考试自动评分系统简介

实现机上无纸考试对于提高学生的上机积极性，全面考核并促进学生编程实践能力的提高具有重要的实用意义和推广价值。然而，传统的程序设计语言考试自动评分系统，要么只能对客观题型，即单选、多选和正误判断等标准化试题进行自动评分，要么只能对没有语法错误的编程题按照程序运行结果的正误进行自动评分，对于有语法错误的编程题只能给 0 分。另一方面，通常都没有考虑学生程序是怎样实现编程任务的，不能从程序文本的语法结构和语义角度衡量学生程序与正确答案的接近程度。因此，评分方法不符合教师人工阅卷的思维习惯。

为了克服以上评分方法的缺陷，从理解、分析程序源代码的角度对程序进行评分，设计实现了基于程序理解的 C 语言编程题自动评分系统。该系统的主要优点是不仅能对客观

题进行自动评分，还能对C语言主观编程题进行自动评分，而且评分时不仅测试程序的运行结果，还分析程序的结构和语义，衡量实现编程任务的正确程度，对于不完整和有语法错误的程序也能给出合理的分数，因此更符合教师人工阅卷的结果。此外，采用机器评分代替教师手工评阅试卷，不仅显著提高了阅卷效率，减轻了教师负担，而且由于系统在学生提交试卷后能实时给出评分，增加了成绩评定的透明度和公正性，以及评分的准确性和实时性，避免了人工阅卷主观人为因素对评分的影响，确保了评分的客观公正。该系统主要采用了以下几种评分策略。

(1) 基于程序语义分析的评分

由教师提供一系列模板程序(相当于提供编程题的参考答案)，记录在答案文本文件中。评分过程是将学生程序进行程序标准化转换，消除程序表达方式多样化后，与模板进行程序匹配，计算它们的语义相似程度，给出学生程序的评分结果。语义相似度越高，学生程序得分就越高。该方法保证程序运行结果有误但基本思路正确的情况下也可以得到合理的分数。

(2) 基于得分语句查找的评分

由教师在模板程序中以程序注释的方式标定关键得分语句，并配置相应的分数。评分过程是对学生源程序进行词法分析后，查找能与关键得分语句相匹配的程序语句，并累计相应的分数。该方法保证在学生程序有语法错误、不能正常运行的情况下，也能得到合理的分数。

(3) 基于动态测试的评分

由教师给定程序相应的测试用例集，即输入、输出对的集合。评分过程是运行学生程序，然后判定对于特定的输入程序能否得到期望的输出，最后根据输出的结果给定分数。该方法保证在教师提供的答案模板不全、学生采用特殊方法正确编程的情况下，也可以得分。

该系统具有如下特点：

(1) C语言主观编程题的自动评分

该系统不但能对客观题型进行自动评分，还能对主观编程题进行自动评分，突破了以往考试系统只能对客观题自动评分的局限性，为程序设计语言考试提供了一种全新的模式。

(2) 丰富的题型，全面的能力考核

系统提供了单项选择、多项选择、判断正误、简单填空、程序填空、普通编程、实验、复杂编程、一般改错和附加改错等10种题型，其典型运行界面如图2-52~图2-57所示。其中后6种题型的运行界面相同，都需要学生在答题区内输入完整的源程序。系统可以进行简单的语法检查，但是鼓励学生在自己熟悉的编译环境下进行程序的调试，然后将调试正确的源程序复制粘贴到系统的答题区内。

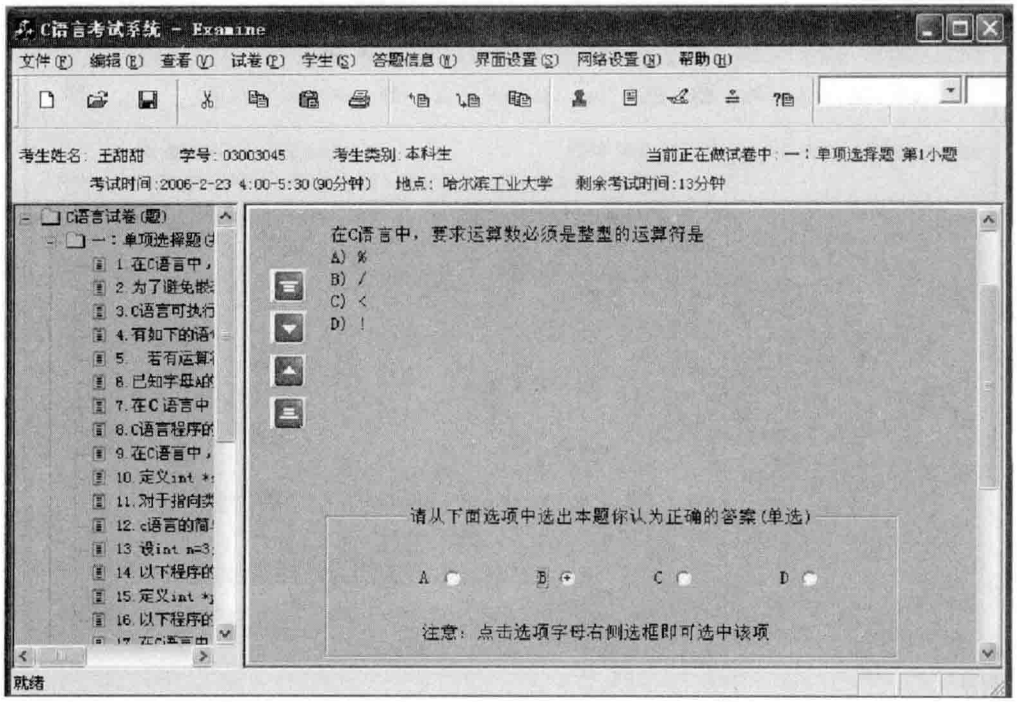


图 2-52 单项选择题答题界面

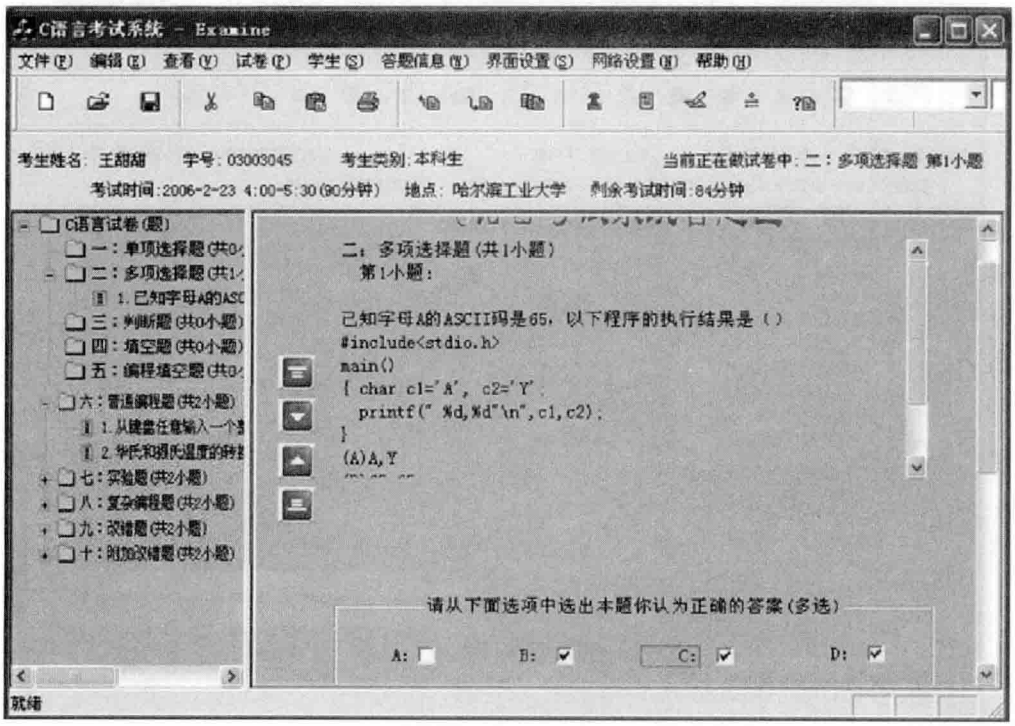


图 2-53 多项选择题答题界面

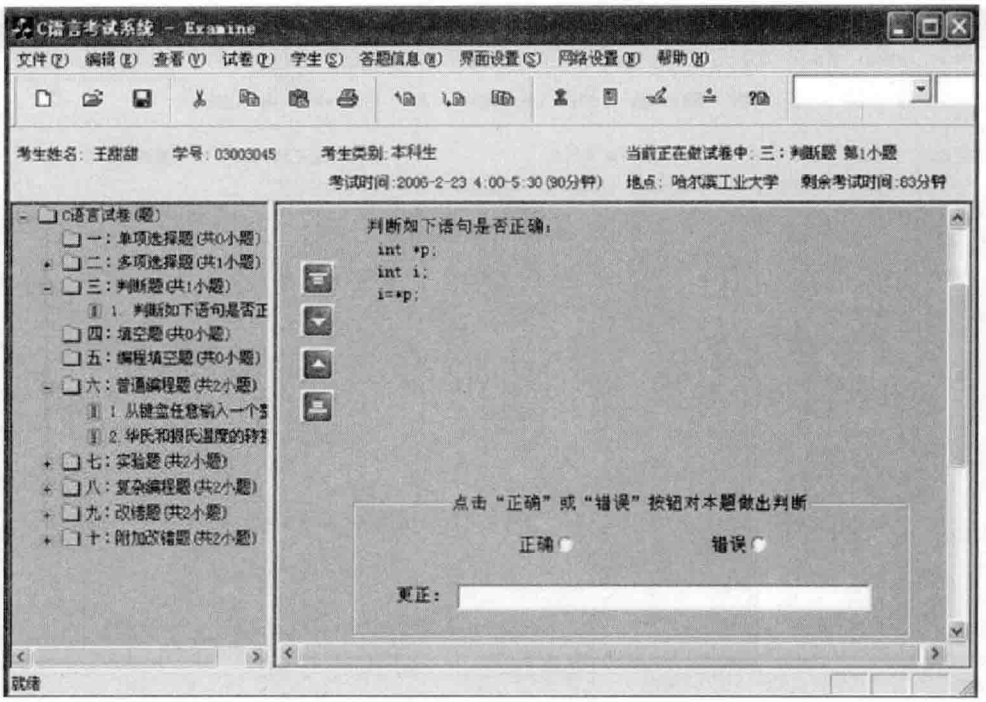


图 2-54 判断正误题答题界面

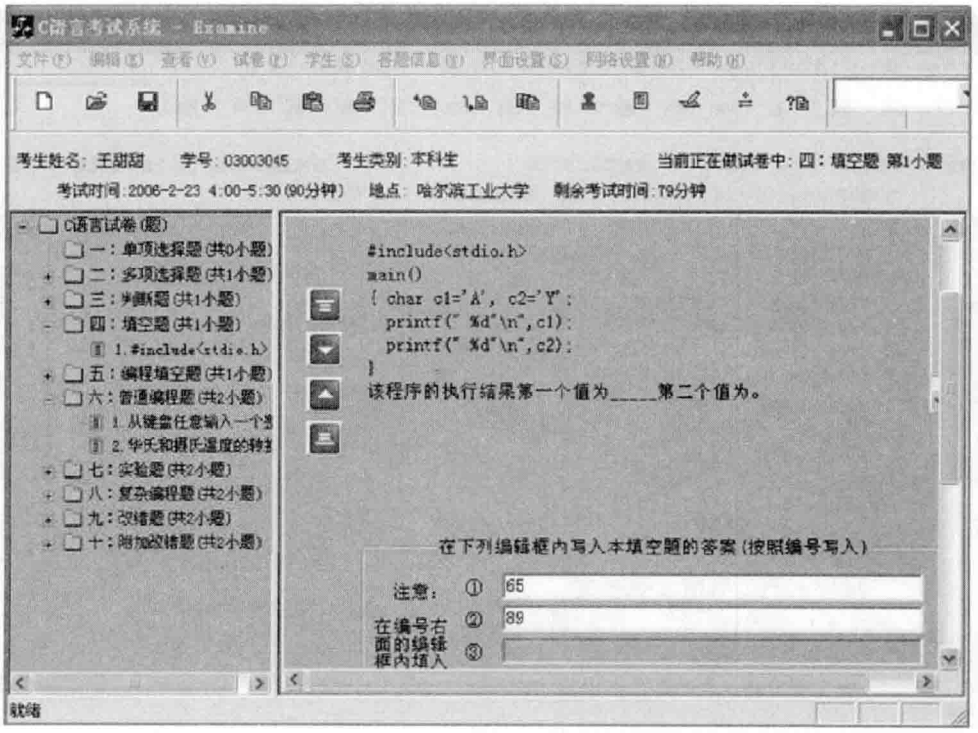


图 2-55 简单填空题答题界面

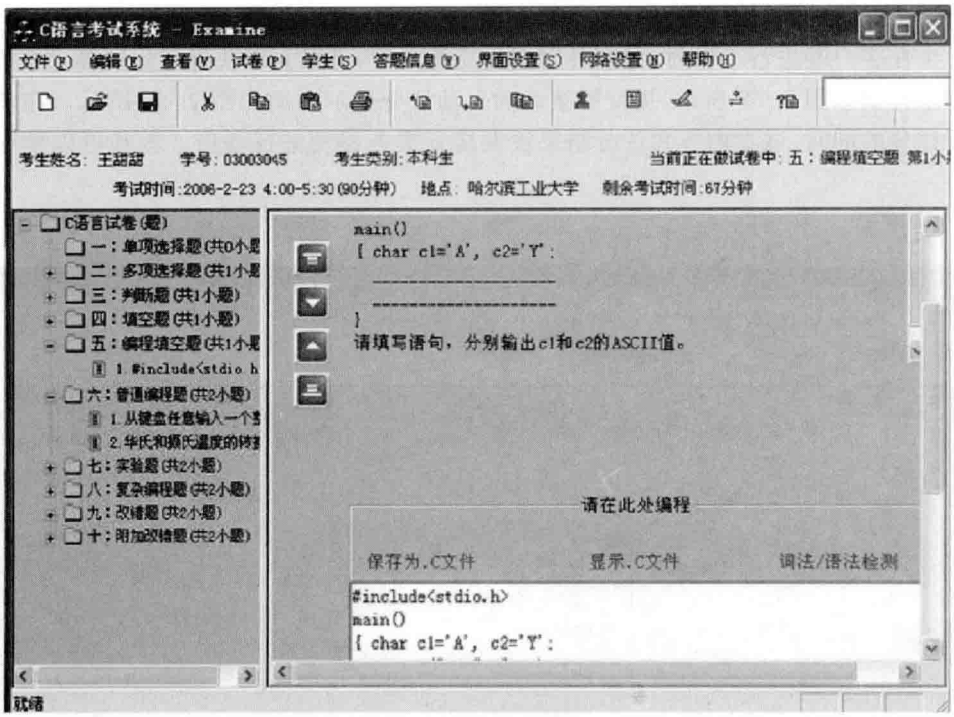


图 2-56 程序填空题答题界面

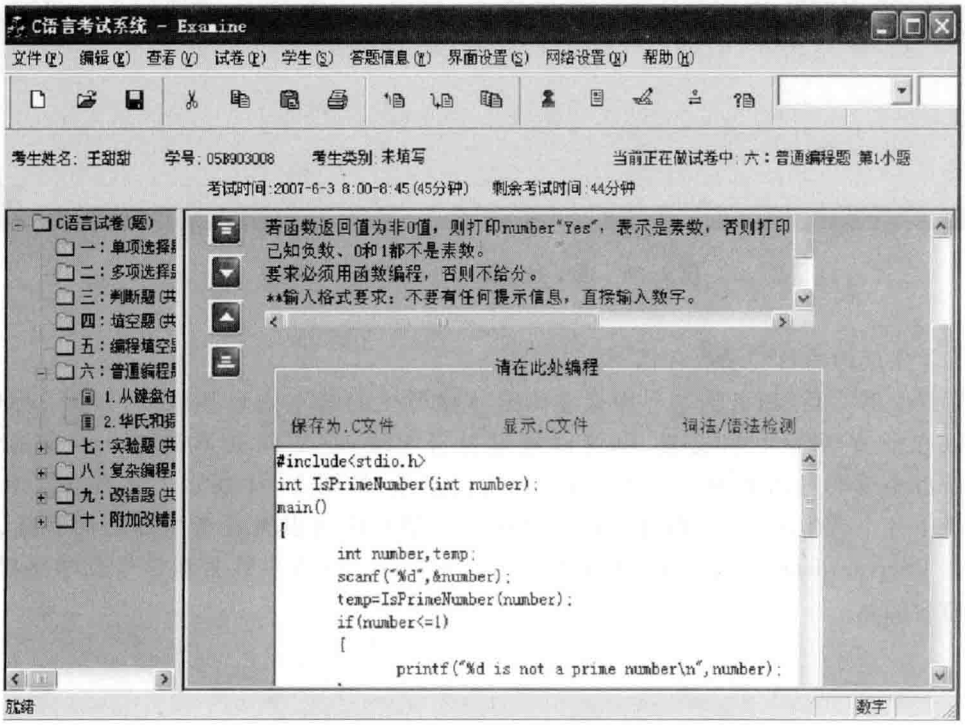


图 2-57 普通编程题答题界面

（3）学生成绩的实时评分反馈

当学生在客户端进行考试时，向服务器端提交答案后，可以立即获得评分结果。系统的客户端用户登录界面如图 2-58 所示，提交答案后的自动评分显示界面如图 2-59 所示。在学生提交试卷获得评分的同时，试卷内容和评分结果被发送到服务器端进行备份。教师可以复制下来单独存档。

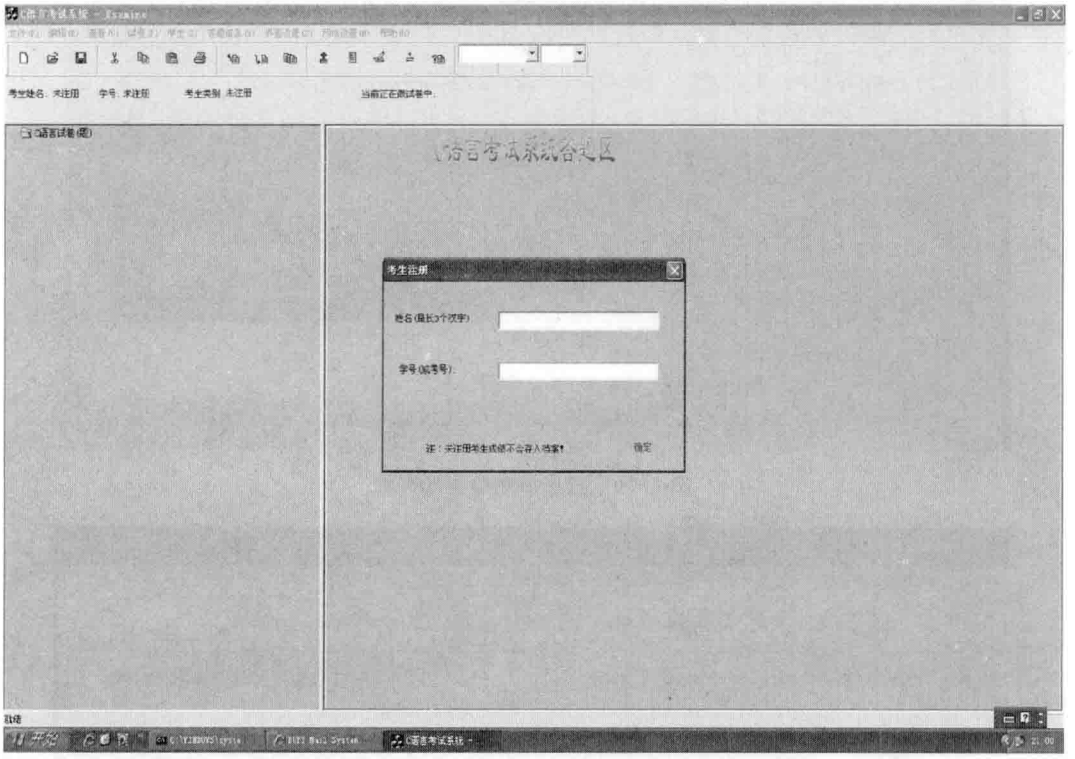


图 2-58 考生登录输入姓名和学号的界面

（4）学生成绩的自动录入存档

考生提交答案后，服务器端界面会立即显示该考生的基本信息和成绩，如图 2-60 所示。教师可以在此查看考生的成绩，也可以通过打开 EServer 文件夹下的 Access 数据库文件 EServer.mdb查看和导出系统自动录入存档的考生成绩，如图 2-61 所示。该数据库中的 score 表中记录了各个考生的考号、姓名以及考试成绩。教师还可以查看考生提交的答案，这些答案存储在 EServer\data\stuProgram 文件夹中。每个文件是以考生姓名和学号及学生提交答案的时间命名的。

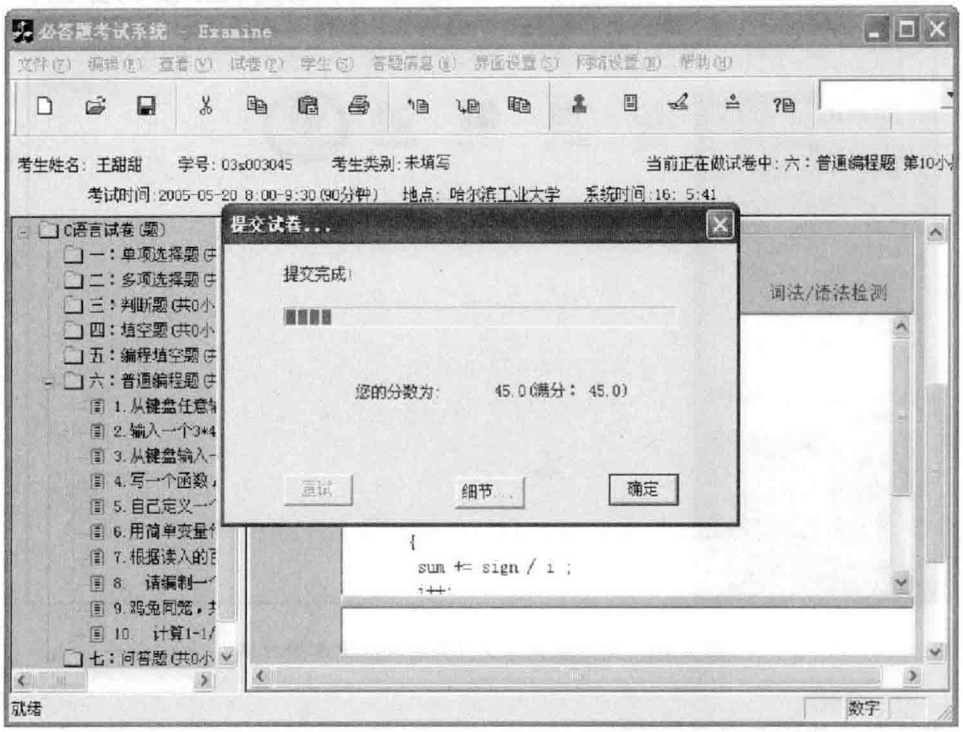


图 2-59 提交答案后的自动评分显示界面

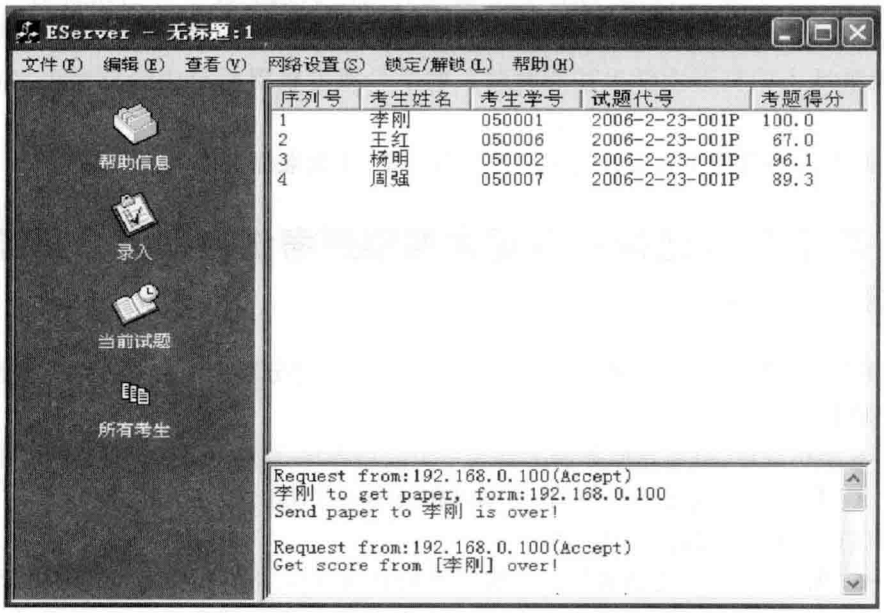


图 2-60 服务器端显示考生成绩

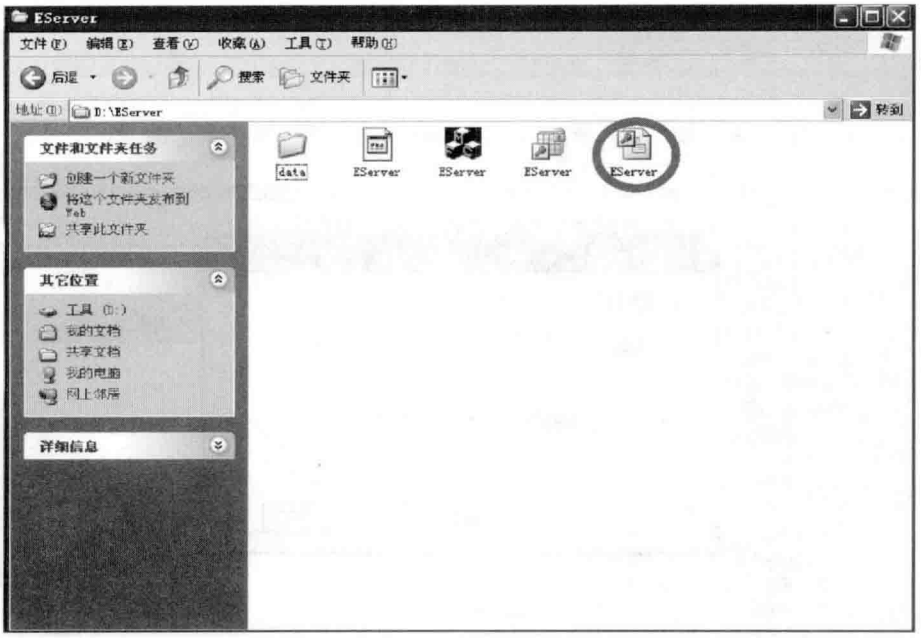


图 2-61 通过数据库文件查看和导出考生成绩

(5) 防止学生使用 U 盘考试作弊

为防止考生在考试过程中使用 U 盘作弊,Examine 学生客户端提供了 U 盘禁用功能,启动 Examine 学生客户端后系统将无法识别 U 盘。如果学生在启动系统前插入了 U 盘,那么启动Examine学生客户端后,系统仍能使 U 盘成为无法识别的设备。当系统对 U 盘的禁用失效时,系统一旦检测到 U 盘插入,将启用第二级防范措施——报警机制,由客户端向服务端报告 U 盘被插入的机器上学生注册时输入的学号或者机器所对应的机器名字(在学生未注册时)。

该系统的主要缺点是不具备试卷和题库管理功能,需安装服务端和客户端。

2.4 基于 B/S 结构的 C 语言编程题考试自动评分和题库管理系统简介

该系统的评分策略与 C/S 结构的 C 语言编程题考试自动评分系统相同,系统的主要特点如下:

(1) 安装和使用方便

采用 B/S 架构设计,只需在服务器上安装一个应用程序,学生可使用浏览器访问系统(如图 2-62所示)。不仅可以在局域网内使用,还可以在公网上使用。

(2) 考试过程的全程监控和防作弊

如图 2-63 所示,教师在考试过程中不仅可以随时查看学生登录系统和试卷提交情况,还能实时查看每个学生的答题进度和答题结果,并对学生机器和提交试卷进行锁定,防止学生反复提交试卷。此外,还具有强大的防作弊功能,不仅能在考试过程中禁用学生机的 USB 接口,实时监

图 2-62 学生使用浏览器进入考试系统的答题界面

控学生 U 盘插拔次数并报警，还可以在考试结束后对学生试卷中的试题进行查重，杜绝考试作弊现象的发生。

（3）试卷管理、题库管理、用户管理、成绩统计分析等功能

题库中有两千余道题目，题型包括单选题、判断题、一般填空题、程序填空题、普通编程题、复杂编程题、一般改错题、附加改错题等。

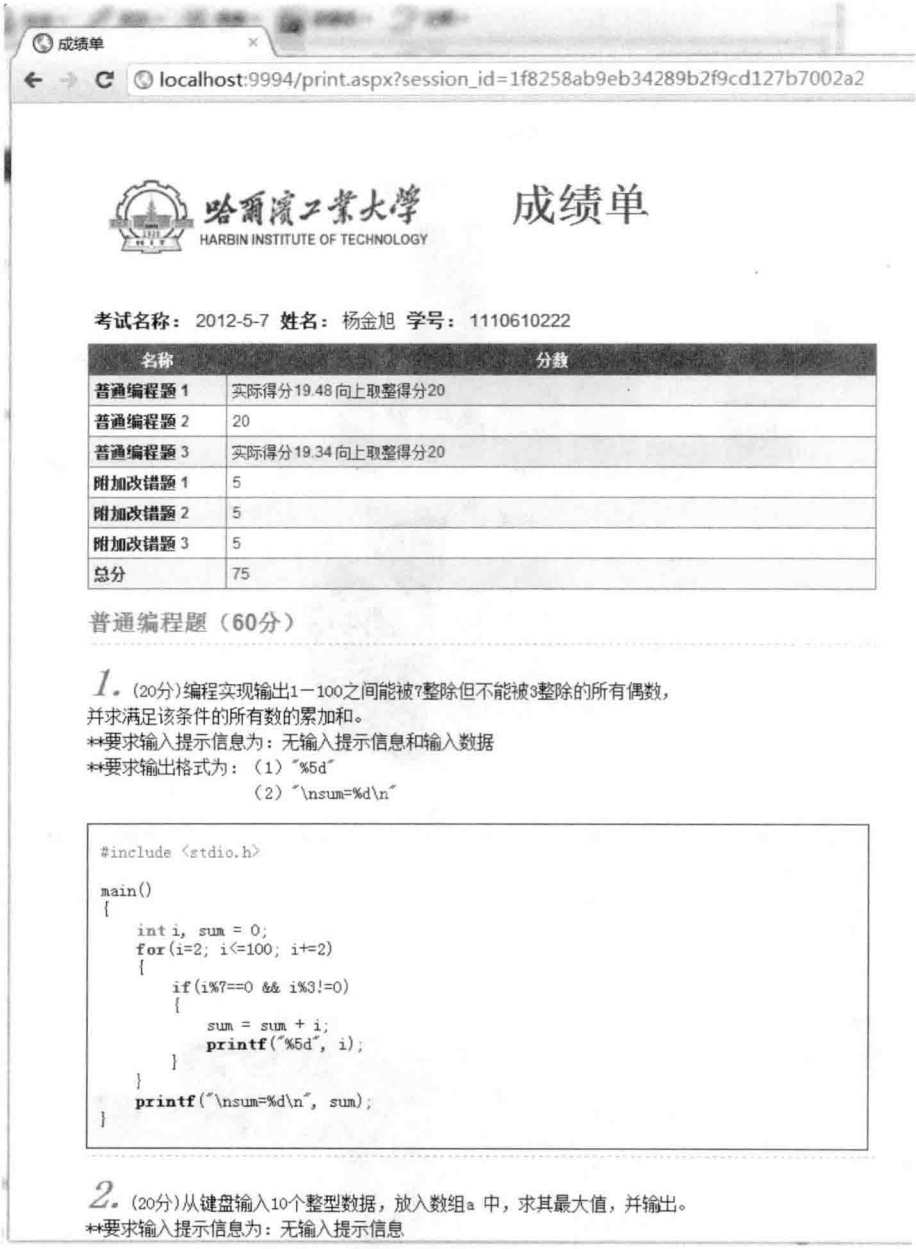
教师出题更加方便，既可以对题库中试题进行增删改，也可以利用题库中的试题进行抽题组卷，形成多张试卷，学生可以任选一套试卷进行考试，但每场考试每台机器和每个学生只能提交一次答题结果。

考试结束后，不仅可以对学生试卷进行自动存档，生成每个学生的 PDF 格式的成绩单（如图

图 2-63 考试过程的全程监控和防作弊界面

2-64所示)以及成绩汇总的数据库文件,还可以按照分数段和各题的得分情况对某场考试的试卷进行统计分析,生成柱状图或饼状图(如图 2-65 所示),帮助教师分析试题的难易程度和学生的考试成绩。

此外,还可以对题库中的试题进行查重(如图 2-66 所示),避免题库中的试题重复,提高题库试题的质量。


成绩单

localhost:9994/print.aspx?session_id=1f8258ab9eb34289b2f9cd127b7002a2

哈爾濱工業大學 HARBIN INSTITUTE OF TECHNOLOGY

成绩单

考试名称： 2012-5-7 **姓名：** 杨金旭 **学号：** 1110610222

名称	分数
普通编程题 1	实际得分19.48 向上取整得分20
普通编程题 2	20
普通编程题 3	实际得分19.34 向上取整得分20
附加改错题 1	5
附加改错题 2	5
附加改错题 3	5
总分	75

普通编程题（60分）

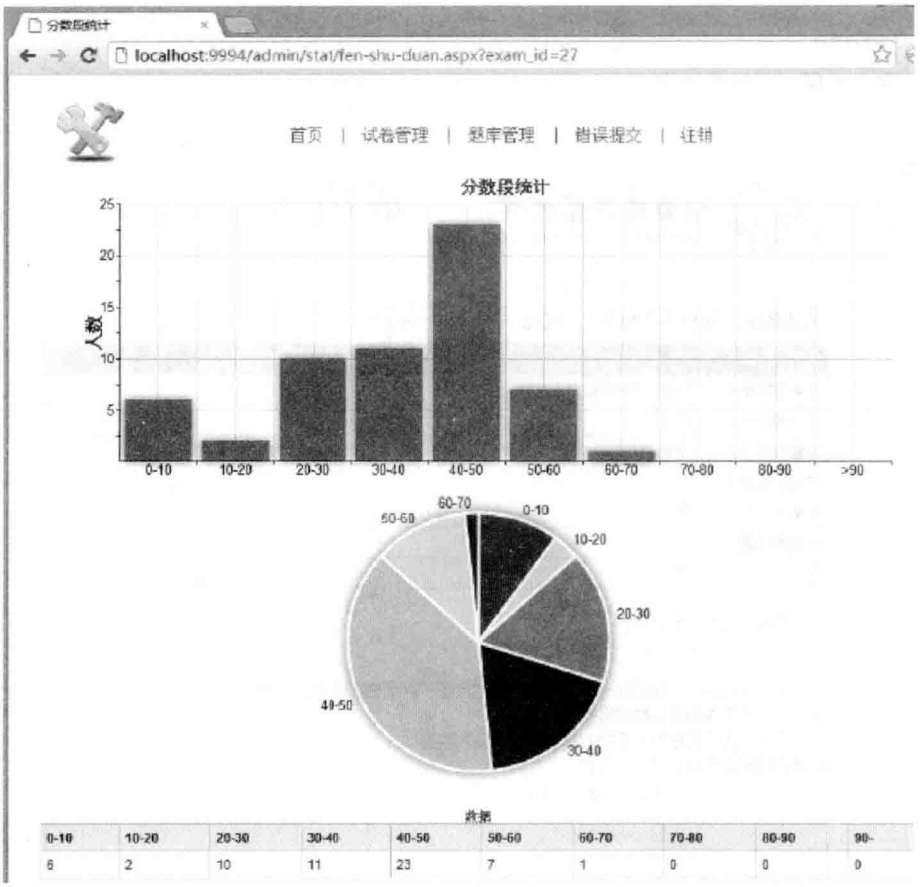
1. (20分)编程实现输出1－100之间能被7整除但不能被3整除的所有偶数，并求满足该条件的所有数的累加和。
**要求输入提示信息为：无输入提示信息和输入数据
**要求输出格式为：（1）"%5d"
（2）"\nsum=%d\n"

```
#include <stdio.h>

main()
{
    int i, sum = 0;
    for(i=2; i<=100; i+=2)
    {
        if(i%7==0 && i%3!=0)
        {
            sum = sum + i;
            printf("%5d", i);
        }
    }
    printf("\nsum=%d\n", sum);
}
```

2. (20分)从键盘输入10个整型数据，放入数组a 中，求其最大值，并输出。
**要求输入提示信息为：无输入提示信息

图 2-64 试卷存档与生成的学生成绩单



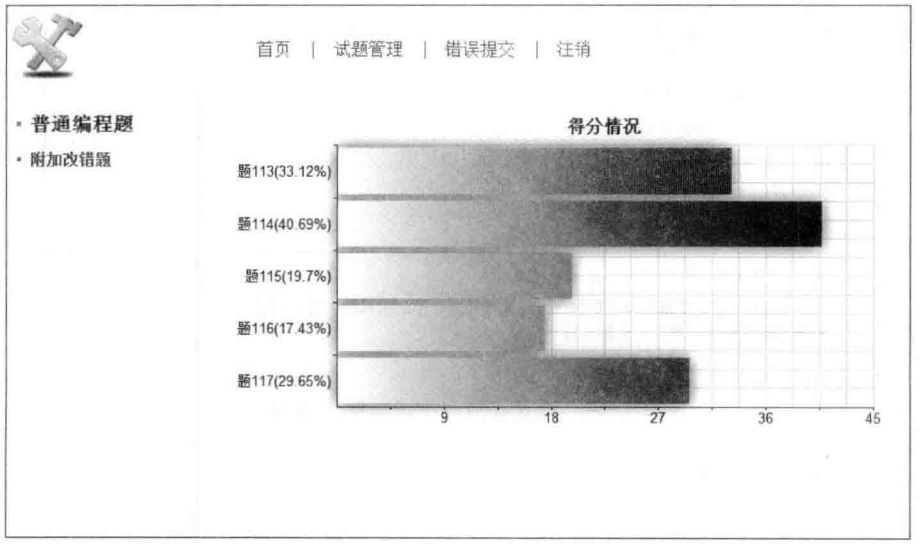


图 2-65 考试成绩分析的界面

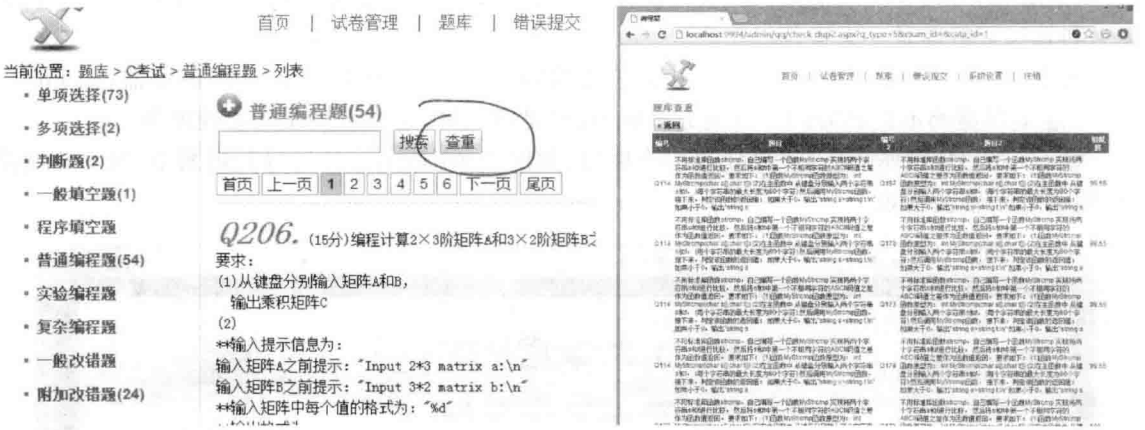


图 2-66 试题查重的界面

2.5 在线评判简介

在线评判(Online Judge,OJ)的基本形式是提供一个庞大的在线题库,参与者在其中寻找感兴趣的题目,并尝试编程解决,之后把源代码提交到网站,网站立刻进行评判,即时给出结果。将OJ引入C/C++语言的教学中,可以为学有余力的学生提供一个自我学习的平台。

OJ起源于一项世界级的大学生程序设计竞赛——ACM/ICPC(ACM International Collegiate Programming Contest,http://icpc.baylor.edu)。这项竞赛由世界最大的计算机协会——ACM(Association for Computing Machinery,http://www.acm.org)主办,每年一次,现在每年全世界都有上千所大学、上万名学生参加这个竞赛。ACM/ICPC比赛所用的评测系统就是一种OJ—PC^2(http://www.ecs.csus.edu/pc2/)。不过这个系统是专门为比赛设计的,不适合平时练习使用。因此,为了训练队员,很多的OJ被开发出来,并面向世界提供服务。

世界范围内最著名的OJ是西班牙的Universidad de Valladolid(http://acm.uva.es)和美国的TopCoder(http://www.topcoder.com)。前者是民间OJ的鼻祖,而后者在成功的商业模式的支持下,成为功能最丰富的程序设计在线竞赛平台。在TopCoder做题不仅免费,甚至还能赚美金。

因速度和网络流量费等原因,中国大学生更喜爱国内的OJ。不仅在这些OJ上做题,更在其上热烈地讨论、互相交流,还自己出题组织一些在线比赛,形成了一种独特的ACM文化圈。

国内有很多OJ,不可能全部列出来,这里只简单介绍几个比较著名且稳定的OJ。

北京大学的OJ(http://acm.pku.edu.cn/JudgeOnline)被称为POJ,是国内的大本营,有3 000余道题目,支持C/C++、Java和Pascal等语言。有奖月赛是它的特色之一。

浙江大学的OJ(http://acm.zju.edu.cn)被称为ZOJ,是国内最早的OJ,现有2 800多道题,支持C/C++、Java、Pascal和Python等语言。

哈尔滨工业大学的OJ(http://acm.hit.edu.cn)被称为HOJ,是本书作者所在学校的OJ,现有

1 400 多道题,支持 C/C++、Java、Pascal 等语言。除了经常举办在线比赛外,正在丰富中的新手指引和算法资料库是一大特色。

现在以 HOJ 为例介绍 OJ 的一般用法。OJ 的使用很简单,只要一个 Web 浏览器,如 Firefox 或 IE,连编译器都不是必需的。但绝大多数 OJ 都是英文的,这一点要做好心理准备。

登入一个 OJ 网站后,首先要注册一个账号,如图 2-67 所示。注册过程很简单,填写相应信息,提交后账号就生效。

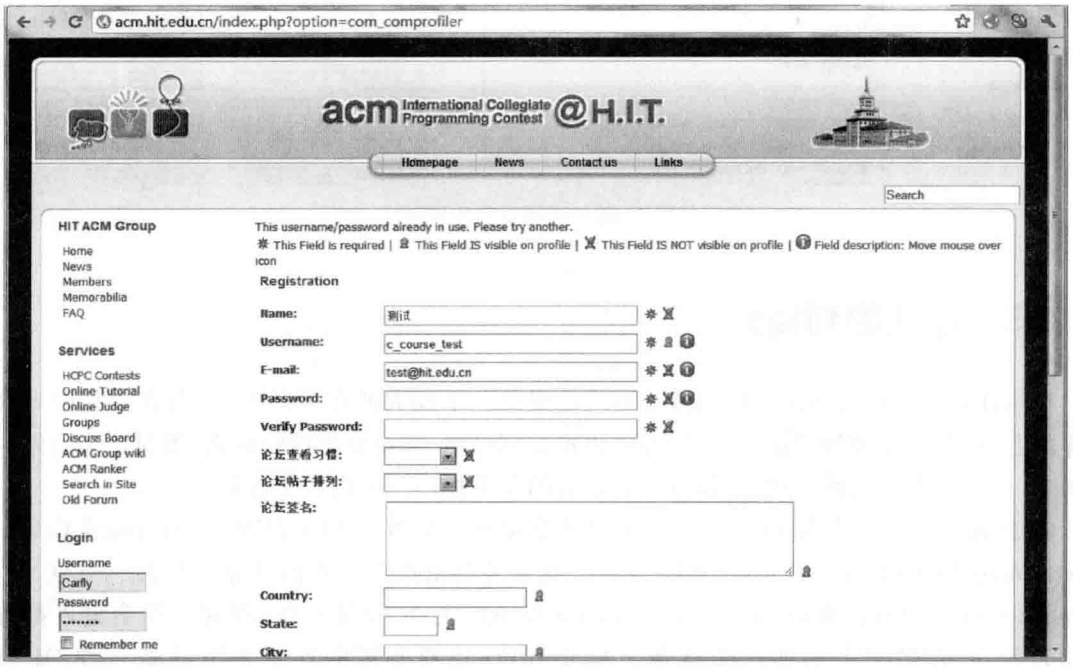


图 2-67 在 HOJ 上注册新账号

接下来找到题库 Problem Set,如图 2-68 所示。最左侧的对号标志着这道题你已经通过(当然,刚注册的用户看不到任何对号)。程序通过自动评判又称为“ac”(Accepted 的缩写)。那么到底什么样的程序算是通过呢? 如果通过分析用户提交的程序来判断该程序是否能够解决题目给出的问题是不可能的,因此 OJ 通过“测试用例”来判断一个程序是否能通过。也就是只要程序能够对给定的输入进行分析和计算,并输出和用例一致的结果,就可以算作通过。可见对于“正确”的程序,都能够通过所有的测试用例,而如果程序“错误”,则很难通过所有的测试用例(出题人在设计测试用例时,往往使用大量的数据或者苛刻数据测试程序对边界条件的处理能力)。

每道题都有唯一的 ID,然后是该题的题目名和链接,最后一列的 Ratio 是一个关键参数,它说明了每道题一共被提交了多少次,其中 ac 了多少次,通过率是多少。通过这个参数可以大体看出每道题的难度。一般来说,通过率越高的题目越简单。

下面以最简单的 1 001 题介绍如何在 HOJ 上提交代码,1 001 题目如图 2-69 所示。

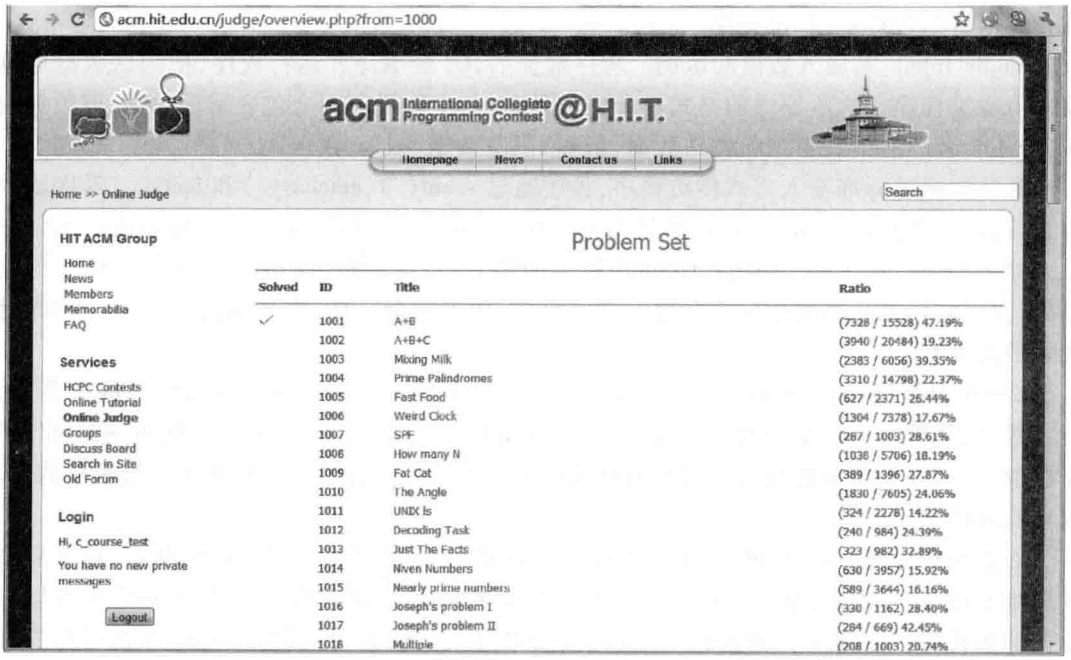


图 2-68 HOJ 的题库

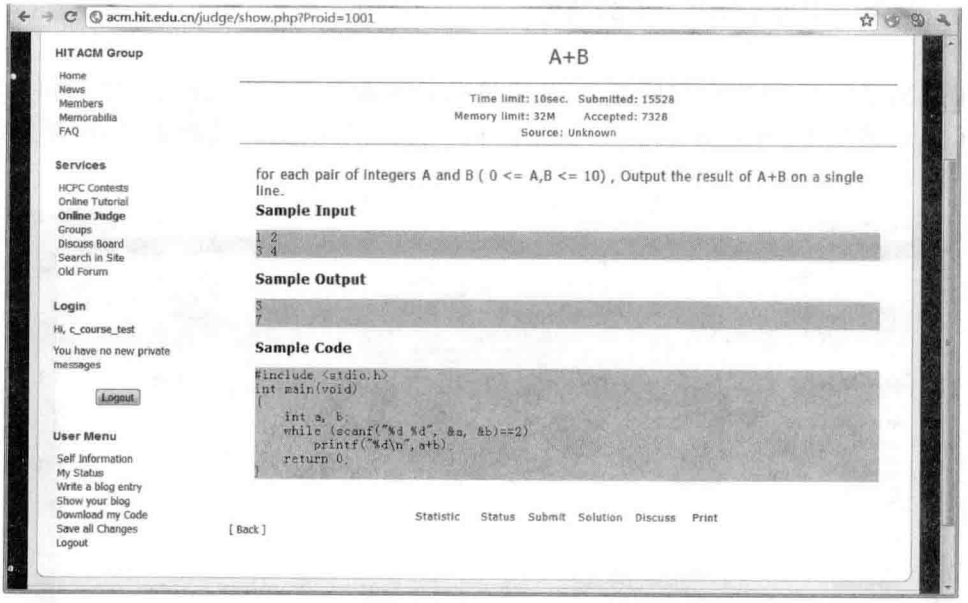


图 2-69 HOJ 编号为 1 001 的题目

1 001 题是 OJ 的“Hello World”,就是计算两个数的和,而且连答案都给出来了。题目标题下有两个重要参数 Time limit 和 Memory limit,是程序的运行时限和内存限制。如果你的程序超出了此限制,就算错。接下来的题目说明了此题需要你解决什么问题。很多题目的描述都图文并

茂且有小故事作为背景，非常有趣。

“Sample Input”是输入数据的示例。程序提交后，OJ 会编译并运行程序，把测试数据通过标准输入传递给程序。这些数据的格式会严格依照示例，中间肯定不会包含错误数据，但数据规模会很大，还会有一些不易想到的特殊数据，俗称“阴险数据”或“e 数据”。

输入数据通过标准输入方式传给程序，程序通过 scanf( )、getchar( ) 和 fgets( ) 等库函数读入数据后，按照题意计算结果，并把结果用 printf( )、putchar( ) 和 puts( ) 等库函数输出到标准输出。输出必须严格按照“Sample Output”所示的格式，一个空格、回车都不能错。OJ 会得到这些输出，并把它们和正确的输出进行逐字节的比较，如果完全一样，那么恭喜，ac 了！否则，就是一次失败的提交。

图 2-69 中的“Sample Code”很恰当地描述了这个过程。首先用 scanf( ) 读入两个整数，如果读入成功，就输出这两个数的和，并在数字后面加上一个回车。如果读入不成功，则退出程序。这里提醒一下，并不需要把所有的测试数据都读入后再计算、输出，只要读进来一组完整数据，就可以计算并输出了。

怎么提交程序呢？图 2-69 页面底端有几个链接。“Status”是查看此题的状态，点开后会详细列出都有谁 ac 了此题，谁的程序效率最高，没 ac 的都犯了什么错误。“Solution”中可以下载你自己的源代码。“Discuss”链接到论坛，在上面可以找到前人对此题的感想，你也可以发问或参与讨论别人的问题。“Submit”就是提交了，如图 2-70 所示。输入用户名和密码，题目编号已经自动填好，选择你使用的语言，粘贴代码，就能提交了。因为程序在服务器编译和运行需要一定的时间，所以提交后并不能马上看到结果。稍等一下，在全站的“Status”就能查看到结果，如图 2-71 所示。

这里列出了最后 20 次提交的状态，从中可以看到谁正在做哪道题，结果如何。我们当然更关心自己的结果，并希望它是“Accepted”的。除此以外，还有几种结果，其含义如表 2-3 所示。

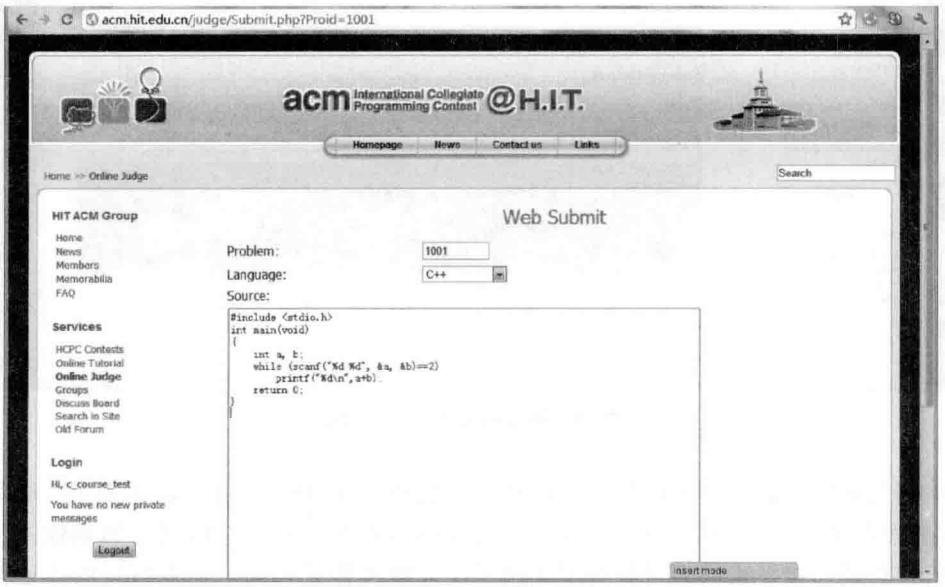


图 2-70 向 HOJ 提交程序

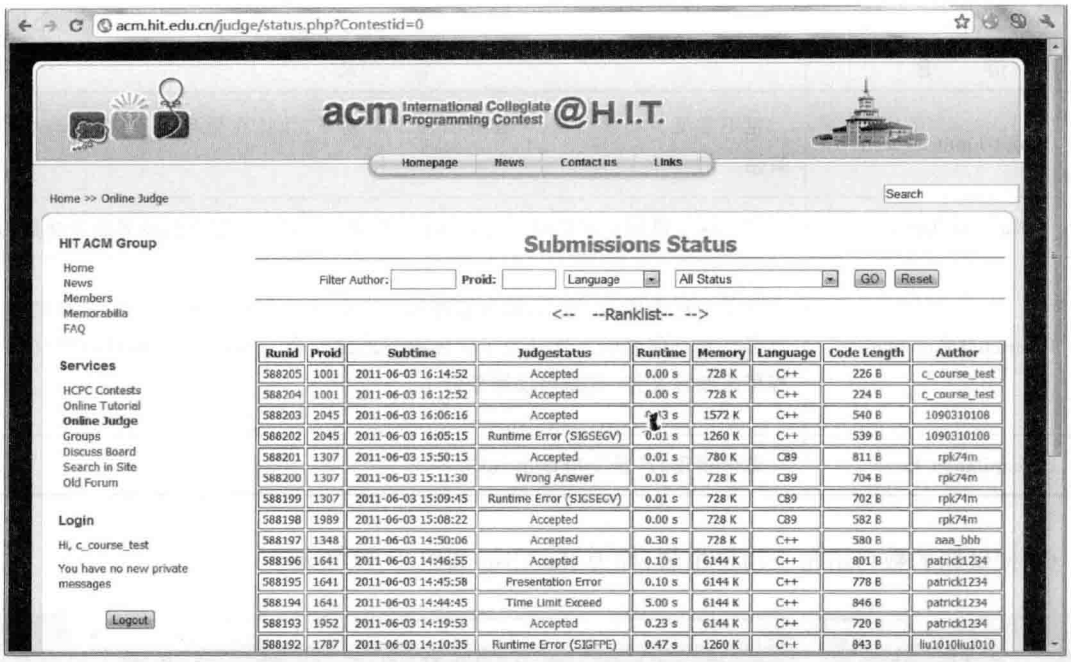


图 2-71 HOJ 上的实时答题状态

表 2-3 HOJ 结果信息表

信 息	含 义
Accepted	程序通过自动评判
Queuing	排队中,表示你的程序提交成功,正在等待处理
Running	程序运行中,请耐心等待结果
Presentation Error	俗称 PE,表明程序的输出结果并不错误,但格式不对,如多了回车或少了空格等
Wrong Answer	俗称 WA,说明程序运算错误。无论是仅有一条测试数据错误还是所有测试数据都错误,OJ 给出的结果都是 WA
Abnormal Termination	程序最后没有 return 0。从工程角度来说,程序返回 0 以外的数就表示程序出错了
Time Limit Exceed	俗称 TLE,简称超时。可能需要优化算法,或者排除程序中的死循环
Memory Limit Exceed	并不常见的错误,仅在使用了超出题目限制的内存时才会出现

续表

信　息	含　义
Output Limit Exceed	也不常见。如果出现，一般说明程序死循环，并且在循环中不停地输出垃圾数据
Runtime Error	出现这个错误一定是程序有大 bug，如使用野指针、数组下标越界或者除数为 0
Restricted Functions	除了与标准输入/输出、数学运算和字符串处理有关的函数之外，其余的库函数都是禁止使用的。这是为了保障服务器的安全，否则任何人都可以提交一个程序来删除文件、重新启动服务器了
Compilation Error	编译错误，这是不应该出现的

OJ 按做题的数目给每个用户排名，称为 Ranklist，如图 2-72 所示。

Rank	Name	Self Introduction	Accepted	Totalsubmit	Ratio (AC/submit)
1	EvilSeraph	Enjoy Your ACM Life!	1369	2404	56.95 %
2	Edelweiss	HIT 终于圆了 Final 梦!	1348	2792	48.28 %
3	weisili	各位加油	1047	2176	48.12 %
4	imems	Baidu牛final带必须请两频 在HOJ用JAVA太容易被鄙视	1010	1767	57.16 %
5	fishcanfly		979	2104	46.53 %
6	wywcgs	bless各位的缓考.....	968	1760	55.00 %
7	NicYun	whoami	935	1878	49.79 %
8	Robin	http://www.robin47.com | Ruby Coder	900	2230	40.36 %
9	GTmac	前途未卜	885	1776	49.83 %
10	sdfond	Bless HIT Teams	862	1964	43.89 %
11	Williammed	前途渺茫，PKU Possible? 这个号AC的题都是大水题	850	2490	34.14 %
12	ZhouGuyue	在有生之年 我应该也能刷到1000题 ...	848	1406	60.31 %
13	PerfectCai	每次用神灵武士都超神 嘎嘎	823	2016	40.82 %

图 2-72　HOJ 的用户排名

2.6　C 语言程序设计学习方法指南

虽然 C 语言问世已近 40 年，但它仍以其精炼的语法、强大的功能和广泛的应用吸引着众多

的初学者和开发者。大到宇宙飞船、航空母舰，小至嵌入式设备，C 语言的身影无处不在。因此，C 语言已成为大学生，尤其是理工科学生应该掌握的一项基本技能。

然而，有很多学生觉得 C 语言难学难用，经常产生一些畏惧感。其实只要掌握一些方法，克服心理上的恐惧，不轻言放弃，是完全可以学好 C 语言的。

在 C 语言的学习过程中，主要有以下几方面需要注意。

（1）注重培养计算思维，不纠结于语法细节

由于 C 语言语法的灵活性，在学习过程中会接触很多技术细节，例如前缀++运算符、运算符的优先级与结合性等问题。但千万不要淹没在各种技术细节的海洋中，如果将主要精力放到死记硬背那些琐碎的语法细节（如运算符的优先级，输入/输出函数的格式转换符等）上，则不仅会只见树木不见森林，捡了芝麻丢了西瓜，还会觉得 C 语言过于晦涩难懂，产生畏惧情绪，渐渐失去学习的兴趣和信心。

学生应注重培养和训练计算思维。所谓计算思维，是运用计算机科学的基础概念进行问题求解、系统设计以及人类行为理解等涵盖计算机科学之广度的一系列思维活动。在 C 语言的学习过程中，主要涉及抽象、递归、基本算法复杂度的估计等思维方法，因此理解并使用 C 语言实现这些计算机学科的基本思维方法，将 C 语言看成是一种用计算机进行问题求解的工具，要比对 C 语言语法的学习重要得多。

（2）勤学勤练，熟能生巧

编程是一个实践性很强的工作，最好的学习方法就是勤学勤练，边看书边调试代码，把书上的例子程序反复地修改调试运行，通过增删语句，观察程序运行结果的变化，理解语句的作用，从中得到自己的体会，并注意总结解决问题的通用方法，形成一定的范式。

卖油翁的故事告诉我们，学习程序设计，只有多多上机实践和自己动手编程，才能提高编程能力，成功是没有捷径的。想写出好的文章，除了掌握语言，还要多读多写，才能自如运用语言写出漂亮的文章。“熟读唐诗三百首，不会吟诗也会诌”。编程也是如此，写出漂亮的程序不是听课听出来的，也不是看书看出来的，而是读别人的程序读出来的，更是自己动手练出来的。

除了熟悉书上的例题，练习书后的习题外，还可以自己多找一些更有挑战性的问题，例如大学生程序设计竞赛（ACM/ICPC）的练习题或者竞赛题等，这些题目往往具有一定的实际应用背景，而且在 HOJ 网站（http://acm.hit.edu.cn/）上提交自己的代码后，会立即获得结果是否正确的反馈。多加练习后，编程能力一定会获得很大的提高，而且会产生浓厚的学习兴趣。

（3）养成良好的代码风格

代码风格包括函数、变量的命名、注释的编写以及缩进等。养成良好的代码风格，对于代码日后的维护、团队的合作等都十分有益，因此养成良好的代码风格非常重要。初学者可以遵守一个较为通用的代码规范，当然最好的方法还是学习别人的代码是怎么写的。学习别人的代码也是提高自己实际编程能力一个很重要的途径，现在网络上有很多高质量的开源代码可供学习。

（4）在错误中学习，学会思考，提高能力

虽然我们提倡不懂就问，但是不要因此而产生一种对教师或他人的依赖心理，在问问题之前要自己先动脑思考一下，在思考以后再问，不只关注问题的答案，更要关注问题分析的过

程，这样才能真正提高自己的思考能力。也不要指望代码输入到计算机后能够“一次通过”，在调试错误代码的过程中往往会学习到更多，编程能力的提高总是伴随着程序调试能力的提高而提高的。

王国维在《人间词话》中说，古今之成大事业、大学问者，必经过三种境界：昨夜西风凋碧树。独上高楼，望断天涯路。此为第一境界。衣带渐宽终不悔，为伊消得人憔悴。此为第二境界。众里寻他千百度，蓦然回首，那人却在灯火阑珊处。此为第三境界。成为编程高手的过程中，同样也要经历这样的三个境界。学好编程不能期望一蹴而就。人生如程序，程序亦人生。

参考文献

[1] SCHILDT H. C 语言大全[M].4 版. 王子恢,戴健鹏,译.北京:电子工业出版社,2001.

[2] KERNIGHAN B W, RITCHIE D M. The C Programming Language[M].2nd ed. 北京:清华大学出版社,1996.

[3] DEITEL H M, DEITEL P J. C 程序设计教程[M].薛万鹏,译. 北京:机械工业出版社,2000.

[4] KELLY P. A guide to C programming[M]. 3rd ed. Dublin: Gill & Macmillan,1999.

[5] KER NIGHAN B W, PIKE R. 程序设计实践[M].裘宗燕,译. 北京:机械工业出版社,2000.

[6] PRATT T W, ZELKOWITZ M V.程序设计语言:设计与实现[M]. 北京:电子工业出版社,2001.

[7] 苏小红,陈惠鹏,孙志岗,等.C 语言大学实用教程[M].3 版.北京:电子工业出版社,2012.

[8] 苏小红,孙志岗.C 语言大学实用教程学习指导[M].3 版.北京:电子工业出版社,2012.

[9] KELLY P,苏小红.双语版 C++程序设计[M].北京:电子工业出版社,2010.

[10] 周海燕,赵重敏,齐华山. C 语言程序设计[M]. 北京:科学出版社,2001.

[11] 陈朔鹰,陈英,乔俊琪. C 语言程序设计习题集[M]. 北京:人民邮电出版社,2000.

[12] 张高煜. C 语言程序设计实例[M]. 北京:中国水利水电出版社,2001.

[13] WIRTH N. 算法+数据结构=程序[M]. 北京:科学出版社,1990.

[14] 杨世明,王雪琴. 数学发现的艺术[M]. 青岛:中国海洋大学出版社,1998.

[15] 胡正国,蔡经球. 程序设计方法学[M].2 版. 西安:西北工业大学出版社,1992.

教育部高等理工教育教学改革与实践项目研究成果

本书特色：

- 实例兼顾趣味性和实用性，所有习题和实验题都有实际的应用背景，面向工程实践和计算思维能力训练，绝非单纯为解释语法而设计。
- 将程序设计方法和数据结构作为两条脉络清晰的主线，避免语法堆砌、舍本求末、只见树木不见森林等问题。
- 精心设计了一些含有隐蔽错误的分析改错题，注重错误程序的分析和讲解、程序设计风格、程序测试和调试方法。
- 配套的教学资源丰富，主教材支持读者在线测试书后习题。
- 本书为国家级精品资源共享课、中国大学MOOC课程的配套教学用书。

ISBN 978-7-04-043320-3

定价 33.00 元

[General Information]
书名=C语言程序设计学习指导　第3版
作者=苏小红，王甜甜，车万翔编著
页数=327
SS号=13868270
DX号=
出版日期=2015.08
出版社=北京高等教育出版社